Ingegneria del Software

Mattia Masina
CdL in Informatica – Università di Bologna

Anno Accademico 2026/2027

Indice

1 Introduzione e Contesto: La Produzione del Software **5**
- 1.1 La Trasformazione Digitale e il Contesto Industriale . . . 5
- 1.2 Definizioni, Classificazioni e Requisiti . . . 5
- 1.3 Dipendenze, Aspetti Legali e Garanzie . . . 6
- 1.4 Rischi, Qualità e AI . . . 6

2 Essence **8**
- 2.1 Origini, Kernel e Alpha . . . 8
  - 2.1.1 Origini e Iniziativa SEMAT . . . 8
  - 2.1.2 Cos'è Essence e i 7 Alpha del Kernel . . . 8
- 2.2 Metodi, Pratiche e Strumenti . . . 9
  - 2.2.1 Strumenti Pratici e Metodologie . . . 9
  - 2.2.2 Metodi, Pratiche e Ciclo di Vita degli Alpha . . . 10
  - 2.2.3 Elementi Complementari e Caso UniMagic . . . 11
- 2.3 Diagnostica di Progetto e Anatomia delle Pratiche . . . 13
  - 2.3.1 Diagnostica di Progetto (Project Health) . . . 13
  - 2.3.2 Anatomia delle Pratiche . . . 14
  - 2.3.3 Esercizio: Code Review come Pratica Essence . . . 14

3 Programmazione e Progettazione Orientata agli Oggetti **16**
- 3.1 Introduzione . . . 16
- 3.2 UML . . . 17
  - 3.2.1 Introduzione . . . 17
  - 3.2.2 Alcune definizioni . . . 22
  - 3.2.3 UML nel dettaglio . . . 25
  - 3.2.4 Object Oriented Concept Modeling . . . 26
  - 3.2.5 Object-Oriented Analysis (OOA) . . . 30
  - 3.2.6 Object-Oriented Design (OOD) . . . 49
- 3.3 Design Patterns . . . 66
  - 3.3.1 Singleton Pattern . . . 69
  - 3.3.2 Builder Pattern . . . 70
  - 3.3.3 Abstract Factory Pattern . . . 70
  - 3.3.4 Factory Method Pattern . . . 72
  - 3.3.5 Prototype Pattern . . . 73
  - 3.3.6 Adapter Pattern . . . 75
  - 3.3.7 Bridge Pattern . . . 75
  - 3.3.8 Decorator Pattern . . . 76
  - 3.3.9 Proxy Pattern . . . 77
  - 3.3.10 Visitor Pattern . . . 77
  - 3.3.11 Strategy Pattern . . . 78
  - 3.3.12 Chain of Responsibility Pattern . . . 79
  - 3.3.13 Mediator Pattern . . . 79

4 UML: esercitazioni **81**
4.1 La Notazione delle Classi (Class Icon) . . . 81
4.2 La Notazione degli Oggetti (Object Icon) . . . 82
4.3 Relazioni Strutturali: Generalizzazione e Realizzazione . . . 83
4.4 I Diagrammi dei Casi d'Uso (Use Case Diagrams) . . . 85
4.5 Esercizi sui Diagrammi delle Classi . . . 90
4.6 Esercizi Complessi (Classi e Casi d'Uso) . . . 94
4.7 Esercizi sugli Statechart . . . 100
4.8 Esercizi sugli Activity Diagram . . . 104
4.9 Esercizi sui Sequence Diagram . . . 106
4.10 Esercizio Completo: il Garage . . . 109
4.11 Esercizi su Componenti e Deployment . . . 114

5 Processi **116**
5.1 Introduzione a Scrum . . . 116
5.2 User Stories . . . 125
5.3 I Processi Agili . . . 135

6 Architetture Software **146**
6.1 Concetti di System Design . . . 146
6.1.1 Introduzione e Concetto di System Design . . . 146
6.1.2 Architettura di Sistema e Livelli di Astrazione . . . 146
6.2 Stili Architetturali . . . 147
6.2.1 Gli Stili Architetturali secondo Garlan e Shaw . . . 147
6.2.2 Stile 1: Pipes and Filters (Tubi e Filtri) . . . 148
6.2.3 Stile 2: Data Abstraction and Object-Oriented Organization . . . 149
6.2.4 Stile 3: Layered Systems (Sistemi Stratificati / a Strati) . . . 150
6.2.5 Stile 4: Event-Based, Implicit Invocation (Invocazione Implicita) . . . 150
6.2.6 Quadro Riassuntivo dei Primi Quattro Stili Architetturali . . . 151
6.2.7 Stile 5: Repositories (Repository e Architetture a Lavagna) . . . 152
6.2.8 Stile 6: Table-Driven Interpreters (Interpreti Guidati da Tabelle) . . . 152
6.2.9 Architettura a Repository Centrale (Central Repository) . . . 153
6.3 Architetture Client-Server, Multi-Livello e a Strati . . . 153
6.3.1 Il Modello Client-Server (C/S) . . . 153
6.3.2 Architetture Basate su Internet e Intranet . . . 154
6.3.3 Architetture Multi-Livello: Dal Two-Tier all'n-Tier . . . 154
6.3.4 Approfondimento sulle Architetture a Strati (Layered Architecture) . . . 156
6.3.5 Architetture di Rete a Strati: OSI e TCP/IP . . . 156
6.4 Architettura Applicativa e Modelli di Controllo . . . 157
6.4.1 Application Architecture e Modelli di Controllo . . . 157
6.4.2 Il Modello Call and Return . . . 158
6.4.3 Il Modello Manager . . . 158
6.4.4 Modelli di Controllo Guidati da Eventi (Event-Driven Control) . . . 159
6.4.5 Il Modello Broadcast . . . 159
6.4.6 Il Modello Guidato da Interrupt (Interrupt-Driven) . . . 160
6.4.7 Processi Multi-Threading e Concorrenza . . . 160
6.4.8 Modelli di Decomposizione Modulare Applicativa . . . 160

6.4.9 Il Modello Data-Flow (Flusso di Dati) . . . 160
6.4.10 Il Modello Orientato agli Oggetti (Object-Oriented) . . . 161
6.5 Principi Guida del Design . . . 163
6.5.1 Principi Guida del Design: Astrazione, Scala e Strategie . . . 163
6.5.2 Valutazione delle Alternative e Raffinamento Iterativo . . . 164
6.5.3 Quadro Sinottico Finale del Modulo 2 . . . 164

7 Java Strategy Pattern **166**
7.1 Motivazioni e Fondamenti: Polimorfismo e Scoping . . . 166
7.2 Classi in Ambiti Annidati (Inner Scopes) . . . 167
7.3 Classi Statiche Annidate (Static Nested Classes) . . . 168
7.4 Classi Interne (Inner Classes) . . . 170
7.5 Classi Locali (Local Classes) . . . 172
7.6 Contesti Statici e Non-Statici nelle Classi Annidate . . . 174
7.7 Cattura delle Variabili Locali: Final ed Effectively Final . . . 175
7.8 Pattern ed Esempi Applicativi di Classi Locali . . . 176
7.9 Significato di `static` e `final` . . . 177
7.10 Errori Comuni e Correzioni: Guida Pratica . . . 178
7.11 Interfacce: Definizione, Costanti ed Ereditarietà . . . 179
7.12 Metodi nelle Interfacce e Risoluzione dei Conflitti . . . 180
7.13 Evoluzione delle Interfacce e Retrocompatibilità . . . 183
7.14 Classi Anonime (Anonymous Classes) . . . 184
7.15 Gestione degli Eventi e Listener in Java GUI . . . 185
7.16 Caso Studio SimpleCalc: I Tre Approcci a Confronto . . . 185
7.17 Sintesi Architetturale: Dalle Classi Anonime alle Lambda . . . 188

1 Introduzione e Contesto: La Produzione del Software

1.1 La Trasformazione Digitale e il Contesto Industriale

La trasformazione digitale consiste nel **far vivere meglio le persone grazie al software**. Oggi ci sono **oltre 180 milioni di sviluppatori** su GitHub (dato 2025).

Leggi Fondamentali

- **Legge di Wirth**: "Il software diventa lento più rapidamente di quanto l'hardware diventi più veloce". Oggi: il codice si moltiplica più rapidamente della nostra capacità di comprenderlo.
- **Legge di Eagleson**: "Qualsiasi codice che non hai rivisto negli ultimi sei mesi ti sembrerà estraneo". Oggi: il codice generato da LLM non rivisto riga per riga ti sembrerà tuo finché non si rompe.

Il Software come Prodotto Industriale. Il software è **sempre il prodotto di un processo di sviluppo** che inizia con un'idea e termina con il ritiro. Il costo di sviluppo cresce con il **quadrato delle dimensioni**; gli assistenti AI abbassano il costo per riga ma non quello di capire, integrare e mantenere sistemi complessi.
Ordini di grandezza (SLOC): NASA Space Shuttle 1,8M; Linux Kernel 20M; Google (monorepo) 2 miliardi.

1.2 Definizioni, Classificazioni e Requisiti

Cosa è il Software: Quattro Prospettive

1. **Prodotto invisibile, intangibile, duplicabile**, costoso: opera dell'ingegno protetta da copyright.
2. **Servizio digitale**: ha un'interfaccia (API), basa su infrastruttura hw/sw/virtuale, garantisce QoS via contratto.
3. **Componente di sistema**: può essere *off-the-shelf* (larga diffusione) o *custom* (commissionato).
4. **Macchina astratta**: trasforma il computer realizzando funzioni utili; ha un'architettura (struttura + comportamento).

Categorie di Software: apps ed ecosistemi (App Store, Play Store), SaaS, embedded/IoT, social software (Web 2.0), **software libero** (4 libertà FSF: uso, studio, distribuzione, miglioramento), API economy, agenti e orchestratori AI.

Classificazione per Tipo di Prodotto

- **Prodotti Generici (OTS – Off The Shelf)**: creati da un produttore, venduti a molti clienti (es. videogiochi, Office).
- **Sistemi Commissionati (Custom)**: sviluppati per un cliente specifico (es. portale universitario).
- **Servizi in Perpetuo Sviluppo**: sistemi 24/7 in continuo cambiamento (es. Facebook, Amazon, chatbot).

Distinzione Fondamentale: Requisiti vs Feature

- **Requisito**: funzione o qualità *testabile* che l'implementazione deve possedere. *Importante per il cliente.*
- **Feature**: insieme di funzioni che permettono l'uso del prodotto in un servizio. *Importante per il fornitore.*

Esempio e-commerce: Requisiti = registrazione, carrello, indirizzi, pagamento. Feature = "Carrello per negozio elettronico" (es. OpenCart).

1.3 Dipendenze, Aspetti Legali e Garanzie

Dipendenze ed Ecosistemi. Ogni prodotto dipende da altri prodotti: si associa al sistema un **grafo di dipendenze** i cui nodi sono pacchetti (librerie) in versioni specifiche; la sua gestione è centrale per sicurezza, riproducibilità e manutenibilità. Un **ecosistema software** è un mercato di prodotti o componenti su una piattaforma definita da un'azienda, che evolvono nello stesso ambiente.

Licenze. Il software è **opera dell'ingegno** protetta da copyright (non brevettabile in Europa). Licenze: Proprietaria / Public Domain / Open Source (GPL, MIT, Apache, BSD...).

Garanzie e Terminologia

Il software commerciale è venduto **"as is"**: l'EULA esclude garanzie di commerciabilità, idoneità, assenza virus. Tutto il rischio è sull'utente.

- **Verifica**: aderenza alla specifica ("abbiamo costruito il prodotto giusto?").
- **Validazione**: accettazione dal cliente ("abbiamo costruito il prodotto che serve?").
- **Certificazione**: aderenza a specifiche di legge/standard.

1.4 Rischi, Qualità e AI

Rischi del software: di sviluppo; difetti nel software operativo; di esercizio; **allucinazioni durante la generazione AI**.

Caso studio – CrowdStrike (19 luglio 2024): aggiornamento difettoso di Falcon Sensor → 8,5 milioni di sistemi Windows bloccati, ≥10 miliardi $ di danni (aerei, banche, ospedali).
Difettosità (difetti/KLoC): peggiori sistemi militari 55; Agile (XP) 1,4; NASA Space Shuttle 0,1.

AI e Software: Punti da Ricordare

- **Sicurezza**: circa il 45% del codice generato da LLM contiene vulnerabilità (Veracode 2025).
- **Studio METR 2025** (16 sviluppatori esperti, 246 task): con strumenti AI risultano **19% più lenti**, ma percepiscono di essere +20% più veloci. *Percezione ≠ realtà misurata.*
- **Gartner**: rischio di "agent-washing"; >40% dei progetti agentici abbandonato entro il 2027.
- L'AI cambia il modo di lavorare, **non l'esigenza di verifica, responsabilità e comprensione profonda**.

Visione della disciplina: il software è infrastruttura cruciale di complessità enorme; servono innovazione continua e processi collaborativi (Agile & DevOps); non è neutrale (responsabilità etica); è una costruzione sociale, fatta da team interdisciplinari *per le persone*.

2 Essence

2.1 Origini, Kernel e Alpha

2.1.1 Origini e Iniziativa SEMAT

L'**ingegneria del software è una disciplina giovane** (nata tra fine anni '60 e anni '70) e senza basi teoriche condivise: esistono **oltre mille metodologie**, spesso variazioni minime l'una dell'altra con vocabolari diversi.

Sintomi della Frammentazione

- **Babele terminologica**: ogni nuovo membro del team (o sostituto a metà progetto) fa perdere settimane per allinearsi su stato e linguaggio.
- **Cultura dei documenti vs dei risultati**: documentazione "write-only" che viene dimenticata e cestinata.
- **Pratiche non validate empiricamente**: efficacia discutibile e risultati disomogenei da team a team.
- **Dilemma dell'insegnamento**: insegnare un metodo (Scrum, XP) a scapito di altri; Essence offre un quadro neutro a monte.

SEMAT (*Software Engineering Method and Theory*), fondata nel **2009** da **Ivar Jacobson**, **Bertrand Meyer** e **Richard Soley**. Prima specifica del Kernel nel 2012; nel **2014** Essence diventa standard **OMG** (come UML e BPMN).

2.1.2 Cos'è Essence e i 7 Alpha del Kernel

Essence è prima di tutto uno **strumento orientato alla conversazione**: permette a persone con background diversi di parlare dello stato di un progetto senza fraintendimenti.

Essence: Metamodello, non Metodo

- **Non è un metodo**: non impone un processo né una sequenza di attività.
- **È un metamodello**: struttura e vocabolario comuni a qualsiasi metodo; si integra con Scrum, Kanban, RUP, SAFe.
- **Diagnosi, non cura**: il Kernel rileva stato e blocchi (come un termometro); **il team decide** quali azioni correttive adottare.

Essence ha due elementi: un **Kernel comune** (concetti minimali sempre presenti) e un **Linguaggio universale** per descrivere e comporre pratiche.

I 7 Alpha. Sono gli aspetti essenziali da far progredire in ogni progetto (*essential things to work with*). Il progresso non si misura con documenti consegnati, ma con **esiti oggettivi**: ogni Alpha attraversa una sequenza di **stati di salute** con criteri verificabili.

Area di Interesse	Alpha	Focus
Customer	Opportunity, Stakeholders	Valore per utenti e committenti.
Solution	Requirements, Software System	Specifica e realizzazione tecnica.
Endeavour	Work, Team, Way of Working	Persone, organizzazione, attività.

1. **Opportunity**: il valore di business che giustifica il prodotto.
2. **Stakeholders**: persone e gruppi interessati all'esito del sistema.
3. **Requirements**: cosa il sistema deve fare e quali vincoli/qualità rispettare.
4. **Software System**: il manufatto in costruzione (codice, test, configurazioni).
5. **Work**: attività e task necessari per raggiungere gli obiettivi.
6. **Team**: il gruppo che realizza le attività.
7. **Way of Working**: pratiche, convenzioni e strumenti adottati dal team.

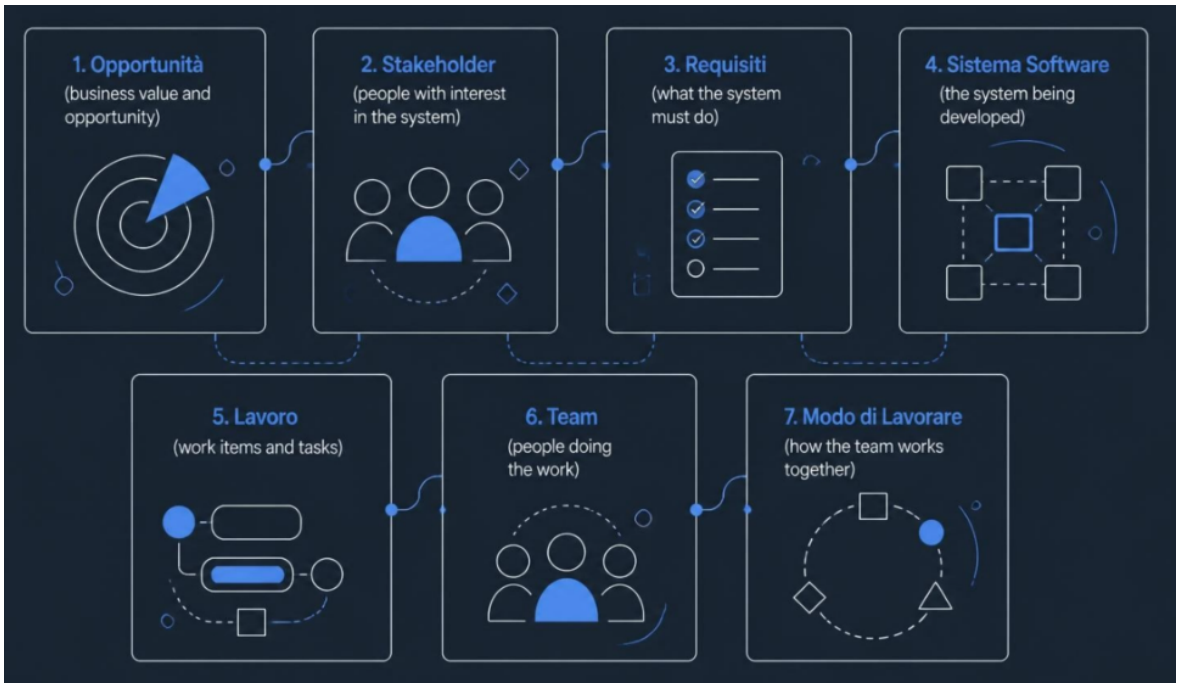


Figura 1: I 7 Alpha del Kernel di Essence e le tre Aree di Interesse.

2.2 Metodi, Pratiche e Strumenti

2.2.1 Strumenti Pratici e Metodologie

Il Kernel è tradotto in **mazzi di carte**, usate per consultazione rapida e dinamiche di team:

- **Progress Poker**: ognuno gioca la carta con lo stato che ritiene corrente per un Alpha; le discrepanze guidano la discussione.
- **Milestone Mapping** (*Objective Go*): si definisce quali stati di quali Alpha servono per un traguardo.
- **Checkpoint**: audit periodici tramite le checklist degli stati.

Valore su metodologie esistenti. Essence non le sostituisce ma le integra: ne evidenzia i confini (es. Scrum copre *Work* e *Team*, non l'architettura del *Software System*), individua gli Alpha trascurati e tratta le pratiche (User Stories, TDD, Code Review...) come **building blocks componibili**.

2.2.2 Metodi, Pratiche e Ciclo di Vita degli Alpha

I metodi tradizionali falliscono spesso per: **elefantiasi prescrittiva** (troppi ruoli e artefatti), **cargo cult** (si adottano i nomi, non la sostanza: chiamare "Standup" una riunione qualsiasi) e **focus sugli artefatti anziché sugli esiti**.

Concetto	Definizione
Processo	**Flusso ordinato di lavoro nel tempo**: sequenze e dipendenze temporali tra attività.
Metodo	**Guida complessiva** per condurre un'iniziativa dall'inizio alla fine; è una **composizione di pratiche**.
Pratica	Risolve un **aspetto specifico del lavoro** (es. stima a punti, retrospettive, TDD); modulare e componibile.

Metafora dell'albero: il Kernel è l'albero (struttura stabile), le pratiche sono le decorazioni scelte da ogni organizzazione.

Anatomia di un Alpha. Un Alpha non è un documento ma una **preoccupazione essenziale** con un ciclo di vita esplicito e valido per qualsiasi metodo (Waterfall, Scrum, XP).

- **Stato**: condizione verificabile lungo il ciclo di vita.
- **Checkpoint**: evidenze tangibili richieste per dichiarare raggiunto uno stato.
- **Cumulatività**: uno stato include i precedenti.

Stati dei 7 Alpha

- **Opportunity**: *Identified* → *Solution Needed* → *Value Established* → *Viable* → *Addressed* → *Benefit Accrued*.
- **Stakeholders**: *Recognized* → *Represented* → *Involved* → *In Agreement* → *Satisfied for Deployment* → *Satisfied in Use*.
- **Requirements**: *Conceived* → *Bounded* → *Coherent* → *Acceptable* → *Addressed* → *Fulfilled*.

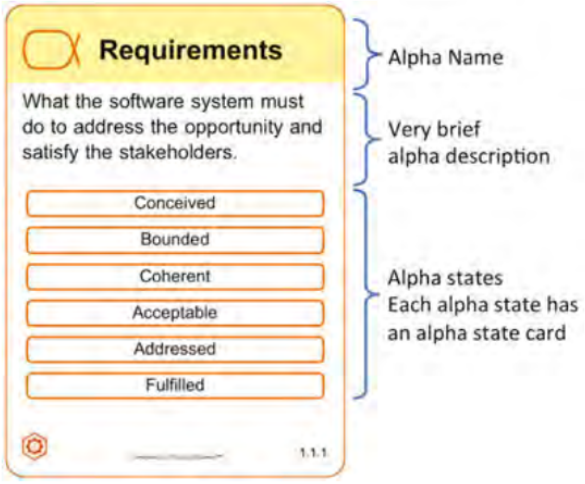


Figura 2: Anatomia di una carta Alpha: stati e checklist.

- **Software System**: *Architecture Selected* → *Demonstrable* → *Usable* → *Ready* → *Operational* → *Retired*.
- **Work**: *Initiated* → *Prepared* → *Started* → *Under Control* → *Concluded* → *Closed*.
- **Team**: *Seeded* → *Formed* → *Collaborating* → *Performing* → *Adjourned*.
- **Way of Working**: *Principles Established* → *Foundation Established* → *In Use* → *In Place* → *Working Well* → *Retired*.

Relazioni tra Alpha: Opportunity guida i Requirements; gli Stakeholders usano e valutano il Software System; il Team pianifica ed esegue il Work; il Way of Working disciplina Team e Work.

Progresso vs Salute. Metriche tradizionali (burndown, ore, task chiusi) possono mascherare problemi: un progetto può sembrare in orario mentre accumula rischio su un Alpha trascurato.

- **Progresso**: quanto un Alpha è avanzato lungo i suoi stati.
- **Salute**: se lo stato attuale sostiene l'iniziativa senza compromettere gli altri Alpha.
- **Ostacoli al prossimo stato**: il team deve individuare cosa impedisce di soddisfare i checkpoint dello stato successivo.

2.2.3 Elementi Complementari e Caso UniMagic

Work Product vs Alpha. Un Work Product è un artefatto contingente (backlog, diagramma delle classi, build). Produrlo **non fa avanzare automaticamente l'Alpha**: un documento di 200 pagine non porta i *Requirements* a *Coherent* se restano contraddizioni. Vale solo come **evidenza** dei checkpoint.

Activity Space e Attività. Lo *Activity Space* è l'area di lavoro universale del Kernel (il "cosa", es. *Explore Possibilities*); l'*Attività* è l'unità concreta definita da una pratica (il "come", es. *Sprint Planning*). Il mapping evidenzia duplicazioni e lacune.

Competenze e Pattern. Le *Competencies* sono le abilità richieste alle persone; i **Ruoli** (Scrum Master, Product Owner…) sono pattern che aggregano responsabilità e competenze.

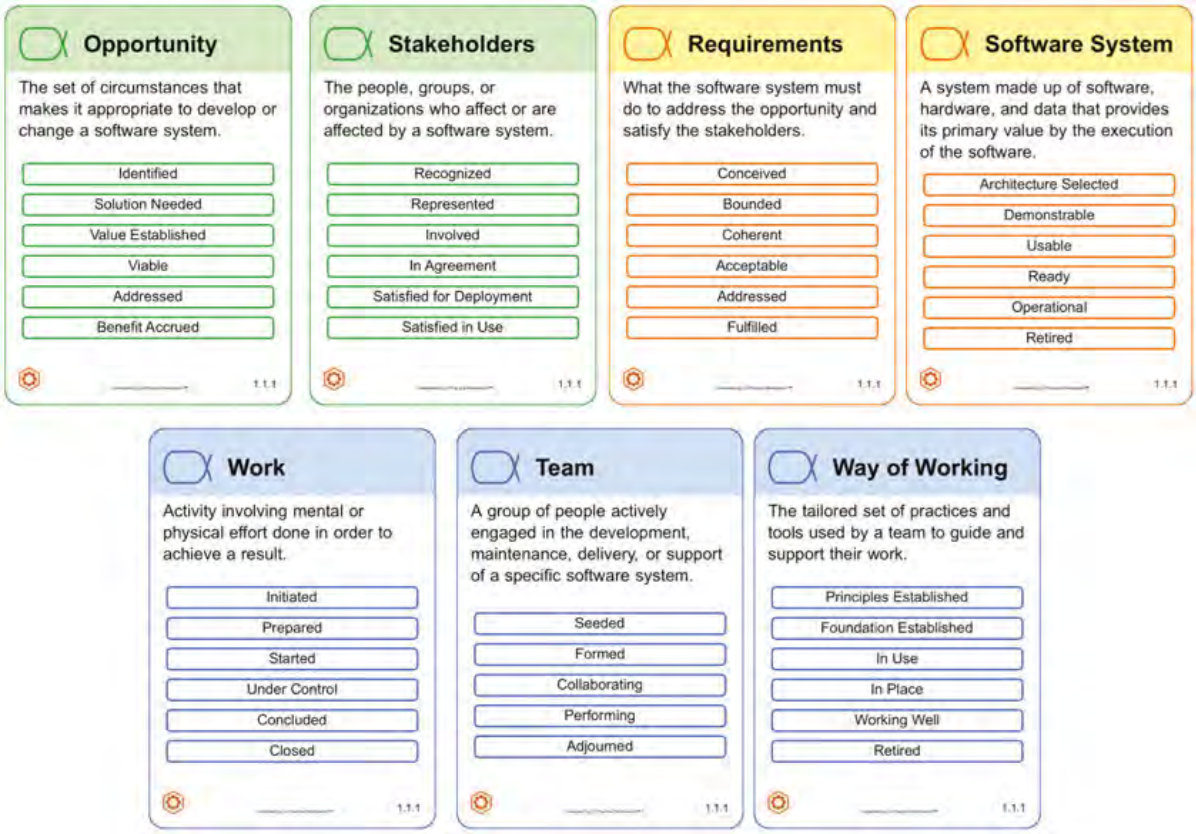


Figura 3: Anatomia di una carta Alpha di Essence e gerarchia degli stati con checklist.

Caso UniMagic. Una software house di 12 persone riceve dall'*Università di Warthog* l'incarico di sviluppare **UniMagic** (supporto ai corsi, prenotazioni, pratiche amministrative) per **2.000 studenti entro un semestre**. Gli stakeholder sono in disaccordo su priorità e privacy; il team di 6 persone non conosce Essence.

Diagnosi Iniziale (Week 0)

- **Requirements** – ***Conceived***: conflitti non risolti su priorità e privacy impediscono *Bounded*.
- **Team** – ***Formed***: membri assegnati ma senza evidenze di vera collaborazione (non *Collaborating*).
- **Way of Working** – ***Principles Established***: intenzione di lavorare in modo iterativo, ma nessuna pratica ancora in uso.

L'asserzione a rischio è la fattibilità per 2.000 utenti in un semestre: mancano evidenze su architettura, capacità misurata del team e accordo tra stakeholder. Essence diagnostica; **la cura** (rinegoziare lo scope, prototipare, spike architetturali) spetta al team.

2.3 Diagnostica di Progetto e Anatomia delle Pratiche

2.3.1 Diagnostica di Progetto (Project Health)

L'avanzamento si misura con le **evidenze dei checkpoint**, non con il tempo trascorso o i documenti prodotti.

- **Evidenze, non aspirazioni**: uno stato è raggiunto solo se **tutti** i suoi checkpoint hanno evidenze tangibili.
- **Il disaccordo ha valore**: discrepanze nel team segnalano ambiguità da investigare.
- **Stato corrente vs prossimo**: il *gap* tra i due definisce le priorità dell'iterazione. **Chasing the State**: isolare i checkpoint scoperti del prossimo stato, raccogliere le evidenze mancanti e rivalutare.
- **Ambiguità**: con evidenze parziali o contestate ci si ferma allo stato inferiore pienamente garantito.

Anomalie Tipiche

- **Software avanzato con Requisiti non *Bounded***: debito di rework.
- **Work "Under Control" con Team solo *Formed***: disallineamenti nascosti.
- **Stakeholder "soddisfatti" senza uso reale**.
- **Way of Working immutato** nonostante blocchi ricorrenti.

Protocollo di valutazione in 30 minuti: audit su 2–3 Alpha rilevanti per la decisione corrente; si formalizza un *Evidence Record* (stato corrente, prossimo stato, evidenze, incertezze, azioni, responsabile, data di revisione).



UniMagic a tre settimane.

Alpha	Team	Docente	Diagnosi
Requirements	Conceived / Bounded	**Bounded** (*at best*)	Interviste e backlog di 46 item, ma privacy non affrontata.
Software System	Demonstrable	**Demonstrable**	Prototipo cliccabile e pipeline attivi; nessuna verifica di prontezza operativa.
Work	Prepared	**Started**	Esecuzione iniziata; incertezza su priorità e perimetro esclude *Under Control*.
Stakeholders	Recognized / Involved	**Represented**	Interviste a 12 studenti e 4 amministrativi, ma nessun accordo stabile.

Decisione: la maturità tecnica (prototipo, pipeline) è sbilanciata rispetto ai requisiti di privacy. Prima si fa la privacy review e si concordano i criteri di accettazione (*una pipeline efficiente non compensa un requisito non bounded*).

2.3.2 Anatomia delle Pratiche

Le 8 Dimensioni di una Pratica

1. **Obiettivo**: il valore che la pratica persegue.
2. **Alpha di interesse**: Alpha su cui interviene.
3. **Work Products**: artefatti prodotti o usati come evidenza.
4. **Attività**: compiti da svolgere.
5. **Competenze**: abilità minime necessarie.
6. **Criteri di avanzamento**: entry ed exit criteria.
7. **Pattern ed esempi**: modelli e convenzioni riutilizzabili.
8. **Supporti ausiliari**: strumenti, AI, vincoli di sicurezza.

Sub-Alpha. Specializzazioni di un Alpha del Kernel (come l'ereditarietà in OOP): es. singole *User Story* come sub-alpha di *Requirements*. Ogni Sub-Alpha ha stati e checklist propri; vanno usati con parsimonia per non appesantire la diagnosi.

Competenze. Tre livelli: **1 – Base**, **2 – Applicativo**, **3 – Esperto**. La competenza si dimostra sul campo, non coi titoli, e può essere tecnica o sociale (negoziazione, consenso con gli stakeholder).

2.3.3 Esercizio: Code Review come Pratica Essence

Esercizio: *essenzializzare* la pratica **Code Review**, cioè descriverla come pratica Essence riusabile e indipendente dagli strumenti (purpose, Alpha primario, work product, attività, competenze,

criteri di avanzamento, pattern, uso dell'LLM). Nelle soluzioni degli studenti c'è buona intuizione, ma confusione tra *Alpha*, *Work Product* e *strumenti*.

Aspetto	**Studenti**	**Soluzione del docente**
Purpose	qualità, sicurezza, manutenibilità	valutare le modifiche prima dell'integrazione e condividere conoscenza
Alpha primario	spesso ambiguo o errato (es. "approvazione del codice" è un *risultato*; "avere un IDE" è uno strumento)	*Software System*, con *Team* e *Way of Working* a supporto
Work product	report / documentazione (troppo generico)	*Review Record* strutturato
Attività	lettura, analisi, correzione	Prepare Review, Examine Change, Resolve Findings, Confirm Integration
Competenze	soprattutto programmazione	development, testing, comprensione del sistema, comunicazione costruttiva
LLM	assente o generico	supporto alla review, con verifica umana

Una sequenza come «in attesa di revisione → da correggere → corretto» descrive meglio gli stati di un Alpha *specifico della pratica* (es. *Code Change*) che di un Alpha del kernel. Le *competencies* sono l'aspetto più sottovalutato. L'LLM può suggerire domande o possibili problemi, ma ogni risultato va verificato da un revisore competente.

3 Programmazione e Progettazione Orientata agli Oggetti

3.1 Introduzione

La programmazione orientata agli oggetti (OO) si è affermata nell'ingegneria del software con l'aspettativa primaria di accrescere in modo tangibile la produttività nello sviluppo dei sistemi. Tale paradigma offre una modalità intuitiva di strutturare la comprensione del mondo reale e del dominio applicativo, basandosi su tre concetti cardine: gli **oggetti**, la trasmissione di **messaggi** e la chiara ripartizione delle **responsabilità**.

Lo sviluppo software orientato agli oggetti si configura come una metodologia onnicomprensiva: consente di osservare il contesto del sistema, di descriverne formalmente il modello concettuale e di adottare un nucleo omogeneo di concetti lungo l'intero ciclo di vita del progetto. Il paradigma fornisce un supporto strutturale al conseguimento di un'elevata qualità del software, forzando lo sviluppatore ad applicare sistematicamente i principi di:

- **Information Hiding**: occultamento dello stato e dei dettagli implementativi interni;
- **Astrazione**: focalizzazione sulle proprietà essenziali dei componenti, tralasciando i dettagli contingenti;
- **Modularizzazione**: suddivisione dell'architettura in elementi coesi e debolmente interdipendenti;
- **Riuso**: condivisione e specializzazione di componenti software consolidati senza duplicazioni.

Decomposizione del Sistema: Sottosistemi e Moduli. La padronanza della complessità architetturale richiede la separazione del sistema in unità computazionali distinte, il cui significato varia in base al paradigma adottato:

- **Sottosistema (*Sub-system*)**: nell'approccio procedurale identifica un'unità responsabile di un compito specifico o di un insieme delimitato di compiti; nell'approccio orientato agli oggetti è costituito da un insieme cooperante di classi (esempi classici sono il DBMS e il modulo centrale di gestione degli errori).
- **Modulo (*Module*)**: costrutto maggiormente dipendente dal linguaggio di programmazione; corrisponde a una libreria o insieme di funzioni nel paradigma procedurale, mentre coincide con una o più classi nel paradigma orientato agli oggetti.

Occultamento dell'Informazione (Information Hiding). Un modulo deve celare la propria implementazione interna ed esporsi verso l'esterno unicamente tramite un'interfaccia pubblica. È vietato l'accesso diretto ai dati interni e ai metodi privati, consentendo la lettura e la manipolazione dello stato unicamente tramite metodi accessori (*accessor methods*). Moduli e interfacce devono essere definiti tramite astrazione: una modifica all'implementazione interna che mantenga invariata l'interfaccia non deve produrre alcun effetto sul resto del sistema.

Accoppiamento (Coupling). L'accoppiamento misura l'intensità e la natura dei legami di dipendenza tra due moduli; l'obiettivo progettuale fondamentale è il **basso accoppiamento** (*loose coupling*), che realizza l'information hiding in modo nettamente più efficace rispetto a un sistema con alto accoppiamento. I livelli di accoppiamento si ordinano in scala di dipendenza crescente:

1. **Invocazione di metodi**: un modulo richiama metodi pubblici dell'interfaccia di un altro (livello minimo raccomandato);
2. **Accoppiamento di dati e di controllo**: passaggio di strutture dati o di flag che guidano il flusso logico del modulo invocato;
3. **Sottoclassificazione tra moduli**: una classe di un modulo eredita direttamente da una classe allocata in un altro modulo;
4. **Dipendenza da compilatore o da chiamate di sistema (OS)**: impiego di primitive o estensioni non standard dell'ambiente;
5. **Accoppiamento di I/O**: vincolo e condivisione diretta sui medesimi flussi fisici o periferiche;
6. **Accoppiamento comune (*Common Coupling*)**: condivisione simultanea di memoria o variabili globali;
7. **Accoppiamento di contenuto (*Content Coupling*)**: un modulo altera arbitrariamente codice o dati interni di un altro (**da evitare rigorosamente**).

Coesione (Cohesion). Un modulo si definisce coesivo se tutti i suoi elementi concorrono allo svolgimento di un singolo e ben identificato compito. La coesione è classificata secondo magnitudo qualitativa crescente:

- **Coincidental**: elementi raggruppati senza alcuna ragione logica;
- **Logical**: elementi uniti solo perché appartenenti alla medesima categoria concettuale;
- **Temporal**: istruzioni eseguite nella stessa finestra temporale (es. routine di inizializzazione);
- **Procedural**: il flusso di controllo passa sequenzialmente da una parte all'altra;
- **Communicational**: componenti vincolati dall'accesso ai medesimi canali di Input/Output;
- **Sequential**: l'output di un elemento costituisce direttamente l'input del successivo;
- **Informational**: routine indipendenti che insistono sulla medesima struttura dati;
- **Functional**: tutti gli elementi concorrono a realizzare una sola funzione (**condizione ideale da perseguire**).

3.2 UML

3.2.1 Introduzione

Verso la fine degli anni '80 e l'inizio degli anni '90, la diffusione della programmazione a oggetti ha visto proliferare metodologie, notazioni grafiche e processi eterogenei tra loro in competizione. I tre approcci predominanti dell'epoca erano riconducibili a:

- Grady Booch (metodo Booch, focalizzato su analisi e design architetturale);
- James Rumbaugh (OMT – *Object-Modeling Technique*, centrato sulla modellazione concettuale dei dati);

- Ivar Jacobson (OOSE – *Object-Oriented Software Engineering*, pioniere dei casi d'uso).

La confluenza di questi autori all'interno di Rational Software ha condotto all'unificazione delle rispettive notazioni, sfociando nella definizione dello **Unified Modeling Language (UML)**, in seguito standardizzato a livello internazionale dall'OMG (*Object Management Group*). L'obiettivo di UML è formalizzare e standardizzare le idee fondazionali della programmazione e progettazione ad oggetti.

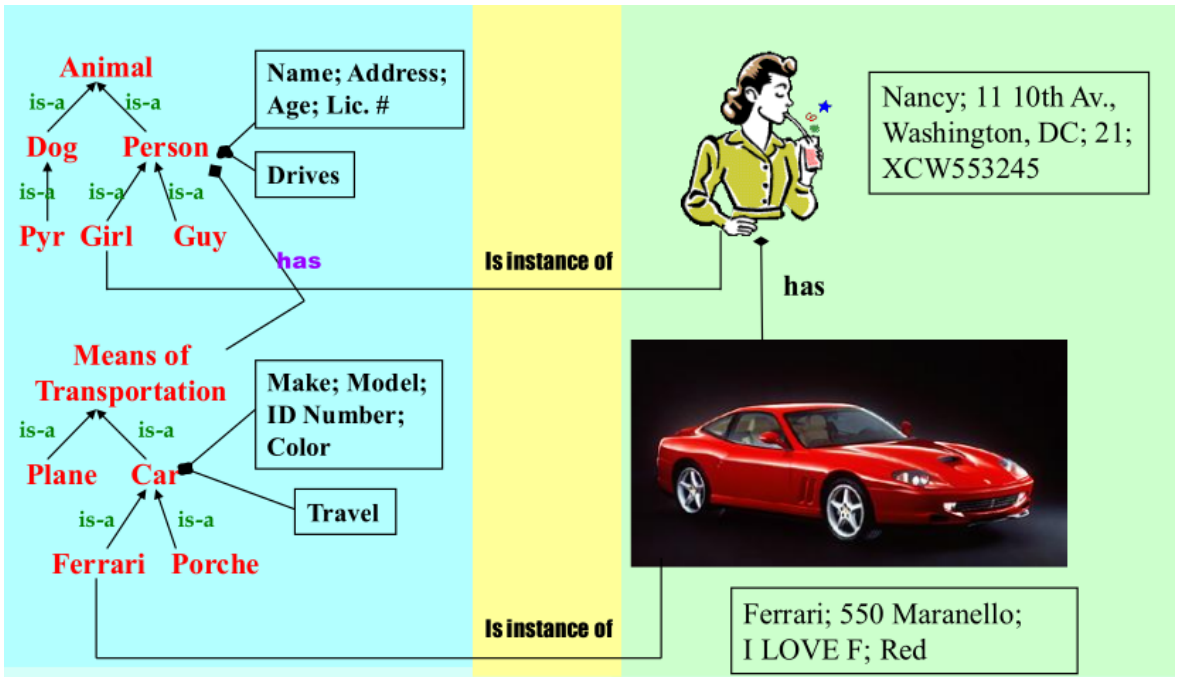
Continuità della Rappresentazione nel Ciclo di Vita. L'idea cardine del paradigma consiste nel modellare il dominio in termini di **oggetti interagenti** e mantenere inalterata tale rappresentazione concettuale attraverso tutte le fasi dello sviluppo:

1. **OO Concept Modeling**: modellazione dei concetti del dominio e del contesto operativo;
2. **OO Analysis (OOA)**: comprensione rigorosa di cosa il sistema deve realizzare;
3. **OO Design (OOD)**: definizione della struttura tecnica e delle interfacce software;
4. **OO Programming (OOP)**: codifica dell'implementazione nel linguaggio di destinazione.

Concetti Fondamentali: Classi, Istanze e Messaggi. Il vocabolario primario si articola attorno alle seguenti entità:

- **Classi e Gerarchie di Classi**: una classe agisce come stampo descrittivo generale (*blueprint*, pattern o template); descrive proprietà (**attributi**) e comportamenti (**metodi**), definendo relazioni e legami di **ereditarietà** con altre classi.
- **Oggetti**: istanze concrete di una classe; possiedono attributi a cui sono assegnati valori definiti e relazioni istanziate con altri oggetti.
- **Messaggi e Metodi**: gli oggetti collaborano scambiandosi messaggi operativi, ai quali il destinatario risponde invocando il corrispondente metodo incapsulato.

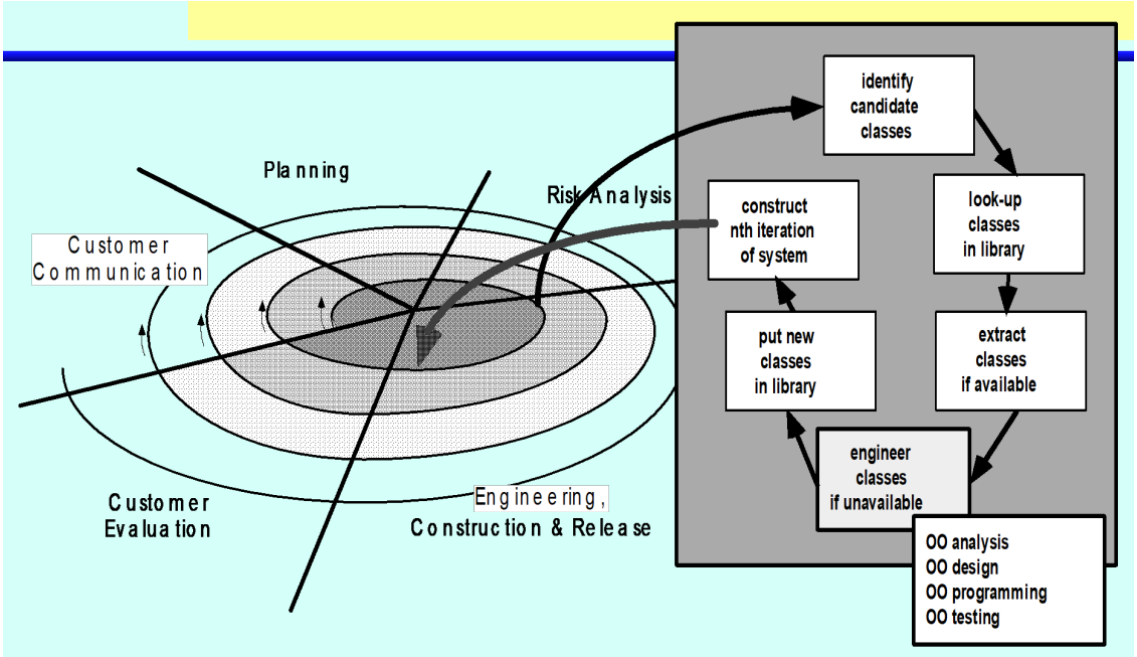
Esempio di Modello Concettuale ed Ereditarietà. Un modello concettuale si articola specificando oggetti, i relativi attributi e le relazioni intercorrenti (ad esempio: l'oggetto persona *Joe* possiede attributi come nome, indirizzo, età e patente, ed è collegato tramite la relazione *owns* all'oggetto automobile *Porsche*, provvisto di costruttore, modello, targa e colore).
Le relazioni *is-a* collegano classi astratte e concrete (*Person* specializza *Animal*; *Girl* e *Guy* specializzano *Person*); l'oggetto *Nancy* è un'istanza di *Girl* e eredita gli attributi di *Person*.

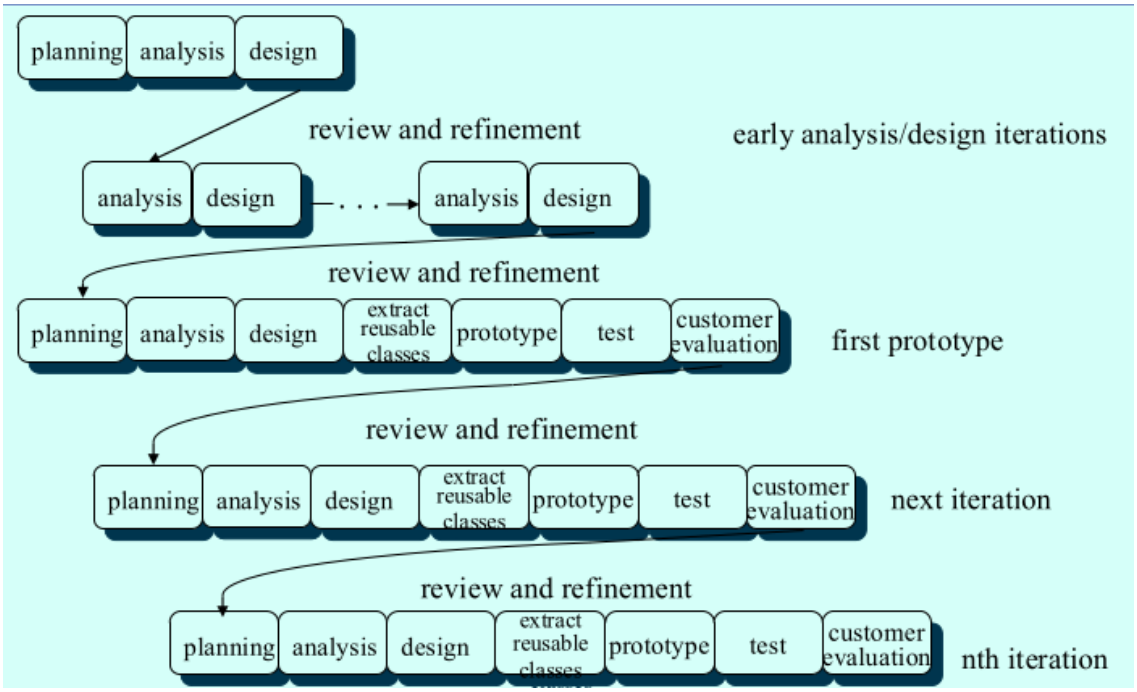

Animal
is-a
is-a
Dog
Person
is-a
is-a
is-a
Pyr
Girl
Guy
Name; Address;
Age; Lic. #
Drives
has
Means of
Transportation
is-a
is-a
Plane
Car
is-a
is-a
Ferrari
Porche
Make; Model;
ID Number;
Color
Travel
Is instance of
Is instance of
Nancy; 11 10th Av.,
Washington, DC; 21;
XCW553245
has
Ferrari; 550 Maranello;
I LOVE F; Red

Il Processo a Spirale nello Sviluppo ad Oggetti

Il modello di processo orientato agli oggetti adotta un paradigma **evolutivo a spirale**. Lo sviluppo non segue una sequenza a cascata rigida, bensì compie cicli concentrici progressivi. In ciascuna spira dell'evoluzione, il sistema attraversa settori cardine di ingegnerizzazione:

- **Customer Communication**: raccolta continua dei requisiti ed esiti attesi;
- **Planning**: pianificazione dell'iterazione corrente e perimetrazione degli obiettivi;
- **Risk Analysis**: identificazione precoce e risoluzione dei fattori di rischio architetturale;
- **Engineering, Construction & Release**: costruzione operativa dell'incremento software;
- **Customer Evaluation**: validazione sul campo con il committente/utente per raccogliere il feedback necessario a guidare l'iterazione successiva.




planning
analysis
design
review and refinement
early analysis/design iterations
analysis
design
analysis
design
review and refinement
planning
analysis
design
extract reusable classes
prototype
test
customer evaluation
first prototype
review and refinement
planning
analysis
design
extract reusable classes
prototype
test
customer evaluation
next iteration
review and refinement
planning
analysis
design
extract reusable classes
prototype
test
customer evaluation
nth iteration

3.2.2 Alcune definizioni

Definizione Formale di Classe e Istanziazione. Una **classe** costituisce l'astrazione cardine della programmazione orientata agli oggetti: essa rappresenta una collezione di oggetti simili e funge da *template*, *pattern*, *blueprint* o descrizione generalizzata di un insieme omogeneo di elementi. Nello specifico, una classe definisce:

- **Proprietà (Attributi)**: le caratteristiche strutturali e lo stato informativo condiviso da tutti gli oggetti appartenenti alla classe;
- **Comportamenti (Operazioni/Metodi)**: l'insieme delle azioni e delle funzioni che ciascun oggetto della classe è in grado di eseguire.

Una volta definita la classe come schema concettuale, è possibile procedere all'**istanziazione**, ovvero alla creazione di una specifica istanza fisica allocata in memoria: l'**oggetto**.

Operazioni, Servizi e Scambio di Messaggi. Un'**operazione** (denominata anche *servizio*) è una procedura eseguibile incapsulata all'interno di una classe, specificamente progettata per agire su uno o più attributi definiti nella classe stessa.

- **Invocazione tramite messaggi (*Message Passing*)**: nei modelli concettuali a oggetti, le operazioni non vengono concepite come chiamate di funzioni slegate, bensì come risposte all'invio di un *messaggio* da parte di altri oggetti o componenti del sistema;
- **Sinonimi e convenzioni terminologiche**: nei vari contesti teorici e linguaggi di programmazione, il termine operazione assume denominazioni equivalenti:
  - *Service* (nei modelli architetturali e orientati ai servizi);
  - *Method* (nei linguaggi a oggetti puri o ibridi come Smalltalk e Java);
  - *Member Function* (nella convenzione del C++);
  - *Function Entry* (in linguaggi per la programmazione concorrente, come Concurrent Pascal).

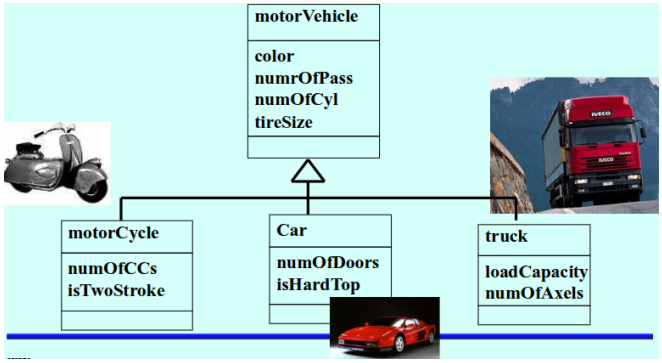
Il Concetto di Ereditarietà (*Inheritance*). L'**ereditarietà** è il meccanismo che consente di definire nuove classi come **estensioni** o specializzazioni di classi preesistenti. La classe derivata (*sottoclasse*) eredita tutti gli attributi e i metodi della classe genitore (*superclasse*), con la facoltà di:

- Aggiungere nuovi attributi e metodi specifici;
- Ridefinire (*override*) il comportamento di operazioni esistenti per adattarle alle proprie necessità.

Questo legame logico può essere descritto attraverso diverse formulazioni equivalenti:

- Relazione ***is-a*** ("è un"): ad esempio, la classe *Dog is-a Animal*;
- Relazione di estensione (***extends***): la classe derivata estende le potenzialità della classe base;
- Relazione insiemistica di inclusione: la classe degli *Animals* contiene concettualmente l'insieme dei *Dogs*;
- Relazione di generalizzazione: la superclasse generalizza i tratti comuni condivisi da tutte le sue sottoclassi.

Rappresentazione Grafica della Gerarchia di Ereditarietà. Nel formalismo grafico di UML, l'ereditarietà viene indicata mediante una linea orientata che risale dalla sottoclasse alla superclasse, terminante con una freccia a triangolo (simbolo di generalizzazione).

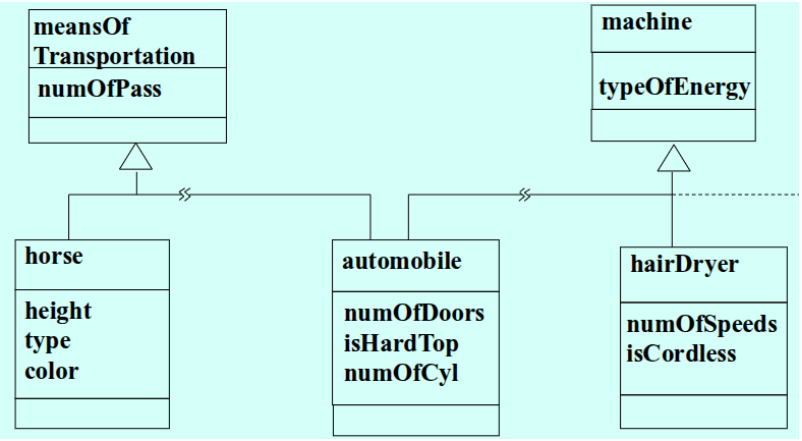


Ereditarietà Multipla (*Multiple Inheritance*). Si definisce **ereditarietà multipla** la caratteristica che consente a una sottoclasse di ereditare contemporaneamente attributi e metodi da due o più superclassi distinte e reciprocamente indipendenti:

- **Superclassi indipendenti**:
  - `meansOfTransportation`: modella l'entità come mezzo per trasportare carichi o persone (con l'attributo `numrOfPass`);
  - `machine`: modella l'entità come dispositivo meccanico o tecnologico (con l'attributo `typeOfEnergy`).
- **Ereditarietà singola a confronto**:
  - La classe `horse` deriva unicamente da `meansOfTransportation`, aggiungendo caratteristiche fisiche come `height`, `type` e `color`;
  - La classe `hairDryer` (asciugacapelli) deriva unicamente da `machine`, aggiungendo parametri funzionali come `numOfSpeeds` e `isCordless`.
- **Ereditarietà multipla**: la classe `automobile` eredita congiuntamente da `meansOfTransportation` e da `machine`. In questo modo, l'automobile acquisisce sia la capacità di trasportare passeggeri (`numrOfPass`), sia il tipo di energia consumata (`typeOfEnergy`), aggiungendo attributi specifici come `numOfDoors`, `isHardTop` e `numOfCyl`.

Sebbene l'ereditarietà multipla garantisca grande flessibilità concettuale, molti linguaggi ad oggetti moderni (come Java o C#) preferiscono limitarne l'impiego diretto sulle classi, sostituendola con l'implementazione di interfacce multiple per evitare conflitti di denominazione o ambiguità di derivazione (il cosiddetto problema del diamante).

Polimorfismo. Il **polimorfismo** rappresenta la capacità di utilizzare lo stesso nome per identificare metodi che compiono operazioni concettualmente simili (*"same kind"*) su oggetti di tipo differente.



- **Analogia matematica**: l'operatore `+` viene impiegato per sommare numeri appartenenti a insiemi eterogenei (naturali, interi, reali, complessi), ma anche strutture algebriche più complesse come vettori e matrici;
- **Vantaggio progettuale**: permette di gestire grandi insiemi di classi con comportamenti affini in modo coerente e uniforme, evitando di dover inventare e memorizzare nomi distinti e specifici per ciascuna variante (come avviene ad esempio nella libreria standard del C con `printf`, `fprintf`, `sprintf`).

Forme di Polimorfismo. Nel paradigma ad oggetti e nei linguaggi moderni si distinguono principalmente tre forme di polimorfismo:

- **Polimorfismo ad-hoc (*Overloading*)**: più funzioni o metodi condividono lo stesso identico nome all'interno dello stesso ambito, ma differiscono per la lista dei parametri (numero o tipo). Ad esempio:
  - `print(file)`
  - `print(string)`
  - `print(number)`
- **Polimorfismo generico (*Parametric / Templates*)**: un modello astratto definisce una struttura o un algoritmo comune a un insieme di tipi, rendendo il codice indipendente dal tipo di dato concreto su cui opera (es. un `swap` valido per qualsiasi tipo);
- **Polimorfismo per inclusione o ereditarietà (*Overriding*)**: ridefinizione della logica di un metodo ereditato dalla classe base all'interno di una classe derivata, consentendo la risoluzione dinamica del comportamento in fase di esecuzione.

Polimorfismo per Ereditarietà ed Esecuzione Dinamica (Overriding). L'overriding sfrutta la gerarchia di ereditarietà per consentire a oggetti di classi derivate di rispondere allo stesso messaggio con comportamenti specializzati:

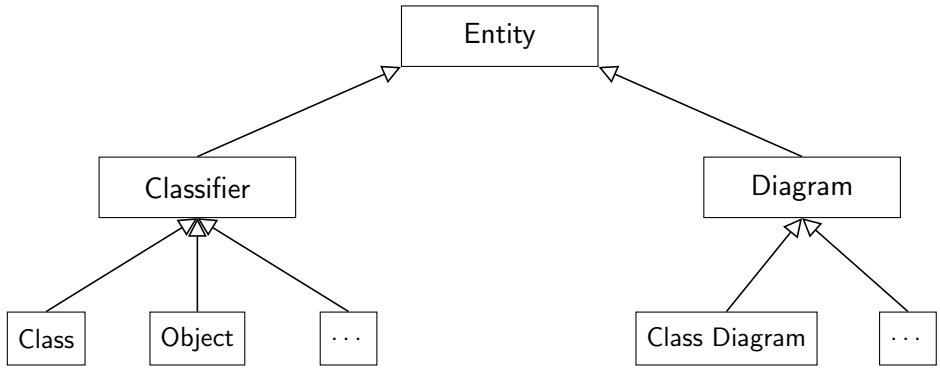
- **Metodi virtuali nella classe base**: la classe radice dichiara il metodo contrassegnandolo come virtuale (`virtual void draw()`), predisponendolo per essere ridefinito dai discendenti;

- **Specializzazione nelle sottoclassi**: classi concrete come `LineGraph` e `PieChart` ereditano da `Graph` ed effettuano l'override di `draw()`, implementando ciascuna la propria logica grafica specifica;
- **Invocazione indipendente dal tipo concreto**: una funzione generica come `DrawCorrect(Graph &t)` accetta un riferimento al tipo base e invoca `t.draw()`; grazie al *dynamic dispatching*, a runtime viene eseguita automaticamente la versione del metodo associata all'effettiva classe dell'oggetto passato;
- **Gestione trasparente di collezioni eterogenee**: è possibile gestire insiemi di oggetti diversi tramite una struttura uniforme (ad esempio un array di puntatori alla classe base, `Graph* list[10]`); ciclando sugli elementi ed invocando `DrawCorrect(*list[i])`, ciascun oggetto disegnerà se stesso correttamente senza che il codice chiamante debba conoscere a priori il suo tipo specifico.

3.2.3 UML nel dettaglio

Tassonomia delle Entità in UML: Classificatori e Diagrammi. L'intero impianto concettuale di UML struttura le proprie entità primarie (*Entity*) secondo una gerarchia fondamentale articolata su due direttrici:

- **Classifier (Classificatori)**: astrazioni logiche che definiscono modelli concettuali, strutture di dati e contratti di comportamento all'interno del dominio applicativo;
- **Diagram (Diagrammi)**: viste grafiche parziali o complete che visualizzano insiemi di entità e le loro relazioni reciproche all'interno del sistema.



I Principali Classificatori. Tra i classificatori cardine previsti dal metamodello UML rientrano:

- **Class**: astrazione che modella attributi di stato e operazioni comuni a un insieme omogeneo di oggetti;
- **Interface**: contratto descrittivo composto da dichiarazioni di operazioni prive di implementazione diretta;
- **Datatype**: tipo descrittivo per valori puri che non necessitano di un'identità autonoma;

- **Component**: unità di packaging modulare e riusabile che implementa e offre specifiche interfacce;
- **Node**: risorsa di calcolo hardware o ambiente computazionale di esecuzione;
- **Use Case**: sequenza di azioni orientate a uno scopo condotte da un attore esterno interagendo con il sistema;
- **Subsystem**: raggruppamento di livello architetturale superiore che racchiude classificatori operativi cooperanti.

Classificazione dei Diagrammi UML. UML definisce oltre 9 tipologie distinte di diagrammi, aggregate in due macro-famiglie complementari a seconda della dimensione osservata:

- **Diagrammi Strutturali (*Structural Diagrams*)**: catturano la configurazione statica delle entità, dei tipi e dei collegamenti fisici o logici:
  1. **Class Diagram**: descrive la struttura statica delle classi, dei relativi membri e delle gerarchie di ereditarietà o associazione;
  2. **Object Diagram**: istantanea a runtime raffigurante oggetti concreti con specifici valori di attributo e legami d'istanza;
  3. **Component Diagram**: evidenzia l'articolazione modulare del software e le dipendenze tra componenti;
  4. **Deployment Diagram**: illustra l'architettura fisica di rilascio, mappando i componenti sui nodi di calcolo operativi.
- **Diagrammi Comportamentali (*Behavioral Diagrams*)**: descrivono l'evoluzione dinamica nel tempo, le reazioni agli eventi e i flussi di controllo tra oggetti:
  5. **Use Case Diagram**: formalizza i requisiti operativi descrivendo le interazioni di alto livello tra attori e funzionalità;
  6. **Sequence Diagram**: evidenzia lo scambio ordinato cronologicamente di messaggi lungo le linee di vita degli oggetti;
  7. **Collaboration Diagram**: rappresenta le collaborazioni tra oggetti valorizzando la topologia strutturale delle relazioni e numerando i messaggi;
  8. **Statechart Diagram**: traccia gli stati interni attraversati da una singola entità reattiva in risposta a stimoli ed eventi;
  9. **Activity Diagram**: visualizza la logica procedurale, le attività concorrenti parallele e le sincronizzazioni di flusso.

3.2.4 Object Oriented Concept Modeling

Obiettivi dell'OO Concept Modeling. La modellazione concettuale orientata agli oggetti persegue due finalità complementari:

- **OO Context Analysis**: comprensione del contesto operativo e dell'ambiente in cui il sistema software andrà a operare;

- **OO Requirement Analysis**: comprensione rigorosa delle reali necessità e dei requisiti effettivi che il sistema deve soddisfare.

Nella prassi, quest'intera fase preliminare viene spesso indicata genericamente come *Requirement Analysis*, ma include concettualmente entrambe le attività di analisi del contesto e dei requisiti.

I Casi d'Uso nella Modellazione Concettuale. Una descrizione di Use Case costituisce uno scenario che definisce un "filo d'uso" (*thread of usage*) del sistema. Essa comprende due componenti cardine:

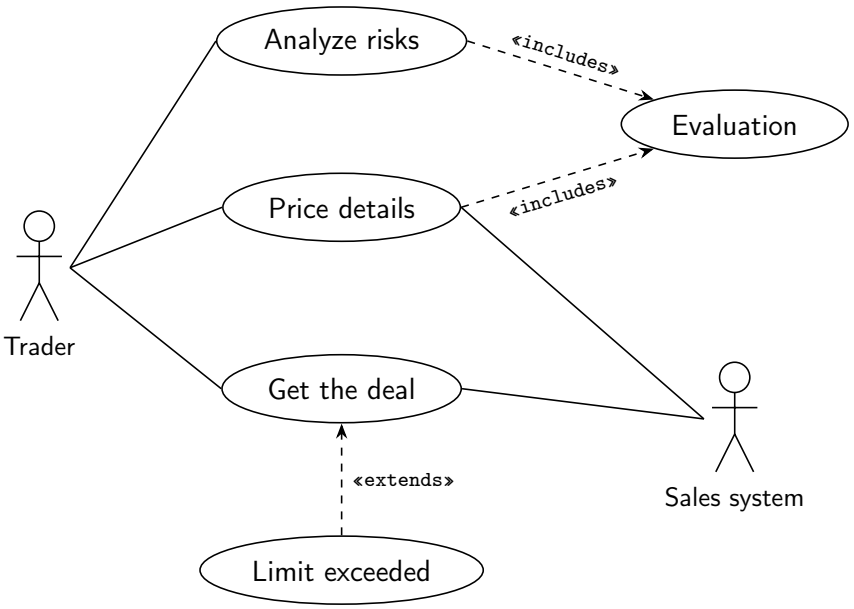
1. **Diagramma (Use Case Diagram)**: vista grafica che mette in relazione gli **attori** (ruoli interpretati da utenti umani o dispositivi esterni durante l'operatività) con i **casi d'uso** (le specifiche unità di utilizzo del sistema);
2. **Descrizione Testuale**: documento o scheda descrittiva che ordina in sequenza logica e temporale le attività previste nello scenario.

Definizione e Caratteristiche di uno Use Case. Uno Use Case rappresenta un'interazione tipica tra attori e sistema, finalizzata a soddisfare un bisogno concreto dell'utente descrivendo come il sistema viene impiegato. Ad esempio, per un programma di elaborazione testi (*word processor*), casi d'uso tipici sono: applicare il grassetto a una porzione di testo, creare un indice o eliminare una parola.
I casi d'uso presentano proprietà strutturali ben definite:

- **Granularità**: può variare da un livello fine (*small*) a un livello più ampio o aggregato (*large*);
- **Visibilità (Often)**: nella maggior parte dei casi catturano funzionalità direttamente percepibili e fruibili dall'utente finale;
- **Obiettivo discreto (Always)**: un caso d'uso persegue e porta a compimento un obiettivo singolo, autonomo e ben delimitato;
- **Requisito esterno (Always)**: descrive costantemente una funzionalità richiesta dall'esterno, senza dettagliare i meccanismi algoritmici interni al software.

Relazioni Avanzate: Inclusione ed Estensione (Esempio NYSE). La modellazione delle transazioni di borsa (NYSE) evidenzia la cooperazione tra attori umani (`Trader`) e sistemi automatizzati esterni (`Sales system`), oltre all'uso delle relazioni standard tra casi d'uso:

- **Relazione** `«includes»`: definisce un'inclusione obbligatoria e incondizionata. Sia `Analyze risks` sia `Price details` incorporano sistematicamente il sotto-caso `Evaluation` per verificare i parametri di congruità finanziaria;
- **Relazione** `«extends»`: modella un comportamento opzionale o condizionale che estende il caso d'uso base solo al verificarsi di specifiche condizioni. Quando si tenta di concludere la transazione (`Get the deal`), il flusso viene esteso dalla gestione dell'eccezione di superamento fido (`Limit exceeded`) qualora il volume d'acquisto ecceda la soglia consentita.



Gli Attori nei Casi d'Uso. Un **attore** rappresenta il ruolo assunto da un'entità esterna nel momento in cui interagisce con il sistema:

- **Esecuzione dei casi d'uso**: gli attori avviano e portano a compimento i casi d'uso; nella prassi metodologica si individuano prima gli attori rilevanti e, solo successivamente, i rispettivi casi d'uso;
- **Natura non esclusivamente umana**: un attore non deve necessariamente essere una persona fisica, ma può essere un dispositivo hardware, un sensore, un timer di sistema o un servizio software esterno;
- **Coinvolgimento**: un attore può ottenere un valore diretto e tangibile dal completamento del caso d'uso, oppure limitarsi a parteciparvi operativamente come tramite o fornitore di supporto.

Relazioni tra Casi d'Uso: Extends e Includes. Per strutturare e collegare i casi d'uso tra loro si impiegano principalmente due relazioni formali:

- **Relazione** `«extends»`: si utilizza quando un caso d'uso è analogo a un altro ma aggiunge comportamenti specifici o gestisce varianti:
  - Si modella innanzitutto il flusso base principale e ordinario;
  - Per ciascun passo del flusso, ci si interroga su cosa potrebbe fallire (*what could go wrong*) o su come l'azione potrebbe svolgersi diversamente (*how might this work out differently*);
  - Ogni variante, eccezione o percorso alternativo viene modellata come estensione del caso d'uso base.
- **Relazione** `«includes»`: si impiega quando una sequenza di passaggi o un frammento di comportamento è comune a più casi d'uso:
  - Consente di fattorizzare la logica condivisa in un unico punto;
  - Evita la duplicazione ridondante (*copy-and-paste*) delle specifiche testuali all'interno dei documenti di analisi.

Descrizione Testuale del Caso d'Uso. Il diagramma fornisce unicamente una visione d'insieme delle interazioni; per comprendere l'effettivo funzionamento del sistema è indispensabile associare a ciascun caso d'uso una **descrizione testuale**. Essa consiste in una specifica scritta, concisa e priva di ambiguità, che documenta la sequenza di passi scambiati tra l'attore e il sistema. A seconda del livello di dettaglio e del contesto, la descrizione può adottare due formati distinti:

1. **Formato sequenziale passo-passo (*Step-by-step*)**: elenca le azioni operative in sequenza cronologica numerata. Esempio per il caso d'uso *Get the deal*:

   (a) L'utente inserisce nome utente e identificativo del conto bancario;

   (b) Il sistema verifica la validità delle credenziali e del conto;

   (c) L'utente inserisce il numero di azioni da acquistare e l'ID del titolo;

   (d) Il sistema calcola il prezzo corrente di negoziazione;

   (e) Il sistema verifica che l'importo non superi il limite o fido consentito;

   (f) Il sistema inoltra l'ordine alla borsa (NYSE);

   (g) Il sistema memorizza e restituisce il codice di conferma della transazione.

2. **Formato narrativo discorsivo (*Narrative paragraph*)**: descrive il flusso sotto forma di paragrafo testuale continuo, utile in fasi esplorative o per descrivere servizi informativi generali. Esempio per il caso d'uso *Frequent Flyer*:

   > *"L'utente accede al sottosistema per ottenere informazioni sul proprio stato nel programma frequent flyer. All'interno del sottosistema, l'utente può consultare (a) informazioni generali sul programma (meccanismo di accumulo punti, modalità di iscrizione, ottenimento miglia con partner) e (b) dettagli specifici sul proprio profilo personale (miglia accumulate, livello di appartenenza). L'utente può inoltre aggiornare il proprio indirizzo di residenza."*

Avvertenze e Buone Pratiche di Modellazione. Nella redazione dei diagrammi dei casi d'uso è fondamentale rispettare tre linee guida metodologiche:

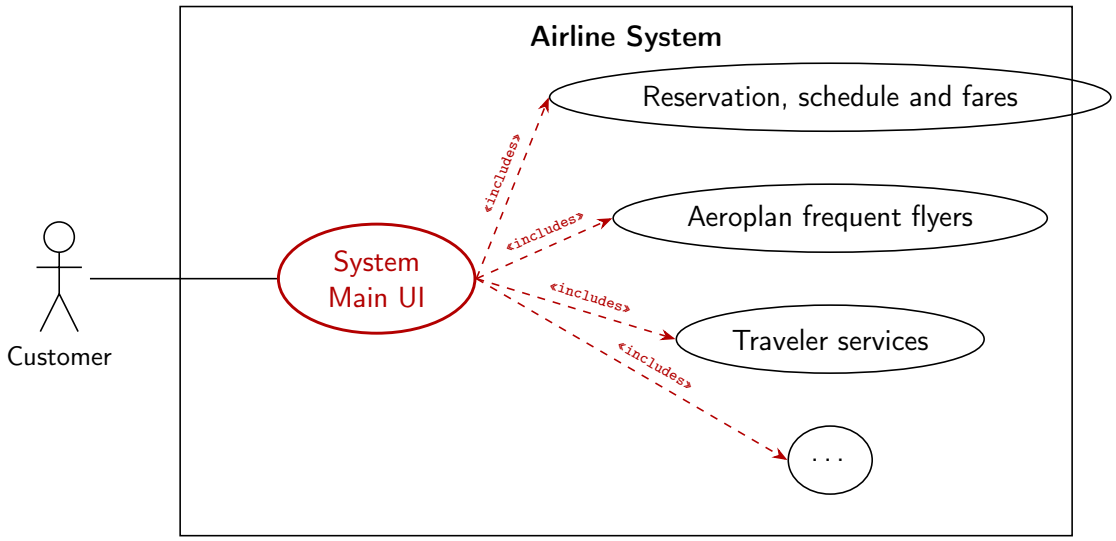
- **Viste parziali**: non è necessario né consigliabile rappresentare l'intero sistema in un solo diagramma affollato; ogni diagramma deve mostrare una vista parziale e circoscritta;
- **Fattorizzazione**: la relazione `«includes»` va usata con disciplina per isolare specifiche condivise ed evitare testo duplicato;
- **Nessuna corrispondenza 1:1 con la UI**: non esiste una corrispondenza diretta tra le schermate grafiche dell'applicazione e gli ovali dei casi d'uso. Un caso d'uso rappresenta un obiettivo logico dell'utente, non una maschera a video o un pulsante d'interfaccia.

L'Anti-Pattern "The KILLER": Decomposizione Strutturale Errata. Uno degli errori più gravi e frequenti nella modellazione OO consiste nel trasformare il diagramma dei casi d'uso in un diagramma di scomposizione a menù o ad albero funzionale:

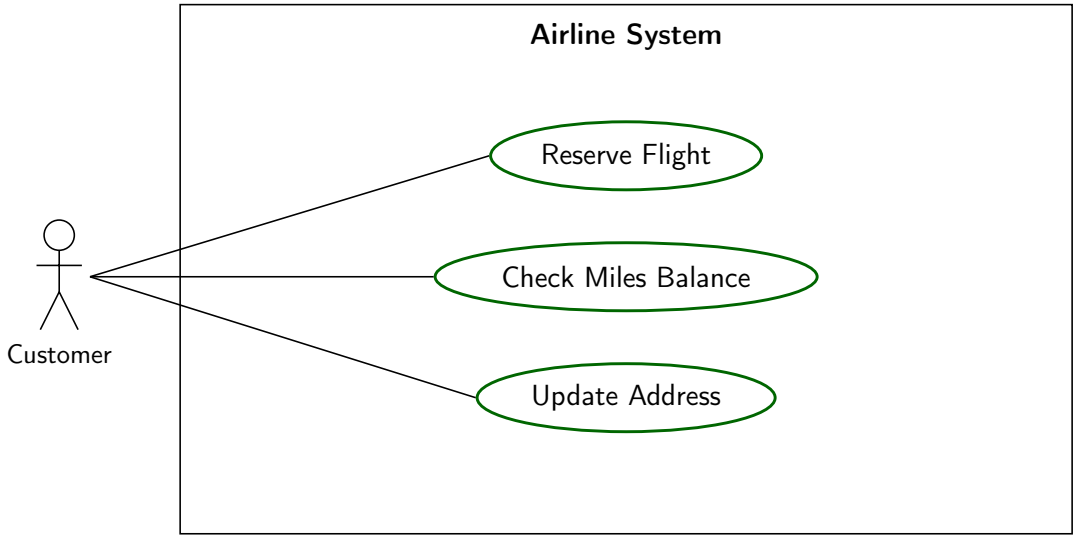
- **L'errore**: definire un caso d'uso generico denominato *System Main UI* collegato all'attore, e fargli includere tramite molteplici `«includes»` i vari sottosistemi applicativi (*Reservation Subsystem*, *Frequent Flyers Subsystem*, *Traveler Services Subsystem*);

- **Perché è errato**: i casi d'uso non devono descrivere la navigazione grafica tra menù né ricalcare l'architettura a moduli software del backend, bensì devono rappresentare singoli processi a valore aggiunto per l'utente esterno.

« ERRATO » Anti-pattern "The KILLER"



« CORRETTO » Un caso d'uso per ogni obiettivo dell'utente



3.2.5 Object-Oriented Analysis (OOA)

Visione Generale dell'OOA. L'analisi orientata agli oggetti (**OOA**) ha lo scopo di comprendere a fondo il dominio del problema e formalizzare i requisiti di sistema prima di passare alle decisioni architetturali o implementative. Il processo si articola in una sequenza iterativa di attività:

1. **Estrarre le classi candidate**: individuare le entità logiche primarie all'interno delle descrizioni del dominio;

2. **Stabilire le relazioni di base tra classi**: identificare i legami concettuali e le connessioni funzionali;

3. **Definire la gerarchia di classi**: strutturare i livelli di generalizzazione e specializzazione (*is-a*);

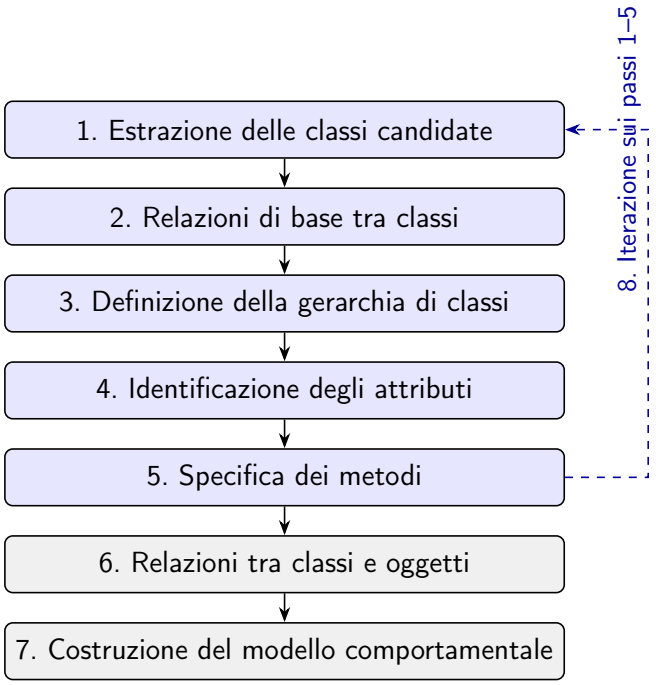
4. **Identificare gli attributi per ciascuna classe**: descrivere le proprietà di stato associate a ogni entità;

5. **Specificare i metodi a supporto degli attributi**: determinare le operazioni incapsulate che operano sui dati interni;

6. **Indicare le relazioni tra classi e oggetti**: chiarire le associazioni operative e i vincoli a livello d'istanza;

7. **Costruire il modello comportamentale**: formalizzare la dinamica temporale e gli scambi di messaggi;

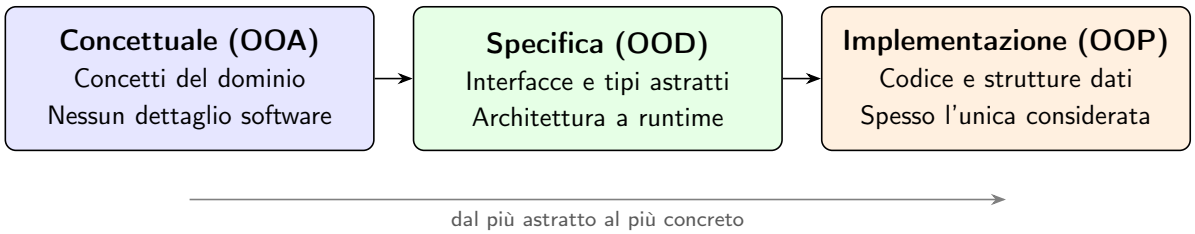
8. **Iterare sui primi cinque passi**: raffinare ciclicamente il modello attraverso revisioni progressive.



Le Tre Prospettive di Modellazione. Durante il ciclo di vita del software, i diagrammi e i concetti orientati agli oggetti possono essere interpretati secondo tre livelli o punti di vista complementari:

- **Prospettiva Concettuale (*Conceptual - OOA*)**: rappresenta unicamente i concetti del dominio del mondo reale; è completamente agnostica rispetto alla tecnologia, al linguaggio e alle scelte implementative;

- **Prospettiva di Specifica (*Specification - OOD*)**: definisce la struttura logica del software in esecuzione, specificando le interfacce e i tipi astratti senza mostrare il codice concreto interno;
- **Prospettiva Implementativa (*Implementation - OOP*)**: dettaglia la reale implementazione nel codice sorgente (nella pratica quotidiana è spesso l'unica ad essere considerata, portando a trascurare le fasi di astrazione precedenti).



Estrazione delle Classi Candidate. La tecnica primaria per estrarre le prime classi candidate consiste nell'**analisi grammaticale** delle descrizioni testuali dei casi d'uso: si isolano i **sostantivi** rilevanti, distinguendo in seguito quali rappresentano vere entità autonome (classi) e quali semplici attributi o dati accessori.

Esempio 1: Flusso "Get the deal" (Trading Azionario). Esaminando la sequenza nominale:

1. Inserimento del nome **user** e del **bank account**;
2. Validazione dei dati inseriti;
3. Inserimento del quantitativo azionario e dello **share ID**;
4. Determinazione del prezzo;
5. Verifica dei limiti di spesa;
6. Invio dell'ordine al **NYSE**;
7. Memorizzazione del **confirmation number**.

I termini evidenziati costituiscono le prime entità candidate: `User`, `BankAccount`, `ShareID`, `NYSE` e `ConfirmationNumber`.

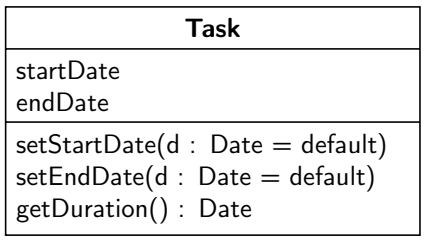
Esempio 2: Portale Viaggiatori Frequent Flyer. Considerando la descrizione narrativa:

> *"L'**utente** accede al sottosistema per consultare lo **stato del programma frequent flyer**. Può visualizzare informazioni generali sul **programma frequent flyer**, sullo **schema premi**, sulle modalità di accumulo **miglia** con le **compagnie partner**, nonché dati sul proprio profilo personale quali **miglia accumulate** e **livello di status**. Può inoltre aggiornare il proprio **indirizzo**."*

Da questa analisi emergono candidati macroscopici (`User`, `FrequentFlyerProgram`, `RewardSchema`, `PartnerCompany`), mentre proprietà quali `milesEarned`, `statusLevel` e `address` vengono modellate come attributi associati alle classi principali.

Definizione di Classe e Struttura Formale. Una **classe** rappresenta un insieme omogeneo di oggetti che condividono identiche caratteristiche e comportamenti. In UML viene schematizzata mediante un rettangolo suddiviso in tre sezioni ben distinte:

- **Nome**: identifica l'astrazione concettuale (ad esempio `Task`);
- **Attributi**: specificano lo stato interno dell'oggetto (ad esempio `startDate`, `endDate`);
- **Operazioni**: definiscono i servizi o metodi invocabili (ad esempio `setStartDate()`, `getDuration()`).



Distinzione tra Classe e Tipo. Nel rigore della progettazione ad oggetti, **tipo** e **classe** non sono sinonimi:

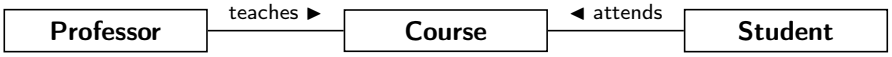
- **Tipo (*Type*)**: rappresenta il **protocollo** compreso da un oggetto, ovvero l'insieme astratto delle operazioni che esso espone all'esterno. Non contiene dettagli implementativi:
  - In Java corrisponde a una `interface`;
  - In C++ corrisponde a una classe base astratta pura (*pure abstract class*);
  - In UML 1.3 viene espresso mediante lo stereotipo formale `«type»`.
- **Classe (*Class*)**: rappresenta il costrutto orientato all'**implementazione**; fornisce la logica concreta, la memorizzazione dello stato e realizza operativamente uno o più tipi.

Il Class Diagram e il Concetto di Associazione. *Le relazioni elencate qui sotto sono riassunte graficamente nella legenda di pag. 36; esercizi svolti nella sezione §4.*

Il **Class Diagram** costituisce il cuore statico della modellazione OO, visualizzando i tipi di oggetti presenti nel sistema e le relazioni intercorrenti:

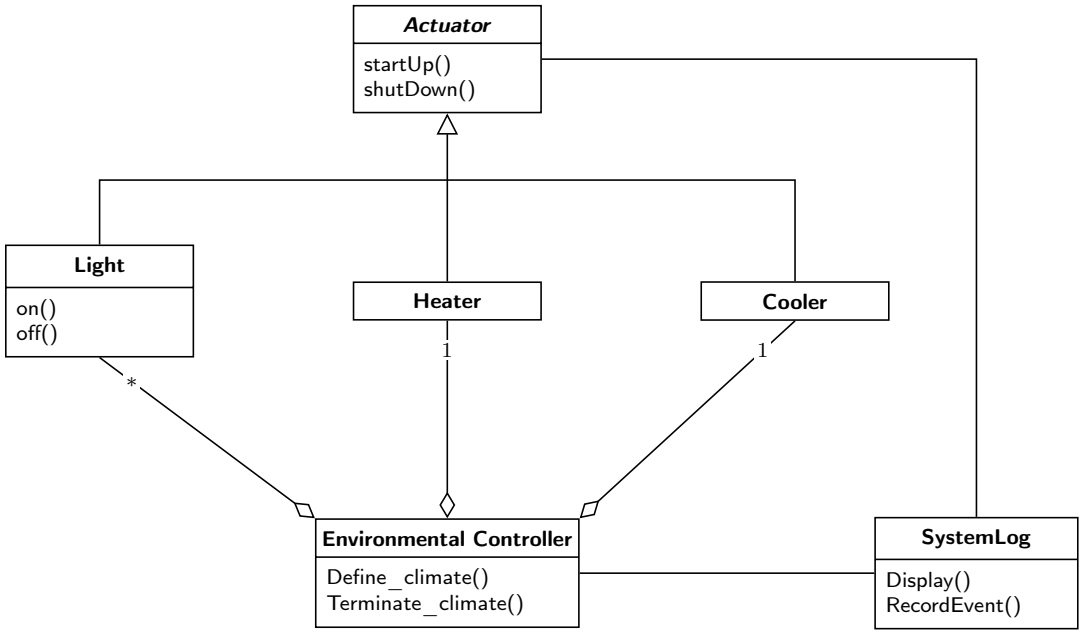
- **Associazione (*Association*)**: relazione semantica tra istanze di classi distinte che collaborano sul piano operativo;
- **Sottotipi / Ereditarietà (*Subtypes / Generalization*)**: catene gerarchiche basate sulla relazione logica *is-a*;
- **Dipendenza (*Dependency*)**: legame di utilizzo in cui una classe impiega temporaneamente i servizi di un'altra.

Un'**associazione** stabilisce un collegamento tra gli oggetti a runtime. Ad esempio: un'istanza di `Professor` eroga didattica per un `Course`, mentre più istanze di `Student` risultano iscritte a quello stesso `Course`.



Esempio Completo di Class Diagram: Sistema di Condizionamento. Il sistema di regolazione termica già esaminato a livello comportamentale può essere descritto nella sua struttura statica attraverso un Class Diagram che combina ereditarietà, composizione/aggregazione e associazione:

- **Gerarchia di attuatori**: la classe base `Actuator` definisce le operazioni generali `startUp()` e `shutDown()`. Le classi concrete `Light`, `Heater` e `Cooler` ereditano da essa specializzandone il funzionamento;
- **Controllore ambientale**: la classe `Environmental Controller` espone i metodi `Define_climate()` e `Terminate_climate()`; controlla direttamente e aggrega un insieme di attuatori con molteplicità distinte (1 controllore gestisce un singolo riscaldatore, un singolo refrigeratore e un numero arbitrario $*$ di punti luce);
- **Modulo di diagnostica**: la classe `SystemLog` registra gli eventi operativi (`RecordEvent()`) ed espone un'interfaccia di visualizzazione (`Display()`), risultando associata sia al controllore che agli attuatori monitorati.



Lettura Guidata del Diagramma: Che Relazione Lega Cosa. Nel diagramma compaiono tre tipi di relazione, riconoscibili dalla decorazione all'estremo della linea:

- **Generalizzazione** (linea con *triangolo vuoto*): significa "*è un*". `Light`, `Heater` e `Cooler` puntano ad `Actuator`: ognuno *è un* attuatore e ne eredita `startUp()` e `shutDown()`. `Light` aggiunge `on()` e `off()`. Il nome `Actuator` in corsivo indica una **classe astratta**: non esistono attuatori "generici", solo le sue sottoclassi concrete;

- **Aggregazione** (linea con *rombo vuoto*): significa "*è composto da / contiene*", cioè una relazione **tutto–parte**. Il rombo si trova sempre dal lato del *tutto* (`Environmental Controller`); le *parti* sono `Light`, `Heater` e `Cooler`. Le parti hanno vita propria: se il controllore sparisce, una `Light` può continuare a esistere (nella **composizione**, rombo *pieno*, le parti morirebbero con il tutto; qui non è il caso). I numeri sono le **molteplicità** dal lato della parte: un controllore ha 1 `Heater`, 1 `Cooler` e un numero qualsiasi ($*$) di `Light`;
- **Associazione** (linea *semplice*, senza rombo né triangolo): significa "*si conoscono / collaborano*". È il legame più debole: non c'è gerarchia né tutto–parte. Qui ce ne sono due: `Environmental Controller`–`SystemLog` e `Actuator`–`SystemLog`. Il `SystemLog` registra gli eventi di controllore e attuatori.

`Heater`, `Light` **e** `Cooler` **sono associati tra loro? No.** Tra di loro non esiste nessuna linea: sono solo "fratelli", perché condividono lo stesso padre `Actuator` (generalizzazione). Ciascuno ha invece relazioni solo con *altre* classi:

- con `Actuator`: generalizzazione (ne è figlio);
- con `Environmental Controller`: aggregazione (ne è parte);
- con `SystemLog`: associazione *ereditata*, perché l'associazione è disegnata su `Actuator` e vale quindi per tutte le sue sottoclassi.

Classi e Diagrammi: Linee Guida di Progettazione. Nello sviluppo di modelli OO, le classi e i diagrammi non hanno una corrispondenza biunivoca: una singola classe può comparire all'interno di molteplici diagrammi differenti. Ciascun diagramma deve focalizzarsi su una specifica vista o prospettiva del sistema, rispettando criteri di chiarezza e leggibilità:

- **Numero contenuto di elementi**: evitare di inserire un numero eccessivo di classi nel medesimo diagramma;
- **Associazioni essenziali**: limitare le associazioni mostrate unicamente a quelle rilevanti per la vista corrente;
- **Information hiding**: nascondere attributi o operazioni secondarie che non contribuiscono alla comprensione dello scenario analizzato;
- **Natura iterativa**: la stesura di un diagramma efficace e corretto (*proper diagram*) richiede sistematicamente molteplici iterazioni di raffinamento.

Notazione Grafica delle Relazioni: Legenda. Prima di entrare nel dettaglio di ogni relazione, la tabella seguente fissa la notazione usata in tutti i diagrammi di queste note (e nelle soluzioni della sezione *UML: esercitazioni*, §4). Ciò che distingue due relazioni è sempre la combinazione di tre elementi: **tipo di linea** (continua o tratteggiata), **decorazione all'estremo** (freccia, triangolo, rombo) e **lato** su cui la decorazione è posta.
Le relazioni si ordinano per **forza del legame**, dalla più debole alla più forte:

$$\text{dipendenza} < \text{associazione} < \text{aggregazione} < \text{composizione} < \text{generalizzazione}$$

Nel dubbio si sceglie la relazione *più debole* che riesce ancora a esprimere il vincolo del dominio: ogni legame più forte aumenta l'accoppiamento tra le classi (principio del basso accoppiamento, sezione sulla progettazione modulare).

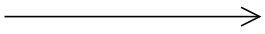
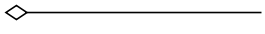
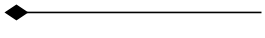
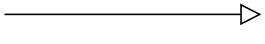
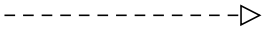
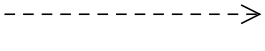
Simbolo	Relazione	Significato
	Associazione	legame semantico tra istanze, *lunga durata*
	Associazione navigabile	come sopra, ma solo il lato *sorgente* conosce quello di destinazione
	Aggregazione (◊)	parte–tutto *debole*: le parti sopravvivono al tutto
	Composizione (♦)	parte–tutto *forte*: le parti nascono e muoiono col tutto
	Generalizzazione	*is-a*: il triangolo punta alla *superclasse*
	Realizzazione	la classe implementa un'*interfaccia*: il triangolo punta all'interfaccia
	Dipendenza	uso *temporaneo*: se il fornitore cambia, il cliente può rompersi

Figura 4: Legenda delle relazioni UML tra classi. Il rombo sta sempre dal lato del *tutto*; il triangolo e la freccia stanno sempre dal lato dell'elemento *più generale* o *usato*.

Come scegliere la relazione giusta. Le tre domande seguenti, poste nell'ordine, selezionano la relazione (con esempi dalle esercitazioni):

1. **Una classe *è un* caso particolare dell'altra?** → generalizzazione (`Manager` *is-a* `Staff`, Esercizio del Pub); se l'elemento generale è solo un contratto di operazioni → realizzazione (`Pistol` realizza `SemiAutomatic`).
2. **Una classe *è parte* dell'altra?** → aggregazione o composizione. Se la parte non ha senso senza il tutto → composizione (`Room` in `Apartment`, Esercizio 4: la stanza non esiste senza l'appartamento); altrimenti aggregazione (`Person` proprietaria di `Apartment`).
3. **Un'istanza *conosce* stabilmente l'altra (la mantiene in un attributo)?** → associazione, eventualmente navigabile (`Customer` *places* `Order`). Se invece l'altra classe compare solo come parametro, variabile locale o valore di ritorno di un metodo → dipendenza.

Associazione: Relazione tra Classi. Un'**associazione** stabilisce un legame semantico e concettuale tra istanze di classi distinte. La notazione grafica prevede tre elementi fondamentali:

- **Nome dell'associazione (*Name*)**: indica il significato logico della relazione (ad esempio `hasCustomer`);
- **Direzione di lettura (*Direction*)**: indicata da una freccia o triangolo pieno accanto al nome, che stabilisce il verso corretto in cui leggere la frase concettuale (es. un ordine "ha come cliente" un dato cliente);
- **Molteplicità agli estremi (*Multiplicity*)**: specifica il numero di istanze di ciascuna classe che possono o devono partecipare al legame;

- **Vincoli cardinali**: nell'esempio tipico, a un'istanza di `Order` è associato esattamente 1 cliente (`Customer`), mentre un singolo cliente può emettere molteplici ordini ($*$).



Convenzioni sui Nomi delle Associazioni. La corretta denominazione delle associazioni è fondamentale per evitare ambiguità descrittive:

- **Nomi generici da evitare**: non impiegare etichette vaghe o tautologiche prive di valore informativo, quali `associated_with`, `has` o `is_related_to`;
- **Uso di sintagmi verbali**: assegnare preferibilmente nomi costituiti da predicati verbali precisi ed espliciti, come `has_part`, `is_contained_in`, `works_for` o `teaches`.

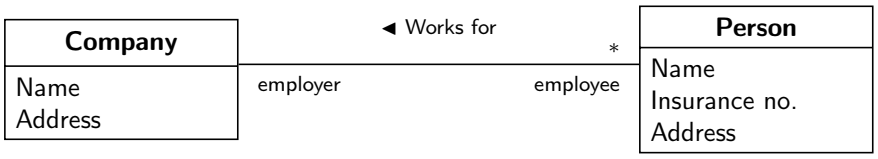
I Ruoli nelle Associazioni (Roles). Ciascuna estremità di un'associazione definisce un **ruolo** (*role*), che specifica la funzione o la prospettiva assunta dall'oggetto di destinazione nell'ambito della relazione:

- **Duplicità del ruolo**: ogni associazione contempla due ruoli, uno per ciascuna direzione di navigazione o di lettura;
- **Ruolo implicito**: se non specificato espressamente, il ruolo coincide convenzionalmente con il nome della classe di destinazione;
- **Ruolo esplicito**: può essere etichettato direttamente all'estremità della linea per chiarire il significato contestuale dell'oggetto collegato (ad esempio, l'estremo di `OrderLine` collegato a `Order` assume esplicitamente il ruolo di `line item`).

Role Names e Associazioni Riflessive. L'esplicitazione dei nomi di ruolo assume un'importanza strutturale vincolante in scenari specifici:

- **Identificazione dei partecipanti**: nell'associazione tra `Company` e `Person` (con nome associazione *Works for*), l'estremo verso l'azienda prende il ruolo di `employer`, mentre l'estremo verso la persona prende il ruolo di `employee` (con molteplicità $*$), definendo i contratti bilaterali del legame;
- **Obbligatorietà nelle associazioni riflessive**: quando un'associazione collega istanze appartenenti alla **medesima classe** (associazione unaria o ricorsiva), la definizione del nome di ruolo è **obbligatoria**;
- **Disambiguazione della gerarchia**: ad esempio, se la classe `Person` modella la relazione aziendale interna *Supervises*, solo la presenza dei ruoli distinti `Manager` e `Salesperson` consente di distinguere chi supervisiona da chi viene supervisionato.

Associazione con ruoli bilaterali



Associazione riflessiva (ruoli obbligatori)



Molteplicità nelle Associazioni. La **molteplicità** specifica quanti oggetti di una determinata classe possono partecipare all'associazione rispetto a una singola istanza della classe opposta. Essa viene indicata in prossimità dell'estremità della linea di collegamento:

- **Esempio d'ordine e cliente**: a un'istanza di `Order` è associato esattamente un `Customer` (1), mentre un singolo cliente può generare molteplici ordini ($*$);
- **Formulazione discorsiva**: l'ordine appartiene a uno e un solo committente, mentre il committente può emettere una pluralità di ordini nel tempo.

Sintassi e Notazioni della Molteplicità. I vincoli cardinali possono essere espressi mediante diverse notazioni formali:

- `*` oppure `0..*`: cardinalità da zero a infinito ($0..\infty$), indicante un numero arbitrario di istanze collegate;
- `1` oppure `1..1`: esattamente una istanza obbligatoria;
- `0..1`: partecipazione facoltativa (zero o una sola istanza);
- `1..100`: intervallo continuo con limite minimo e massimo definiti;
- `2,4,5`: enumerazione discreta di valori ammissibili.

Navigabilità delle Associazioni (Navigability). La **navigabilità** stabilisce la possibilità per un'istanza di accedere direttamente a runtime all'oggetto associato:

- **Navigabilità unidirezionale**: rappresentata da una **freccia aperta** (a V, non un triangolo) all'estremo navigabile della linea continua di associazione;



- **Accesso asimmetrico**: una freccia da `Order` verso `Customer` indica che l'ordine possiede la conoscenza diretta del proprio cliente, mentre il cliente non mantiene visibilità o collezioni dirette dei propri ordini;
- **Associazione bidirezionale**: in assenza di frecce, la navigabilità è garantita in entrambi i versi; in questo scenario l'uso dei nomi di ruolo diventa fondamentale per identificare chiaramente i contratti di accesso reciproci.



Dipendenza (Dependency). La **dipendenza** è la relazione più debole tra classi e si distingue dall'associazione per la *durata* del legame:

- **Notazione**: linea **tratteggiata** con **freccia aperta** (a V) che parte dalla classe *cliente* e punta alla classe *fornitore* (*supplier*). Può avere uno stereotipo, ad esempio «*use*», «*create*» o «*instanceOf*»;
- **Semantica**: una modifica al fornitore può richiedere modifiche al cliente (stessa definizione usata per le dipendenze tra package, §3.2.6). Non esiste un attributo che memorizzi il fornitore: il cliente lo *usa* soltanto, in modo **temporaneo**, come parametro, variabile locale o valore di ritorno di un'operazione, oppure lo istanzia;
- **Differenza con l'associazione**: l'associazione (linea continua) implica un collegamento che sopravvive alle singole chiamate e di norma si traduce in un attributo; la dipendenza (linea tratteggiata) vale solo per la durata dell'uso;
- **Molteplicità e ruoli**: non si indicano, perché non c'è un legame strutturale tra le istanze.

Classi di Associazione (Association Classes). Una **classe di associazione** si introduce quando uno o più attributi e operazioni non appartengono concettualmente a nessuna delle due classi correlate prese singolarmente, bensì appartengono intrinsecamente all'atto stesso della loro associazione:

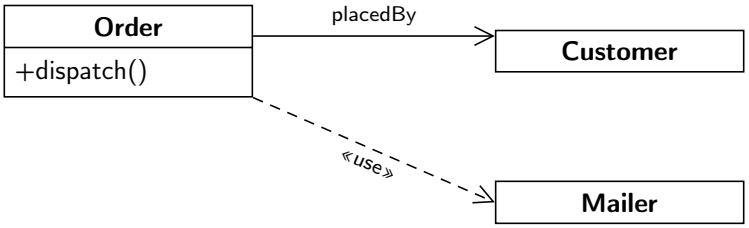
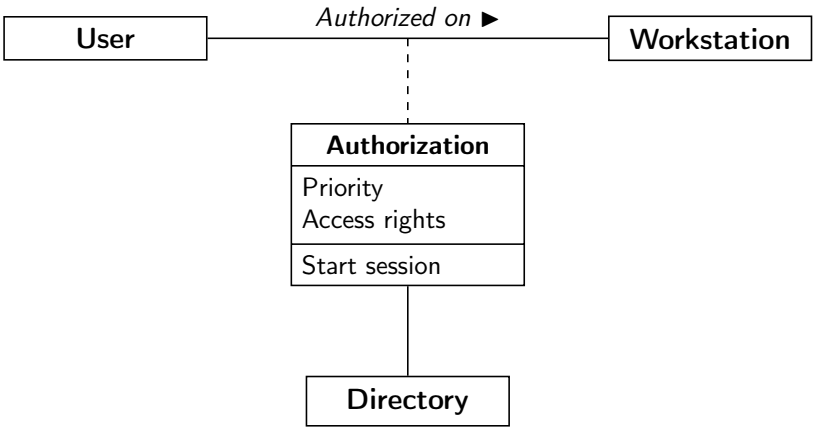


Figura 5: `Order` *ha* un `Customer` (associazione: attributo permanente) ma *usa* un `Mailer` solo dentro `dispatch()` (dipendenza).

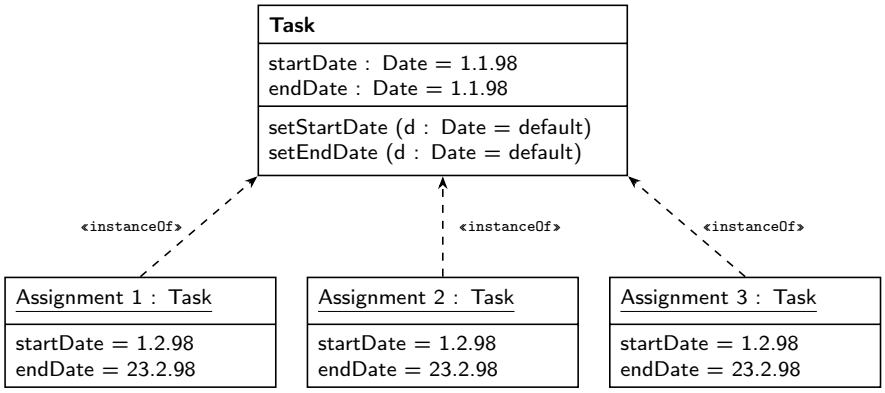
- **Esempio di autenticazione**: la relazione *Authorized on* tra un utente (`User`) e una postazione di lavoro (`Workstation`) definisce attributi contestuali quali il livello di priorità (`Priority`) e i diritti di accesso (`Access rights`), nonché l'azione `Start session`;
- **Notazione grafica**: la classe di associazione (`Authorization`) viene agganciata mediante una linea tratteggiata ortogonale alla linea di associazione principale;
- **Partecipazione ad altre relazioni**: trattandosi a tutti gli effetti di una classe, una classe di associazione può a sua volta intrattenere ulteriori associazioni ordinarie con altre classi del sistema (ad esempio verso `Directory`).



Relazione tra Classi e Istanze di Oggetti. Una classe definisce la struttura astratta condivisa da un gruppo omogeneo di oggetti, dichiarandone il nome, gli attributi e le operazioni ammissibili con eventuali valori di default:

- **La Classe**: definisce il pattern generale (ad esempio `Task` con attributi tipizzati `startDate : Date` e metodi d'impostazione);
- **Gli Oggetti**: rappresentano le entità concrete istanziate a runtime; la notazione grafica UML richiede che il nome dell'oggetto sia obbligatoriamente **sottolineato**, nella forma `nomeOggetto : NomeClasse`;
- **Rappresentazione dello stato**: nei box degli oggetti non si indicano i metodi (poiché appartengono alla definizione della classe), bensì i valori concreti assunti dai campi di stato per quella specifica istanza;

- **Legame di istanziazione**: la dipendenza tra istanze concrete e classe modello viene rappresentata mediante una freccia tratteggiata caratterizzata dallo stereotipo formale `«instanceOf»`.



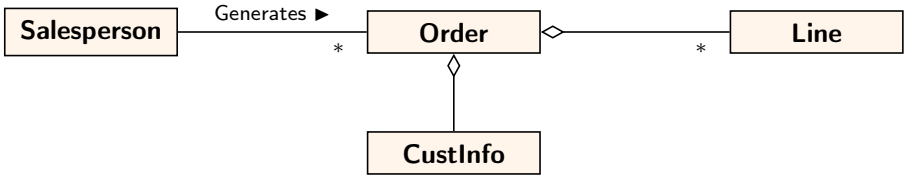
Corrispondenza tra Diagramma delle Classi e Diagramma degli Oggetti. Il diagramma delle classi definisce lo schema strutturale astratto del dominio, mentre il diagramma degli oggetti (*Object Diagram*) ne illustra una configurazione concreta a runtime (istanze specifiche e relativi collegamenti):

- **Modello a livello di classe**: la classe `Salesperson` genera (*Generates*) un insieme di ordini (`Order`, con molteplicità $*$). Ogni ordine aggrega le informazioni sul cliente (`CustInfo`) e una collezione di righe d'ordine (`Line`, con molteplicità $*$);
- **Modello a livello di istanza**: a runtime, una singola persona addetta alle vendite (`curtisClyde:`) può aver acquisito due istanze di ordine distinte (`order121:` e `order122:`);
- **Configurazione dei componenti**: `order121:` contiene il profilo cliente `ace furniture:` e quattro righe di vendita (`line1:` ... `line4:`), mentre `order122:` aggrega `harmon assoc:` e due sole righe (`line1:`, `line2:`).

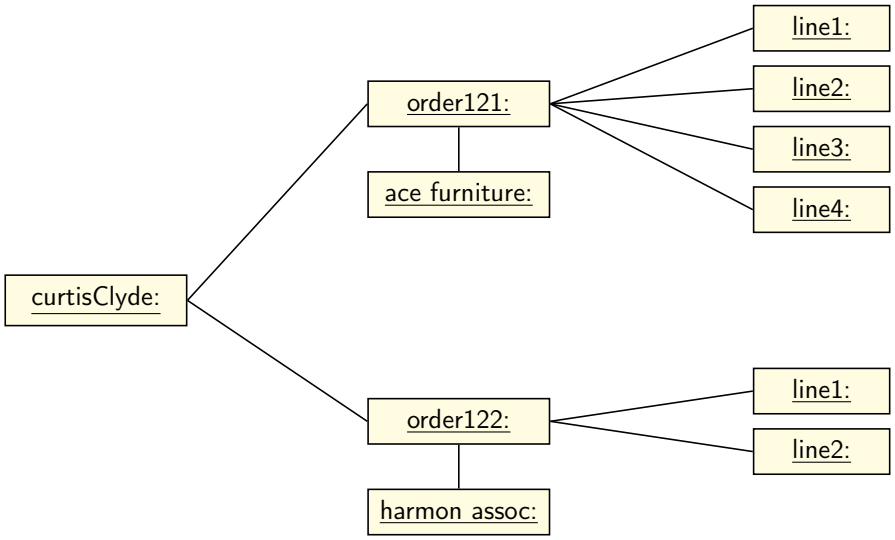
Differenza tra Attributi e Associazioni. Sebbene concettualmente possano apparire simili, attributi e associazioni assolvono compiti differenti:

- **Sul piano concettuale**: la distinzione è minima; un attributo può essere visto come una forma degenere di associazione a valore singolo con molteplicità 0..1;
- **Sul piano di specifica e implementazione**:
  - **Navigabilità implicita**: la navigabilità procede sempre unidirezionalmente dalla classe verso il tipo dell'attributo;
  - **Memorizzazione per valore, non per riferimento**: un attributo conserva un valore concreto (*value*), non un puntatore o riferimento logico condivisibile. Non vi è condivisione di istanza tra oggetti differenti (*no sharing between instances*);
  - **Tipi valore semplici**: gli attributi accolgono tipicamente oggetti privi di identità autonoma, quali numeri, stringhe, date o importi monetari (`Money`).

Class diagram



Object diagram



Le Operazioni: Responsabilità e Interfaccia. Le **operazioni** (o servizi) modellano le attività che una classe è in grado di svolgere ed equivalgono ai messaggi che essa può ricevere:

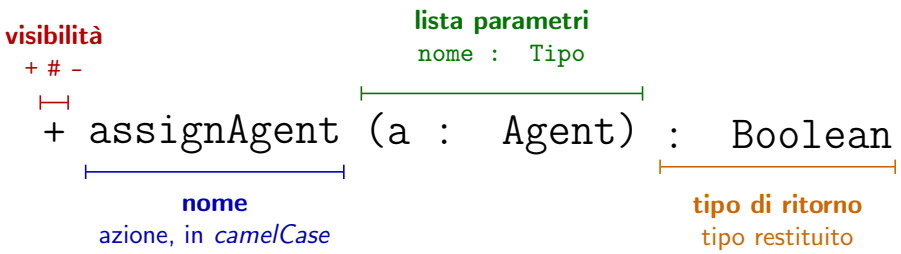
- **A livello concettuale**: corrispondono alle **responsabilità primarie** assegnate alla classe all'interno del dominio;
- **A livello di specifica**: rappresentano l'insieme dei messaggi pubblici accettati, costituendo l'**interfaccia** visibile verso l'esterno;
- **Convenzione di sintesi**: nei modelli concettuali non si indicano mai le banali operazioni di accesso o modifica dei singoli campi (*getter* e *setter*), concentrandosi solo sulla logica rilevante.

Sintassi Formale UML per le Operazioni. La specifica UML formalizza la firma delle operazioni secondo la sintassi:

visibilità **nome** (lista_parametri) : tipo_ritorno

Ad esempio: `+ assignAgent (a : Agent) : Boolean`.

- **Visibilità**: definita dai simboli `+` (*public*), `#` (*protected*) e `-` (*private*). La sua interpretazione dipende dal linguaggio di programmazione e risulta del tutto opzionale a livello concettuale;
- **Nome**: stringa identificativa che esprime l'azione (spesso in notazione *camelCase*);
- **Lista parametri**: argomenti formali espressi con la notazione `nomeParametro : Tipo`;
- **Tipo di ritorno**: specifica del tipo di dato o dell'oggetto restituito al chiamante.



Classificazione delle Operazioni: Query vs Modifier. Dal punto di vista dell'effetto prodotto sullo stato interno dell'oggetto, le operazioni si ripartiscono in due categorie nette:

- **Query**: interroga l'oggetto e restituisce un valore informativo **senza alterare** in alcun modo il suo stato interno (operazione priva di effetti collaterali, *side-effect free*). Le query possono essere eseguite in qualsiasi ordine senza modificare il comportamento futuro del sistema. Corrispondono tipicamente alle richieste di lettura (*getting*);
- **Modifier**: altera lo stato interno dell'oggetto modificando uno o più attributi. La sequenza e l'ordine temporale di invocazione dei modificatori sono determinanti per il corretto funzionamento. Corrispondono ai comandi di modifica (*setting*).

Aggregazione: Semantica e Proprietà. L'**aggregazione** costituisce una forma specializzata di associazione binaria che esprime una relazione concettuale di tipo "parte-tutto" (*has-a / part-of*):

- **Rapporto contenitore-componente**: i componenti rappresentano parti costitutive dell'oggetto aggregato (ad esempio, un'automobile ha motore e ruote come parti integranti);
- **Transitività**: l'aggregazione gode della proprietà transitiva; se la classe A aggrega la classe B e B a sua volta aggrega C, allora A aggrega indirettamente C;
- **Scenari tipici**: modellazione di distinte base o scomposizione fisica (*parts explosion*) e gerarchie organizzative aziendali.

Aggregazione e Composizione (Composizione Forte). Nel metamodello UML si distingue l'aggregazione semplice (o debole) dalla **composizione** (o aggregazione forte), la quale introduce vincoli stringenti di ciclo di vita e possesso esclusivo:

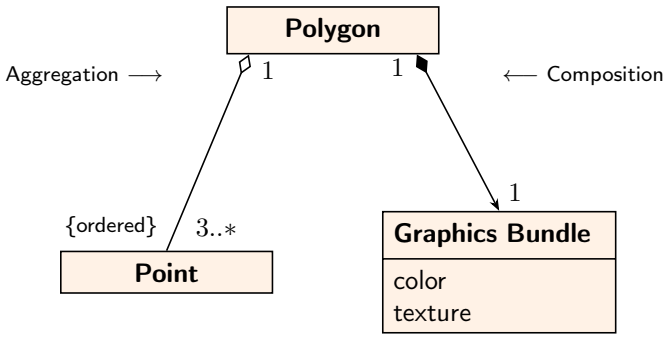
- **Appartenenza esclusiva**: nella composizione ciascuna parte componente appartiene a uno e un solo oggetto contenitore (*"components belong only to one whole"*);
- **Coincidenza del ciclo di vita**: le parti nascono e muoiono con il contenitore (*"parts live and die with the whole"*). La distruzione dell'oggetto principale implica obbligatoriamente la cancellazione a cascata (*cascading delete*) di tutte le sue parti costitutive;
- **Associazioni 1..1**: la composizione si rende necessaria anche per modellare associazioni biunivoche esclusive in cui i due estremi condividono la medesima esistenza;
- **Esempio di contrasto concettuale**: in una squadra sportiva (ad es. i Calgary Flames), i singoli atleti sono aggregati alla squadra ma **non** cessano di esistere né vengono eliminati se il club scompare; al contrario, le risorse interne a un poligono grafico vivono unicamente in sua funzione.

Esempio di Modellazione: Il Poligono Grafico. La classe `Polygon` illustra contemporaneamente entrambe le tipologie di relazione:

- **Aggregazione debole verso** `Point`: il poligono è definito da un insieme ordinato di punti geometrici (molteplicità $3..*$, con vincolo `{ordered}`); i punti possono esistere indipendentemente nello spazio cartesiano e sopravvivere alla rimozione della figura;
- **Composizione verso** `Graphics Bundle`: il poligono possiede in modo esclusivo il proprio descrittore grafico (attributi di colore e texture); la distruzione del poligono distrugge contestualmente il relativo pacchetto grafico.

Criteri per Riconoscere l'Aggregazione. Per stabilire se una relazione tra classi debba essere formalizzata come semplice associazione oppure come aggregazione, l'analista verifica due criteri diagnostici fondamentali:

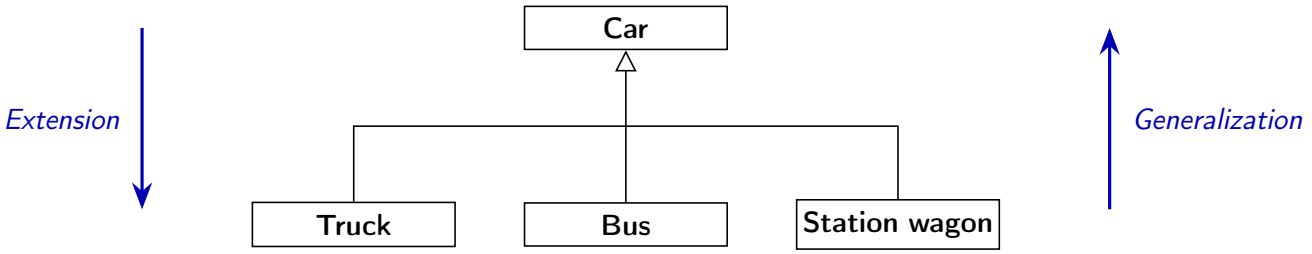
1. **Criterio concettuale di contenimento**: ha pienamente senso, nel vocabolario e nelle regole del dominio reale, affermare che l'entità figlia è "parte di" (*"part of"*) dell'entità aggregante?



2. **Propagazione dei comportamenti**: un'operazione o una trasformazione di stato invocata sull'intero sistema influenza e ricade direttamente anche sulle singole parti costitutive?

Generalizzazione ed Estensione. La modellazione concettuale gerarchica può essere percorsa in due direzioni complementari:

- **Generalizzazione (*Generalization*, verso l'alto)**: processo di astrazione induttiva che estrae proprietà e comportamenti condivisi da più entità affini (ad esempio `Truck`, `Bus` e `Station wagon`) sintetizzandoli in un'unica classe comune di livello superiore (`Car`);
- **Estensione o Specializzazione (*Extension*, verso il basso)**: processo deduttivo che declina un'entità generale introducendo caratteristiche e varianti specifiche per ciascun sottotipo;
- **Fattorizzazione**: le entità figlie conservano molti elementi comuni ereditati dalla radice, differenziandosi unicamente per le peculiarità del rispettivo ambito operativo.



Distinzione tra Istanziazione e Generalizzazione: Proprietà di Transitività. Un errore insidioso nell'analisi orientata agli oggetti consiste nel confondere il legame ontologico di **istanziazione** con quello di **generalizzazione**:

- **La Generalizzazione è transitiva (*is kind of*)**: se A è un sottotipo di B e B è un sottotipo di C, allora A è formalmente anche un sottotipo di C;
- **L'Istanziazione NON è transitiva (*is instance of*)**: appartenere come individuo concreto a una classe non autorizza salti categoriali tra metadati ed entità fisiche.

Considerando la sequenza di asserzioni descrittive:

1. *Shep* è un Border Collie (istanziazione: individuo $\in$ classe);

2. Un Border Collie è un Cane (generalizzazione: sottoclasse $\subseteq$ classe);

3. I Cani sono Animali (generalizzazione: sottoclasse $\subseteq$ classe);

4. Un Border Collie è una Razza (categorizzazione ontologica: classe $\in$ meta-concetto);

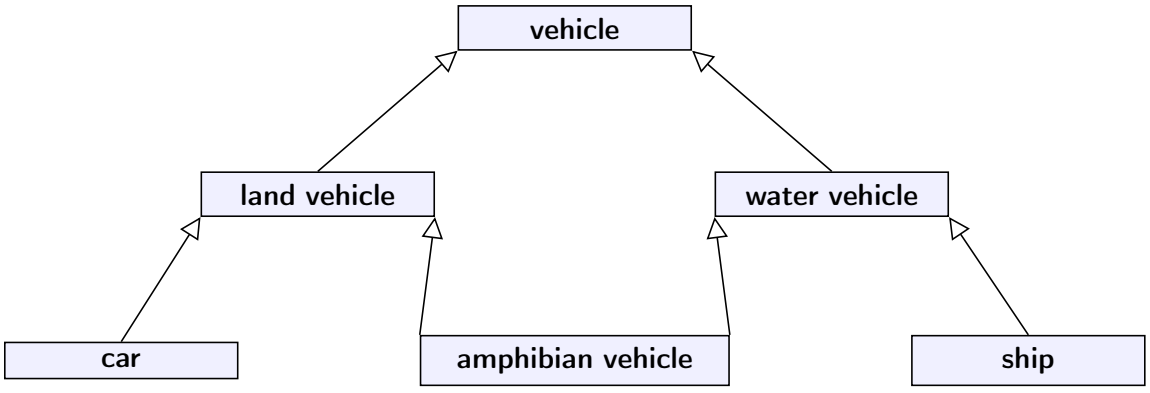
5. Un Cane è una Specie (categorizzazione ontologica: classe $\in$ meta-concetto).

Combinando le prime tre asserzioni, la transitività della generalizzazione è valida: *Shep* è a tutti gli effetti un cane $(1 + 2)$ ed è anche un animale $(1 + 2 + 3)$. Al contrario, non è logicamente ammissibile combinare l'istanza con i concetti di livello meta ($1+4$ e $2+5$): *Shep* **non** è una razza canina, né un Border Collie rappresenta una specie tassonomica.

Generalizzazione: Meccanismi di Estensione e Restrizione. Nelle gerarchie di ereditarietà, la sottoclasse eredita integralmente attributi e operazioni dalla classe antenata (*ancestor*), potendo evolvere secondo due direzioni:

- **Estensione (*Extension*)**: aggiunta di nuovi attributi o nuovi metodi specializzati all'interno della sottoclasse;
- **Restrizione (*Restriction*)**: applicazione di vincoli supplementari e più rigidi sui campi ereditati dall'antenato. Ad esempio, la classe `Circle` può essere modellata come specializzazione di `Ellipse` imponendo il vincolo di semiassi di uguale lunghezza;
- **Avvertenza di consistenza**: variazioni arbitrarie dello stato (come la modifica della lunghezza di un singolo asse nel cerchio) possono violare la restrizione geometrica, compromettendo l'integrità del modello.

Ereditarietà Multipla (Multiple Inheritance). L'ereditarietà multipla si verifica quando una classe deriva simultaneamente caratteristiche e comportamenti da più superclassi distinte.



Nella gerarchia sopra illustrata, `vehicle` fa da radice a `land vehicle` e `water vehicle`. La classe specialistica `amphibian vehicle` eredita congiuntamente da entrambi i rami, originando la classica struttura a diamante.

Il Problema del Diamante e l'Ereditarietà Virtuale in C++. Quando due classi derivate ereditano dalla medesima classe base e una quarta classe le eredita entrambe, si presenta il problema della duplicazione dei membri. Si consideri la gerarchia composta da `ItemToSell`, `EducationalTool`, `Book` e `TextBook`:

Ereditarietà ordinaria (senza `virtual`):

```
class ItemToSell {
    double price;
};

class EducationalTool :
    public ItemToSell { };

class Book :
    public ItemToSell { };

class TextBook :
    public EducationalTool,
    public Book { };
```

Ereditarietà virtuale (`virtual`):

```
class ItemToSell {
    double price;
};

class EducationalTool :
    virtual public ItemToSell { };

class Book :
    virtual public ItemToSell { };

class TextBook :
    public EducationalTool,
    public Book { };
```

Valutazione dell'Ereditarietà Multipla: Vantaggi e Svantaggi. L'ereditarietà multipla consente a una classe di derivare contemporaneamente proprietà e comportamenti da più superclassi, introducendo opportunità progettuali ma anche sensibili complessità:

- **Vantaggi**:
  - **Aderenza al modello concettuale**: riflette da vicino il modo naturale di classificare della mente umana, in cui un'entità appartiene spesso a molteplici categorie contemporaneamente;
  - **Flessibilità tassonomica**: offre grande libertà nella composizione e specializzazione dei tipi;
  - **Massimizzazione del riuso**: favorisce il recupero congiunto di porzioni di stato e logica già implementate in alberi gerarchici indipendenti.
- **Svantaggi e criticità**:
  - **Perdita di chiarezza nell'esecuzione**: insorgono ambiguità nel determinare quale specifico metodo debba essere invocato a runtime qualora più superclassi offrano la medesima firma;
  - **Complessità implementativa**: richiede strutture di memoria elaborate a livello di compilatore (come la tabella dei metodi virtuali a offset multipli o l'ereditarietà virtuale);
  - **Risoluzione dei conflitti**: impone l'onere esplicito di disaccoppiare attributi o comportamenti omonimi duplicati lungo i rami di derivazione.

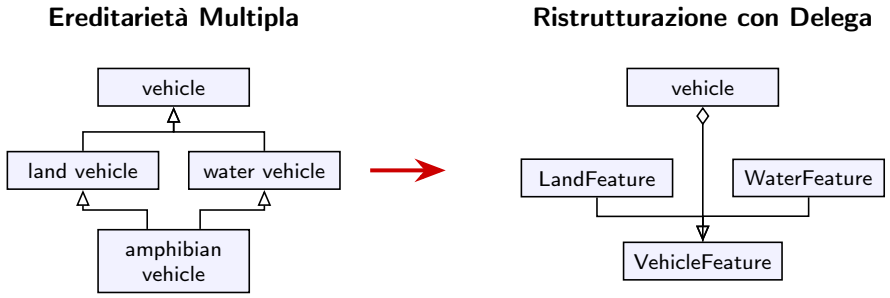
Strategie per Evitare l'Ereditarietà Multipla. Poiché molti linguaggi di programmazione moderni (come Java o C#) vietano l'ereditarietà multipla di classi concrete per scongiurare ambiguità, il problema viene risolto a livello architetturale ristrutturando il modello concettuale attraverso tre strategie principali:

1. **Delega e aggregazione (*Delegation & aggregation*)**: si sostituisce la relazione di specializzazione (*is-a*) con una relazione di possesso e delega (*has-a*), aggregando componenti che incapsulano le singole caratteristiche;

2. **Ereditarietà del tratto primario e delega dei tratti secondari**: si identifica la caratteristica concettualmente dominante su cui basare l'ereditarietà singola, delegando le restanti proprietà tramite aggregazione o composizione;

3. **Generalizzazione su dimensioni distinte**: si struttura una tassonomia gerarchica rigorosamente ordinata secondo criteri ortogonali successivi.

Tecnica 1: Sostituzione con Delega e Aggregazione. Nel caso del veicolo anfibio (che nell'ereditarietà multipla deriva sia da `land vehicle` sia da `water vehicle`), il modello ristrutturato elimina la doppia derivazione: la classe `Vehicle` aggrega un'astrazione `VehicleFeature`, la quale viene a sua volta specializzata nelle componenti concrete `LandFeature` e `WaterFeature`. Un veicolo anfibio viene quindi modellato come un veicolo generico che possiede entrambe le feature delegate.



Quando Utilizzare i Diagrammi delle Classi. I diagrammi delle classi costituiscono la colonna portante (*backbone*) dell'intero approccio concettuale orientato agli oggetti. Tuttavia, la loro utilità dipende strettamente da come vengono impiegati:

- **Non sovraccaricare la notazione**: iniziare sempre con i costrutti essenziali, evitando di forzare ogni simbolo o ornamento previsto dallo standard UML;

- **Rispettare la prospettiva adottata**: nella fase di analisi concettuale non bisogna inserire dettagli implementativi; la prospettiva di specifica (focalizzata su interfacce e contratti) risulta spesso più duratura ed efficace della mera implementazione;

- **Focalizzarsi sulle aree critiche**: è preferibile produrre pochi diagrammi mirati, precisi e costantemente aggiornati, piuttosto che mantenere una vasta documentazione obsoleta e disallineata rispetto al codice.

Procedura Pratica per Creare un Class Diagram. La costruzione di un diagramma delle classi procede secondo una sequenza incrementale:

1. **Partire dalla struttura minima**: individuare le classi principali del dominio applicativo e tracciare le associazioni più evidenti ed esplicite;
2. **Arricchire progressivamente il modello**:
   - Introdurre gli **attributi** salienti necessari a caratterizzare lo stato;
   - Definire la **molteplicità** numerica agli estremi di ciascuna linea di collegamento;
   - Specificare le **operazioni** primarie che esprimono le responsabilità e i servizi erogati.

Regole di Buona Pratica (Rules of Thumb). Per garantire la massima leggibilità e manutenibilità dei modelli, è opportuno attenersi ad alcune regole pratiche consolidate:

- **Pluralità delle viste**: una medesima classe può legittimamente comparire in diagrammi diversi, ciascuno focalizzato su un aspetto o sottosistema differente;
- **Contenimento della densità**: ogni diagramma deve descrivere un aspetto specifico; è buona norma evitare un affollamento di troppe classi o un intreccio fitto di associazioni;
- **Occultamento del dettaglio superfluo**: nascondere attributi, operazioni o parametri di dettaglio che non risultano direttamente rilevanti per la comprensione della specifica vista corrente;
- **Raffinamento iterativo**: la realizzazione di un diagramma formalmente corretto ed efficace (*"proper diagram"*) non scaturisce mai al primo tentativo, ma richiede sempre molteplici cicli di revisione.

3.2.6 Object-Oriented Design (OOD)

Obiettivi e Problematiche di Progettazione (Design Issues). Un metodo di progettazione orientato agli oggetti efficace deve governare cinque dimensioni qualitative:

- **Decomposability (decomponibilità)**: quanto il metodo aiuta a scomporre un problema complesso in sotto-problemi indipendenti e più semplici da risolvere.
- **Composability (componibilità)**: quanto i componenti (moduli/classi), una volta realizzati e collaudati, possono essere riutilizzati e ricomposti in nuovi sistemi.
- **Understandability (comprensibilità)**: quanto un modulo può essere compreso isolatamente, senza analizzare l'intero sistema circostante.
- **Continuity (continuità)**: capacità dell'architettura di assorbire piccole variazioni dei requisiti con modifiche localizzate in uno o pochi moduli.
- **Protection (protezione)**: capacità di confinare la propagazione degli effetti di un errore o di un'eccezione verificatasi all'interno di un modulo.

I Quattro Componenti Fondamentali dell'OOD. La decomposizione architetturale isola quattro aree di interesse ortogonali:

1. **Problem Domain Component**: sottosistemi che implementano i requisiti funzionali e la logica di dominio del cliente.
2. **Human Interaction Component**: moduli responsabili delle interfacce grafiche (GUI) e dell'esperienza d'uso.
3. **Task Management Component**: componenti che controllano e coordinano compiti concorrenti e thread paralleli.
4. **Data Management Component**: infrastruttura per persistenza, memorizzazione e recupero degli oggetti (DBMS o file system).

Transizione da OOA a OOD: Una Differenza Qualitativa. Il passaggio dall'analisi (OOA) alla progettazione (OOD) non è un semplice incremento di dettaglio, ma una **differenza qualitativa**: si passa dal modellare *cosa* il sistema deve fare nel dominio reale al definire *come* il software viene costruito ed eseguito.

Modello di Analisi (OOA)		**Modello di Design (OOD)**
Classi concettuali	$\longrightarrow$	Classi concrete e istanze di oggetti
Attributi generici	$\longrightarrow$	Tipi di dato specifici del linguaggio
Operazioni astratte	$\longrightarrow$	Metodi con firma, visibilità e algoritmo
Associazioni logiche	$\longrightarrow$	Link navigabili, puntatori e messaggi
Comportamento atteso	$\longrightarrow$	Controllo di flusso, concorrenza e gestione eventi

In OOD le decisioni implementative si consolidano:

- i nomi e le firme dei metodi vengono fissati;
- la molteplicità si realizza con strutture dati concrete (array, puntatori, collezioni);
- la visibilità è formalizzata: `+` public, `-` private, `#` protected, `~` package;
- i diagrammi di interazione vengono raffinati nel dettaglio temporale ed esecutivo.

Statechart Diagrams vs Interaction Diagrams. La distinzione tra le due famiglie risiede nel perimetro di osservazione:

- **Diagrammi di interazione**: visione *inter-oggetto* (macro), mostrano come più oggetti cooperano scambiandosi messaggi per realizzare uno scenario d'uso.
- **Diagrammi di stato (Statechart)**: visione *intra-oggetto* (micro), a grana fine. Descrivono il ciclo di vita di **un solo oggetto alla volta**, mostrando come i messaggi ricevuti ne alterano lo stato.
- **Sinonimi**: state diagrams, state transition diagrams, macchine a stati finiti, *Harel diagrams* (da David Harel, che ne ha definito la semantica estesa per sistemi reattivi).

Lo Stato di un Oggetto e le Transizioni. Lo **stato** di un oggetto è determinato dall'insieme dei valori correnti dei suoi attributi in un dato istante. Nella modellazione gli stati non rappresentano ogni piccola variazione di valore, ma raggruppano le configurazioni qualitativamente significative per la logica di business.
Ad esempio, per una classe `Account` con attributo `balance: Float` si distinguono due stati: `ok` (quando balance ≥ 0) e `overdrafted` (quando balance < 0).

- Gli stati mutano in risposta a eventi (es. invocazione di `deposit` o `withdraw`).
- Come nelle macchine a stati finiti, il sistema può tornare ciclicamente allo stato di partenza (`ok` $\rightarrow$ `overdrafted` $\rightarrow$ `ok`).
- Un evento può causare una transizione tra stati diversi oppure una **transizione riflessiva** (*self-transition*) che lascia l'oggetto nello stesso stato (es. un `deposit` su un conto già `ok`).

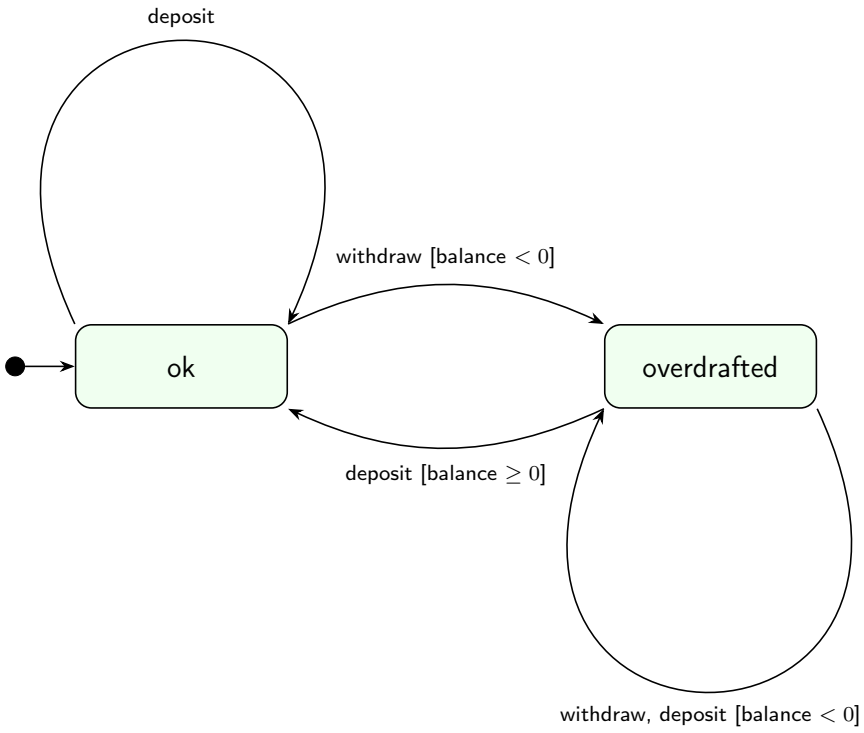


Figura 6: Statechart della classe `Account`: stati `ok` e `overdrafted`, con transizioni riflessive.

Sintassi Formale dello Stato (State Notation 1). In uno Statechart il rettangolo con angoli arrotondati può essere suddiviso in due compartimenti:

1. **nome dello stato** ed eventuali variabili di stato significative;
2. **azioni e attività interne**:
   - `entry / azione`: eseguita istantaneamente all'ingresso nello stato;

- `do / attività`: eseguita in modo continuativo finché l'oggetto resta nello stato;
- `on evento / azione`: eseguita internamente al verificarsi di un evento, senza uscire dallo stato;
- `exit / azione`: eseguita istantaneamente all'uscita dallo stato.

Distinzione Fondamentale: Azione vs Attività

- **Azione (*action*)**: processo quasi istantaneo, atomico e **non interrompibile**. Esempi: azzerare un contatore, emettere un segnale acustico, impostare una variabile.
- **Attività (*activity*)**: processo che dura nel tempo, richiede risorse significative ed è **interrompibile** da qualsiasi evento che inneschi una transizione uscente. Esempi: rendering grafico, scansione continua di una porta seriale.

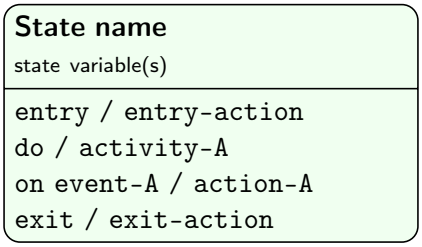


Figura 7: Notazione di uno stato con compartimenti interni.

Sintassi Formale della Transizione (State Notation 2 e 3). Una transizione tra due stati modella il passaggio causale ed è etichettata dalla sintassi

Event (argomenti) [**guard condition**] / **action**

- **Event**: lo stimolo scatenante (messaggio ricevuto, evento di clock, segnale hardware);
- **Guard condition**: espressione booleana tra parentesi quadre. La transizione scatta solo se la guardia è vera nel momento in cui si riceve l'evento. Le guardie di transizioni uscenti dallo stesso stato con lo stesso evento devono essere **mutuamente esclusive**;
- **Action**: procedura atomica eseguita contestualmente alla transizione;
- *Omissione*: ciascun elemento della terna (evento, guardia, azione) può essere omesso; una transizione senza evento (*completion transition*) scatta non appena terminano le attività interne dello stato sorgente.

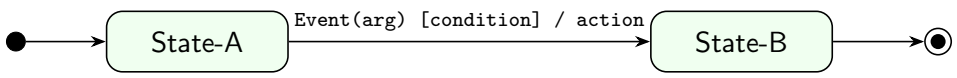


Figura 8: Sintassi completa di una transizione.

Semantica degli Eventi (Run-to-Completion). Qualunque classificatore UML può avere una macchina a stati che ne descrive il comportamento.

- Gli eventi in arrivo finiscono in una **coda** e vengono estratti **uno alla volta**.
- **Run-to-completion**: un nuovo evento è estratto solo quando la macchina ha finito di elaborare il precedente.
- Se più transizioni sono eseguibili insieme (stesso evento, guardie entrambe vere), **ne scatta una sola**.
- **Submachine state**: uno stato può rimandare a un altro diagramma di stati; un'icona indica uno stato composito di cui non si mostra l'interno.

Esempio Completo: Gestione di un Ordine (Order Processing). Il ciclo di vita di un ordine e-commerce illustra l'interazione tra guardie e attività:

1. dallo stato iniziale si passa a `Checking` eseguendo l'azione `/ get first item`;
2. in `Checking` si esegue l'attività `do: check item`; se restano articoli da verificare scatta la transizione riflessiva `get next item [not all items checked]`;
3. se tutti gli articoli sono stati controllati ma alcuni non sono disponibili (`[all items checked && some items not in stock]`) si entra in `Waiting`;
4. se invece sono tutti disponibili (`[all items checked && all items in stock]`), oppure quando in `Waiting` arrivano le merci mancanti (`item received [all items available]`), si passa a `Dispatching`; se l'arrivo è parziale (`item received [some items not in stock]`) si resta in `Waiting` con una transizione riflessiva;
5. in `Dispatching` si esegue `do: initiate delivery` e, al termine, si raggiunge `Delivered`.

Semplificazione mediante Superstati (Composite States). Nello scenario dell'ordine sorge un'esigenza comune: **il cliente deve poter cancellare l'ordine in qualsiasi momento**.

- **Soluzione naive**: una transizione `cancelled` da ciascuno stato intermedio (`Checking`, `Waiting`, `Dispatching`) verso `Cancelled`. Genera disordine visivo e riduce la manutenibilità del diagramma.
- **Soluzione con superstato**: gli stati cooperanti si racchiudono in uno stato composito `Active`. Dal bordo del superstato parte **un'unica transizione** `cancelled` verso `Cancelled`: se l'evento arriva mentre l'oggetto è in uno qualsiasi dei sotto-stati (*substates*), il superstato viene interrotto e l'oggetto passa a `Cancelled`.

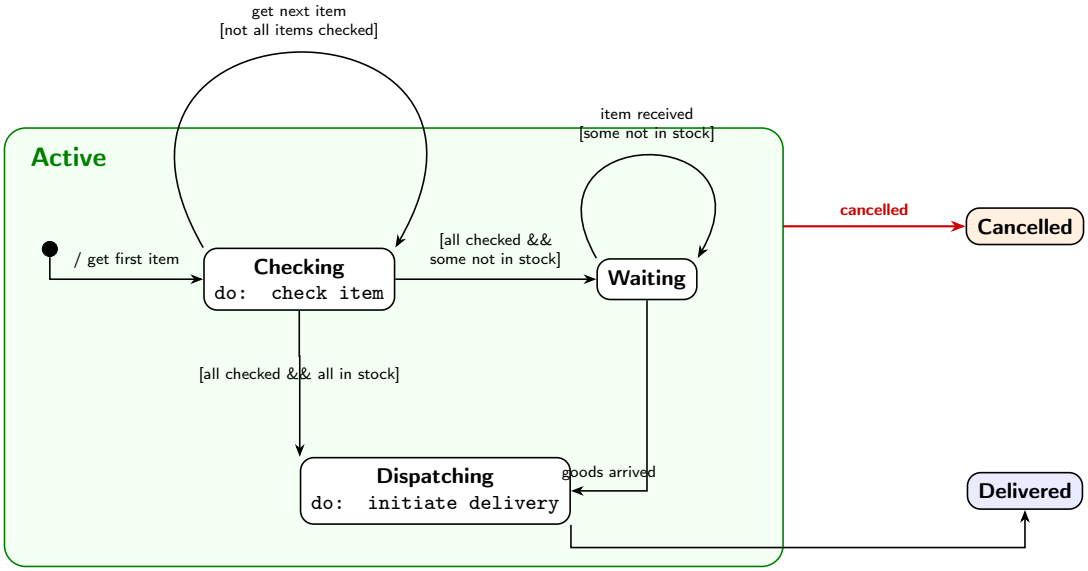


Figura 9: Superstato `Active`: una sola transizione `cancelled` sostituisce una transizione per ogni sotto-stato.

Diagrammi di Attività (Activity Diagrams). L'**Activity Diagram** modella il comportamento procedurale concentrandosi sul flusso di controllo sequenziale e parallelo tra attività. A differenza dello Statechart (che descrive un oggetto in funzione degli eventi esterni), rappresenta il flusso di lavoro interno al sistema (*workflow*).

- **Attività**: a livello concettuale è un compito da portare a termine; a livello di specifica o implementazione coincide spesso con un metodo di una classe.
- **Fondamento formale**: la semantica dei flussi concorrenti è analoga a quella delle **Reti di Petri** (evoluzione tramite token lungo archi e transizioni).

Costrutti Sintattici: Fork, Join e Branch. I nodi di controllo governano diramazioni e parallelismo:

- **Barra di fork**: barra nera con un arco entrante e due o più archi uscenti. Avvia rami concorrenti eseguiti in parallelo (in qualsiasi ordine o interleaving).
- **Barra di join**: barra con più archi entranti e un solo arco uscente. È un punto di rendezvous: per essere superata richiede che **tutti i rami concorrenti** in ingresso abbiano terminato.
- **Ramo decisionale (branch)**: rombo con un flusso entrante e più percorsi alternativi mutuamente esclusivi, controllati da **guardie** (`[condition 1]`, `[condition 2]`).
- **Ordinamento parziale**: l'Activity Diagram esprime un ordinamento parziale delle operazioni, senza imporre una sequenza rigida dove c'è concorrenza.

Nodi e Modello a Token (Activity Diagram).

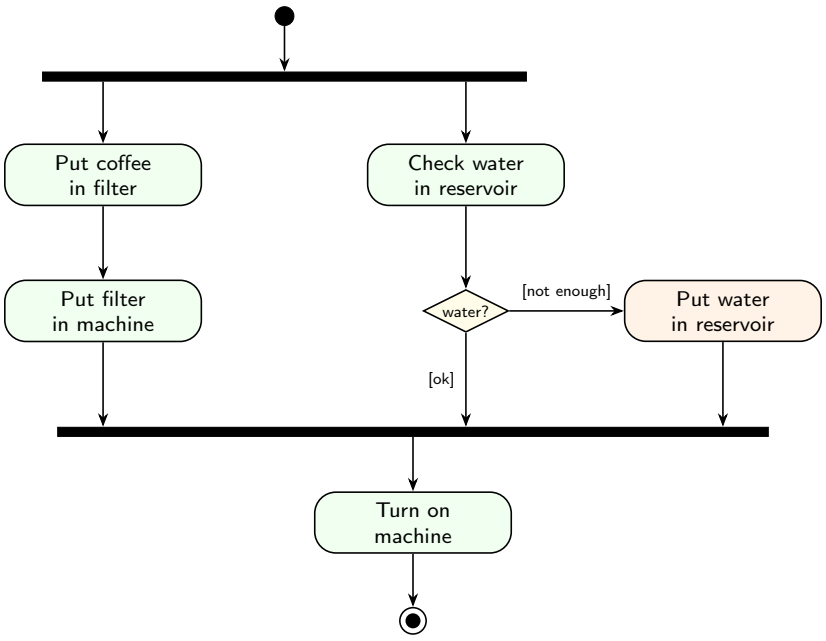


Figura 10: Activity Diagram della preparazione del caffè: fork, branch e join.

- **Tre tipi di nodo**: *action* (unità di comportamento: può invocare un'attività, un comportamento o un'operazione), *object* (oggetto in ingresso/uscita di un'azione), *control* (guidano il flusso: decision, merge, fork, join, finali).
- **Token**: la semantica si capisce immaginando **gettoni** che viaggiano nel diagramma. Un'azione parte quando c'è un token su **tutti** gli archi entranti e le **precondizioni** sono vere; alla fine mette un token su tutti gli archi uscenti. Il movimento di un token è atomico. Le **postcondizioni** devono valere perché il token esca.
- **Decision**: un arco entrante, più uscite **mutuamente esclusive** (il token va su una sola). **Merge**: più ingressi, una sola uscita (nessuna sincronizzazione).
- **Fork**: duplica il token su tutte le uscite. **Join**: genera il token in uscita solo quando ce n'è uno su **ogni** ingresso.
- **Nodi finali**: il *flow final* ($\otimes$) termina solo il ramo che lo raggiunge ("kill thread"); l'*activity final* (cerchio pieno con bordo) termina **tutti** i flussi ("kill process").
- **Usi**: modellare il flusso di un caso d'uso (analisi), il comportamento di un'operazione di classe o un algoritmo (design).

Diagrammi di Interazione: Il Dialogo tra Oggetti. I diagrammi di interazione descrivono il comportamento dinamico concentrandosi sugli **oggetti**: dettagliano come le istanze si scambiano messaggi per realizzare le funzionalità dei casi d'uso. L'interazione si osserva da due prospettive duali:

1. **Time-based view (Sequence Diagram)**: orientata alla cronologia degli scambi;
2. **Organization-based view (Collaboration Diagram)**: orientata alla topologia delle relazioni tra oggetti.

Sequence Diagrams: Struttura e Sintassi. Nei diagrammi di sequenza:

- gli **oggetti** sono disposti in orizzontale in alto, in riquadri con nome sottolineato (`nomeOggetto : NomeC`
- ogni oggetto ha una **linea di vita** (*lifeline*) verticale tratteggiata, lungo la quale il tempo scorre dall'alto verso il basso;
- i **messaggi** sono frecce orizzontali dall'oggetto mittente (client) al destinatario (supplier);
- la **barra di attivazione** (*activation bar*, o *focus of control*) è un rettangolo sottile sulla lifeline che indica l'intervallo in cui l'oggetto ha il controllo dell'esecuzione.

Tipologie di Messaggio nei Sequence Diagrams. UML definisce notazioni distinte a seconda della semantica:

- **Messaggio sincrono (*call*)**: **freccia a punta piena**. Il mittente invoca l'operazione e si blocca finché il destinatario non termina e restituisce il controllo (eventuale messaggio di ritorno tratteggiato).
- **Messaggio asincrono (*signal*)**: **freccia aperta o mezza freccia**. Il mittente invia il segnale e **non si blocca**. L'arrivo di un messaggio asincrono può:
  1. avviare un nuovo thread di esecuzione;
  2. creare una nuova istanza di oggetto;
  3. comunicare con un thread già attivo.
- **Creazione** («create»): freccia diretta al riquadro dell'oggetto creato, che compare all'altezza corrispondente al momento della nascita.
- **Distruzione** («destroy»): grande croce (**X**) al termine della lifeline.
- **Auto-delegazione (*self-call*)**: freccia che parte e ritorna sulla stessa lifeline, con barra di attivazione nidificata.
- **Iterazioni e guardie**: annotate nel testo del messaggio (es. `2: * prepare()`, `4: [check == true] remove()`).

Gestione della Complessità: Principio KISS

I diagrammi di sequenza devono chiarire la logica di scambio, non trasformarsi in labirinti illeggibili:

- applicare la regola **KISS** (*Keep It Small and Simple*);
- se una ramificazione è semplice, annotarla con una guardia `[condition]`; se i flussi alternativi sono complessi, **non affollare un unico diagramma**: disegnare diagrammi di sequenza separati per ciascuno scenario significativo.

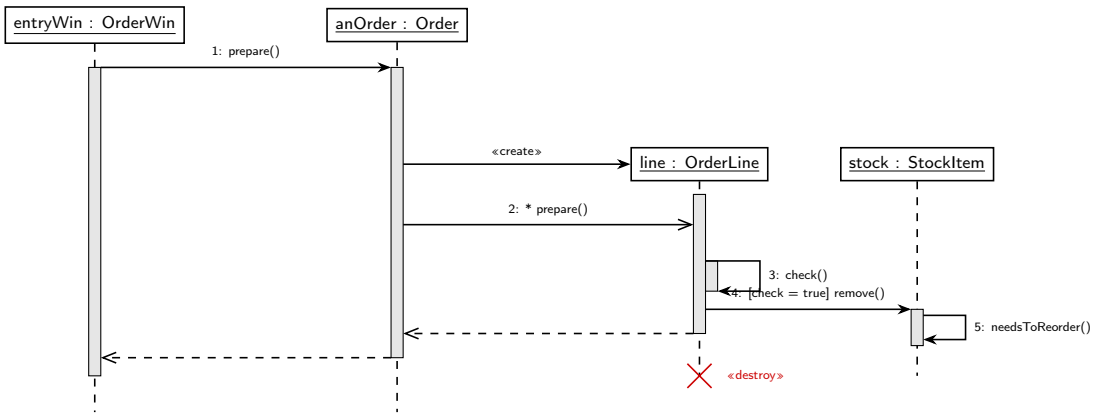


Figura 11: Sequence Diagram: creazione, messaggio asincrono iterato, self-call e distruzione di un oggetto.

Lifeline e Istanze. Nei diagrammi di interazione compaiono **istanze** di classificatori (oggetti, attori) oppure **lifeline**, che rappresentano un **ruolo** e non un oggetto preciso. Si preferiscono le lifeline.

- Un'**istanza** è *un* oggetto specifico; una **lifeline** è un'istanza **arbitraria** del classificatore.
- Entrambe usano la notazione del classificatore, ma la lifeline **non è sottolineata**.
- Notazione completa: `nome [selettore] : Tipo`. Il **selettore** è una condizione booleana opzionale che sceglie un rappresentante (es. `[id = "1234"]`).

Sync vs Async, Sintassi dei Messaggi.

- Il messaggio **sincrono** blocca il mittente fino al *return*; l'**asincrono** no. I return sono opzionali: si mostrano se non peggiorano la leggibilità o per indicare valori di ritorno.
- In **analisi** conviene indicare tutti i messaggi come sincroni (il caso più restrittivo); la distinzione emerge in design.
- Tipi di messaggio: chiamata sincrona, asincrona, return, creazione, distruzione, *found message* (mittente sconosciuto), *lost message* (destinatario sconosciuto).
- In analisi basta il **nome**; per più dettaglio si aggiungono i parametri tra parentesi (formali, attuali o entrambi: `getArea(shape=g)`) e le guardie.

Combined Fragment. Un **combined fragment** è un'area del diagramma che racchiude parte dell'interazione e ne dice **come** viene eseguita. È composto da:

- un **operatore** (nell'angolo in alto a sinistra, in un pentagono);
- uno o più **operandi**: fasce orizzontali separate da linee tratteggiate;
- zero o più **guardie** tra parentesi quadre, dopo l'operatore o in cima all'operando.

Il rettangolo si estende orizzontalmente fino a includere le lifeline coinvolte. Gli operatori più usati (corrispondono ai costrutti di controllo dei linguaggi):

- `opt`: un solo operando, eseguito se e solo se la guardia è vera (*if*);
- `alt`: più operandi, ne esegue **al più uno** (*if/else*);
- `loop min,max [cond]`: esegue min volte e poi al massimo (max−min) volte finché la condizione è vera; min e max si possono omettere, la condizione può essere in linguaggio naturale;
- `break`: come `break`;
- `gate`: punto sul bordo esterno del diagramma, origine o destinazione di una freccia;
- `ref`: rimanda a un altro sequence diagram, per modularizzare e riusare.

Classi di Confine (Boundary Classes). Una **boundary class** modella la frontiera di comunicazione tra il sistema e il mondo esterno (attori umani, periferiche, altri sistemi). Nei diagrammi di interazione conviene evidenziarle perché:

- permettono di formalizzare e isolare i requisiti di interfaccia verso l'esterno;
- **non** vincolano in anticipo la tecnologia (schermate grafiche, REST, RPC, driver seriali).

Collaboration Diagrams: La Prospettiva Organizzativa. I **diagrammi di collaborazione** (*communication diagrams* nelle versioni recenti di UML) descrivono la stessa interazione dei Sequence Diagrams, ma eliminano l'asse temporale per valorizzare la **topologia dei collegamenti** tra oggetti:

- gli oggetti sono nodi di una rete collegati da archi continui (*link*);
- l'ordine dei messaggi è espresso dalla **numerazione decimale (di Dewey)** accanto alla freccia:

  $$1,\ 1.1,\ 1.2,\ 2,\ 2.1,\ 2.1.1,\ 2.2,\ 3,\ \ldots$$

  che evidenzia gerarchia e causalità delle chiamate (l'operazione 1.1 è scatenata durante l'esecuzione di 1).

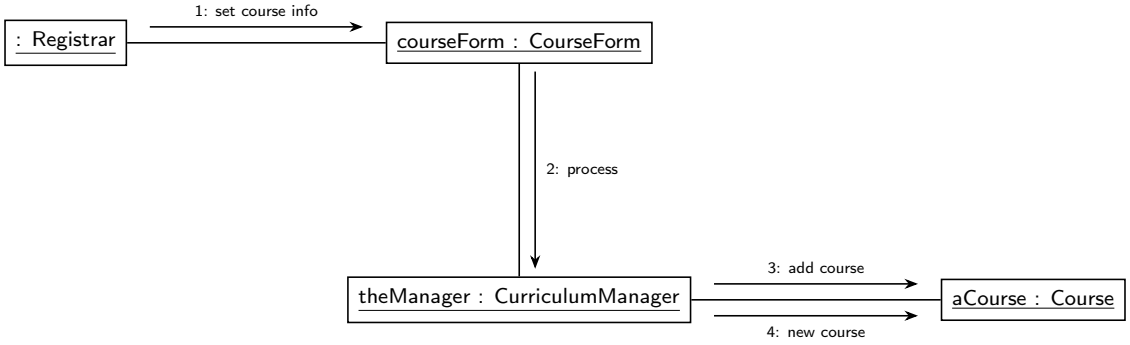


Figura 12: Collaboration Diagram per la registrazione di un corso.

Confronto Finale: Sequence vs Collaboration Diagrams

- **Sequence**: insostituibili quando conta il **flusso del tempo** e l'ordine degli eventi; diventano pesanti all'aumentare degli oggetti.
- **Collaboration**: ideali per valutare l'**architettura**, l'accoppiamento tra classi e i link necessari alla comunicazione; più difficili da leggere per flussi lunghi o molto annidati.

System Design e Scomposizione in Sottosistemi. All'aumentare delle dimensioni del software non è praticabile modellare il sistema come un insieme indifferenziato di centinaia di classi tra loro connesse. L'ingegneria del software adotta il principio latino del **Divide et Impera** (*divide & conquer*), ripartendo la complessità in sottosistemi cooperanti dotati di responsabilità circoscritte:

- **Nei metodi strutturati**: la divisione avviene per **decomposizione funzionale** discendente (*top-down*), scomponendo i processi in sotto-funzioni procedurali.
- **Nel paradigma ad oggetti**: la scomposizione raggruppa insiemi coesi di classi e strutture correlate in unità di livello superiore, denominate **Packages** e **Components**.

Modelli di Interazione tra Sottosistemi. La collaborazione tra sottosistemi si struttura secondo due macro-modelli architetturali:

1. **Modello Client-Server**: una relazione asimmetrica in cui il sottosistema client emette richieste operative (*requests*) subordinate a un contratto di servizio (*contract*) garantito dal sottosistema server.
2. **Modello Peer-to-Peer**: una collaborazione simmetrica e paritetica in cui ciascun sottosistema agisce sia da fruitore sia da erogatore di funzionalità verso l'altro, con contratti bilaterali vincolanti per entrambi gli estremi.



Figura 13: Topologie di interazione architetturale: Client-Server e Peer-to-Peer.

Un tipico esempio di cooperazione distribuita tra sottosistemi è costituito da un'infrastruttura di monitoraggio: il `Sensor Subsystem` invia periodicamente check-in di stato e segnalazioni di allarme al `Central Communication Subsystem`, mentre il `Control Panel Subsystem` interroga la centrale per verificare lo stato delle zone e configurare le tipologie di rilevamento.

I Package in UML e le Dipendenze. Un **Package** in UML rappresenta un meccanismo di raggruppamento generico utilizzato per organizzare gli elementi del modello in blocchi logici gestibili. Nei linguaggi di programmazione esso mappa direttamente sui costrutti nativi di modularizzazione: i `package` in Java e Python, oppure i `namespace` in C++ e C#.

Graficamente, il package viene rappresentato come una cartella con una linguetta superiore sinistra. Tra i package possono intercorrere relazioni di **dipendenza** (*dependency*), tracciate mediante una freccia tratteggiata aperta:

Definizione Formale di Dipendenza tra Package

Si verifica una dipendenza tra due elementi quando **una modifica alla definizione dell'elemento fornitore (*supplier*) può comportare modifiche a cascata nell'elemento cliente (*client*)**. Lo scopo primario del design su larga scala consiste nel **minimizzare le dipendenze** e massimizzare la coesione interna, limitando la propagazione degli effetti collaterali dovuti ai cambiamenti.

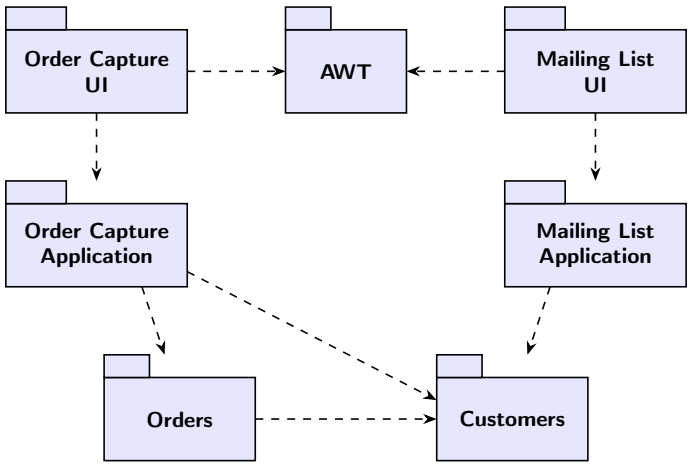


Figura 14: Diagramma dei package per un sistema di evasione ordini e spedizioni.

Package Annidati e Controllo della Visibilità. I package possono essere strutturati gerarchicamente mediante annidamento (*nested packages*), come nel caso di un package `Domain` che racchiude al suo interno i sotto-package `Orders` e `Customers`. Esistono due interpretazioni formali sull'accessibilità degli elementi interni:

- **Trasparente (*transparent*)**: tutti i contenuti e le classi presenti nei sotto-package interni risultano visibili e direttamente accessibili ai package esterni.
- **Opaca (*opaque*)**: i sotto-package rimangono incapsulati all'interno; l'esterno ha visibilità esclusivamente sulle classi dichiarate direttamente a livello del container genitore.

Restrizione delle Interfacce di Package: Il Pattern Facade. Se le classi esterne si accoppiano direttamente a molteplici classi interne di un package, l'accoppiamento globale degrada. La soluzione canonica (GoF) consiste nell'introdurre il design pattern **Facade**:

1. Si impone a tutte le classi interne del package la **visibilità package** (*package-private*, contrassegnata da `~`), rendendole inaccessibili dall'esterno.
2. Si espone un'unica classe pubblica (**Facade**) che definisce l'interfaccia e il comportamento complessivo del modulo verso il mondo esterno.

3. La classe Facade riceve i messaggi esterni e ne delega l'elaborazione interna alle classi opportune all'interno del package.

Regole di Buona Pratica per i Package (Rules of Thumb). La progettazione della struttura dei package segue criteri consolidati:

- **Evitare cicli di dipendenza**: la rete delle dipendenze tra package deve costituire un grafo aciclico orientato (DAG). Se due package dipendono mutuamente l'uno dall'altro, non possono essere compilati né distribuiti indipendentemente.
- **Troppe dipendenze → Refactoring**: un package che importa un numero eccessivo di altri package è sintomo di cattiva ripartizione delle responsabilità; richiede un intervento di refactoring per disaccoppiare la logica.
- **Criterio di leggibilità (Letter-size Rule)**: si ricorre formalmente ai package quando il diagramma delle classi complessivo del sistema diventa talmente esteso da non risultare più comodamente leggibile su un unico foglio standard.

Distinzione Fondamentale: Packages vs Diagrams

- **Packages**: rappresentano **divisioni fisiche e organizzative** dello sviluppo software. Il loro scopo è agevolare la pianificazione industriale del lavoro (*workpackages*) e governare i moduli software da compilare e distribuire.
- **Diagrams**: rappresentano un **assemblaggio logico** di informazioni concettuali. Il loro scopo è puramente cognitivo: facilitare la comprensione del dominio del problema e guidare l'analisi.

I Componenti in UML. Un **Componente** (*Component*) rappresenta una parte fisica, sostituibile e modulare di un sistema software, che incapsula lo stato e il comportamento esponendo un insieme di interfacce contrattuali. A differenza delle classi (entità puramente logiche), il componente coincide con l'unità fisica effettivamente rilasciata e installata presso l'ambiente del cliente finale (ad esempio una libreria compilata `.dll`, un archivio `.jar` o un eseguibile `.exe`).

Notazione Grafica e Interfacce (Lollipop Notation). Nella notazione classica di UML il componente è raffigurato come un rettangolo provvisto di due piccole linguette orizzontali sporgenti dal bordo sinistro. I componenti comunicano tra loro attraverso contratti formali denominati **interfacce**:

- **Interfaccia Fornita (Realization)**: il componente implementa ed espone un insieme di operazioni verso l'esterno. Graficamente è indicata mediante la notazione a lecca-lecca (*lollipop*, linea terminante con un cerchietto vuoto), oppure mediante una freccia di realizzazione tratteggiata verso il box dell'interfaccia.
- **Interfaccia Richiesta (Dependency)**: un altro componente richiede i servizi dichiarati in quell'interfaccia per funzionare, legandosi ad essa tramite una freccia di dipendenza tratteggiata.

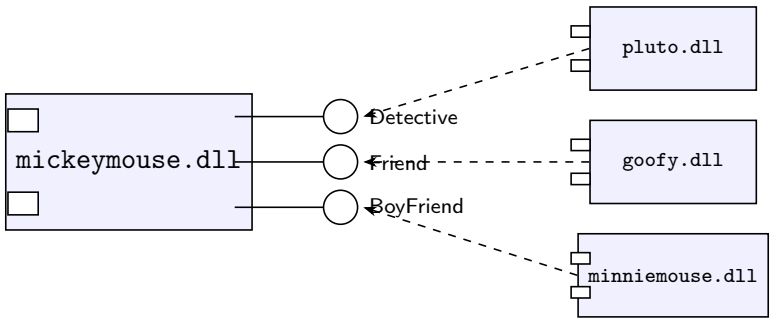


Figura 15: Componenti, interfacce fornite e dipendenze esterne.

Relazione tra Componenti e Classi. È imperativo distinguere l'impacchettamento fisico dei componenti dall'ereditarietà concettuale tra classi:

- Un componente software (ad es. `mickeymouse.dll`) **racchiude e realizza concretamente** al proprio interno molteplici classi logiche (ad es. `Cartoon`, `Character`, `Mouse`) mediante una relazione di manifestazione/dipendenza.
- Ciò è concettualmente differente dal modellare una classe derivata (`MickeyTheMouse`) che eredita le proprietà delle classi base via specializzazione (*is-a*): l'ereditarietà attiene al dominio logico, mentre il componente attiene al rilascio fisico del codice.

I Nodi Computazionali (Nodes). Mentre i componenti rappresentano elementi software eseguiti, il **Nodo** (*Node*) modella la macchina concreta su cui tali componenti risiedono:

Definizione Formale di Nodo (Booch, Rumbaugh, Jacobson)

Un **Nodo** è un elemento fisico presente a run-time che rappresenta una **risorsa computazionale**, dotata tipicamente di memoria e capacità di elaborazione.

Graficamente il nodo viene rappresentato mediante la prospettiva di un **parallelepipedo tridimensionale** (cubo). Può essere identificato attraverso due convenzioni di denominazione:

- **Nome semplice**: indica il singolo elaboratore (es. `kiosk_7`, `sales`).
- **Nome esteso e qualificato**: identifica la gerarchia del dominio e le proprietà di configurazione, come `server::backup {remoteAdministrationOnly}`.

I Deployment Diagrams. Il **Deployment Diagram** mappa la topologia dell'infrastruttura fisica hardware e l'allocazione software a runtime:

- I nodi ospitano al proprio interno i componenti distribuiti mediante lo stereotipo di dipendenza «*deploys*» o per annidamento grafico.
- Le connessioni fisiche di rete tra i nodi sono tracciate mediante linee continue decorate con lo stereotipo del protocollo di comunicazione impiegato (es. «10-T Ethernet», «RS-232»).
- Ciascun nodo può dichiarare vincoli e caratteristiche hardware, come velocità del processore (`processorSpeed = 300 MHz`) e quantitativo di memoria primaria (`memory = 128 MB`).

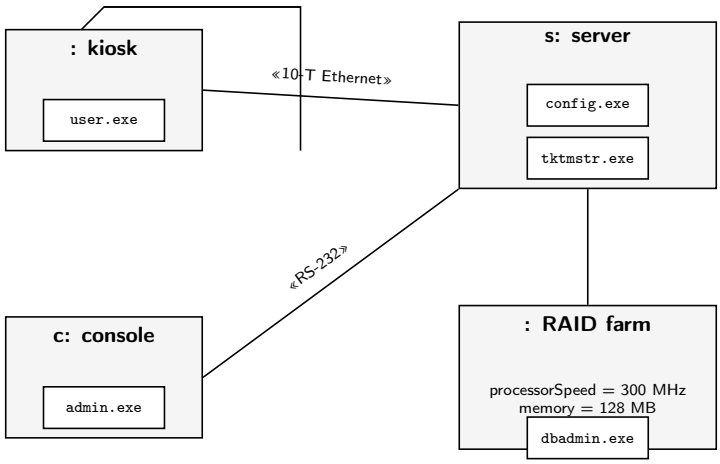


Figura 16: Deployment Diagram: mappatura fisica di nodi, componenti e canali di comunicazione.

Sistemi Real-Time e Modellazione in UML. Nei sistemi in tempo reale (*Real-Time Systems*) la correttezza del software non dipende unicamente dall'esattezza logica dei risultati prodotti, ma anche dall'**istante temporale** in cui tali risultati vengono resi disponibili. Il mancato rispetto di una scadenza temporale (*deadline*) equivale a un malfunzionamento del sistema (critico nei sistemi *hard real-time*, degradante nei sistemi *soft real-time*).
Nell'affrontare la dimensione temporale all'interno di UML si delineano due prospettive complementari:

1. **Costrutti nativi di UML**: elementi e annotazioni predefiniti nello standard per esprimere eventi temporali, scadenze e vincoli di esecuzione all'interno dei diagrammi comportamentali.

2. **Estensioni per il Real-Time (UML-RT)**: impiego dei meccanismi di estensibilità nativi di UML (stereotipi, valori etichettati e vincoli) per importare concetti derivati da metodologie specializzate (come ROOM).

Eventi Temporali e di Variazione (Time and Change Events). Nei sistemi reattivi le transizioni di stato possono essere innescate da stimoli temporali che specificano quando un'azione deve essere eseguita a un determinato istante:

- **Evento di Variazione (`when`)**: specifica una data o un istante orario **assoluto** al verificarsi del quale l'evento viene generato.

```
when (12:00 PM) / goForLunch()
```

- **Evento Temporale Relativo (`after`)**: specifica un intervallo di tempo **relativo** trascorso dall'istante di ingresso nello stato corrente prima che la transizione scatti automaticamente.

```
after (5 minutes) / putTheEggInThePot()
```

Vincoli Temporali nei Diagrammi di Interazione (Timing Constraints). Nei diagrammi di sequenza e di collaborazione è possibile associare vincoli quantitativi ai messaggi scambiati, specificando requisiti di latenza, scadenze o periodicità di campionamento. Per formalizzare questi vincoli UML mette a disposizione tre variabili temporali native:

- `startTime`: istante in cui il messaggio viene inviato o l'esecuzione ha inizio.
- `stopTime`: istante in cui l'elaborazione del messaggio viene completata.
- `executionTime`: durata complessiva dell'operazione, definita implicitamente dalla relazione:

$$\text{executionTime} = \text{stopTime} - \text{startTime}$$

Accanto alle variabili di esecuzione, il costrutto formale `every` identifica messaggi **periodici** che devono essere re-inviati a intervalli regolari prefissati (ad esempio routine di polling ciclico o battiti cardiaci di keep-alive).

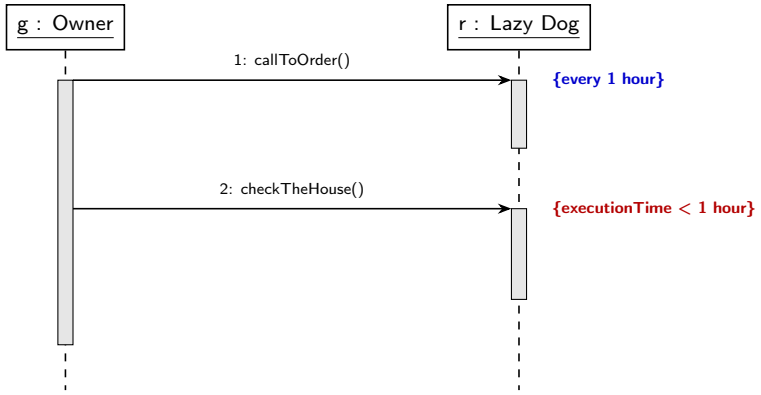


Figura 17: Sequence Diagram con vincoli temporali di periodicità (`every`) e di durata massima.

Il Divario tra Notazione ed Esecuzione: Mancanza di Enforcement

Nonostante UML fornisca costrutti espressivi per formalizzare requisiti real-time:

- **Nessun compilatore o tool ordinario garantisce l'enforcement**: né gli ambienti di programmazione commerciali né i tool CASE storici (come Rational Rose 2000) applicano o verificano automaticamente tali vincoli a runtime.
- **Ruolo di specifica**: i vincoli operano primariamente come documentazione contrattuale e guida per l'analisi di schedulabilità (es. RMA – *Rate Monotonic Analysis*), ma l'effettiva garanzia temporale deve essere realizzata a valle mediante sistemi operativi real-time (RTOS) dedicati e architetture deterministiche.

L'Object Constraint Language (OCL). L'**OCL** è il linguaggio di espressioni di UML. Ha tre proprietà caratteristiche:

- è un linguaggio **puro**: le espressioni non hanno effetti collaterali;
- è un linguaggio di **modellazione**: non è eseguibile;
- è un linguaggio **formale**, con semantica ben definita.

Il suo scopo è definire in modo chiaro e non ambiguo le **guardie** e, più in generale, i vincoli dei diagrammi UML. Esempio di riferimento: classe `Dog` (attributi `weight, age : Real`; operazioni `eat(f : Real)`, `bark()`) associata a `Person` con ruoli `owner` (lato `Person`) e `pet` (lato `Dog`, molteplicità $*$); istanze `Joe` e `Fido`.

Caratteristiche Principali di OCL.

1. **Sintassi degli attributi** come in Java, con `self` al posto di `this`: `self.weight < 100`.
2. **Tipizzazione forte**: la conformità dei tipi viene controllata; ad esempio `self.weight < "abc"` è errata.
3. **Pre- e post-condizioni** associate alle operazioni:

   `Dog::bark()` pre: `neighborhood is happy` post: `neighborhood is angry`
   `Dog::eat(f : Real)` post: `weight = weight@pre + f`

   dove `@pre` indica il valore all'ingresso nell'operazione. Le slide usano `:=` per l'assegnazione; in OCL standard la post-condizione è un'uguaglianza.
4. **Associazioni**: quelle di molteplicità 1 sono trattate come attributi (`Fido.owner = Joe`); quelle di molteplicità diversa da 1 come **insiemi** (i `pet` di `Joe` sono un insieme di cani), con operazioni ben definite. Gli operatori `.` e `->` possono essere usati in modo polimorfo.

Tipi e Collezioni in OCL. Il sistema dei tipi comprende enumerazioni, stringhe, booleani, `Real` e `Integer` (con `Integer` sottotipo di `Real`), tutti sotto `OclAny`, più `OclType` e il tipo `Collection`, che si specializza in `Set` (senza duplicati, non ordinato), `Bag` (con duplicati) e `Sequence` (ordinata). Le operazioni principali sulle collezioni sono:

`size, includes, count, includesAll, isEmpty, notEmpty, sum, exists, forAll, iterate, select, reject, collect`

Introduzione al Refactoring. Chiave di lettura: i sistemi software sono entità in evoluzione che richiedono continui aggiornamenti e *housekeeping*; servono modi per renderli semplici (cioè "non orribilmente difficili") da eseguire. L'approccio è lo sviluppo incrementale con cicli di vita brevi e refactoring costante (fonte: tutorial di Don Roberts).

Il **refactoring dei diagrammi UML** consiste nel rivedere il sistema e ristrutturarne il design e quello dei suoi componenti, puntando a maggiore semplicità e maggiore aderenza alla struttura del dominio, **mantenendone il comportamento**.

Tecniche di Refactoring. Le operazioni elementari si applicano a quattro tipi di elementi (celle non applicabili in grigio nelle slide):

	Class	**(Static) data**	**(Static) function**	**Temp. data**
Add	✓	✓	✓	✓
Delete	✓	✓	✓	✓
Rename	✓	✓	✓	✓
Push Up/Down	✓	✓	✓	–
Break Down / Condense	✓	–	✓	–
Re-qualify	–	–	✓	–

Criteri e Inibitori del Refactoring.

- **Quando**: rifattorizzare quando il programma mostra carenze, non per "miglioramento concettuale".
- Scrivere il codice **una sola volta** (analogo della relazione «include» nei casi d'uso).
- Eliminare le entità troppo grandi (metodi, classi, package/namespace).
- **Inibitori** ("Inhibitor$$"): "il sistema funziona" e non ci sono tempo o soldi; oppure si lavora sulle astrazioni di progetto di qualcun altro.

3.3 Design Patterns

Origini e Fondamenti dei Design Pattern. Nello sviluppo di sistemi orientati agli oggetti emergono frequentemente problemi di progettazione ricorrenti. Anziché riscoprire ogni volta soluzioni *ad-hoc*, l'ingegneria del software ricorre ai **Design Patterns**, formalizzati storicamente da Erich Gamma, Richard Helm, Ralph Johnson e John Vlissides (la celebre *Gang of Four* o GoF):

> *«I Design Pattern sono descrizioni di classi e oggetti comunicanti, personalizzate per risolvere un problema di progettazione generale all'interno di un contesto particolare.»* — E. Gamma

Il concetto affonda le proprie radici teoriche nell'architettura civile ed edilizia, in particolare nelle opere dell'architetto Christopher Alexander (1977):

> *«Un pattern descrive un problema che si manifesta ripetutamente nel nostro ambiente, e descrive il nocciolo della soluzione a tale problema in modo tale che sia possibile impiegare questa soluzione un milione di volte senza mai farlo due volte nello stesso identico modo.»* — C. Alexander

I Pattern nello Sviluppo Software: Il Riuso Orientato agli Oggetti. Trasponendo questa visione all'ingegneria del software orientata agli oggetti:

- I **mattoni** (*bricks*) elementari sono rappresentati dai **design objects** (classi e istanze);
- I **pattern** costituiscono le **configurazioni collaudate** di tali mattoni, formalizzando le relazioni reciproche tra gli oggetti (ereditarietà, associazione, aggregazione, composizione, delega e creazione);
- L'orientamento agli oggetti offre un supporto espressivo ideale per formulare i pattern;
- I design pattern offrono un potenziale straordinario come unità ottimali per il **riuso di architetture e strategie**, superando i limiti del banale riuso di frammenti di codice sorgente.

I Quattro Elementi Costitutivi di un Design Pattern

Ogni pattern GoF è rigorosamente descritto da quattro elementi cardine:

1. **Nome (*Pattern Name*)**: sintesi di una o due parole che crea un vocabolario architetturale condiviso tra progettisti e sviluppatori;

2. **Problema (*The Problem*)**: descrive il contesto operativo, i vincoli e le condizioni in cui ha senso applicare il pattern;

3. **Soluzione (*The Solution*)**: specifica gli elementi astratti di design (classi, ruoli, responsabilità e collaborazioni), senza vincolarsi a una specifica libreria o linguaggio di programmazione;

4. **Conseguenze (*The Consequences*)**: i compromessi tecnici (*trade-offs*), i costi operativi (spazio, tempo, complessità) e i benefici tangibili (flessibilità, estendibilità) derivanti dalla sua adozione.

La Tassonomia della Gang of Four. I 23 pattern classici GoF vengono suddivisi in tre macrofamiglie ortogonali in base al loro intento primario:

- **Pattern Creazionali (*Creational*)**: governano l'istanziazione degli oggetti, rendendo il sistema indipendente da come gli oggetti vengono creati, composti e rappresentati;
- **Pattern Strutturali (*Structural*)**: governano la composizione di classi e oggetti per realizzare strutture logiche più ampie e flessibili;
- **Pattern Comportamentali (*Behavioral*)**: governano le interazioni tra oggetti, l'assegnazione delle responsabilità e i flussi di controllo dinamici.

Scenario Guida: Il Sistema Contabile Municipale (*Township Accounting*). Per motivare empiricamente l'introduzione dei pattern, si considera la progettazione del sistema di bilancio e gestione contabile di un piccolo comune (*township accounting system*):

- Il comune dispone di un bilancio complessivo (`Budget`) composto da svariati conti di spesa ed entrata (`Account`);
- Il sistema deve poter ricavare informazioni contabili aggregate calcolate sui singoli conti;
- L'attenzione progettuale è focalizzata sulla **creazione e analisi della struttura**, mantenendola stabile rispetto alle modifiche computazionali.

I Tre Requisiti di Progetto dello Scenario

1. **Unicità del Bilancio**: il bilancio comunale deve essere **unico** per l'intera municipalità; non possono esistere istanze multiple concorrenti di `Budget`;

2. **Gerarchia dei Conti**: i conti possono essere aggiunti o rimossi; ciascun conto può essere **monolitico** (atomico) oppure **composto da altri sotto-conti**;

3. **Scansione Sequenziale**: deve essere possibile scandire sequenzialmente tutti i conti presenti all'interno del bilancio senza dipendere dalla struttura dati interna di contenimento.

Requisito 2: Struttura dei Conti → Introduzione al Pattern Composite. Il secondo requisito richiede che il bilancio possa contenere conti singoli (es. capitolo di spesa elementare) e conti aggregati (es. bilancio delle opere pubbliche, suddiviso in manutenzione strade, scuole, illuminazione).

- L'anti-pattern consisterebbe nel trattare le due tipologie in modo asimmetrico con rami condizionali (`if-else`);
- La soluzione ideale prevede un'astrazione comune `Account` che dichiara le operazioni uniformi (`GlobalOperation()`, `AtomicOperation()`, `Add()`, `Remove()`);
- `SimpleAccount` (foglia) esegue l'operazione sul conto elementare;
- `CompositeAccount` (nodo composito) mantiene una collezione di riferimenti polimorfici `children : Account[*]` e delega ricorsivamente l'esecuzione a tutti i suoi discendenti:

```
forall g in children:  g.GlobalOperation();
```

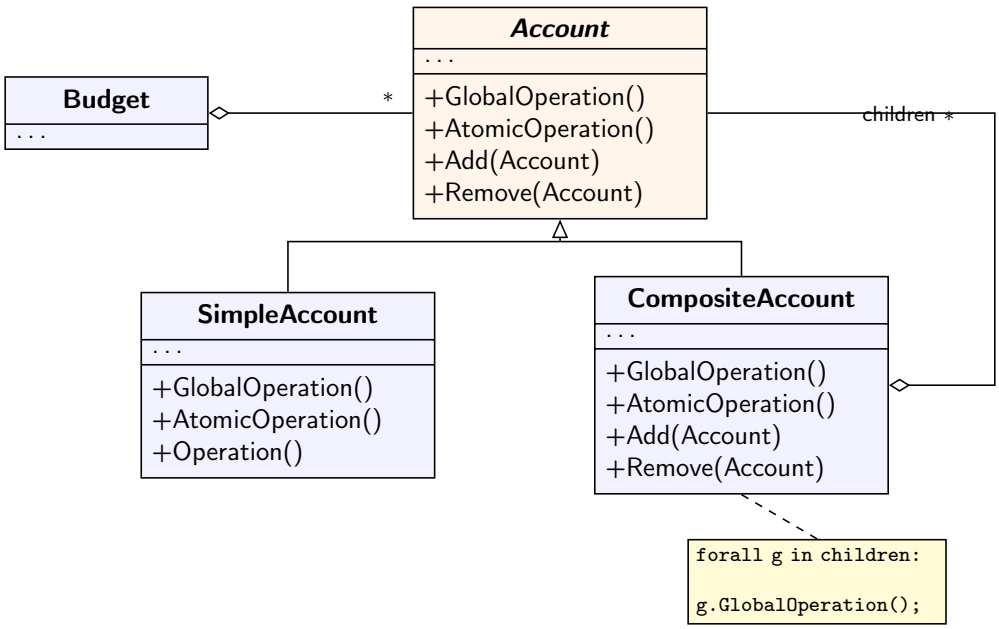


Figura 18: Modellazione dei conti del bilancio mediante il pattern Composite.

Requisito 3: Scansione dei Conti → Introduzione al Pattern Iterator. Per esaminare tutti i conti registrati senza costringere il client `Budget` a conoscere se la struttura interna sia un array, una lista o una mappa:

- La classe `ListOfAccounts` implementa il metodo factory `CreateIterator()` restituendo una nuova istanza di `AccountsIterator`;
- L'interfaccia astratta `Iterator` espone le quattro primitive canoniche di navigazione: `First()`, `Next()`, `IsDone()` e `CurrentItem()`;
- La logica di scorrimento e lo stato del cursore sono completamente separati dalla struttura aggregata di memorizzazione dei dati.

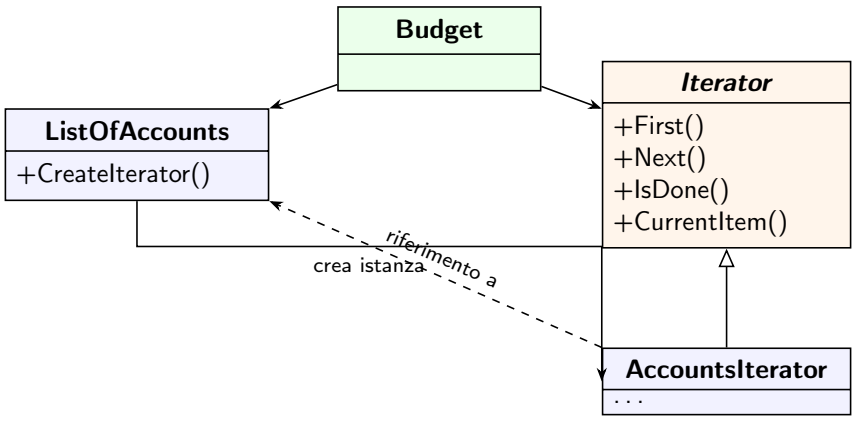


Figura 19: Disaccoppiamento della scansione dei conti mediante il pattern Iterator.

La Famiglia dei Pattern Creazionali (*Creational Patterns*). I pattern creazionali astraggono il meccanismo di istanziazione del software:

- Rendono il sistema indipendente da come i suoi oggetti vengono concretamente creati, composti e rappresentati;
- Incapsulano la conoscenza delle classi concrete, celandone l'implementazione dietro interfacce astratte;
- Sostituiscono la creazione rigida via costruttore esplicito con strategie modulari e dinamiche.

Le slide coprono sistematicamente i cinque pattern creazionali classici della GoF: **Singleton**, **Builder**, **Abstract Factory**, **Factory Method** e **Prototype**.

3.3.1 Singleton Pattern

Intento e Struttura Formale. Garantisce che una classe possieda un'unica istanza accessibile globalmente all'interno dell'intero ciclo di vita dell'applicazione.

- **Costruttore privato**: preclude categoricamente l'istanziazione libera dall'esterno;
- **Istanza statica unica**: conservata come campo privato di classe (`uniqueInstance`);
- **Metodo di accesso statico**: fornisce la *lazy initialization* (istanziazione differita alla prima richiesta effettiva) e restituisce il riferimento condiviso.

```
class Singleton {
    // Riferimento statico privato all'unica istanza
    private static Singleton theInstance = null;

    // Costruttore privato
    private Singleton() {}

    // Metodo factory statico
    public static Singleton getInstance() {
        if (theInstance == null) {
```

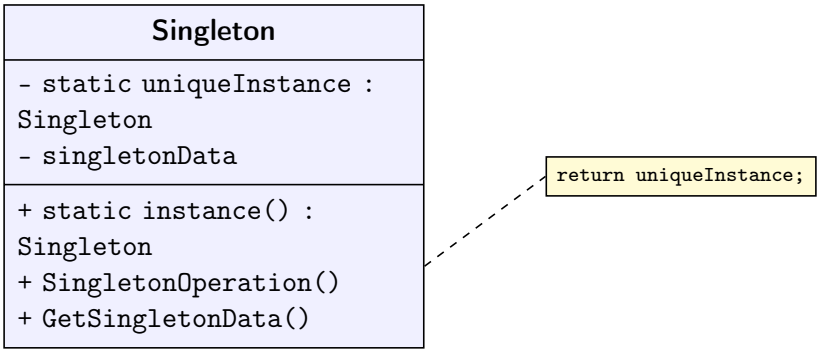


Figura 20: Diagramma delle classi del pattern Singleton con metodo di accesso statico.

```
11              theInstance = new Singleton();
12          }
13          return theInstance;
14      }
15  }
```

3.3.2 Builder Pattern

Intento e Ruolo del Director. Il pattern Builder separa la **costruzione di un oggetto complesso** dalla sua **rappresentazione**, consentendo allo stesso processo di costruzione di generare rappresentazioni differenti.

- Il processo di costruzione è delegato a un'entità di coordinamento detta **Director**;
- Il Director governa la sequenza logica dei passi necessari ad assemblare il prodotto, ma demanda la realizzazione fisica dei singoli componenti a un'interfaccia astratta **Builder**;
- I **ConcreteBuilder** realizzano le parti specifiche assemblando il prodotto finale (`Product`);
- Garantisce un **controllo molto più fine e modulare** sul processo di assemblaggio rispetto a una fabbrica monolitica.

3.3.3 Abstract Factory Pattern

Intento e Famiglie di Prodotti. Fornisce un'interfaccia per creare **famiglie di oggetti correlati o dipendenti tra loro** senza specificare le rispettive classi concrete.

- **Famiglie di prodotti**: si applica quando il sistema è configurato per supportare insiemi molteplici di componenti coerenti (ad esempio le famiglie $A \dots Z$, ciascuna dotata dei componenti $1 \dots n$);
- **Consistenza garantita**: impedisce l'errore accidentale di combinare elementi appartenenti a famiglie discordanti (es. finestre grafiche in stile Motif combinate con pulsanti Windows);
- **Scambio immediato**: l'intera famiglia di prodotti può essere sostituita a runtime istanziando una fabbrica concreta differente.

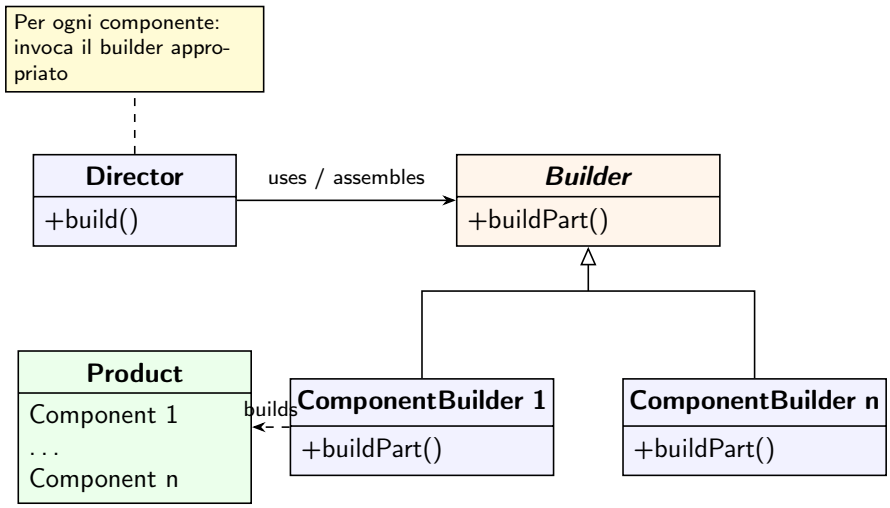


Figura 21: Diagramma di classe formale del pattern Builder.

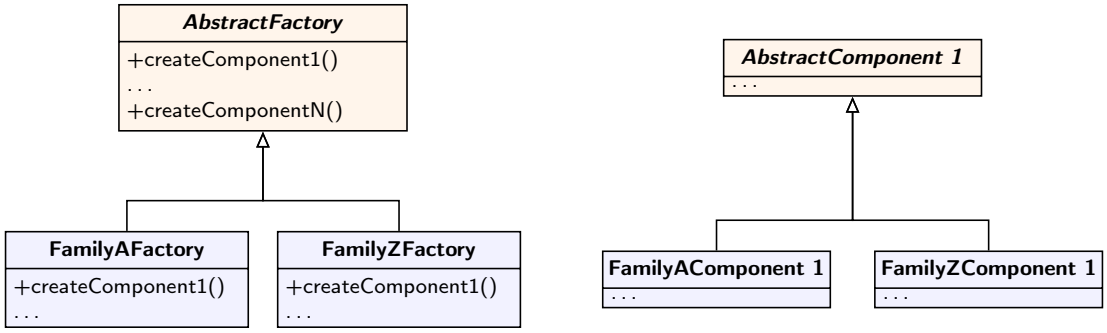


Figura 22: Architettura a famiglie e componenti del pattern Abstract Factory.

```
// Configurazione del client con una specifica famiglia:
AbstractFactory f;
AbstractComponent1 c1;
AbstractComponent2 c2;

// Prelievo del componente 1 appartenente alla Famiglia A:
f = new FamilyAFactory();
c1 = f.createComponent1();

// Passaggio trasparente alla Famiglia C:
f = new FamilyCFactory();
c2 = f.createComponent2();
```

Limitazione Fondamentale dell'Abstract Factory

Sebbene l'Abstract Factory renda banale scambiare famiglie preesistenti, presenta una marcata rigidità evolutiva: **è estremamente difficile supportare nuovi tipi di prodotto all'interno delle famiglie**. Introdurre un nuovo `ComponentN+1` impone infatti di modificare l'interfaccia dell'`AbstractFactory` e aggiornare forzatamente tutte le classi concrete derivate (`FamilyAFactory`, `FamilyBFactory`, ...).

3.3.4 Factory Method Pattern

Intento e Meccanismo di Delega. Definisce un'interfaccia per creare un oggetto, lasciando che siano le **sottoclassi a decidere quale classe concreta istanziare**. Il Factory Method consente a una classe di posticipare l'istanziazione alle proprie sottoclassi:

- **Decisione a runtime**: ideale quando le informazioni necessarie per identificare la classe corretta sono note solo durante l'esecuzione;
- **Collegamento tra gerarchie parallele**: connette una gerarchia di creatori (`Creator`) a una gerarchia parallela di prodotti (`Product`);
- Elimina la necessità di legare il codice di alto livello a classi implementative specifiche.

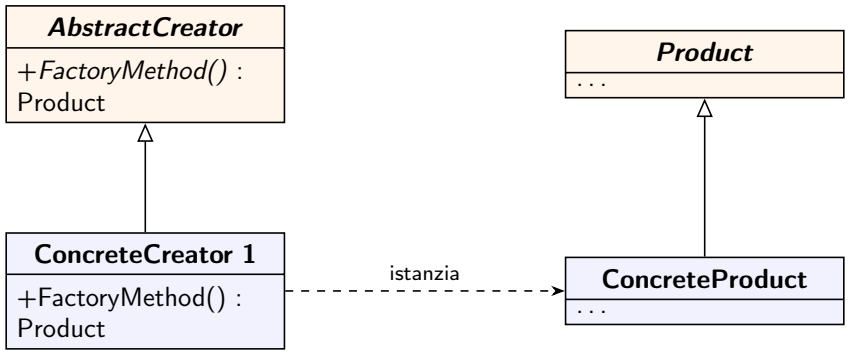


Figura 23: Factory Method: delega dell'istanziazione alle sottoclassi.

```
// Il client invoca il Factory Method senza conoscere la classe concreta
   istanziata:
Product x = ConcreteCreator1.FactoryMethod();
Product y = ConcreteCreator2.FactoryMethod();
```

3.3.5 Prototype Pattern

Intento e Clonazione di Istanze. Specifica i tipi di oggetti da creare impiegando un'**istanza prototipale come modello**, generando nuovi oggetti tramite la **copia/clonazione** di tale prototipo.

- **Evita gerarchie parallele**: scongiura il proliferare incontrollato di fabbriche e creatori tipico del Factory Method;
- **Dinamismo a runtime**: consente di aggiungere, configurare o rimuovere famiglie di oggetti a runtime semplicemente clonando istanze opportunamente pre-configurate;
- Riduce sensibilmente il numero complessivo di sottoclassi necessarie nell'architettura.

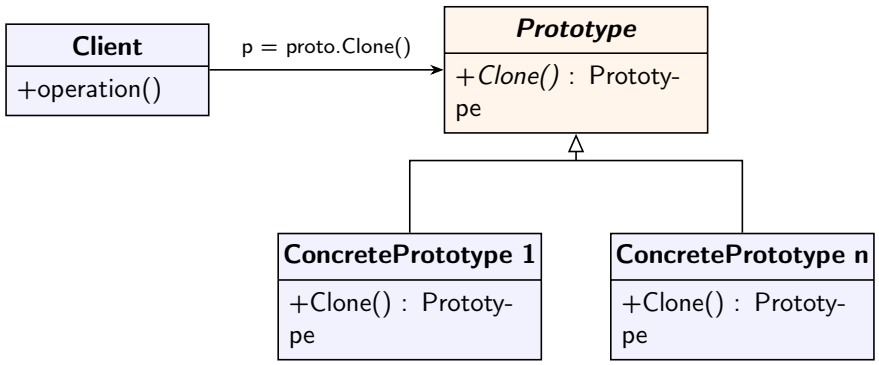


Figura 24: Diagramma delle classi del pattern Prototype con metodo `Clone()`.

Sintesi Comparativa dei Pattern Creazionali

Pattern	Meccanismo Primario	Finalità e Vantaggio Chiave
Singleton	Incapsulamento statico e costruttore privato	Garantisce l'esistenza di un'unica istanza con punto di accesso globale controllato.
Builder	Delega a interfaccia di assemblaggio passo-passo	Separa l'algoritmo di costruzione dalla rappresentazione finale; controllo fine e granulare.
Abstract Factory	Composizione di oggetti interfaccia	Crea famiglie complete di prodotti coerenti senza legarsi a classi concrete; scambio rapido di famiglie.
Factory Method	Ereditarietà e override di metodo virtuale	Demanda la decisione di istanziazione alle sottoclassi; connette gerarchie parallele.
Prototype	Clonazione dinamica di oggetti pre-configurati	Crea nuovi oggetti duplicando prototipi; evita l'esplosione di sottoclassi e gerarchie di factory.

La Famiglia dei Pattern Strutturali (*Structural Patterns*). I pattern strutturali governano le modalità con cui classi e oggetti vengono composti per formare strutture logiche e architetturali più complesse e flessibili:

- **Pattern strutturali delle classi (*Structural Class Patterns*)**: impiegano l'**ereditarietà** per comporre interfacce o implementazioni multiple;
- **Pattern strutturali degli oggetti (*Structural Object Patterns*)**: impiegano la **composizione e l'aggregazione di oggetti** a runtime per realizzare nuove funzionalità, garantendo maggiore dinamismo e minore accoppiamento.

I pattern strutturali analizzati nel percorso comprendono: **Adapter**, **Bridge**, **Decorator** e **Proxy** (oltre a richiamare **Composite**, già introdotto nello scenario del bilancio).

3.3.6 Adapter Pattern

Intento e Risoluzione dei Disallineamenti. Il pattern **Adapter** converte l'interfaccia di una classe in un'altra interfaccia attesa dai client, permettendo la cooperazione tra classi che altrimenti non potrebbero interagire a causa di interfacce incompatibili:

- **Scopo primario**: colmare disallineamenti di denominazione (*naming mismatches*) o di segnatura dei metodi tra componenti preesistenti e nuove architetture;
- **Separazione tra interfaccia e implementazione**: consente a una classe client orientata all'interfaccia astratta `Target` di dialogare con una classe fornitrice esistente (`Adaptee`), la quale espone metodi con nomi o parametri differenti (`SpecificRequest()`).

Variante 2: Object Adapter (con Delega e Composizione). Nei contesti a ereditarietà singola (come Java o C#), o per consentire all'adapter di funzionare con tutte le sottoclassi di `Adaptee`:

- La classe `Adapter` estende o realizza l'interfaccia `Target` e mantiene al proprio interno un riferimento o puntatore a un'istanza concreta di `Adaptee`;
- L'invocazione di `Request()` si traduce nella delega esplicita `adaptee->SpecificRequest()`.

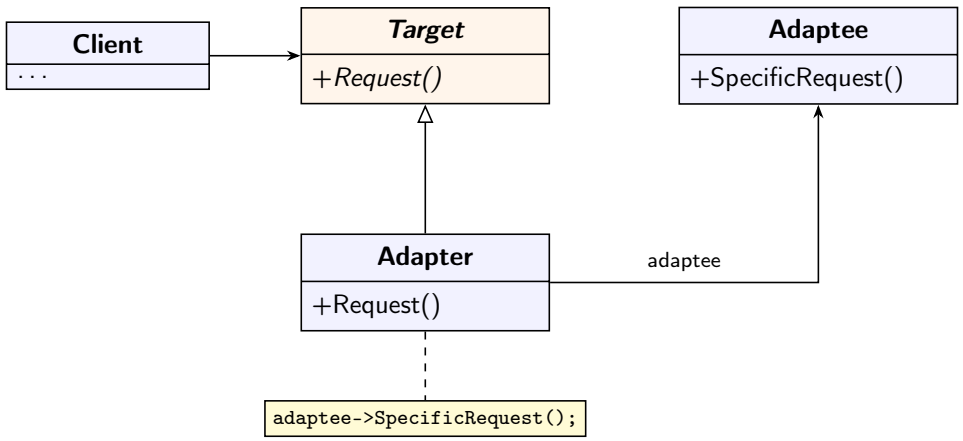


Figura 25: Object Adapter: disaccoppiamento mediante composizione d'oggetto e delega.

3.3.7 Bridge Pattern

Intento e Disaccoppiamento. Disaccoppia un'astrazione dalla sua implementazione, in modo tale che le due dimensioni possano **variare in modo completamente indipendente**:

- **Scopo primario**: evitare un legame permanente tra astrazione e implementazione concreta, scongiurando l'esplosione combinatoria delle sottoclassi;
- **Differenza concettuale con l'Adapter**:
  - L'**Adapter** interviene *a posteriori* per risolvere discrepanze tra interfacce già esistenti non progettate per lavorare insieme;

- Il **Bridge** viene introdotto *a priori* durante la progettazione per far evolvere autonomamente l'albero dei concetti di dominio rispetto all'albero delle piattaforme tecniche sottostanti.

- **Ruolo nei linguaggi moderni**: data la presenza nativa del costrutto `interface` in linguaggi come Java, il Bridge risulta meno critico a livello di pura codifica rispetto al C++, ma conserva un'importanza architetturale fondamentale.

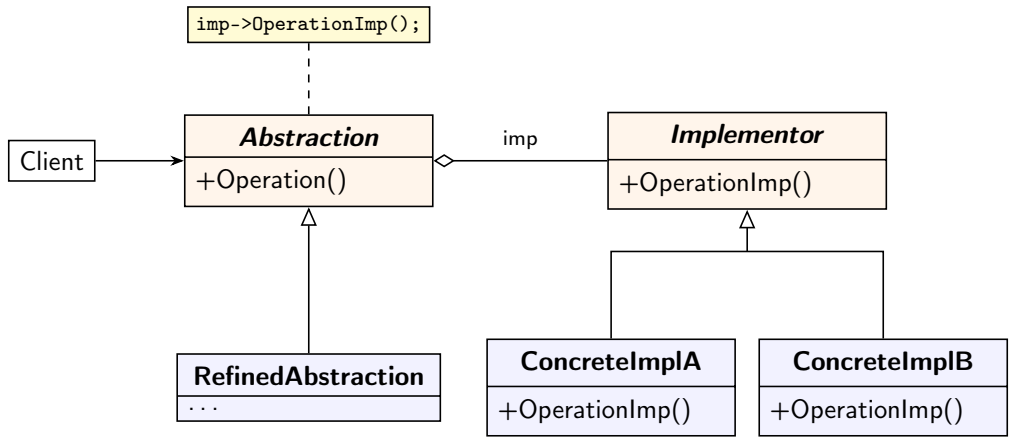


Figura 26: Diagramma delle classi del pattern Bridge: separazione tra gerarchia di astrazione e implementazione.

3.3.8 Decorator Pattern

Intento e Confronto con il Composite. Il pattern **Decorator** consente di allegare dinamicamente responsabilità e funzionalità aggiuntive a un singolo oggetto a runtime, senza alterarne l'interfaccia:

- **Alternativa flessibile al subclassing**: estendere le funzionalità tramite ereditarietà statica produce una proliferazione ingestibile di sottoclassi per ogni combinazione di feature; il Decorator compone invece le funzionalità a runtime;

- **Affinità con il Composite**: la struttura del Decorator è analoga a quella del Composite, con una differenza fondamentale:

  - Nel **Composite**, un nodo aggregatore racchiude un insieme arbitrario ($*$) di figli per formare strutture ad albero;

  - Nel **Decorator**, il componente decoratore racchiude **esattamente un solo componente** (1) alla volta, aggiungendo una specifica caratteristica trasparente prima o dopo aver delegato l'operazione base;

- **Identità non trasparente**: dal punto di vista del codice il decoratore e il componente interno non sono identici; il decoratore funge da guscio (*wrapper*) trasparente all'esterno.

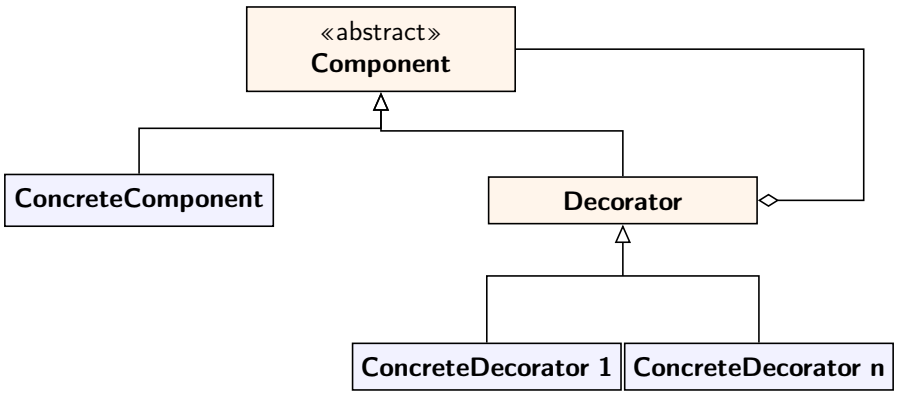


Figura 27: Diagramma strutturale del pattern Decorator.

3.3.9 Proxy Pattern

Intento e Tipologie di Impiego. Fornisce un surrogato o un oggetto segnaposto (*placeholder*) per controllare l'accesso a un altro oggetto:

- **Meccanismo operativo**: l'effettiva implementazione e la risorsa concreta risiedono nell'oggetto reale (`RealEntity`); il `Proxy` intercetta le richieste esponendo la medesima interfaccia e governando l'inoltro;
- **Casi d'uso canonici**:
  1. **Remote Proxy**: gestisce la comunicazione di rete codificando e decodificando parametri verso un oggetto allocato in un diverso spazio di indirizzamento o server remoto;
  2. **Virtual Proxy**: ritarda l'istanziazione costosa di oggetti pesanti (*lazy loading*), ad esempio caricando un'immagine ad alta risoluzione solo quando effettivamente visualizzata sullo schermo;
  3. **Protection Proxy**: verifica i diritti di accesso e i privilegi di sicurezza prima di inoltrare l'operazione all'oggetto protetto.

La Famiglia dei Pattern Comportamentali (*Behavioral Patterns*). I pattern comportamentali si concentrano sugli **algoritmi** e sulla corretta **ripartizione delle responsabilità** tra oggetti:

- Non descrivono semplicemente configurazioni statiche di classi o dati, ma formalizzano schemi complessi di **comunicazione e coordinamento** tra oggetti a runtime;
- Aiutano a padroneggiare flussi di controllo intricati che risulterebbero altrimenti opachi e difficili da tracciare durante l'esecuzione del sistema;
- I pattern comportamentali approfonditi nelle lezioni includono: **Visitor**, **Strategy**, **Chain of Responsibility** e **Mediator**.

3.3.10 Visitor Pattern

Intento e Principio del Double Dispatching. Rappresenta un'operazione da eseguire sugli elementi di una struttura complessa di oggetti, consentendo di **definire nuove operazioni senza modificare le classi degli elementi** su cui opera:

- **Separazione tra dati e algoritmi**: la struttura dati degli elementi (`Element`, `Leaf`, `Composite`) rimane stabile nel tempo; i molteplici algoritmi applicabili a tale struttura vengono incapsulati all'interno di classi esterne specializzate (`Visitor`);
- **Controllo delegato**: è l'oggetto visitato a determinare quale compito eseguire attraverso il meccanismo del *double dispatching*. L'elemento espone il metodo `accept(Visitor v)`, il quale invoca a sua volta l'operazione opportuna sul visitatore (`v.visitX(this)`);
- **Navigazione ricorsiva**: nel caso di elementi compositi, `accept` provvede a propagare la visita a tutti i sotto-elementi contenuti:

```
for each contained object o:  o.accept(v);
```

- **Compromesso (Conseguenza negativa)**: per poter svolgere operazioni complesse dall'esterno, il Visitor può imporre una **rottura dell'incapsulamento** (*break in encapsulation*), costringendo gli elementi a esporre parti del proprio stato interno.

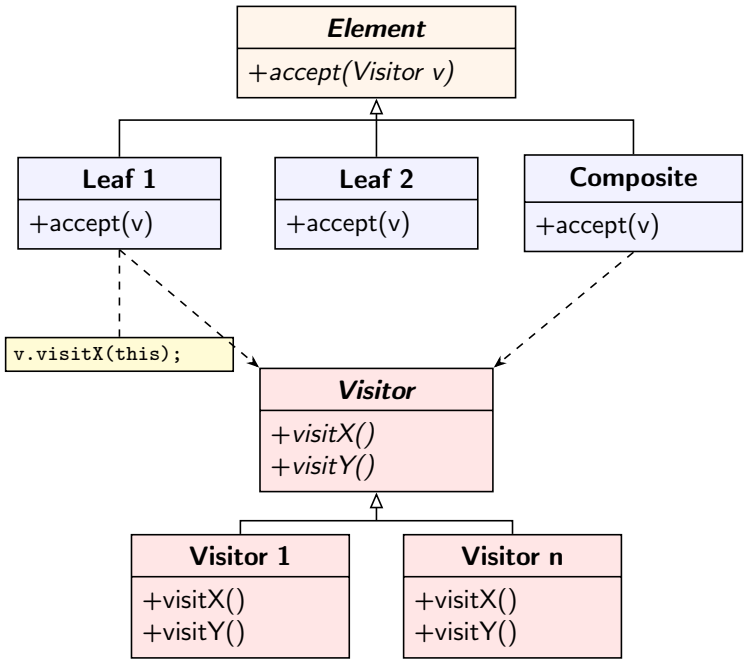


Figura 28: Diagramma del pattern Visitor: separazione degli algoritmi di visita dalla struttura ad albero.

3.3.11 Strategy Pattern

Intento e Parametrizzazione a Runtime. Definisce una famiglia di algoritmi, incapsula ciascuno di essi e li rende intercambiabili a runtime. La **Strategy** permette all'algoritmo di variare in modo indipendente dai client che lo impiegano:

- **Parametrizzazione comportamentale**: consente a un oggetto di mutare la propria logica esecutiva dinamicamente semplicemente sostituendo l'istanza della strategia associata;

- **Eliminazione di rami condizionali**: sradica la presenza di blocchi `switch` o lunghe sequenze di `if-else` sparsi nel codice, racchiudendo ciascun caso logico in una specifica classe concreta;
- **Vantaggi e conseguenze**:
  - Consente la libera scelta dell'implementazione tecnica (es. ordinamento rapido vs ordinamento conservativo);
  - Riduce sensibilmente il numero di sottoclassi di dominio necessarie;
  - Produce, come compromesso architetturale, un **incremento del numero complessivo di oggetti** istanziati nel sistema.

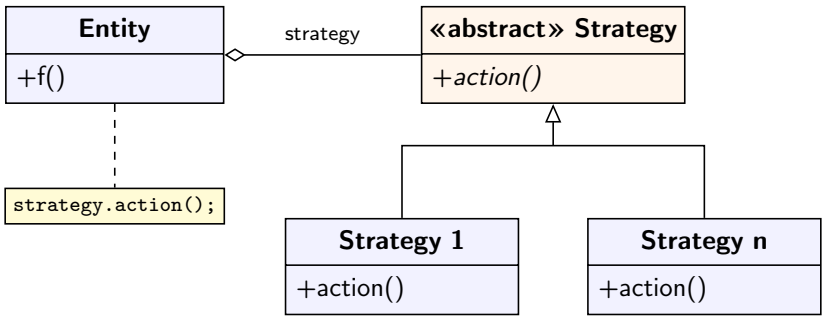


Figura 29: Diagramma del pattern Strategy: incapsulamento ed intercambiabilità degli algoritmi.

3.3.12 Chain of Responsibility Pattern

Intento e Inoltro Dinamico della Richiesta. Consente di disaccoppiare il mittente di una richiesta dai suoi potenziali destinatari, offrendo a più oggetti la possibilità di gestire la richiesta stessa. Gli oggetti destinatari vengono concatenati in una catena lungo la quale la richiesta viene inoltrata finché uno di essi non la prende in carico:

- **Ignoranza del gestore finale**: si adotta tipicamente quando una richiesta deve essere servita da uno tra molteplici componenti concorrenti, ma il sistema non può determinare a priori quale specifico oggetto gestirà l'evento;
- **Disaccoppiamento totale**: il mittente non mantiene puntatori espliciti a tutti i possibili destinatari, ma dialoga esclusivamente con il primo anello della catena;
- **Ricezione non garantita**: poiché non vi è un destinatario vincolato a monte, la richiesta **può cadere a vuoto** qualora attraversi l'intera catena senza che alcun componente se ne faccia carico.

3.3.13 Mediator Pattern

Intento e Controllo Centralizzato. Definisce un oggetto che incapsula e controlla il modo in cui un insieme di oggetti interagisce, prevenendo collegamenti diretti e disordinati (*spaghetti code*) tra componenti cooperanti:

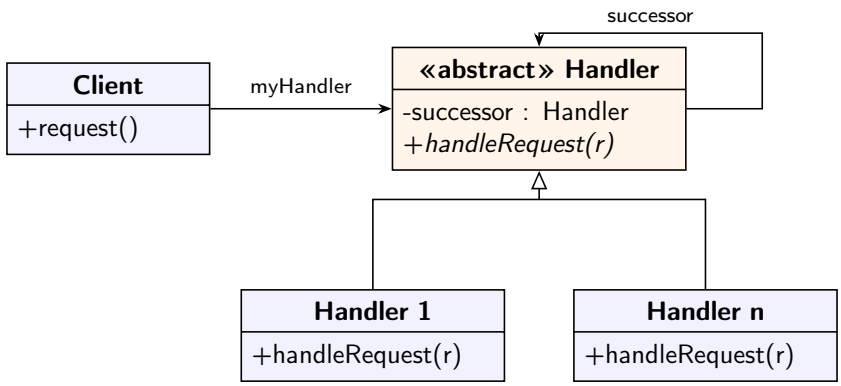


Figura 30: Diagramma del pattern Chain of Responsibility con catena ricorsiva di successori.

- **Disaccoppiamento delle interazioni**: ideale in presenza di reti complesse e fitte di dipendenze incrociate; evita che i singoli oggetti debbano conoscersi o referenziarsi esplicitamente a vicenda;
- **La Metafora della Lavagna (*Blackboard*)**: il concetto di Mediator agisce in modo del tutto analogo a una lavagna condivisa, utilizzata da molteplici soggetti per scambiare informazioni e coordinare le azioni attraverso un controllo centralizzato;
- **Insieme fisso di primitive**: il Mediator formalizza un vocabolario fisso e concordato di primitive operative; ciascun partecipante (*colleague*) deve unicamente conoscere tali primitive per poter interagire all'interno del sistema comune.

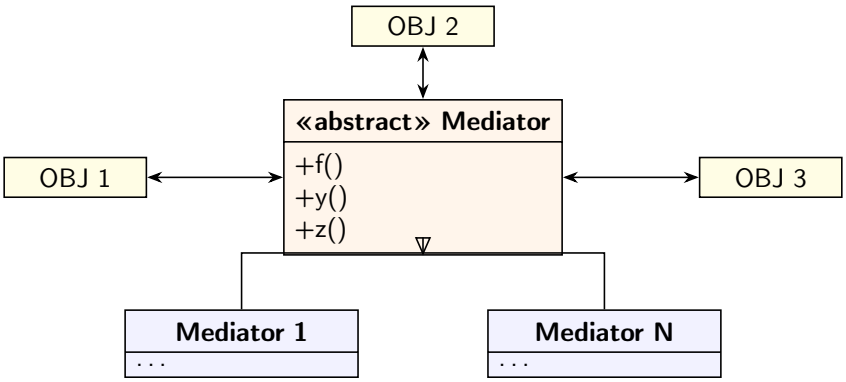


Figura 31: Diagramma del pattern Mediator: accentramento delle relazioni tra colleghi.

4 UML: esercitazioni

4.1 La Notazione delle Classi (Class Icon)

In UML una classe viene rappresentata graficamente mediante un rettangolo suddiviso orizzontalmente fino a tre scomparti distinti (*slot*):

1. **Nome della classe ed eventuale stereotipo**: costituisce l'unico elemento strettamente obbligatorio e va scritto per convenzione in `UpperCamelCase` (es. `Person`, `BankAccount`). Può essere preceduto da uno stereotipo racchiuso tra doppi apici uncinati (es. «*boundary*»).
2. **Attributi e proprietà**: opzionali, definiti per convenzione in `lowerCamelCase` (es. `age`, `accountNumber`).
3. **Operazioni e metodi**: opzionali, definiti anch'essi in `lowerCamelCase` (es. `getAge()`, `deposit()`).

Un principio fondamentale di UML risiede nella flessibilità della rappresentazione: la medesima classe può comparire all'interno di diagrammi differenti con diversi livelli di dettaglio e decorazione. Nei diagrammi concettuali ad alto livello è comune mostrare il solo riquadro con il nome, mentre nei diagrammi di dettaglio progettuale vengono esplicitati tipi, visibilità e valori predefiniti.

Sintassi Formale di Attributi e Metodi. Per descrivere compiutamente i membri di una classe, UML adotta un formato standardizzato in cui soltanto il nome dell'elemento è mandatorio:

- **Formato degli Attributi**:

  ```
  visibilità nome molteplicità : Tipo = valoreIniziale
  ```

  Esempi validi: `-age: int`, `+names [1..*]: string`, `#count: int = 0`.

- **Formato delle Operazioni (Metodi)**:

  ```
  visibilità nome(parametro1: Tipo1, ...) : TipoRitorno
  ```

  Esempi validi: `+getAge(): int`, `+setAge(newAge: int): void`, `+calculate(val: float): float`.

Tipi di Visibilità. La visibilità regola l'accessibilità di attributi e metodi dall'esterno della classe, mappandosi direttamente sui principali modificatori dei linguaggi ad oggetti:

- **Public (`+`)**: l'elemento è accessibile da qualsiasi altra entità del sistema in grado di accedere alla classe stessa;
- **Private (`-`)**: l'elemento è accessibile esclusivamente dalle operazioni interne definite all'interno della classe stessa;
- **Protected (`#`)**: l'elemento è accessibile unicamente dalle operazioni della classe stessa o dalle sue classi derivate (discendenti lungo la gerarchia di ereditarietà);
- **Package (`~`)**: l'elemento è accessibile da qualunque entità allocata nel medesimo package in cui risiede la classe (o nei suoi eventuali sotto-package annidati).

4.2 La Notazione degli Oggetti (Object Icon)

Le istanze di una classe (gli **oggetti**) si disegnano come le classi, con tre differenze:

- il titolo è sottolineato e ha formato `nome : NomeClasse`;
- il nome dell'oggetto è **opzionale** (`: Person` è un oggetto anonimo);
- **niente slot per le operazioni**, ma gli attributi hanno un **valore** (`age = 10`).

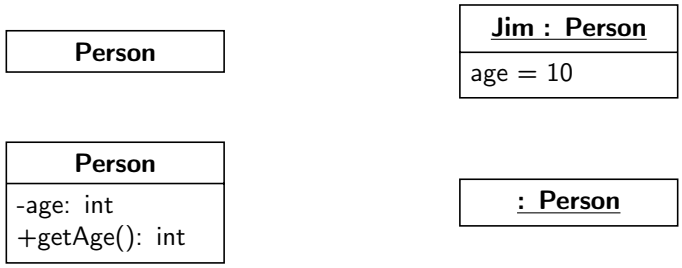


Figura 32: Classe (a sinistra) e oggetti (a destra).

Relazione Classe–Oggetto. L'oggetto *istanzia* la classe: si disegna una **dipendenza** (linea tratteggiata con freccia aperta) dall'oggetto alla classe, con stereotipo «*instanceOf*».

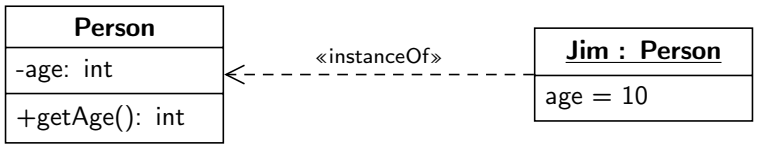


Figura 33: Dipendenza «instanceOf» tra oggetto e classe.

Esercizio 1: i bicchieri. **Testo**: *Definire la classe* `Glass` *con almeno 3 attributi e 2 metodi. Poi istanziare 2 bicchieri: uno pieno di vino, l'altro mezzo pieno di gin tonic.*

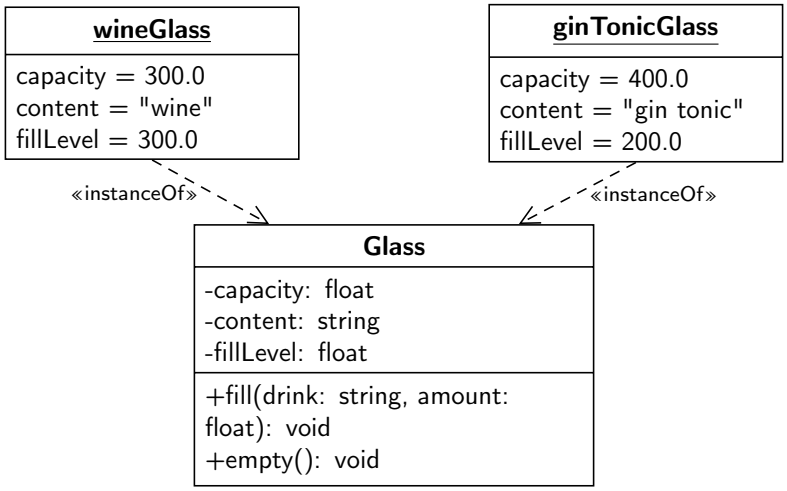


Figura 34: Soluzione dell'Esercizio 1.

4.3 Relazioni Strutturali: Generalizzazione e Realizzazione

I legami concettuali ed operativi tra le classi all'interno di un sistema si esprimono attraverso specifiche relazioni standardizzate.

Ereditarietà (Generalizzazione). L'ereditarietà nel paradigma a oggetti viene formalizzata in UML mediante la relazione di **Generalizzazione** (*Generalization*):

- **Notazione grafica**: una linea continua orientata con una punta a forma di **triangolo vuoto (chiuso)** che parte dall'elemento specifico (*child* o sottoclasse) e punta all'elemento generale (*parent* o superclasse).
- **Semantica**: tra tutte le relazioni previste da UML, la generalizzazione è la più forte, rigida e vincolante. Indica che la sottoclasse acquisisce integralmente attributi, metodi e associazioni della superclasse, potendoli estendere o specializzare.
- **Ereditarietà Multipla**: UML supporta nativamente l'ereditarietà multipla. Una classe figlia può derivare simultaneamente da più superclassi indipendenti. Ad esempio, una classe `automobile` può specializzare congiuntamente la superclasse `meansOfTransportation` (ereditando `numOfPass`) e la superclasse `machine` (ereditando `typeOfEnergy`).

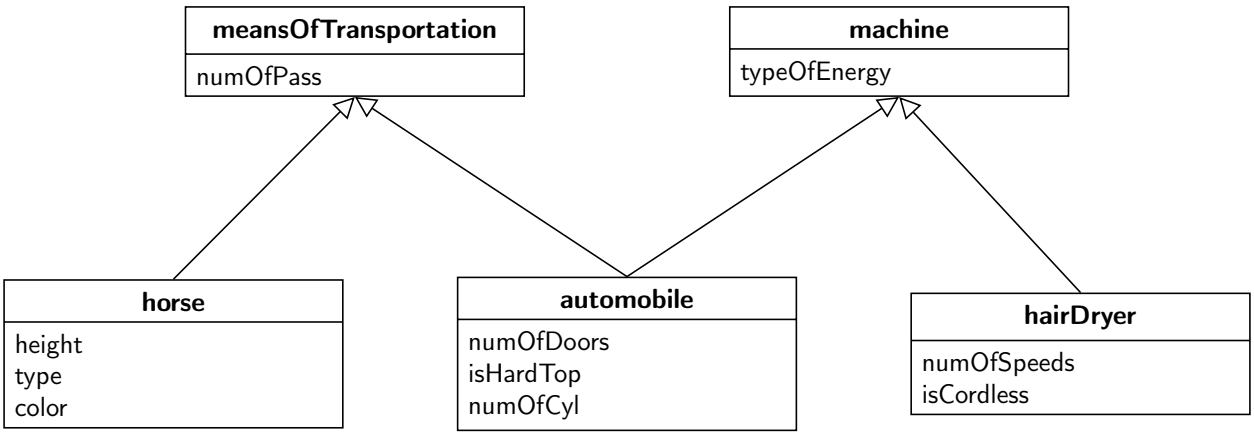


Figura 35: Relazione di generalizzazione singola e multipla in UML.

Relazione di Realizzazione (Realization). La realizzazione descrive una relazione semantica in cui un elemento fornitore (*supplier*) stabilisce un contratto o specifica funzionale pura, e un elemento client (*client*) si fa carico di realizzarla concretamente, implementandone ed eseguendone il codice.

- Il caso canonico è rappresentato dall'**interfaccia** (concetto analogo alle interfacce Java) implementata da una classe concreta.
- **Notazione grafica**: una linea tratteggiata con punta a forma di **triangolo vuoto** orientata verso l'interfaccia. L'interfaccia viene tipicamente decorata con lo stereotipo «*interface*».

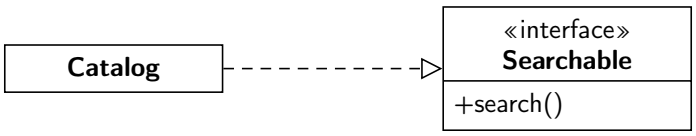


Figura 36: Relazione di realizzazione tra classe concreta e interfaccia.

Classi e Metodi Astratti. Una classe si definisce astratta quando definisce un'astrazione incompleta e non può essere direttamente istanziata, demandando l'implementazione di uno o più metodi alle sottoclassi:

- **Convenzione standard**: in UML, il nome di una classe astratta e dei relativi metodi astratti viene scritto in *corsivo* (*italic*).
- **Notazioni alternative**: poiché nei diagrammi cartacei o in schemi molto complessi il corsivo risulta difficile da identificare o tracciare con chiarezza, UML ammette convenzioni esplicite equivalenti:
  - Inserire lo stereotipo «*abstract*» o la proprietà `{abstract}` in prossimità del nome della classe;
  - Dichiarare il costruttore della classe con visibilità protetta (`#`);
  - Annotare i singoli metodi astratti con il tag `<abstract>` o `{abstract}`.

Esercizio 2: le armi da fuoco. **Testo**: *Modellare un sistema di armi e meccanismi di sparo.*

- `Firearm` (astratta): attributi `caliber`, `weight`, `manufacturer`; metodo astratto `fire()`; metodo concreto `reload()`.
- `TriggerMechanism` (astratta): attributo `triggerType`; metodo astratto `pullTrigger()`.
- Interfacce: `SemiAutomatic` (`ejectCasing()`) e `FullyAutomatic` (`burstFire()`).
- `Pistol`: eredita da `Firearm`, usa un `TriggerMechanism`, implementa `SemiAutomatic`.
- `AssaultRifle`: eredita da `Firearm`, usa un `TriggerMechanism`, implementa entrambe le interfacce.
- `SilencedPistol`: specializzazione di `Pistol`.

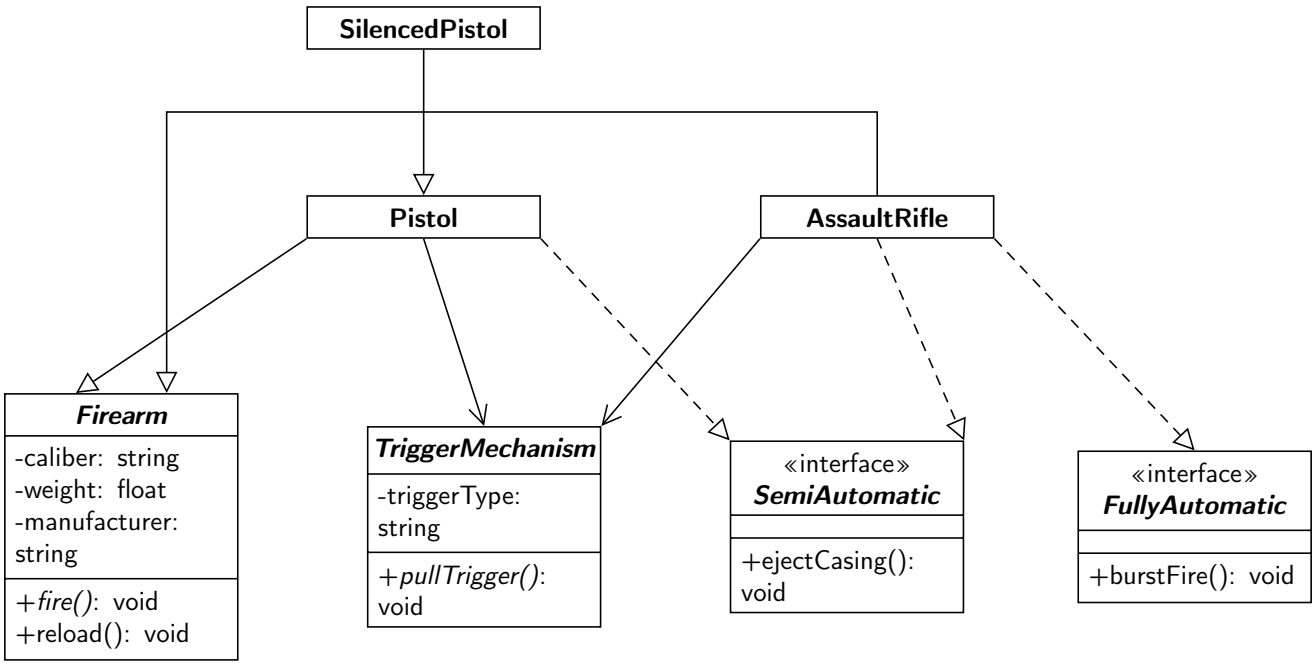


Figura 37: Soluzione dell'Esercizio 2. Corsivo = astratto; freccia piena aperta = associazione "usa".

4.4 I Diagrammi dei Casi d'Uso (Use Case Diagrams)

Il **diagramma dei casi d'uso** (*Use Case Diagram*) descrive il comportamento, sia esso desiderato o effettivamente offerto, di un sistema o di un suo sottosistema. Si tratta tipicamente del primo modello formale prodotto durante il processo di sviluppo nell'ambito della **raccolta e analisi dei requisiti funzionali**. Esso consente inoltre di validare l'architettura logica e verificare successivamente il rispetto dei requisiti.

Il diagramma risponde essenzialmente a due interrogativi fondamentali:

1. **Chi o cosa interagisce con il sistema?** → Gli **Attori** (*Actors*);
2. **Cosa possono compiere tali entità?** → I **Casi d'Uso** (*Use Cases*).

Principio Chiave: Obiettivo del Caso d'Uso

Un caso d'uso specifica **cosa ci si attende che il sistema faccia**, occultando deliberatamente **come** tale comportamento verrà implementato all'interno. Rappresenta una sequenza coerente di azioni (comprensiva di flussi primari, alternativi e di gestione degli errori) in grado di produrre un **risultato osservabile di valore tangibile** per l'attore coinvolto.

Gli Attori (Actors). Un attore specifica un ruolo ricoperto da un'entità esterna che dialoga attivamente con il sistema nell'ambito di un caso d'uso.

- **Esternalità assoluta**: gli attori risiedono sempre ed esclusivamente all'**esterno** del confine di sistema.
- **Natura eterogenea**: un attore non coincide necessariamente con una persona umana; può trattarsi di un apparato hardware, di un sensore, di un agente software remoto, di un sistema

bancario o di un demone interno che innesca compiti schedulati (es. trigger temporale per backup).

- **Attori Primari e Secondari**: l'attore che fa partire l'interazione viene denominato *attore primario* (in UML ogni evento è causalmente ricondotto all'entità che lo genera). Gli altri attori coinvolti per fornire dati di supporto o ricevere notifiche operative sono definiti *attori secondari*.
- **Ordine di lavoro**: prima si trovano gli attori, poi i loro casi d'uso.

Come trovare gli attori? Domande da porsi:

- Chi/cosa usa il sistema? Che ruolo ha?
- In quale parte dell'organizzazione è usato?
- Chi/cosa avvia il sistema? Chi lo supporta e mantiene?
- Altri sistemi interagiscono con questo?
- Ci sono funzioni periodiche (es. backup)?
- Chi/cosa fornisce o riceve informazioni dal sistema?
- Un attore ha più ruoli? Lo stesso ruolo è di più attori?

I Casi d'Uso (Use Cases). Un caso d'uso descrive un'esigenza o un obiettivo che l'attore intende conseguire interagendo con il sistema:

- **Innesco**: è sempre attivato dall'azione di un attore.
- **Nomenclatura**: viene rappresentato graficamente da un'ellisse contenente un'etichetta basata su un **verbo all'infinito** o su un sostantivo che esprima un'azione compiuta (es. *Select/Delete Courses*, *Print Timetable*, *Pay for Food*).
- **Classificatore comportamentale**: il caso d'uso è a tutti gli effetti un classificatore provvisto di comportamento (*classifier with behavior*). Può essere descritto analiticamente mediante diagrammi di interazione, diagrammi di sequenza, diagrammi a stati, corredato da formali precondizioni e postcondizioni.

Come trovare i casi d'uso? Domande da porsi:

- Quali funzioni si aspetta ogni attore?
- Il sistema gestisce informazioni (le memorizza/fornisce)?
- Qualche attore va avvisato quando lo stato del sistema cambia? E viceversa, gli attori devono avvisare il sistema di cambiamenti improvvisi?
- Eventi esterni influenzano il sistema?
- Ci sono casi d'uso di manutenzione?
- Tutti i requisiti funzionali sono coperti da casi d'uso?

Il Confine del Sistema (System Boundary) e le Associazioni. Nel diagramma dei casi d'uso la delimitazione tra ciò che appartiene all'architettura e ciò che fa parte del contesto circostante viene resa esplicita:

- **System Boundary**: rappresentato da un riquadro rettangolare recante in alto il nome del sistema (es. `Restaurant`). I casi d'uso sono racchiusi all'interno della scatola, mentre le icone stilizzate degli attori (*stickman*) stazionano rigorosamente all'esterno.
- **Associazioni**: l'interazione tra attore e caso d'uso è tracciata mediante una linea continua. Regola generale: un attore può essere associato a casi d'uso, classi o componenti. Due casi d'uso che insistono sul medesimo soggetto non si collegano direttamente tra loro tramite semplici associazioni generiche.
- **Caratteristiche facoltative dell'associazione**:
  - **Nome**: un'etichetta verbale che qualifica la relazione (es. `consumes`), eventualmente provvista di un piccolo indicatore di direzione di lettura;
  - **Molteplicità**: indica quanti attori o quante esecuzioni concorrono al legame (es. `1`, `0..n`);
  - **Ruoli**: descrive il ruolo contestuale assunto lungo la linea (es. `sommelier`, `critic`).

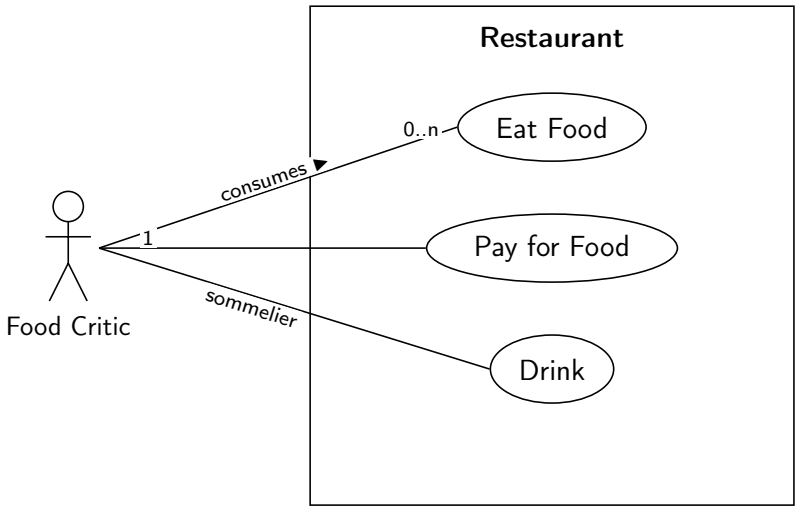


Figura 38: Confine del sistema, casi d'uso e associazioni decorate con nome, molteplicità e ruoli.

Esercizio Pratico: Il Sistema "Stupid Craps". **Testo**: *Si consideri la definizione di un gioco di dadi denominato "Stupid Craps". Il giocatore piazza una scommessa selezionando un numero compreso tra 1 e 6. Successivamente tira un dado: se compare il numero selezionato, vince cinque volte la posta iniziale. Si disegni un diagramma dei casi d'uso minimale per questo sistema.*

Le Trappole Comuni e l'Errore di Sovra-modellazione (*Overdoing*)

Nel modellare un diagramma dei casi d'uso elementare, gli sviluppatori cadono frequentemente nell'errore di voler rappresentare dettagli logici e temporali non pertinenti a questa

vista:

- **Nessuna sequenzialità temporale**: il diagramma dei casi d'uso fotografa le capacità offerte dal sistema, **non** è un diagramma di flusso. Non si devono inserire frecce sequenziali tra un'azione e la successiva.
- **Il dado non è un attore**: il dado è un oggetto interno (o un algoritmo generatore di numeri casuali), non un'entità esterna consapevole che avvia o riceve un servizio.
- **Nessuna classe né costrutto condizionale**: dettagli algoritmici come "se il numero coincide allora paga" appartengono alla logica interna del codice o alle pre/postcondizioni, non alla notazione grafica dei casi d'uso.

Soluzione Minimale ed Esatta:

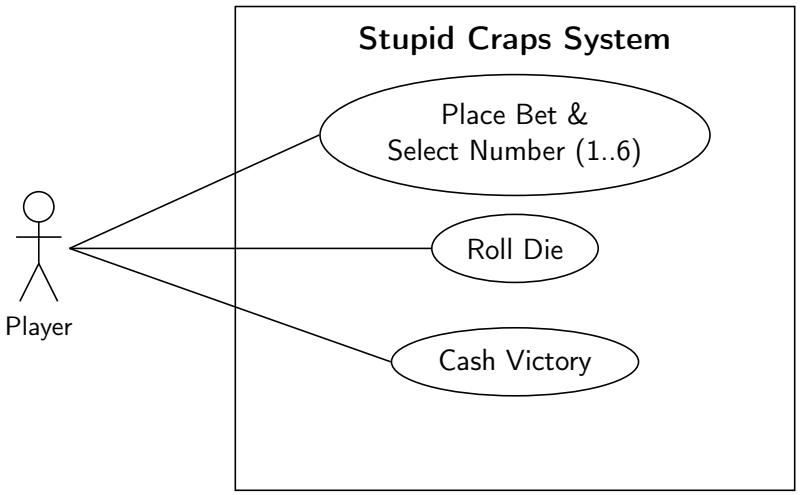


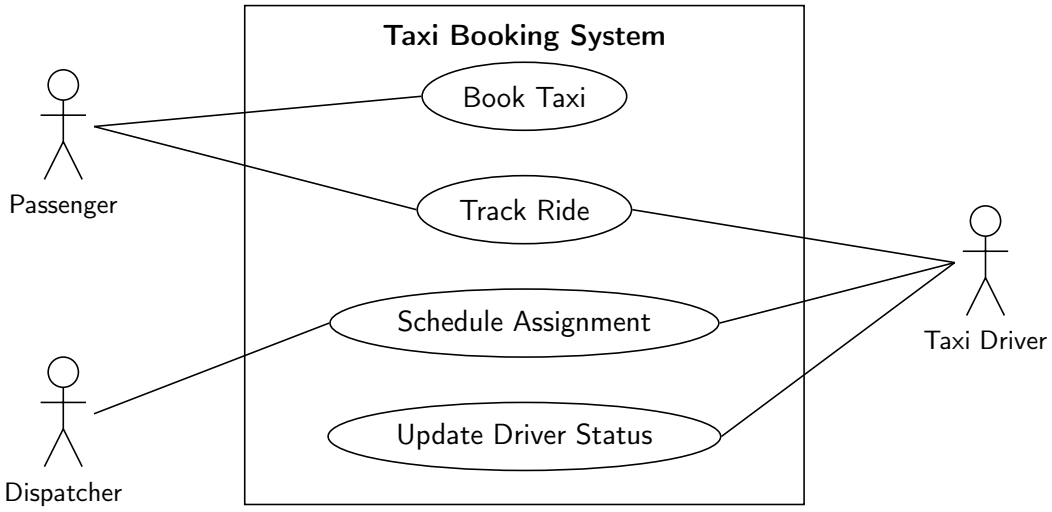
Figura 39: Soluzione corretta e minimale del diagramma dei casi d'uso per il sistema `Stupid Craps`.

Esercizi di Modellazione Concettuale (casi d'uso). Di seguito vengono analizzati e formalizzati i requisiti operativi per i sistemi proposti negli esercizi, identificando per ciascuno gli attori coinvolti, i casi d'uso primari e le relative relazioni strutturali.

Esercizio Taxi: Sistema di Prenotazione e Pianificazione Taxi. Il sistema deve gestire l'interazione tra passeggeri e vettori, ottimizzando l'assegnazione delle corse:

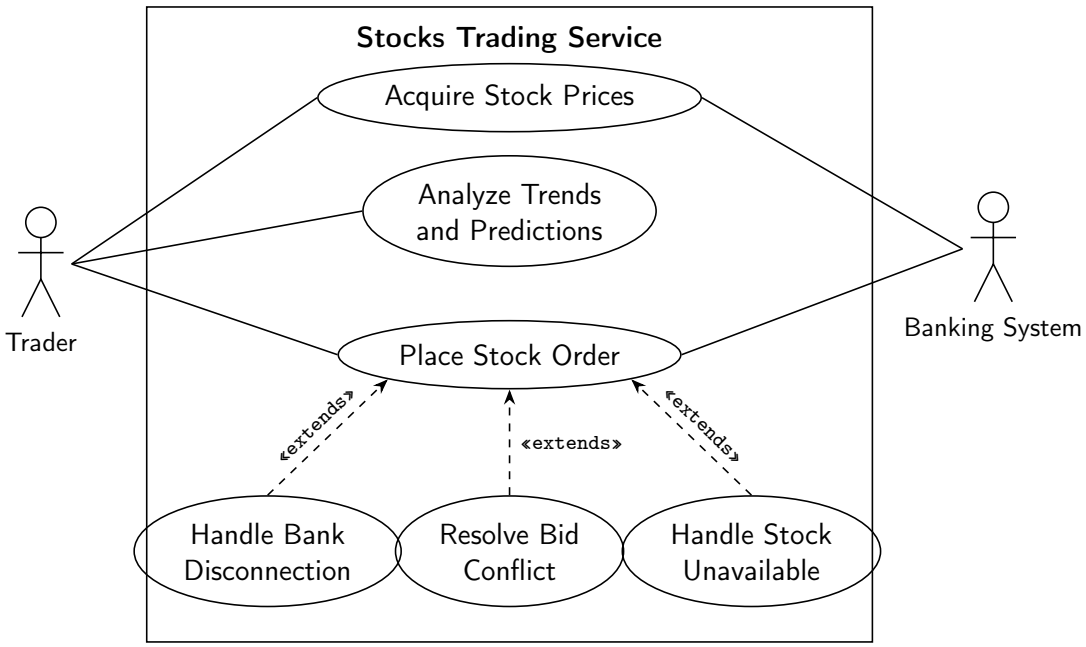
- **Attori identificati**:
  - `Passenger`: utente che richiede una corsa immediata o prenota un viaggio futuro;
  - `Taxi Driver`: operatore che riceve gli incarichi, aggiorna la propria disponibilità e conferma la presa in carico;
  - `Dispatcher / System Scheduler`: entità logica o attore di centrale addetto al coordinamento e all'ottimizzazione dei percorsi.

- **Casi d'uso primari**:
  - *Book Taxi*: inserimento del punto di partenza, destinazione e orario desiderato;
  - *Schedule Assignment*: algoritmo di pianificazione per abbinare la richiesta al taxi più vicino disponibile;
  - *Update Driver Status*: notifica dello stato operativo (libero, occupato, fuori servizio);
  - *Track Ride*: monitoraggio della posizione del veicolo in tempo reale.



Esercizio Stocks (Complex 1): Piattaforma di Negoziazione Titoli (Stocks Trading Service). La piattaforma connette investitori e istituti finanziari per la quotazione e la compravendita di titoli sul mercato:

- **Attori**:
  - `Trader / User`: utente operativo che consulta i corsi azionari ed emette ordini di acquisto;
  - `Banking System`: sistema esterno remoto che fornisce i dati di mercato ed esegue la transazione finanziaria.
- **Casi d'uso e varianti**:
  - *Acquire Stock Prices*: recupero in tempo reale delle quotazioni presso le banche collegate;
  - *Analyze Trends*: analisi predittiva e andamento storico dei prezzi;
  - *Place Stock Order*: emissione dell'ordine (con parametri quali esecuzione immediata vs differita, prezzo secco vs intervallo di prezzo).
- **Estensioni ed eccezioni (`«extends»`)**:
  - *Handle Bank Disconnection*: attivato se il canale verso l'istituto bancario si interrompe;
  - *Resolve Bid Conflict*: risoluzione di conflitti tra offerte sovrapposte;
  - *Handle Stock Unavailable*: notifica nel caso in cui il titolo non sia più negoziabile o esaurito.



4.5 Esercizi sui Diagrammi delle Classi

Notazione usata nelle soluzioni

Riepilogo operativo della teoria esposta in OOA (legenda a pag. 36; associazione, molteplicità, navigabilità, aggregazione, dipendenza).

- ◊ **rombo vuoto** sul lato del tutto = **aggregazione** (le parti vivono anche senza il tutto);
- ♦ **rombo pieno** sul lato del tutto = **composizione** (le parti muoiono col tutto);
- **freccia aperta** (linea continua) = associazione navigabile; **triangolo vuoto** = generalizzazione;
- **freccia aperta tratteggiata** = dipendenza; **triangolo vuoto tratteggiato** = realizzazione.

Come procedere. Si parte dalla struttura minima (classi e associazioni ovvie), poi si aggiungono attributi, molteplicità e operazioni. Le classi candidate sono i **sostantivi** del testo; i **verbi** suggeriscono operazioni e associazioni.

Esempio: il Pub. **Testo**: *Un pub è popolato da clienti e da personale. Il personale raccoglie gli ordini e consegna le birre. Il pagamento avviene alla cassa. Il manager si occupa del servizio, controlla la disponibilità di ogni birra nel frigo e, se serve, lo rifornisce.*
Classi: `Staff` (gestisce i clienti), `Manager` (sottoclasse di `Staff`, gestisce frigo e birre), `Customer`, `Order`, `Beer`, `Fridge` (contiene birre).

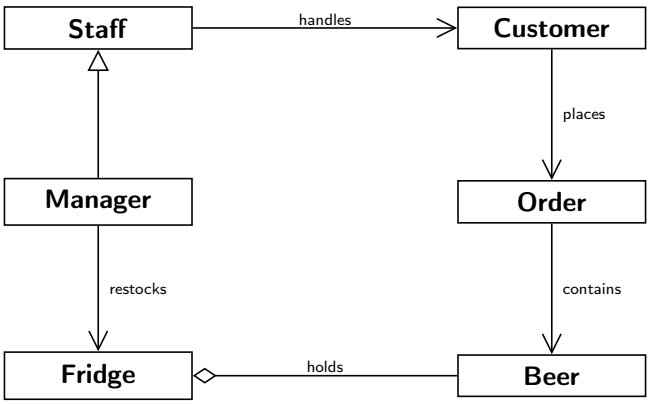


Figura 40: Soluzione: classi del Pub.

Esercizio 4: Edificio. **Testo**: *Un appartamento è composto da una o più stanze, ognuna con lunghezza e larghezza. Un appartamento è posseduto da una o più persone. Un edificio ha un certo numero di piani ed è composto da uno o più appartamenti.*

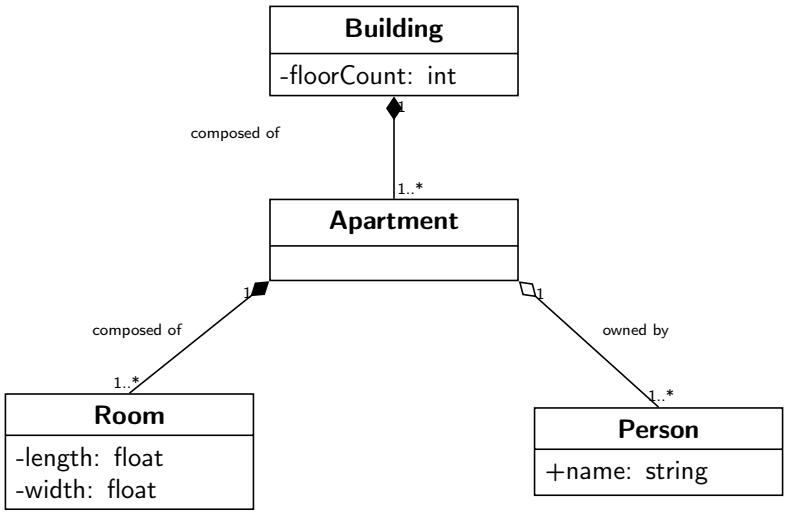


Figura 41: Soluzione dell'Esercizio 4. Aggregazione o composizione? Le stanze non esistono senza l'appartamento (composizione); le persone sì (aggregazione).

Esercizio 5: De Montfort University (DMU). **Testo**: *DMU offre programmi di studio, ognuno porta a un titolo di laurea. Ogni titolo è pubblicizzato nel prospetto informativo. Ogni programma include diversi moduli. Gli studenti iscritti a un programma seguono fino a 8 moduli all'anno. Alcuni titoli sono congiunti: uno studente può iscriversi a due programmi (es. accounting e finance).*

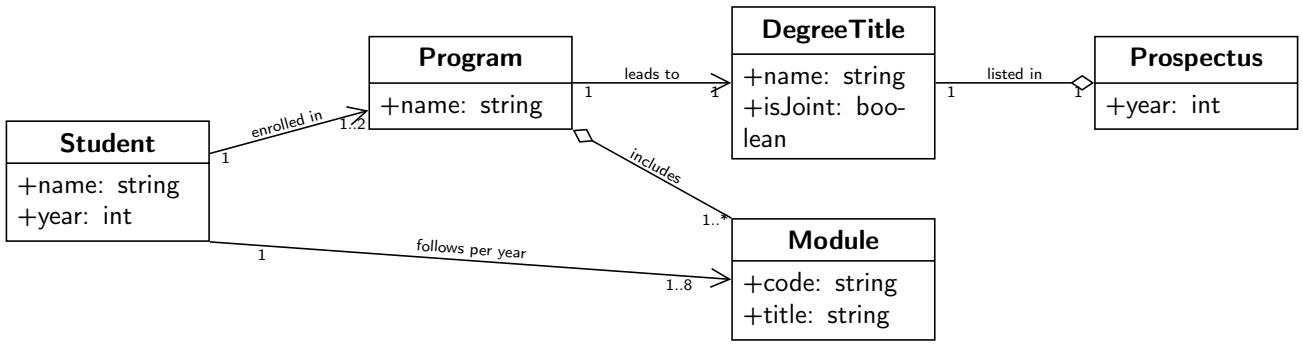


Figura 42: Soluzione dell'Esercizio 5.

Esercizio 5a: estensione di DMU. Parte 1: *DMU è composta da 6 Facoltà. Ogni Facoltà definisce un certo numero di Materie (es. Accounting, Auditing), gestite da diversi Moduli e insegnate in diversi Programmi.*
Parte 2: *Il Consiglio di Facoltà è composto da Studenti e Personale Accademico o Amministrativo. Il Personale Accademico insegna un numero arbitrario di Moduli e supervisiona più Studenti, ognuno iscritto a un Programma. Parte del Personale Amministrativo è nel Consiglio ma non insegna.*

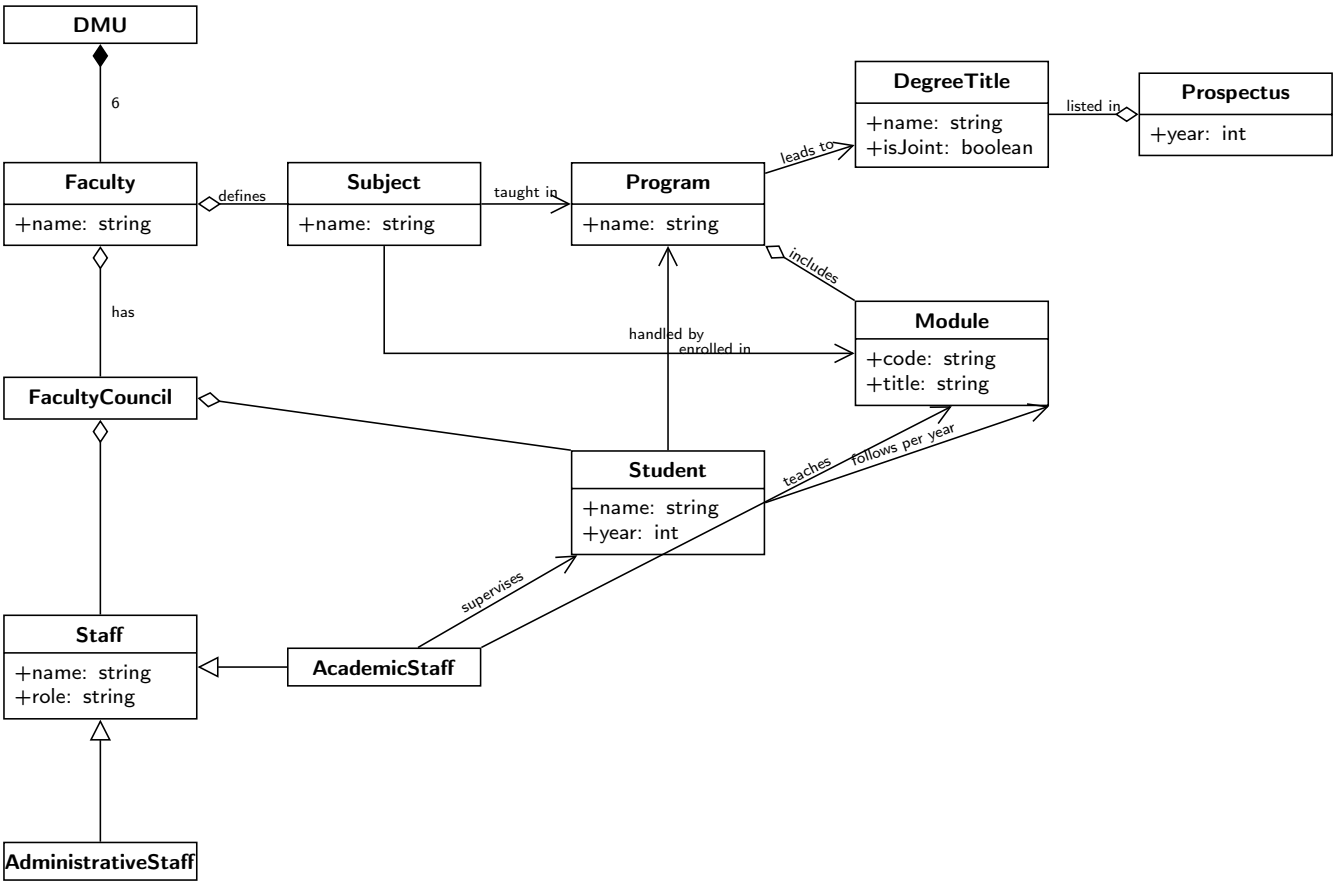


Figura 43: Soluzione completa dell'Esercizio 5a. Il rombo pieno `DMU`–`Faculty` (6 facoltà) è aggiunto rispetto alla slide.

Esercizio 6 (proposto): Nightclub. Testo: *Un sistema gestisce le feste private di un locale. Una festa ha un host (chi invia gli inviti), molti ospiti, un DJ, dei camerieri e un buttafuori. Il sistema deve permettere di invitare le persone, ricevere le risposte (accetta/rifiuta), definire il menù di cocktail e bevande, creare una playlist. I camerieri servono i drink, il DJ gestisce la playlist e accetta richieste durante la festa. Il buttafuori fa entrare solo gli invitati.*
La slide non riporta la soluzione: la proposta seguente è ricostruita dal testo.

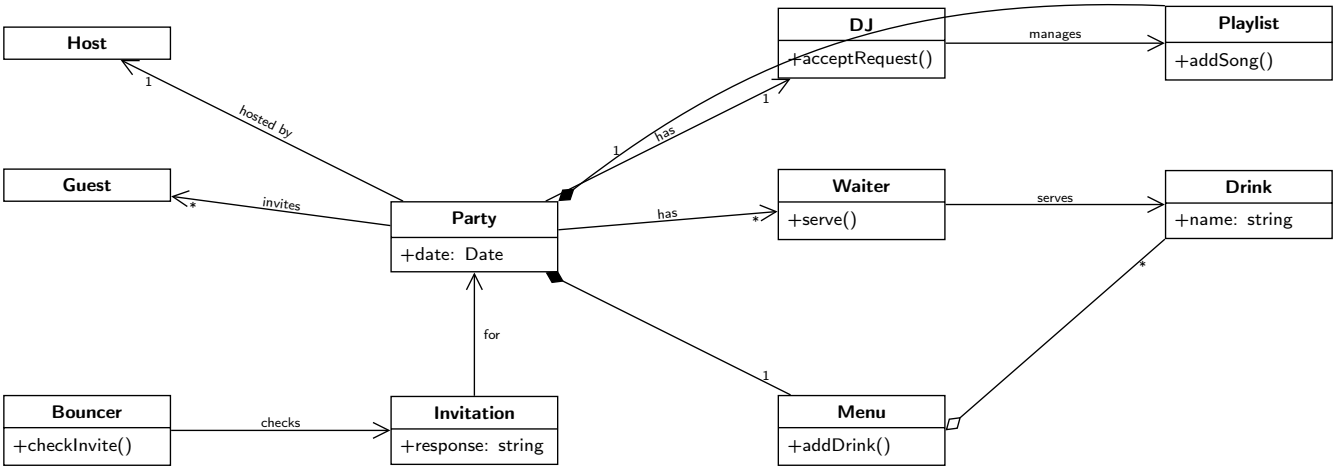


Figura 44: Proposta di soluzione per l'Esercizio 6 (la superclasse `Person` è omessa per leggibilità).

4.6 Esercizi Complessi (Classi e Casi d'Uso)

Complex 1: Stocks Trading Service (diagramma delle classi). Testo e diagramma dei casi d'uso: esercizio Stocks nella sezione sui Casi d'Uso. Qui la parte strutturale.

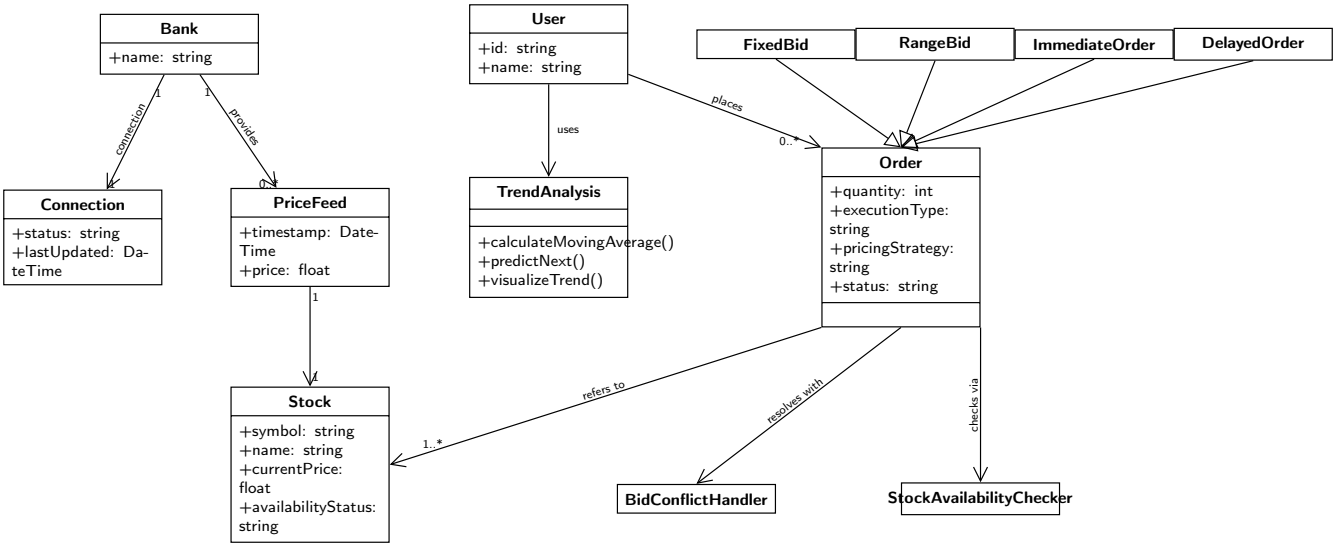


Figura 45: Soluzione Complex 1 – classi.

Complex 2: Network Printing Service. Testo: *Una stampante laser a colori ad altissima risoluzione è disponibile in rete. Il servizio permette di vedere l'anteprima del documento. L'utente può vedere lo stato della stampante (documenti in coda, problemi come inceppamenti, carta o toner finiti) e monitorare i propri lavori di stampa, ritardandoli o cancellandoli. Per stampare servono autorizzazione e quota di stampa. Un amministratore gestisce utenti e quote.*

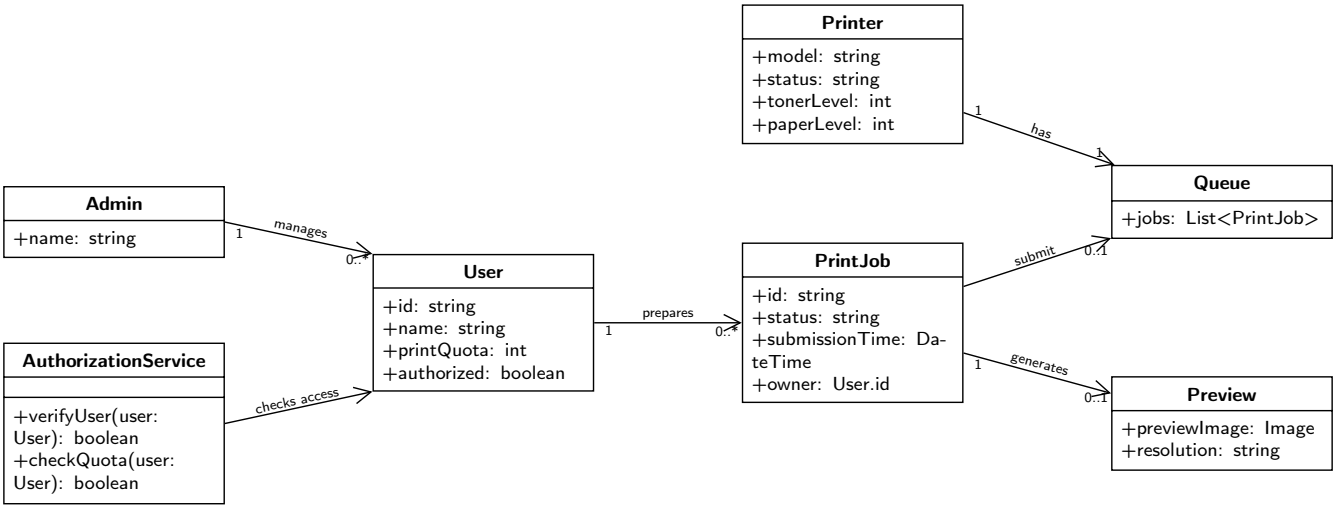


Figura 46: Soluzione Complex 2 (ricostruita dalla slide, che è tagliata a destra).

Complex 3: Bread Machine (classi e casi d'uso). Testo: *La macchina del pane prepara pagnotte tonde o a cassetta. Si toglie la vaschetta, si aggiungono gli ingredienti, si reinserisce la vaschetta, si seleziona il programma e si avvia. A fine cottura la macchina tiene in caldo il pane per un'ora: va tolto e fatto raffreddare, altrimenti indurisce. Prima di affettarlo si aspettano altri 30 minuti. Ingredienti base: liquido (acqua o latte), farina (bianca o integrale), sale, zucchero, lievito (fresco o secco). Si possono aggiungere ingredienti extra (dopo un beep): olive, semi, pomodori secchi. Tutti i pani seguono lo stesso ciclo, tranne quello integrale: preriscaldamento, impasto, due riposi (lievitazione) e cottura.*

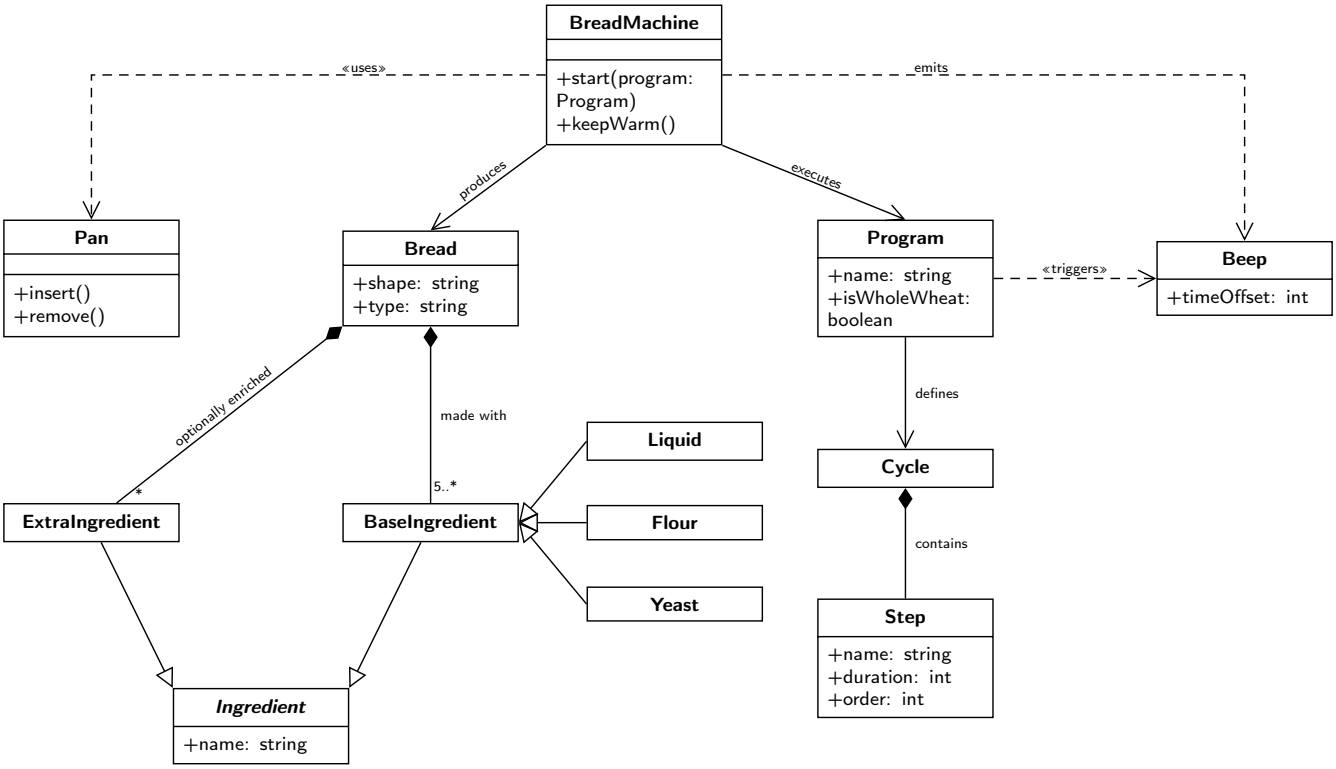


Figura 47: Soluzione Complex 3 – diagramma delle classi.

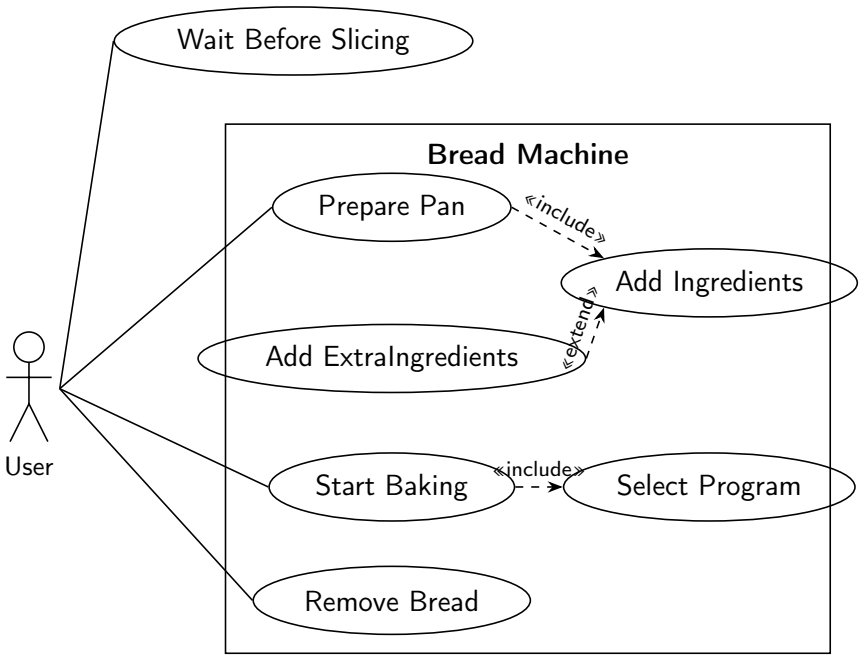


Figura 48: Soluzione Complex 3 – casi d'uso (come nella slide, *Wait Before Slicing* è fuori dal confine).

Complex 4: Component Brokerage System. **Testo**: *Il sistema fa da broker di componenti software. Gli sviluppatori, a fine sviluppo, li sottomettono via Internet. Un amministratore ne verifica funzionalità e modalità di connessione, li categorizza e li pubblica in un'area pubblica. I clienti (altri sviluppatori) navigano/cercano i componenti e li scaricano. Poi i fornitori vedono le statistiche di download e possono aggiungere/rimuovere componenti.*

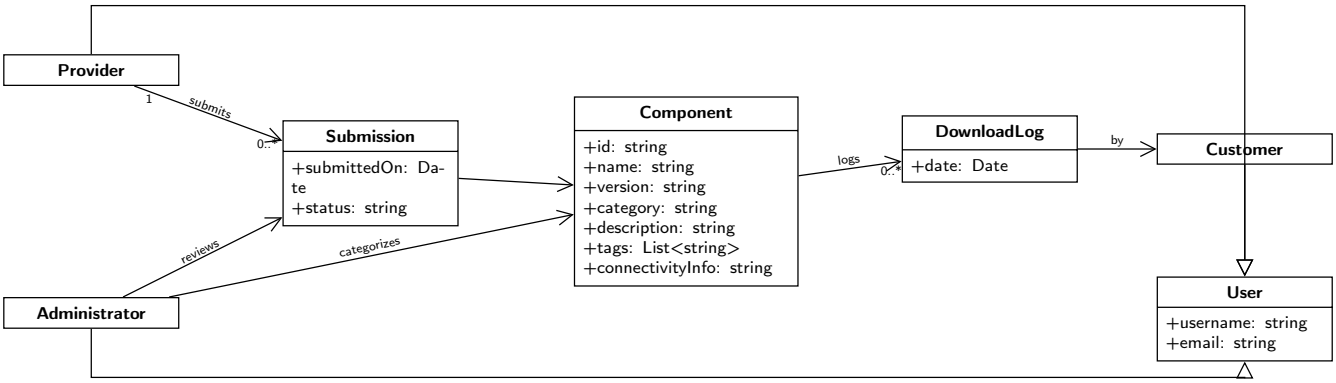


Figura 49: Soluzione Complex 4.

Complex 5: Telepass. Testo: *Un dispositivo sul parabrezza permette di pagare il pedaggio senza fermarsi; in alcune sedi anche il parcheggio. Il servizio è per persone fisiche. Ogni dispositivo è assegnato a un cliente tramite un contratto che definisce il metodo di pagamento: addebito periodico su conto corrente, BancoPosta o carta di credito abilitata. Il dispositivo ha un codice univoco e si associa a al più tre veicoli, le cui targhe sono registrate. Al casello un sistema ottico (CTV) riconosce il tipo di veicolo e fa emettere un segnale di richiesta identificazione; il dispositivo risponde col codice. Dopo mezzo secondo, se il codice è valido, la centralina alza la sbarra; altrimenti una telecamera fotografa la targa. In ogni caso un secondo sistema ottico conferma la classificazione.*

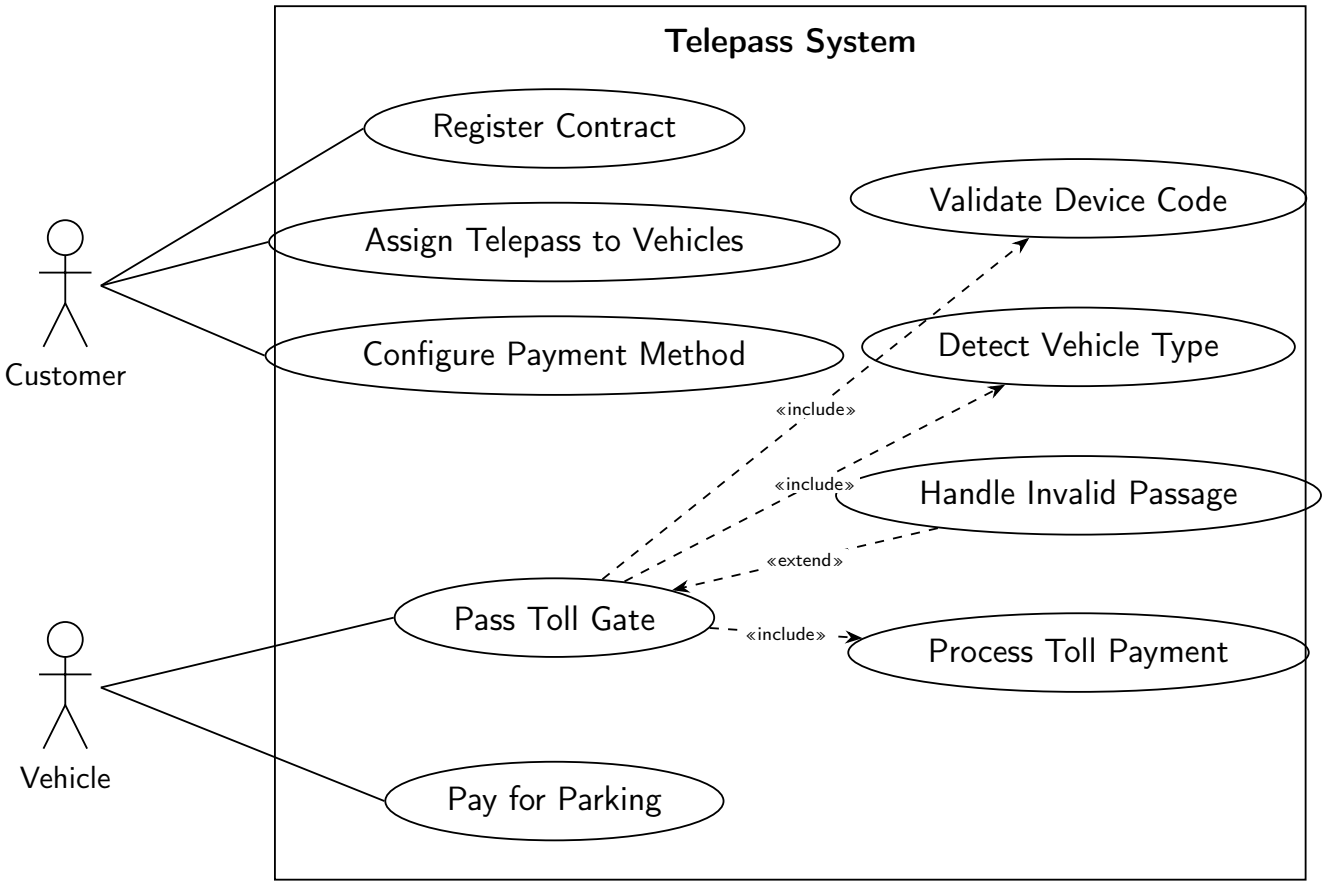


Figura 50: Soluzione Complex 5 – casi d'uso. La freccia «extend» parte da *Handle Invalid Passage* (la slide la disegna al contrario).

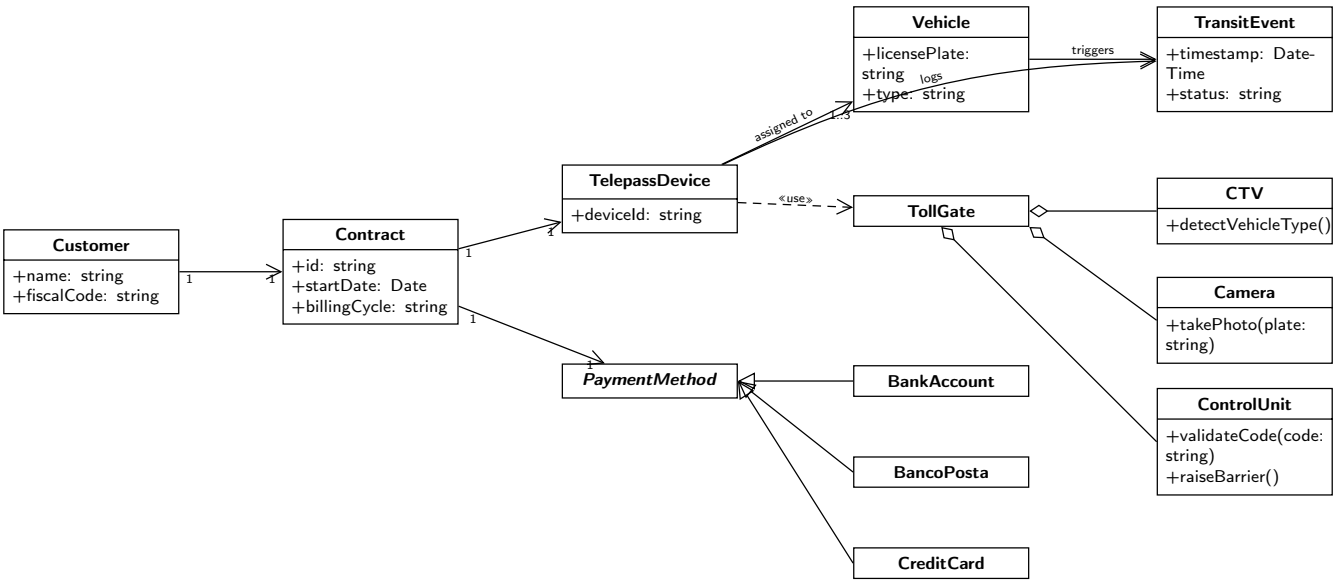


Figura 51: Soluzione Complex 5 – diagramma delle classi.

Esercizio proposto: Bug Tracking System. Testo: *Gli sviluppatori usano il sistema per tracciare i bug di un progetto. Chi trova un bug può inviare una segnalazione. Altri sviluppatori possono assegnare il bug a uno sviluppatore specifico (in particolare il responsabile del modulo) perché lo risolva. Gli utenti possono navigare/cercare tutti i bug. Un amministratore gestisce gli utenti per limitare l'accesso. L'amministratore deve anche generare report sullo stato del progetto, sotto forma di pagine web aggiornate ogni giorno alle 2:00.*
La slide non riporta la soluzione: caso d'uso ricostruito dal testo. Il Timer (evento temporale) è un attore esterno che innesca il report.

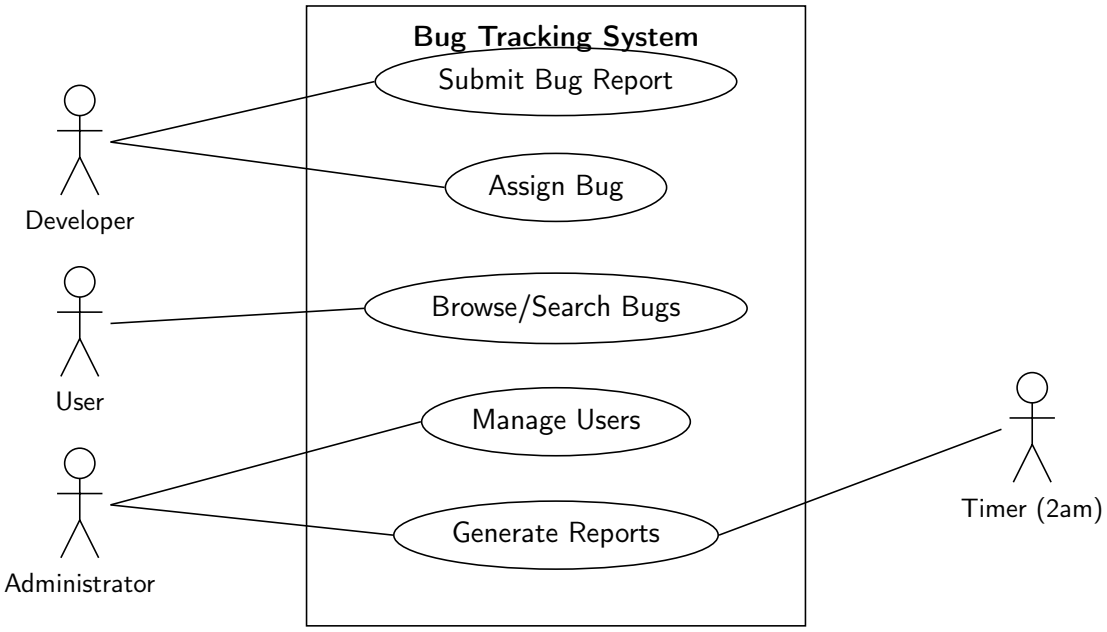


Figura 52: Proposta di soluzione per il Bug Tracking System.

4.7 Esercizi sugli Statechart

Ripasso: sintassi di una transizione

`evento [guardia] / azione`: tutti gli elementi sono opzionali. Cerchio nero pieno = stato iniziale, cerchio con bordo = stato finale. Dentro uno stato: `entry/`, `exit/`, `do/`.

Esercizio 1: il tuo processo di sviluppo software. **Testo**: *Definire lo statechart del proprio processo di sviluppo software.* ("Your mileage may vary": esistono molte soluzioni valide.)

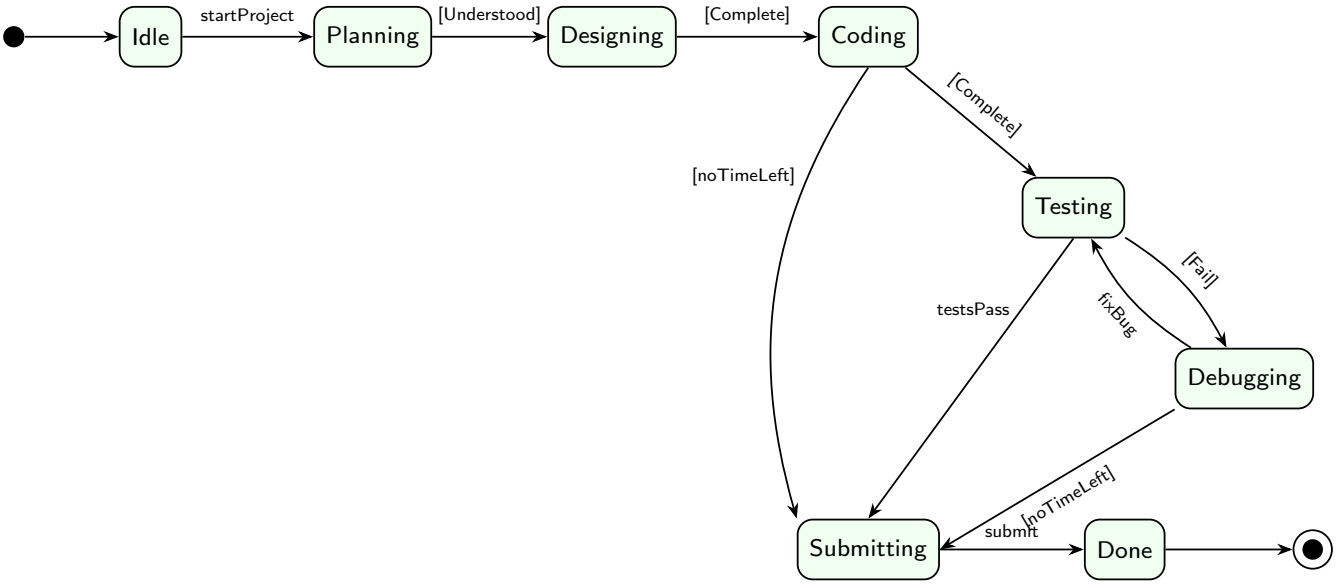


Figura 53: Soluzione dell'Esercizio 1 (Statechart).

Esercizio 2: una partita di basket. **Testo**: *Definire lo statechart di una partita di basket.* **Soluzione semplice**: 4 quarti da 15 minuti con pausa a metà; se pari dopo il 4° quarto, supplementare (che si ripete finché pari).

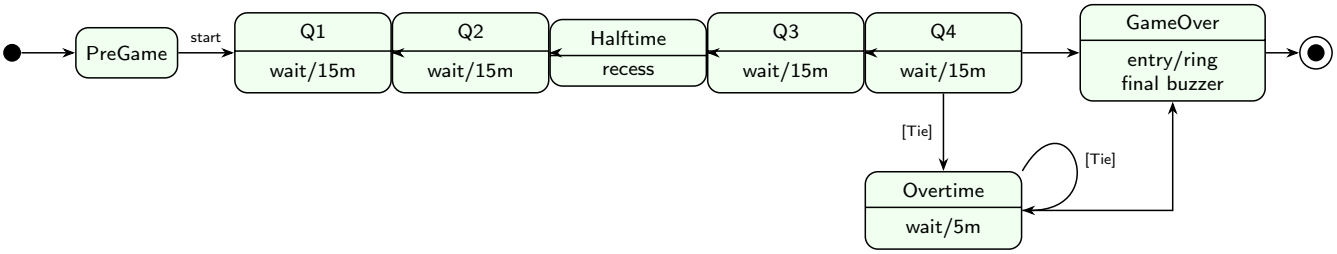


Figura 54: Esercizio 2: soluzione semplice.

Con sottostati: Q1 e Q2 raggruppati in `FirstHalf`, Q3 e Q4 in `SecondHalf` (stati compositi).

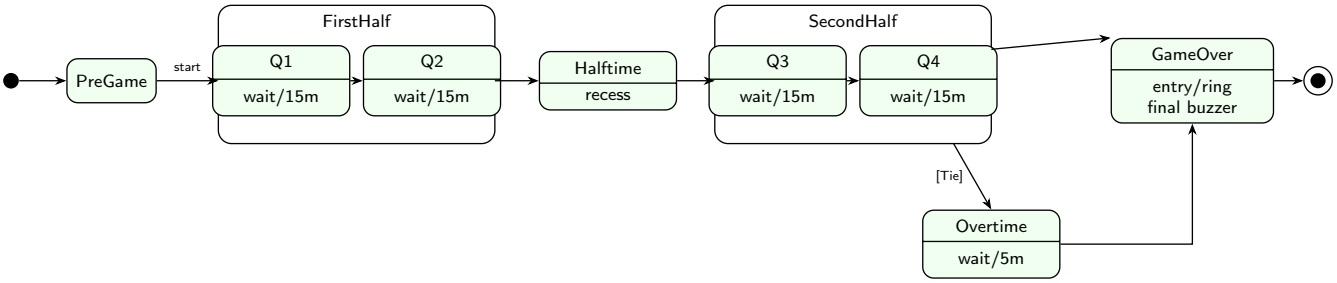


Figura 55: Esercizio 2: soluzione con stati compositi.

Con timeout: ogni quarto (e l'overtime) è uno stato composito con due sottostati, `InPlay` e `Timeout` (1 minuto). Lo schema vale per Q1–Q4 e Overtime.

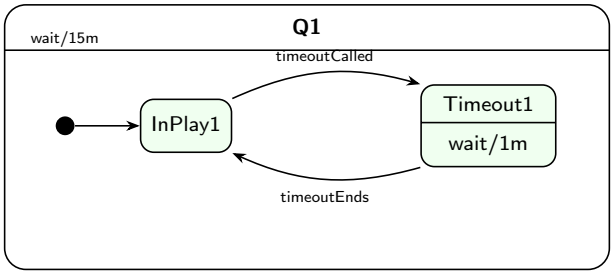


Figura 56: Esercizio 2: dettaglio con timeout (stesso schema per ogni quarto e per l'overtime).

Esercizio 3: stato civile di una persona. **Testo**: *Stato civile dalla nascita alla morte. In Italia ci si sposa a 18 anni; il tribunale dei minorenni può concedere l'emancipazione a chi ha almeno 16 anni, permettendo il matrimonio.*

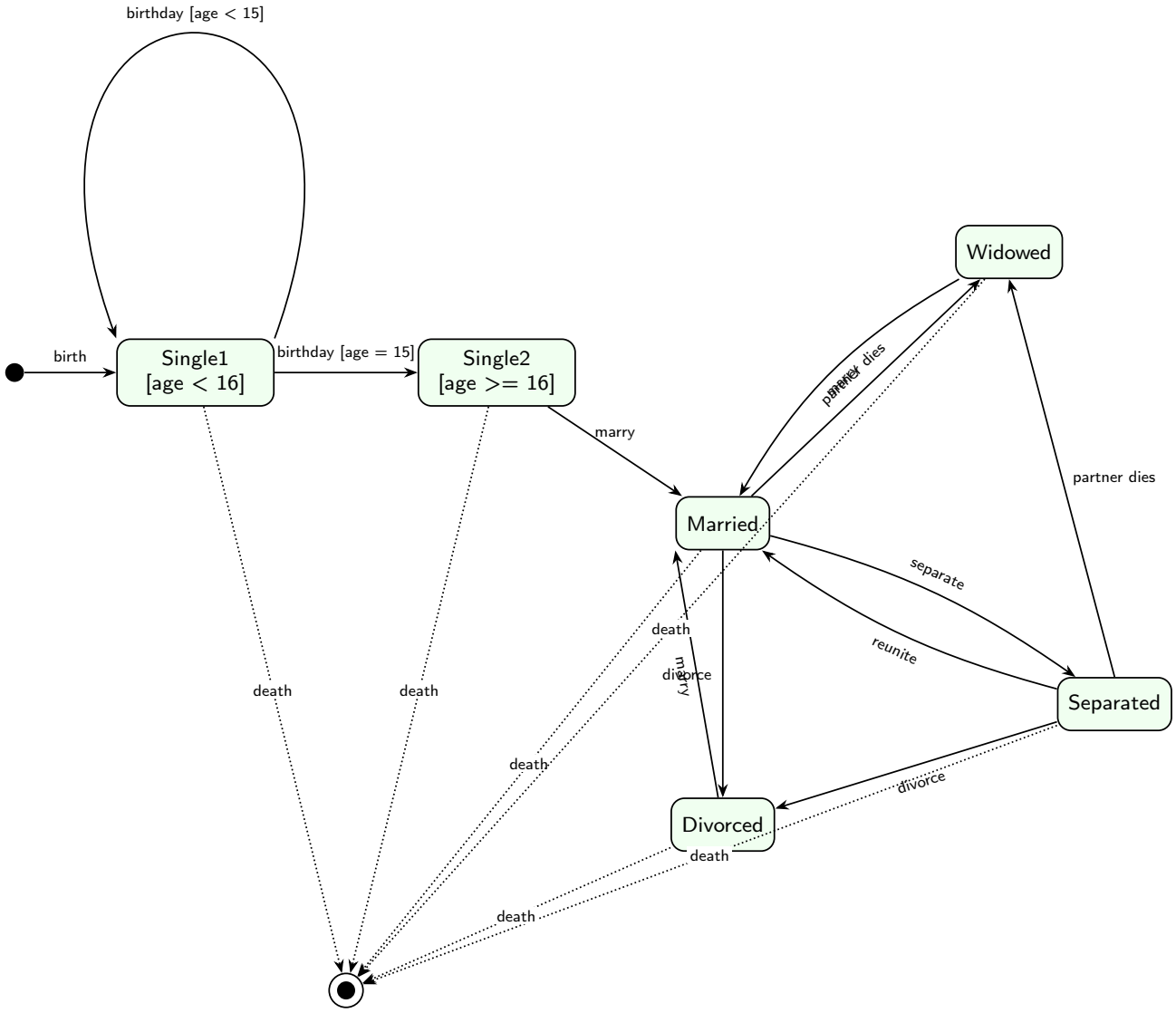


Figura 57: Soluzione dell'Esercizio 3. Ogni stato ha una transizione `death` verso lo stato finale (tratteggiata per leggibilità).

Nota dalla lezione: nessuna delle soluzioni proposte è davvero conforme al diritto italiano. Ad esempio, qui `Single2` (età ≥ 16) può sposarsi con un semplice `marry`, ignorando la regola dei 18 anni e l'emancipazione del tribunale dei minorenni. In aula la soluzione è stata accettata come approssimazione.

Esercizio 4: conto bancario. **Testo**: *Un utente può aprire un conto, prelevare e versare. Disegnare lo statechart che tiene traccia del saldo.*

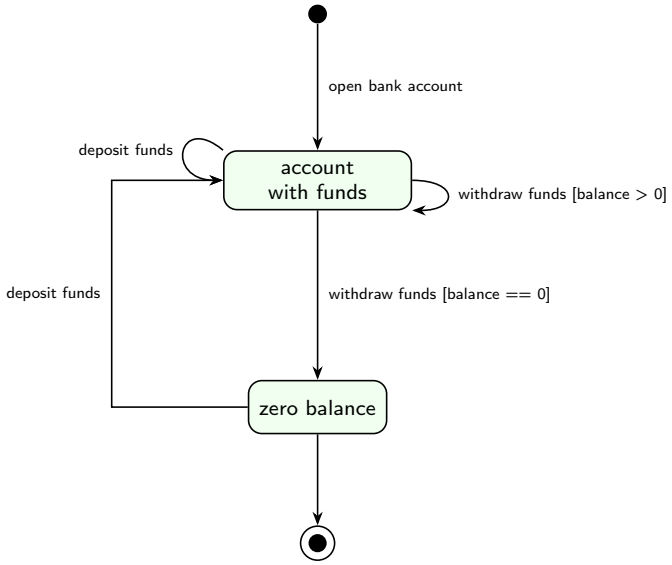


Figura 58: Soluzione dell'Esercizio 4.

Esercizio 5: la spesa al supermercato. **Testo**: *Statechart del cliente: cerca parcheggio, poi fa la spesa; deve comprare surgelati e carne, facendo la coda al banco macelleria; alla fine paga alla cassa, ma può decidere di andarsene in qualsiasi momento.*

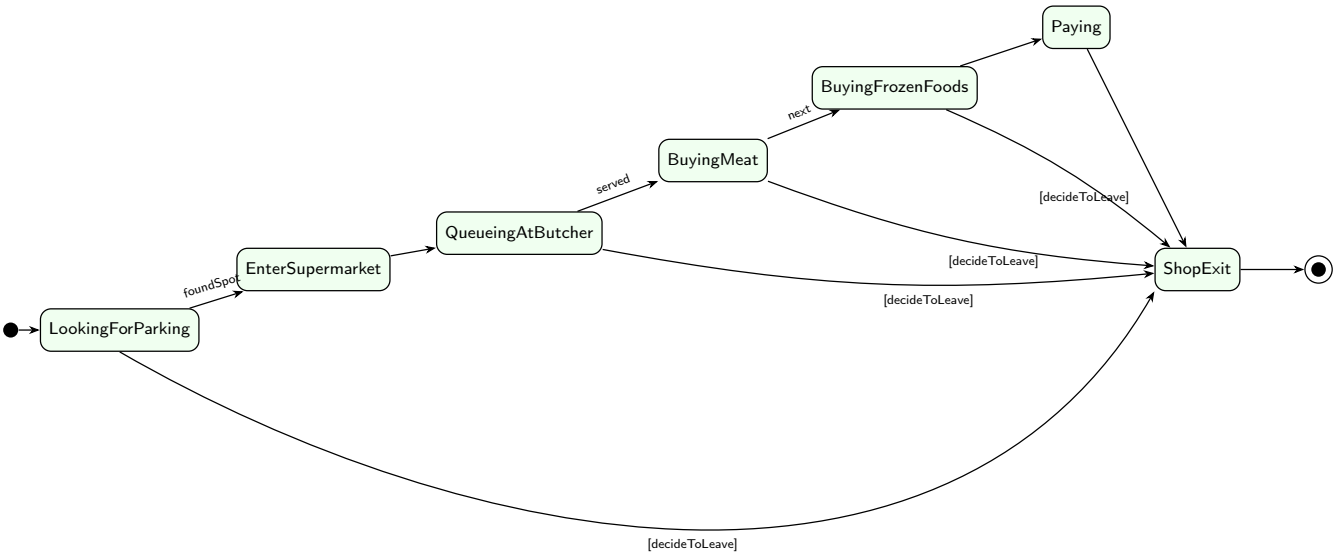


Figura 59: Soluzione dell'Esercizio 5 (la slide mostra anche `[decideToLeave]` da `EnterSupermarket`, omesso per leggibilità).

4.8 Esercizi sugli Activity Diagram

Esercizio 1: biblioteca. **Testo**: *Una biblioteca permette il prestito di libri e riviste. Il sistema tiene traccia dei prestiti in corso e verifica che non si superi il massimo di 6 volumi per persona. Solo i soci registrati possono prendere in prestito. La disponibilità di ogni opera è aggiornata di continuo e va controllata al momento della richiesta.*

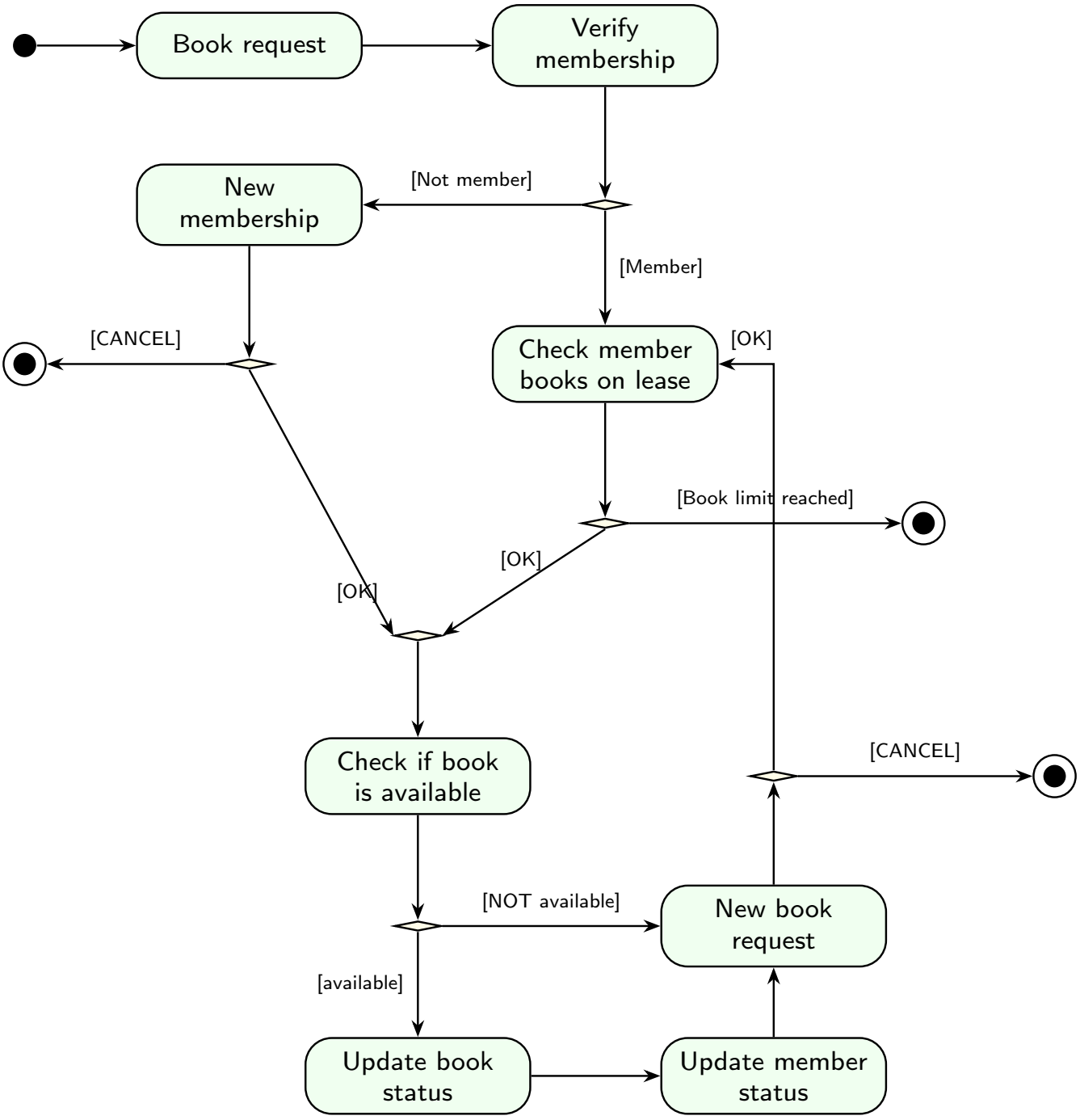


Figura 60: Soluzione dell'Esercizio 1 (rombi = decision/merge; la slide usa una barra al posto del merge, "solution. . . sort of").

Esercizio 2: partita di basket. **Testo**: *Activity diagram della partita di basket. Attenzione: i cicli sono brutti.*

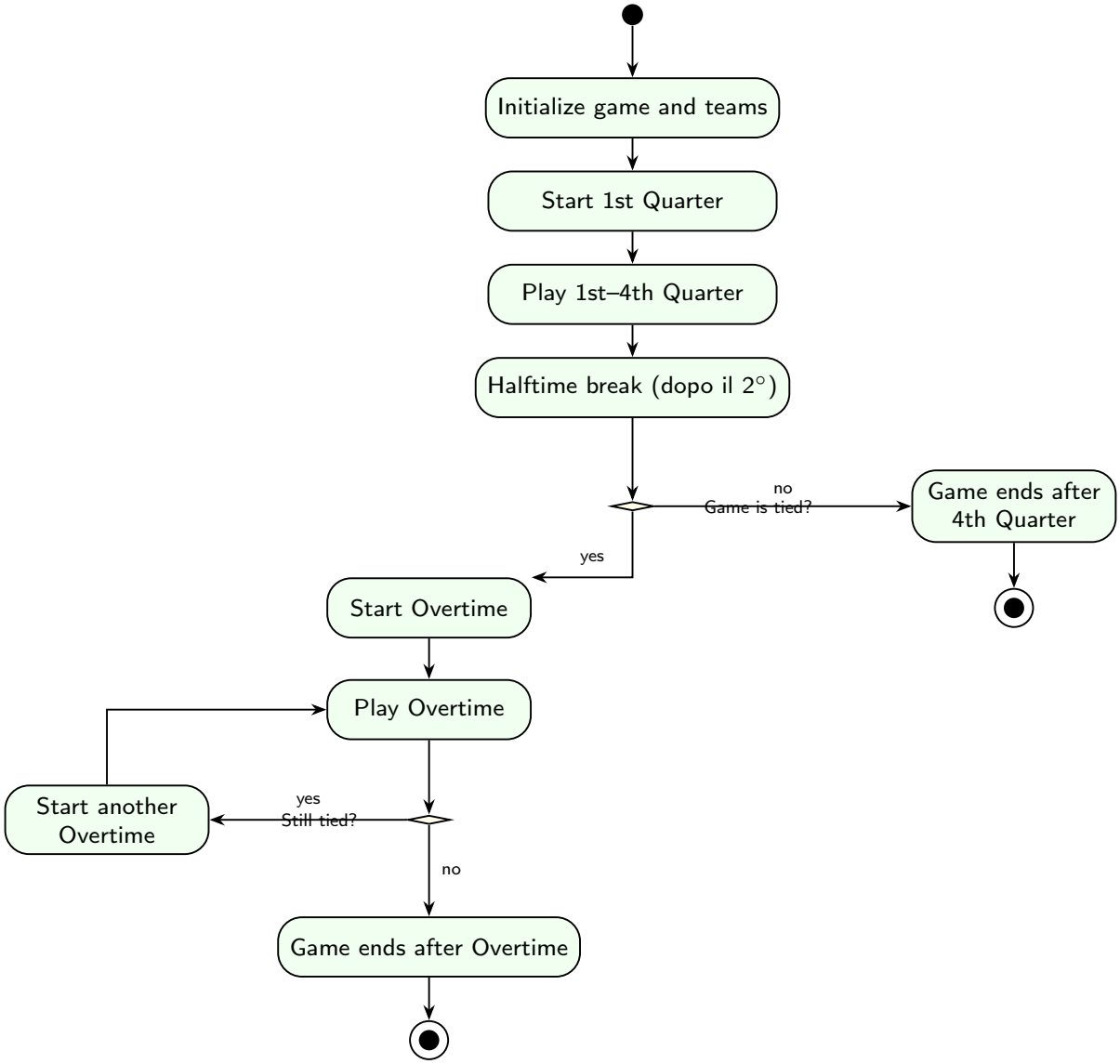


Figura 61: Soluzione dell'Esercizio 2 (il ciclo dell'overtime è un ramo di ritorno dopo *Still tied?*).

Esercizio proposto: il tuo processo di sviluppo (Activity). *La slide non riporta la soluzione: proposta ricostruita.* Dopo la fase di design, codice e test si scrivono in parallelo (fork/join); i test falliti riportano al codice.

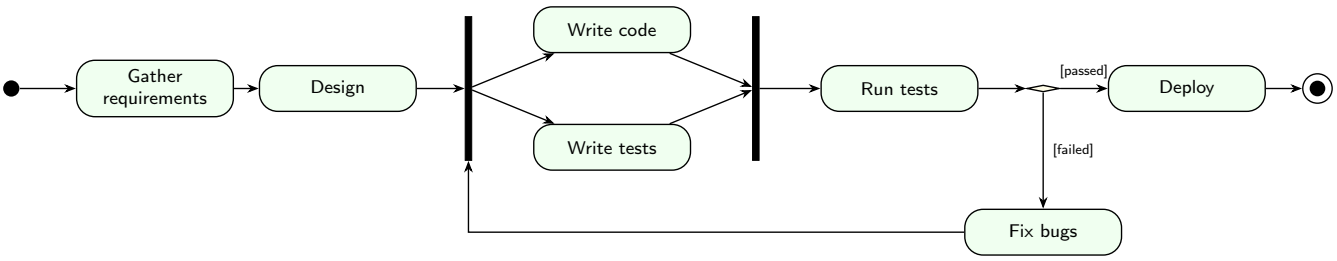


Figura 62: Proposta: processo di sviluppo come Activity Diagram.

4.9 Esercizi sui Sequence Diagram

Esercizio 1: avvio di un ATM. Testo: *Il sistema si attiva quando l'operatore porta l'interruttore su "on". Poi gli si chiede di inserire il contante disponibile nell'ATM e quindi si stabilisce la connessione con la banca. A questo punto si possono servire i clienti.*

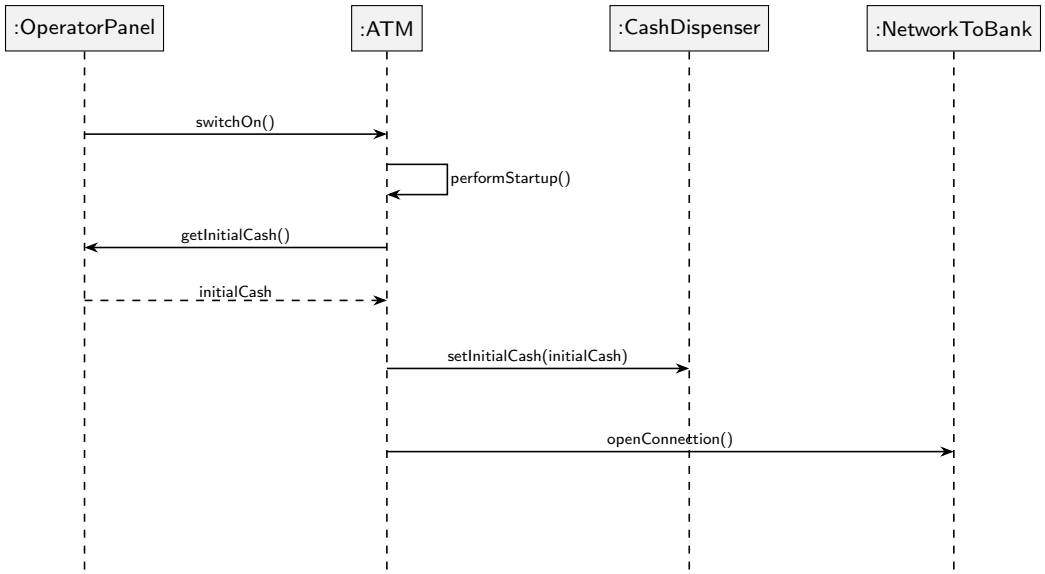


Figura 63: Soluzione dell'Esercizio 1: System Startup Sequence Diagram.

Esercizio 2: richiesta di nuovo codice. Testo: *Modellare come si riceve la richiesta di sviluppare un nuovo pezzo di codice.*

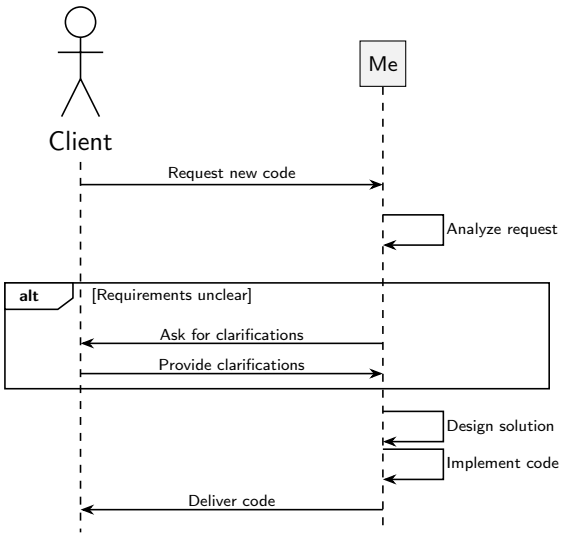


Figura 64: Soluzione dell'Esercizio 2.

Esercizio 3: invio di un pacchetto TCP su Ethernet. Testo: *Modellare con un sequence diagram come un pacchetto TCP viene inviato su una connessione Ethernet.*

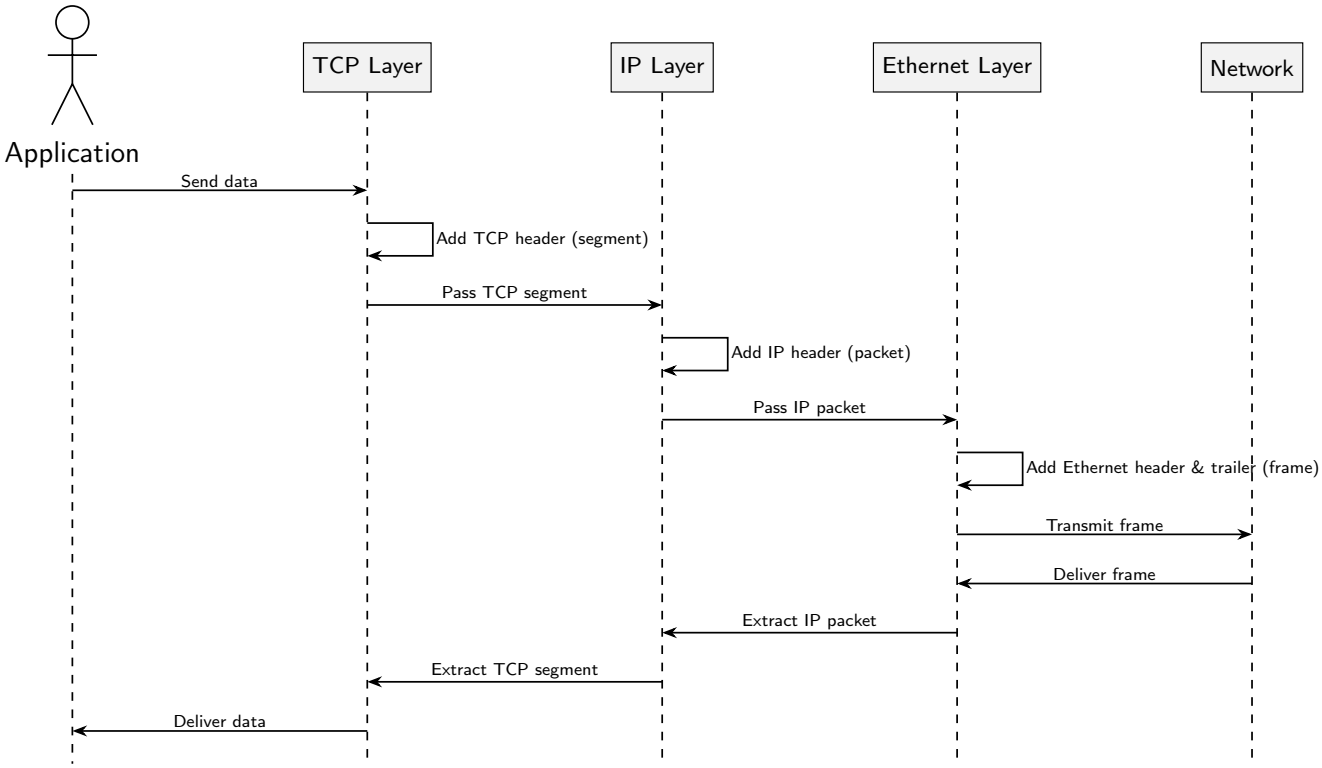


Figura 65: Soluzione dell'Esercizio 3.

Esercizio 4: consegna di un progetto (con alt). Testo: *Modellare come si riceve la richiesta di sviluppare un nuovo pezzo di codice. Qual è la differenza con prima?* (Qui il processo coinvolge più attori/lifeline e un ramo di fallimento test.)

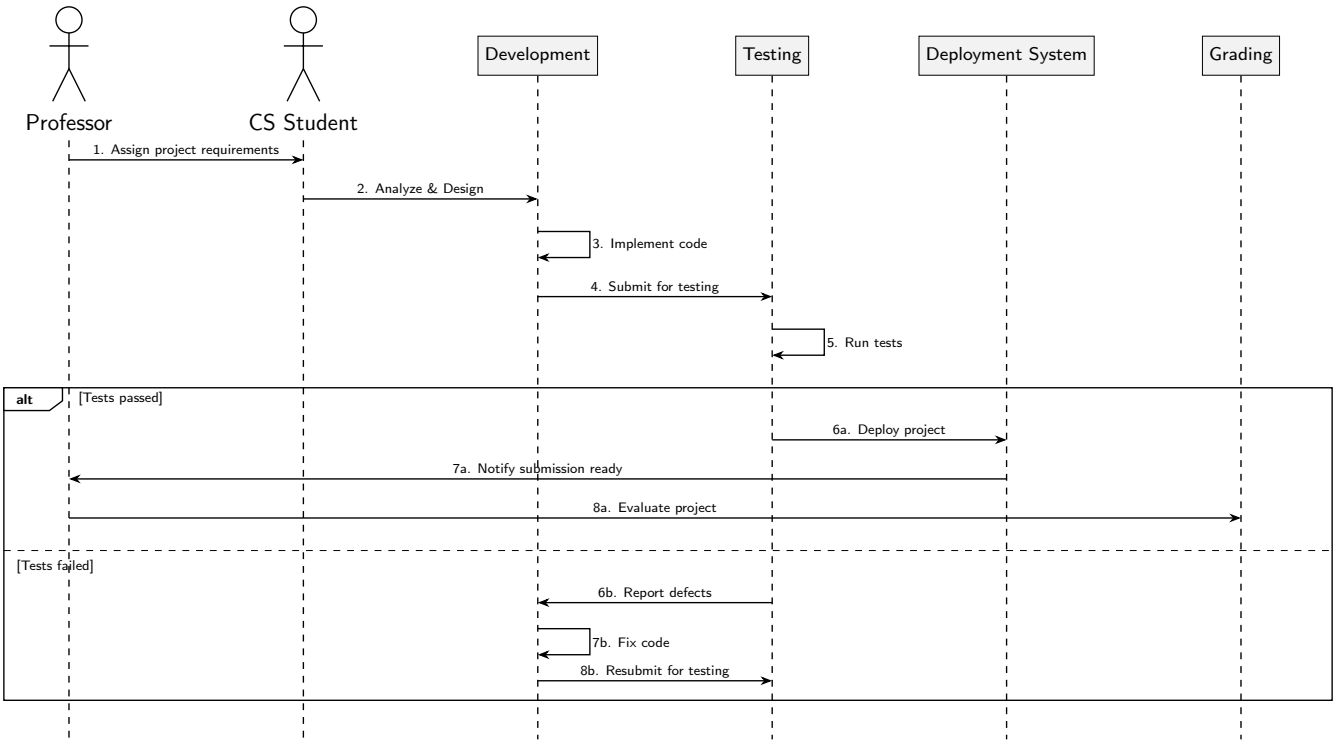


Figura 66: Soluzione dell'Esercizio 4.

Esercizio 5: telefono cellulare (cell to cell e roaming). Testo: *Rappresentare come un cellulare passa da una cella all'altra e gestisce il roaming.*

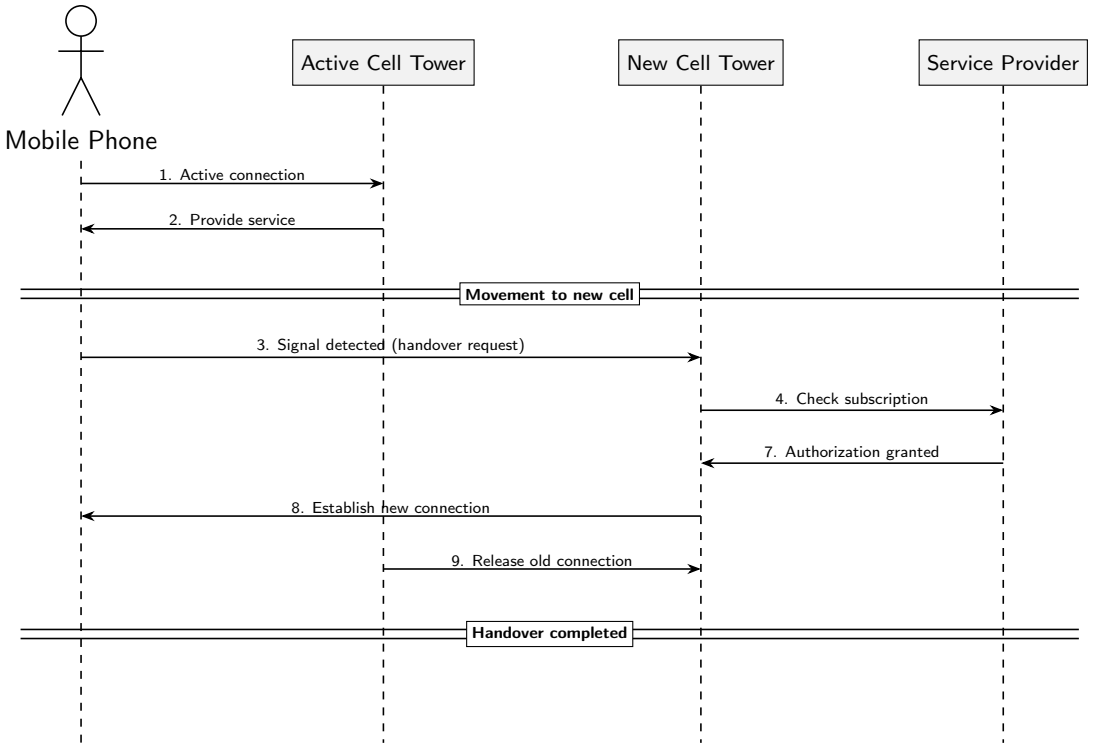


Figura 67: Esercizio 5 – cell to cell.

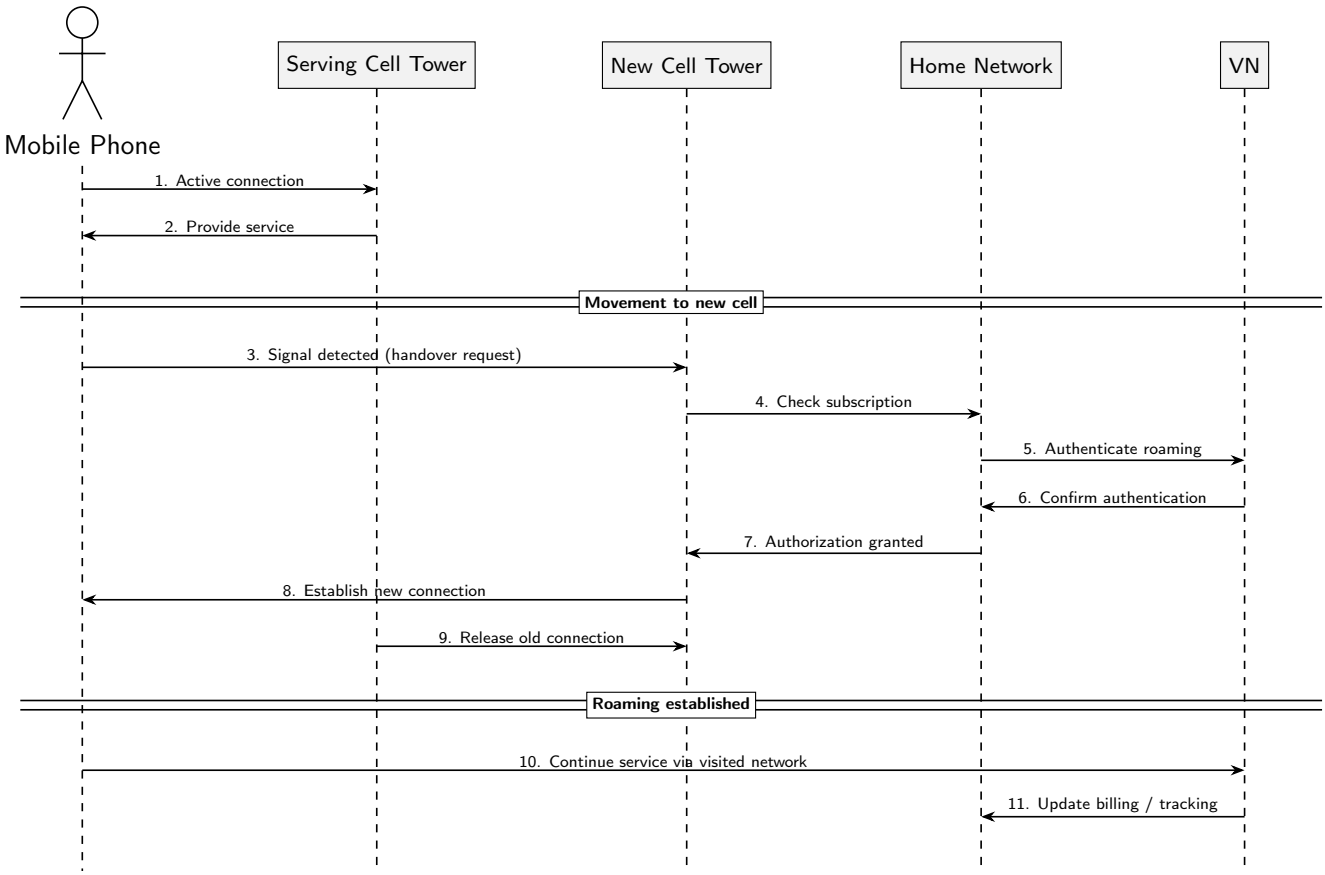


Figura 68: Esercizio 5 – roaming.

4.10 Esercizio Completo: il Garage

Testo: *Un garage è fatto di più livelli, ognuno con un certo numero di posti, di tipo diverso: auto normali, veicoli grandi (furgoni...), auto di lusso. Le auto a GPL possono parcheggiare solo al primo piano. Si può affittare un posto su base mensile, se disponibile; non più del 50% dei posti di ogni categoria può essere affittato. I posti non affittati servono per il parcheggio orario, fino a un massimo di otto ore; oltre le otto ore si applica una penale al ritiro dell'auto. Gli utenti sono i conducenti e il gestore del sistema, che fornisce le informazioni di configurazione (es. il numero di posti per categoria).* Richiesti: casi d'uso, classi, sequence e statechart.

Diagramma dei casi d'uso. `Regular Client` e `Occasional Client` sono specializzazioni dell'attore `Driver` (generalizzazione tra attori).

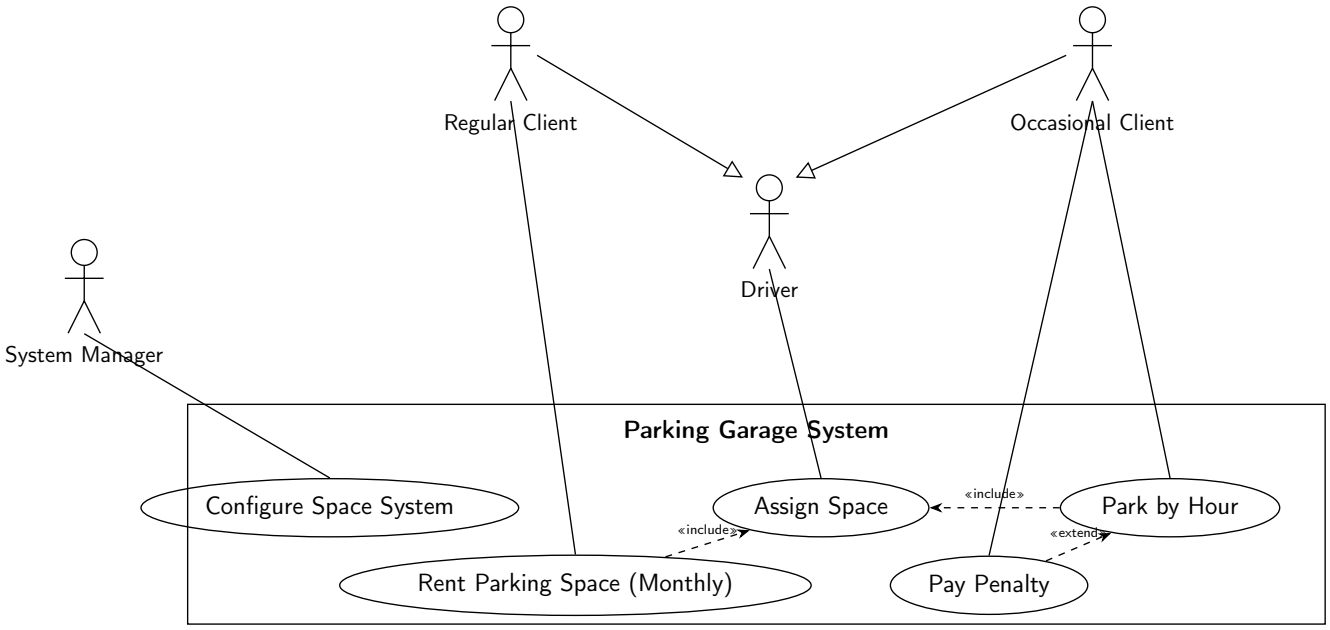


Figura 69: Garage – casi d'uso.

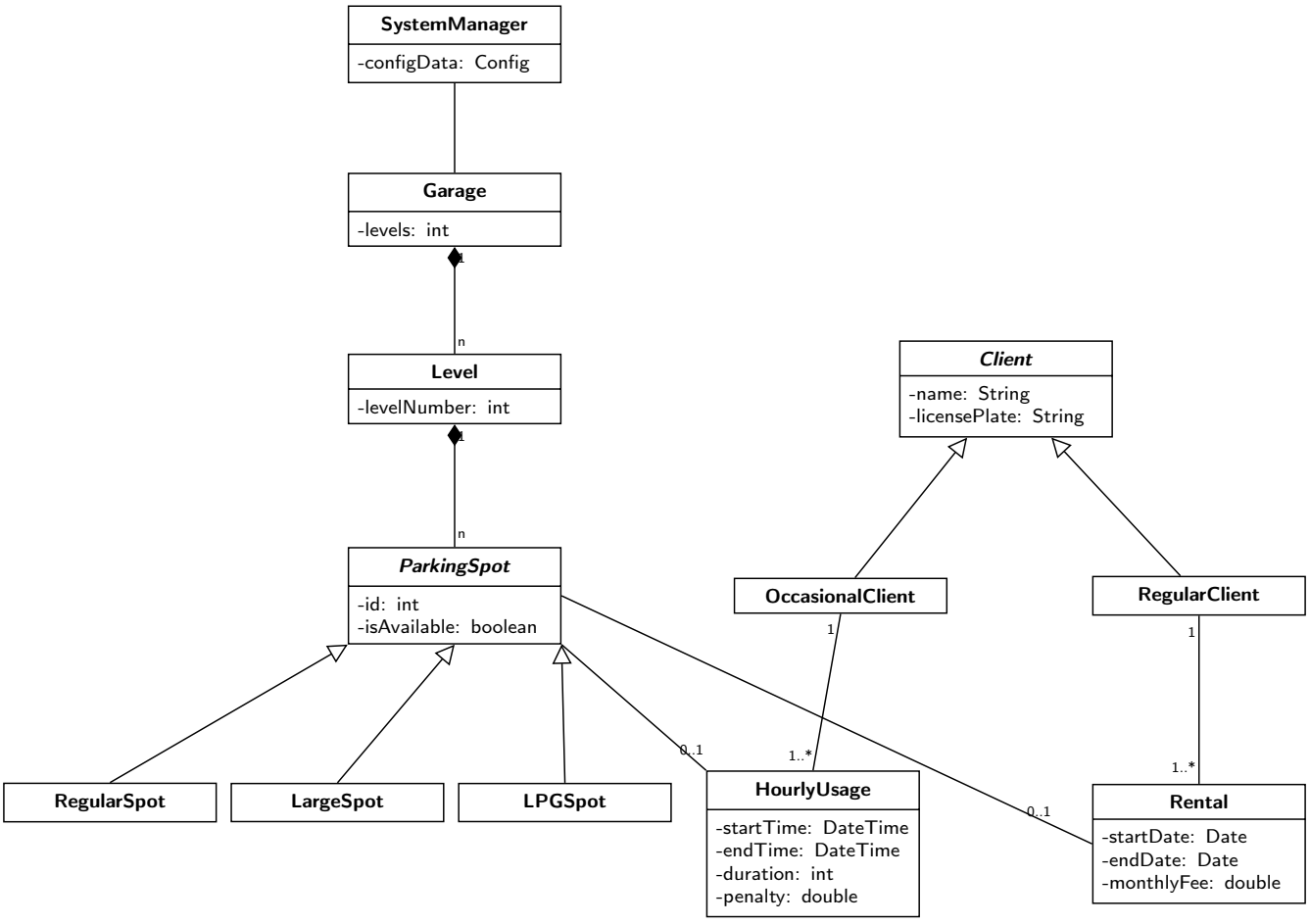


Figura 70: Garage – classi.

Diagramma delle classi.

Diagrammi di sequenza. Tre scenari: abbonamento mensile per auto GPL (ramo `alt`), abbonamento mensile per auto non-GPL (`loop` sui livelli + `alt` sulla regola del 50%), parcheggio orario.

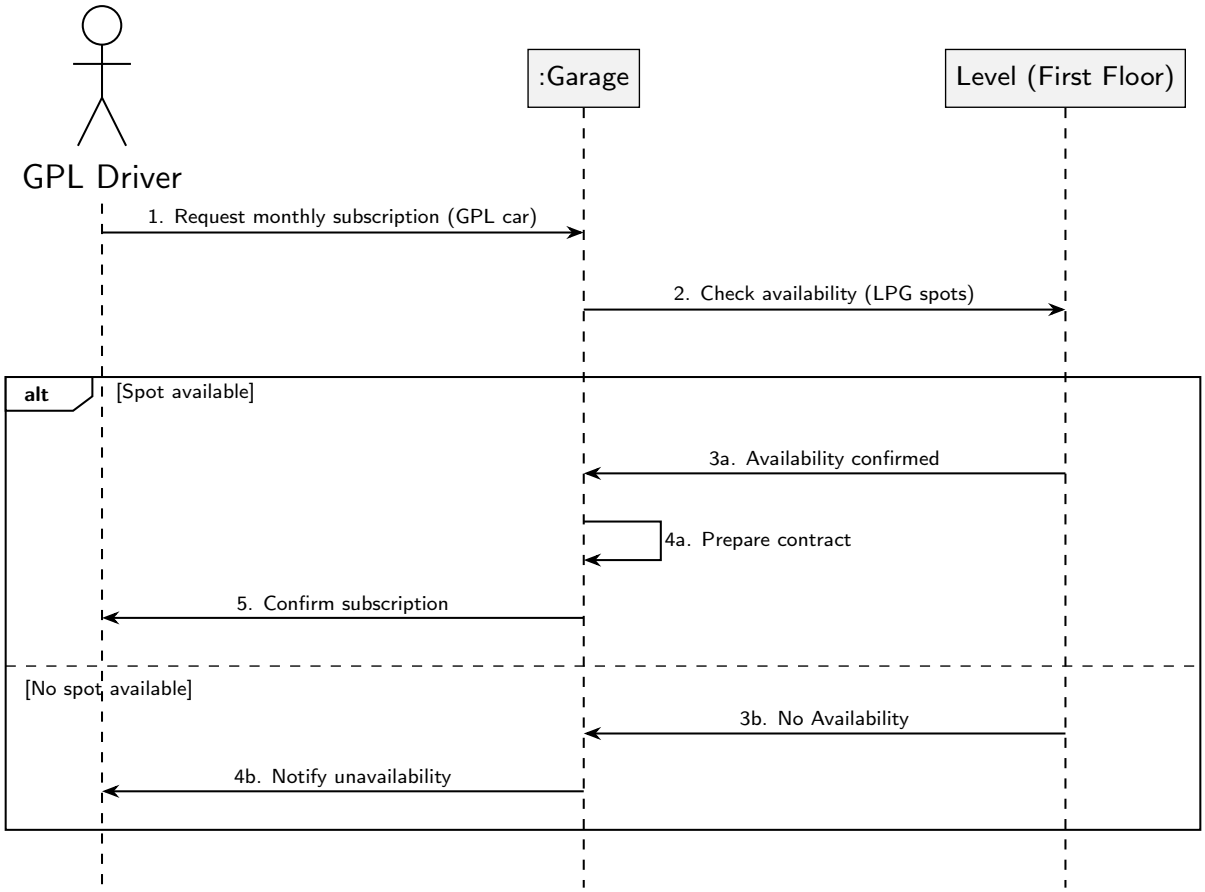


Figura 71: Garage – abbonamento mensile, auto GPL.

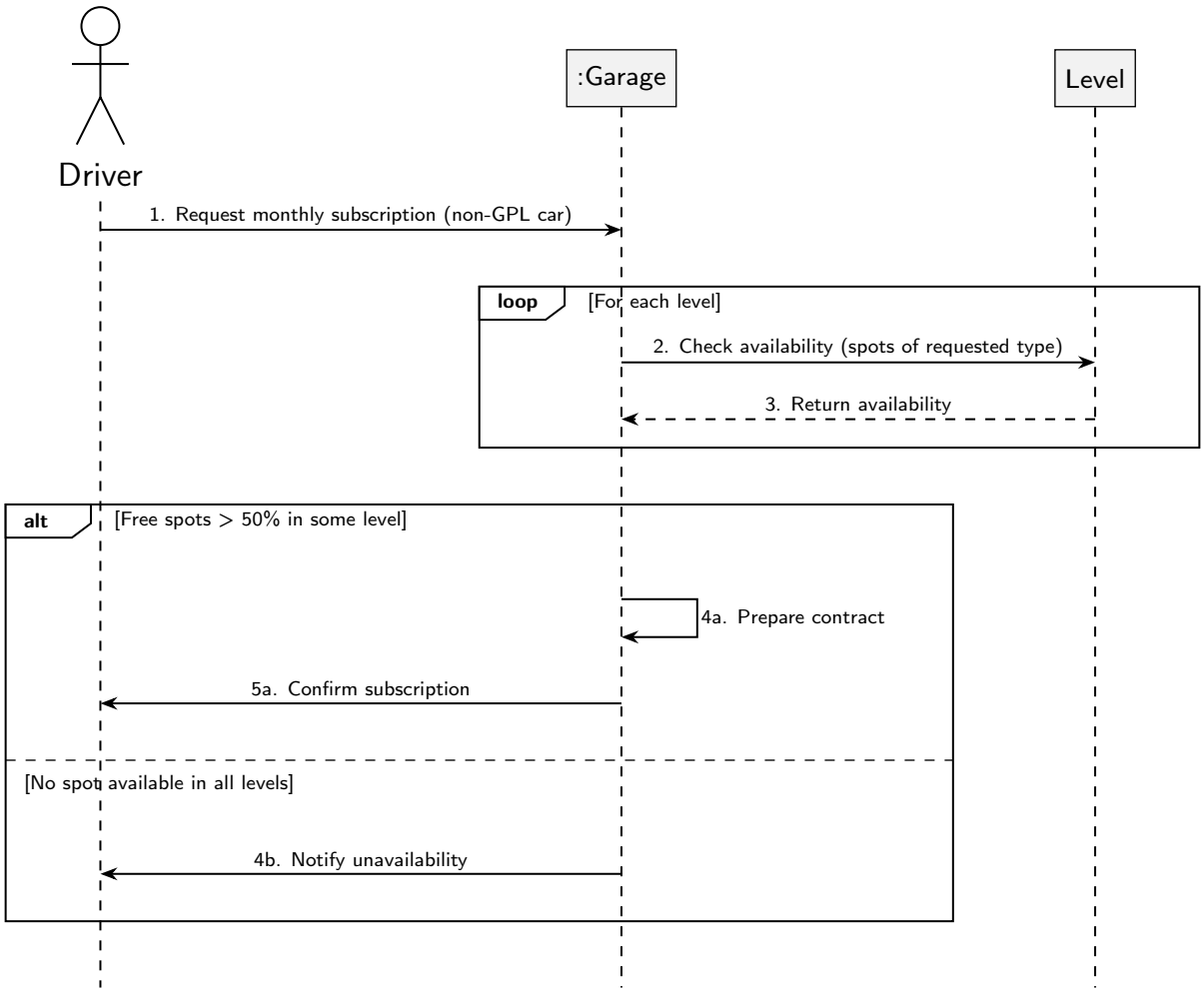


Figura 72: Garage – abbonamento mensile, auto non-GPL.

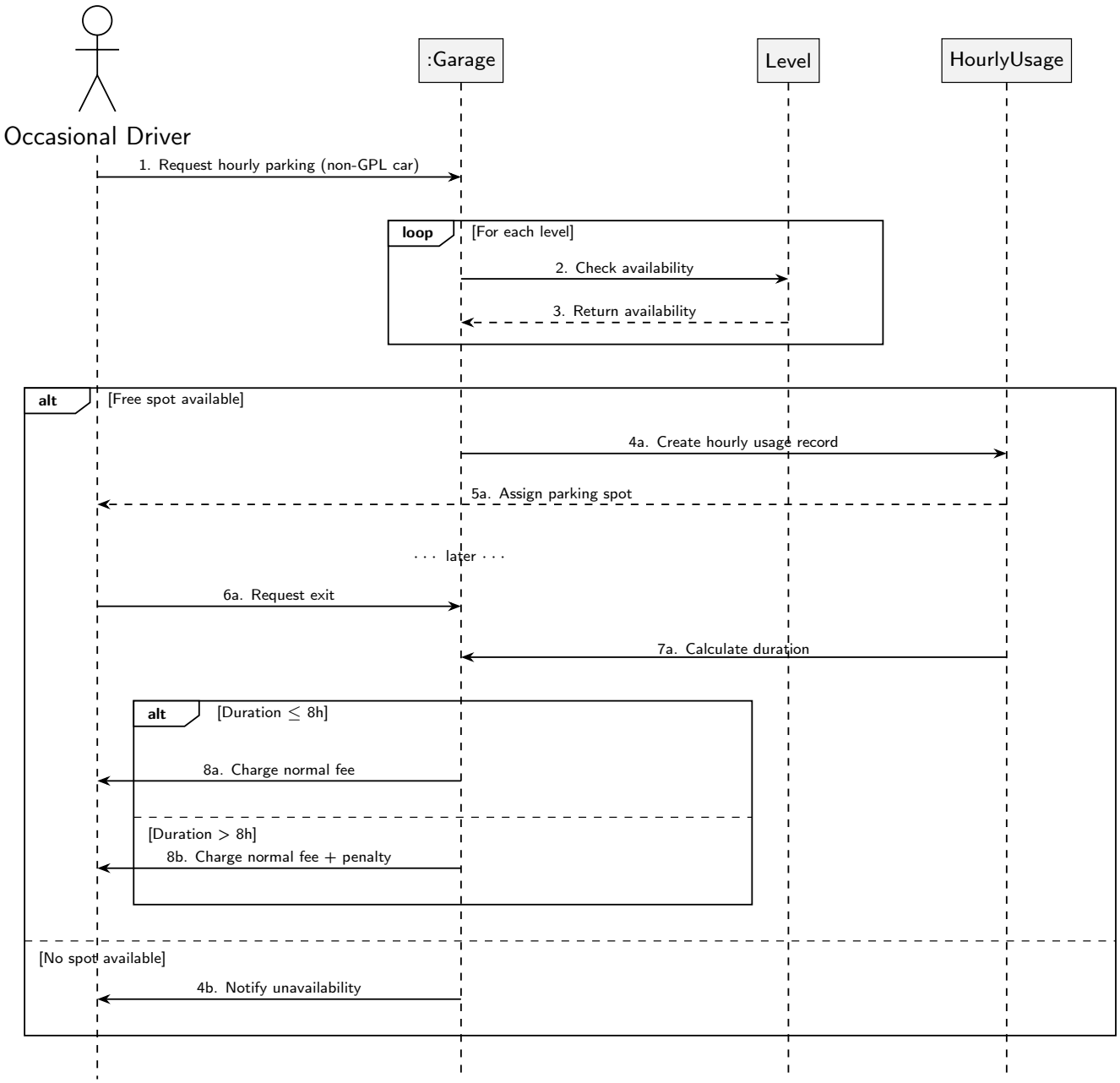


Figura 73: Garage – parcheggio orario (cliente occasionale).

Statechart di un posto auto (proposta). *La slide richiede anche lo statechart ma non lo mostra: proposta ricostruita per* `ParkingSpot`*.*

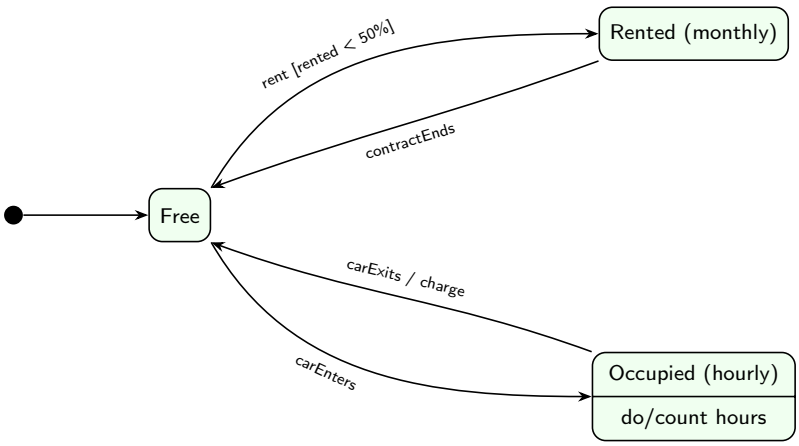


Figura 74: Proposta: statechart di `ParkingSpot`.

4.11 Esercizi su Componenti e Deployment

Esercizio: Online Bookstore (componenti). Testo: *Una software house costruisce un sistema di libreria online con tre componenti principali:* ***`Catalog Service`*** *(libri, categorie, disponibilità),* ***`Shopping Cart Service`*** *(aggiunta/rimozione libri, stato del carrello) e* ***`Payment Service`*** *(pagamenti tramite gateway esterni). Una* ***`Web UI`*** *dipende da Catalog e Shopping Cart. Lo Shopping Cart dipende dal Payment Service. Il Catalog usa un componente* ***`Book Database`****.*

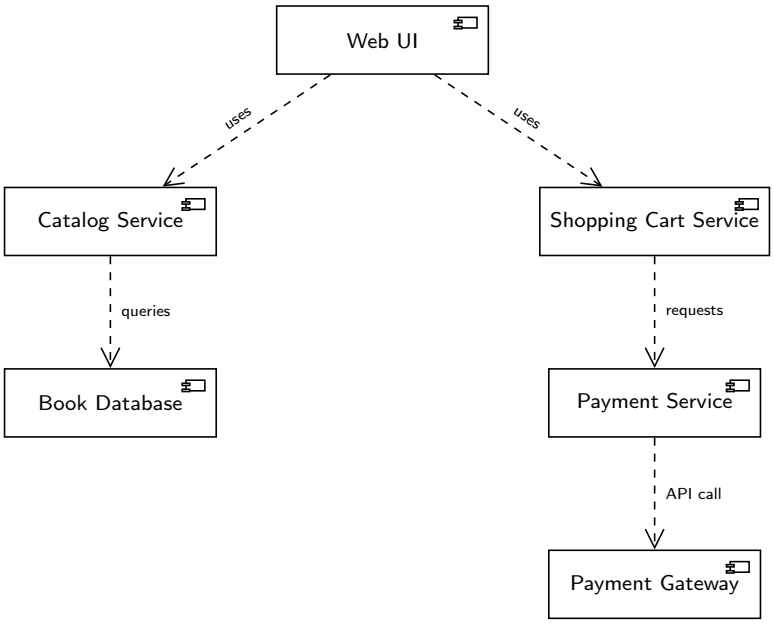


Figura 75: Soluzione: diagramma dei componenti (frecce tratteggiate = dipendenze).

Esercizio: piattaforma d'esame Unibo (deployment). Testo: *Unibo vuole una piattaforma d'esame online: Web Application, Database Server e Authentication Service. La Web Application gira su un Application Server che comunica con Database Server e Authentication Service. Studenti e docenti accedono da PC e dispositivi mobili via Internet. Il Database Server è su una macchina fisica separata, l'Authentication Service è ospitato all'esterno (es. Google OAuth). Mostrare dove sono distribuiti gli artefatti (ExamApp.war, ExamDB, AuthAPI) sui nodi.*

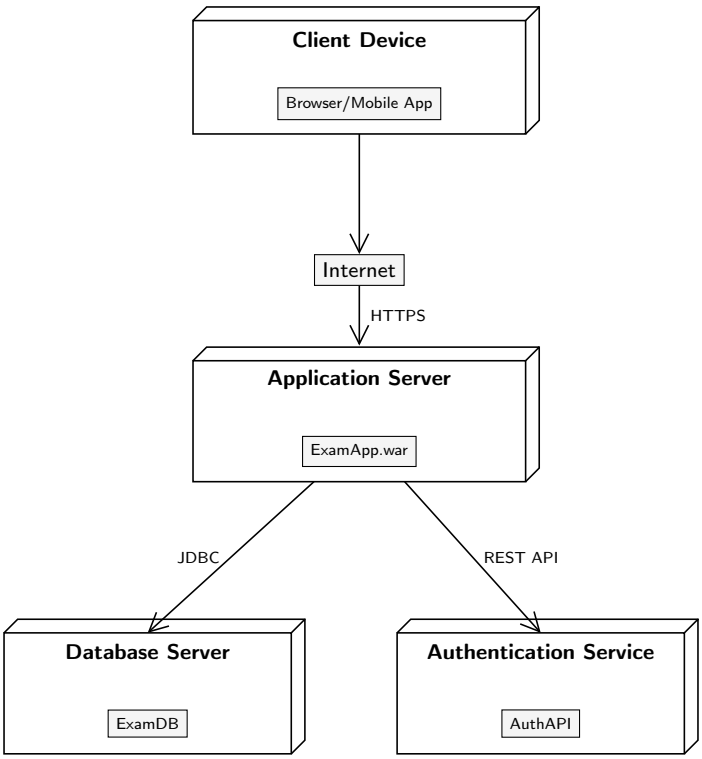


Figura 76: Soluzione: diagramma di deployment.

5 Processi

5.1 Introduzione a Scrum

Perché i piani tradizionali falliscono nel software. Nei progetti ingegneristici tradizionali (come l'edilizia) si può pianificare tutto in anticipo perché i vincoli fisici e i materiali sono noti e stabili. Nello sviluppo software moderno, invece, la maggior parte dei problemi è **complessa**:

- La soluzione ottimale non è chiara all'inizio;
- I requisiti cambiano inevitabilmente durante il tragitto (cambia il mercato, cambiano le idee del cliente);
- Il lavoro richiede il contributo coordinato di molte figure specialistiche.

Tentare di congelare requisiti e scadenze a lungo termine porta al fallimento. L'approccio vincente consiste nello scomporre il lavoro in **incrementi successivi**, collaborando a stretto contatto con gli utenti finali per raccogliere feedback continui.

Il Manifesto Agile (2001)

Nel 2001, diciassette pionieri del software sintetizzano questo cambio di rotta nel **Manifesto Agile**, stabilendo quattro contrapposizioni fondamentali:

1. **Individui e interazioni** più dei processi e degli strumenti;
2. **Software funzionante** più della documentazione esaustiva;
3. **Collaborazione con il cliente** più della negoziazione contrattuale;
4. **Rispondere al cambiamento** più del seguire un piano prefissato.

Attenzione: gli elementi a destra conservano valore, ma gli elementi a sinistra hanno priorità assoluta.

Dalla Staffetta al Rugby: l'origine del termine Scrum. L'approccio convenzionale a cascata (*Waterfall*) ragiona come una **staffetta atletica**: una fase passa il testimone alla successiva in rigida sequenza temporale (Requisiti → Design → Codifica → Collaudo). Se i requisiti iniziali contenevano errori, l'intero sistema fallisce alla consegna.

Nel celebre articolo del 1986 (*The New New Product Development Game*), Hirotaka Takeuchi e Ikujiro Nonaka propongono invece una metafora sportiva alternativa: il **rugby**.

- La squadra si muove e avanza sul campo come una **singola unità compatta**;
- I giocatori si passano la palla avanti e indietro continuamente per superare gli ostacoli;
- Il termine **Scrum** (la mischia ordinata del rugby) descrive proprio il gruppo coeso che lavora insieme per far ripartire il gioco.

In Scrum, quindi, non si affronta una fase per volta: **in ogni singolo ciclo di lavoro (Sprint) il team fa contemporaneamente requisiti, design, codice e test.**

Cos'è Scrum (e cosa non è).

- **È un framework, non un processo**: Scrum non dice passo-passo quali righe di codice scrivere, né impone una sequenza rigida di diagrammi. Fornisce un contenitore strutturato e minimale di regole e ruoli dentro cui inserire le pratiche tecniche più adatte (come TDD, Continuous Integration o Pair Programming mutuate da XP).
- **Uno specchio diagnostico**: come ha sintetizzato il suo co-fondatore Ken Schwaber:

  «Scrum non ti porta all'eccellenza, ma ti mostra il tuo livello di inefficienza.»

  Scrum non risolve magicamente i problemi tecnici, ma toglie gli alibi e rende immediatamente visibili i colli di bottiglia organizzativi e comunicativi.
- **Team al centro vs gerarchie piramidali**: nei modelli tradizionali le decisioni scendono dall'alto in modo burocratico (CEO → Project Manager → Analisti → Programmatori). In Scrum il processo decisionale è distribuito: chi realizza il software decide in autonomia come organizzarlo.

La Metafora di Galline e Maiali (*Pigs vs Chickens*)

Per chiarire il livello di responsabilità e appartenenza al progetto, la tradizione Scrum ricorre alla storiella del ristorante:

> *Una gallina propone a un maiale: «Perché non apriamo un ristorante insieme? Potremmo chiamarlo Uova e Pancetta!». Il maiale ci pensa e risponde: «No grazie: per preparare uova e pancetta tu saresti solo **coinvolta**, mentre io sarei completamente **impegnato** (ci rimetto la pelle!)».*

Da questa metafora discende una netta distinzione nei ruoli di progetto:

- **Ruoli Pig (I Veri Impegnati)**: hanno la responsabilità diretta dell'esito del progetto e sono i soli membri operativi dello Scrum Team (**Product Owner**, **Scrum Master**, **Development Team**). Possono intervenire e votare nelle riunioni operative.
- **Ruoli Chicken (I Coinvolti / Interessati)**: sono gli stakeholder esterni (utenti finali, clienti, management aziendale, esperti di dominio, auditor). Hanno interesse nel successo del prodotto ma non compiono il lavoro quotidiano nello sprint.

Lo Scrum Team: Dimensioni e i Tre Ruoli Chiave. Uno Scrum Team tipico è composto da un numero ridotto di persone (generalmente $\mathbf{5 \pm 2}$, o comunque tra 3 e 9 membri). Non esistono gerarchie interne: l'intero gruppo è egualmente responsabile del risultato finale. Le responsabilità sono distribuite su tre ruoli distinti:

1. Product Owner (PO) – La Voce del Cliente e del Valore. Il Product Owner rappresenta gli interessi del committente e degli utenti finali:

- Definisce la visione e le caratteristiche funzionali del prodotto;
- Identifica e massimizza il valore economico e pratico per l'utente finale;
- Gestisce e ordina per priorità gli elementi del *Product Backlog*;
- Decide le date e i contenuti dei rilasci;
- Accetta o rifiuta i risultati del lavoro al termine dello sprint, verificando il rispetto dei requisiti;
- Fa da ponte tra il team tecnico e gli stakeholder esterni.

2. Scrum Master (SM) – Il Facilitatore e Difensore del Team. Lo Scrum Master non è un capo progetto o un controllore gerarchico (non assegna task né impartisce ordini):

- Supporta il Development Team e il Product Owner nell'applicazione quotidiana di Scrum;
- Facilita i meeting e le cerimonie di processo;
- Rimuove attivamente gli ostacoli operativi (*impediments*) che bloccano gli sviluppatori;
- Protegge il team dalle interruzioni e dalle pressioni esterne del management aziendale;
- Aiuta l'organizzazione a comprendere e adottare la cultura agile.

3. Development Team (DT) – I Costruttori della Soluzione. Il gruppo degli sviluppatori realizza concretamente il software:

- **Auto-organizzato**: decide autonomamente *come* raggiungere l'obiettivo concordato con il PO, ripartendosi i compiti internamente su base volontaria;
- **Cross-funzionale**: possiede al proprio interno tutte le competenze necessarie a consegnare un incremento completo (analisi, architettura, scrittura codice, test, grafica/UI);
- **Competenze "a T" (*T-shaped*)**: ciascun componente ha una specializzazione primaria, ma possiede conoscenze trasversali sufficienti per affiancare e aiutare i colleghi in caso di necessità;
- **Dedizione a tempo pieno**: lavora in modo continuativo e preferibilmente colocalizzato per minimizzare gli attriti di comunicazione.

I Tre Pilastri del Controllo Empirico

Scrum non poggia su previsioni a tavolino, bensì sul **controllo empirico di processo** (apprendere dall'esperienza e dai dati osservati). Tale meccanismo si regge su tre pilastri:

1. **Trasparenza (*Transparency*)**: tutti gli aspetti cruciali del lavoro devono essere visibili e chiari a chi ne risponde (i *pigs*). Richiede l'adozione di un linguaggio comune e condiviso, a partire dalla definizione univoca di cosa si intende per lavoro completato (*Definition of Done*).

2. **Ispezione (*Inspection*)**: il software e gli artefatti vengono esaminati con cadenza regolare per intercettare per tempo scostamenti rispetto all'obiettivo dello sprint. L'ispezione non deve trasformarsi in una pesantezza burocratica che rallenta lo sviluppo.

3. **Adattamento (*Adaptation*)**: se l'ispezione evidenzia anomalie, difetti o derive nei processi o nel prodotto, il team corregge la rotta il più velocemente possibile per limitare danni ulteriori.

I Cinque Valori Fondamentali di Scrum. I pilastri empirici funzionano solo se sostenuti da una cultura etica e professionale condivisa da tutti i membri del gruppo:

- **Focus (Concentrazione)**: il team dedica tutte le proprie energie esclusivamente sugli obiettivi dello sprint in corso e sui task concordati, evitando dispersioni e cambi di rotta improvvisi;
- **Coraggio (*Courage*)**: il coraggio dello Scrum Master nel proteggere il gruppo dai manager; il coraggio del Product Owner nel fidarsi del team e negoziare con il cliente; il coraggio degli sviluppatori nell'affrontare sfide tecniche complesse e nel segnalare apertamente problemi e blocchi;
- **Apertura (*Openness*)**: trasparenza totale sullo stato di avanzamento, disponibilità ad accogliere i cambiamenti esterni e curiosità verso nuove soluzioni tecniche per migliorare la qualità del lavoro;
- **Impegno (*Commitment*)**: ciascuno si assume la responsabilità morale di dare il massimo per raggiungere l'obiettivo comune concordato nello sprint planning;
- **Rispetto (*Respect*)**: stima e fiducia reciproca tra colleghi, riconoscendo la professionalità e l'indipendenza di ciascuno.

La Prima Direttiva delle Retrospettive. Il rispetto reciproco trova la sua massima espressione nel principio enunciato da Norman Kerth, fondamentale per evitare la ricerca di capri espiatori durante le sessioni di analisi degli errori:

> *«A prescindere da cosa scopriremo, comprendiamo e crediamo fermamente che ciascuno abbia svolto il miglior lavoro possibile, date le conoscenze di cui disponeva in quel momento, le sue capacità e abilità, le risorse a disposizione e la situazione contingente.»*

Il Ciclo di Vita del Processo: Visione d'Insieme. Il framework Scrum trasforma la visione del prodotto in codice funzionante attraverso un ciclo iterativo regolare e ripetibile. Il percorso tipico si articola attraverso una sequenza definita di passaggi:

1. **Vision e Requisiti**: gli input provenienti da committenti, utenti e mercato vengono convogliati verso il Product Owner, che li sintetizza nel **Product Backlog**.
2. **Pianificazione dell'Iterazione**: all'inizio di ogni ciclo il team si riunisce nello **Sprint Planning** per concordare l'obiettivo (**Sprint Goal**) e selezionare gli elementi prioritari da realizzare (**Sprint Backlog**).

3. **Esecuzione dello Sprint**: per un periodo fisso compreso tra 1 e 4 settimane il team realizza le funzionalità, sincronizzandosi quotidianamente nel **Daily Scrum** e monitorando l'avanzamento.

4. **Ispezione del Prodotto**: al termine dell'iterazione si tiene la **Sprint Review**, una sessione dimostrativa con gli stakeholder per validare l'incremento software rilasciabile.

5. **Miglioramento del Processo**: il ciclo si chiude con la **Sprint Retrospective**, dedicata ad analizzare cosa ha funzionato e come ottimizzare il modo di lavorare del gruppo nello sprint successivo.

Artefatto: Il Product Backlog. Il **Product Backlog** è l'elenco ordinato, dinamico e in continua evoluzione di tutto ciò che potrebbe essere necessario all'interno del prodotto. È gestito e ordinato esclusivamente dal Product Owner in base al valore di business. I singoli elementi che lo compongono prendono il nome di **PBI** (*Product Backlog Items*) e si dividono in quattro macro-categorie:

- **Funzionalità (Features / User Stories)**: nuove capacità operative richieste dagli utenti o dal cliente;
- **Difetti (Defects / Bug)**: correzioni di anomalie o malfunzionamenti riscontrati nelle versioni precedenti;
- **Lavoro Tecnico (Technical Work)**: interventi su architettura, infrastruttura server, migrazioni tecnologiche o refactoring;
- **Acquisizione di Conoscenza (Knowledge Acquisition / Spike)**: attività di ricerca esplorativa, proof-of-concept o studio di fattibilità per chiarire requisiti ancora ambigui.

Lo Sprint: Il Motore di Scrum. Lo **Sprint** è l'unità di base dello sviluppo in Scrum. Possiede proprietà operative imprescindibili:

- **Durata Costante (*Timebox*)**: dura da 1 a 4 settimane (la convenzione industriale più diffusa è di 2 settimane). La durata non varia mai in corsa: mantenere un ritmo costante migliora la prevedibilità e facilita la sincronizzazione del team;
- **Tutte le attività in ogni ciclo**: non esiste uno sprint dedicato solo al design o uno solo ai test; ogni iterazione include analisi, progettazione, codifica e collaudo;
- **Obiettivo di Rilascio**: ogni sprint deve produrre un **incremento di prodotto potenzialmente rilasciabile** (*Potentially Shippable Product Increment* o MVP), ovvero software funzionante, testato e pronto per andare in produzione.

La Regola del "No Change during Sprint". Una volta che lo sprint è stato pianificato e avviato, **l'ambito e gli obiettivi dello sprint backlog non possono essere modificati da richieste esterne**. Se durante le due settimane emergono nuove idee, richieste del cliente o urgenze del management, queste non entrano nello sprint in corso: il Product Owner le inserisce nel Product Backlog generale, valutando se ordinarle con alta priorità per lo sprint successivo. È compito primario dello Scrum Master proteggere il Development Team da interruzioni e cambi di rotta improvvisi, garantendo la concentrazione necessaria a portare a termine quanto promesso.

Il Rituale dello Sprint Planning. Ogni sprint si apre con una riunione di pianificazione a cui partecipano obbligatoriamente PO, Scrum Master e Development Team. La durata indicativa è di **circa un'ora per ciascuna settimana di sprint** (es. 2 ore per uno sprint bisettimanale). L'incontro si articola in due fasi logiche distinte:

1. **Prima Parte: Cosa fare? (*What?*)**
    - Il PO illustra lo stato attuale del prodotto e presenta gli elementi in cima al Product Backlog;
    - Insieme al team si definisce lo **Sprint Goal**, ovvero il senso di business e l'obiettivo generale dell'iterazione;
    - Valutando la propria capacità produttiva (*Capacity* / *Velocity*), il team seleziona le storie che si impegna a completare.
2. **Seconda Parte: Come farlo? (*How?*)**
    - Il solo Development Team analizza le storie selezionate e stabilisce l'architettura tecnica e il design della soluzione;
    - Ogni storia viene scomposta in compiti tecnici elementari (**task** di sviluppo, test, configurazione);
    - I task vengono stimati in ore lavorative (indicativamente da poche ore a 1–2 giorni ciascuno) formando lo **Sprint Backlog**.

Artefatto: Lo Sprint Goal. Lo **Sprint Goal** è una breve dichiarazione che sintetizza l'impatto o il valore pratico che lo sprint deve portare. Fornisce al team una direzione comune e flessibilità d'azione qualora l'implementazione dettagliata richieda aggiustamenti tecnici durante la lavorazione.

- *Esempio gestionale*: «Rendere l'applicazione compatibile anche con database Microsoft SQL Server oltre al supporto attuale per Oracle»;
- *Esempio finanziario*: «Supportare il calcolo in streaming real-time degli indicatori tecnici di borsa per pareggiare le funzionalità del concorrente X».

Criteri di Qualità: Definition of Ready (DoR) vs Definition of Done (DoD)

Per evitare fraintendimenti tra committente e sviluppatori, Scrum separa nettamente la condizione di ingresso di un'attività dalla sua condizione di uscita:

- **Definition of Ready (DoR) – Criterio di Ingresso**: stabilisce quando una storia è sufficientemente matura per essere ammessa nello Sprint Planning. Una storia è *Ready* se:
    1. È compresa chiaramente da tutti i membri del gruppo;
    2. È sufficientemente dettagliata e provvista di criteri di accettazione testabili;
    3. Il valore per l'utente finale è chiaro al PO per poterne fissare la priorità;
    4. Il team è in grado di stimarne l'effort e completarla entro un singolo sprint;

5. È indipendente da blocchi esterni o decisioni ancora in sospeso.

- **Definition of Done (DoD) – Criterio di Uscita**: è il contratto formale condiviso che definisce quando una storia può considerarsi **veramente finita**. Evita l'anti-pattern del lavoro "quasi fatto" (*Almost Done*) che accumula debito invisibile. Una tipica checklist di DoD prevede:

  1. Codifica della funzionalità completata e revisionata;
  2. Test di unità (*Unit Tests*) scritti, eseguiti e superati con successo;
  3. Test di integrazione superati senza regressioni;
  4. Eventuali test prestazionali o di sicurezza verificati;
  5. Documentazione tecnica essenziale e note di rilascio aggiornate.

Gestione Visuale: Taskboard e Flusso di Lavoro. Durante lo svolgimento dello sprint, il team monitora visivamente i task su un tabellone murale o digitale denominato **Taskboard** (o *Scrumboard*). Il board è tipicamente suddiviso in colonne che tracciano lo stato di avanzamento:

Story $\longrightarrow$ **To Do** $\longrightarrow$ **In Progress** $\longrightarrow$ **Review / Test** $\longrightarrow$ **Done**

Il funzionamento del flusso si basa su precise convenzioni agili:

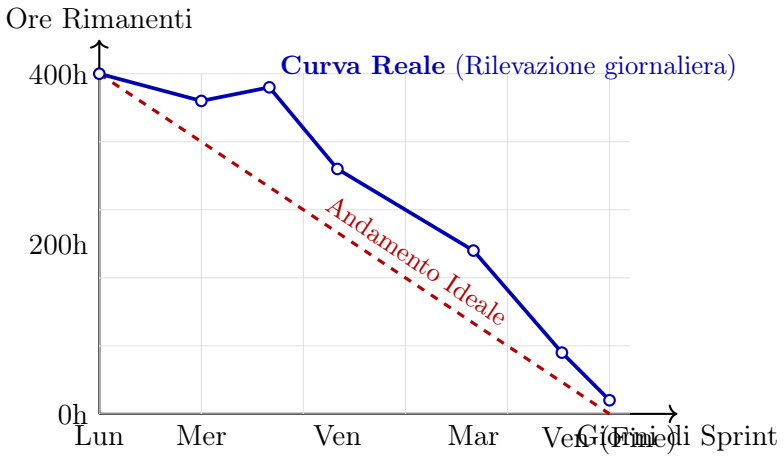
- **Logica "Pull" (su base volontaria)**: il lavoro non viene assegnato da un coordinatore o dal Product Owner. Ciascun membro del team preleva autonomamente il compito successivo dalla colonna *To Do* spostandolo in *In Progress*;
- **Il lavoro emerge in itinere**: se durante lo sviluppo ci si accorge che un compito è più complesso del previsto, il team può ridefinirlo o scomporlo inserendo nuovi cartellini nello Sprint Backlog;
- **Aggiornamento quotidiano dell'effort**: alla fine di ogni giornata lavorativa, gli sviluppatori aggiornano la stima delle ore rimanenti per ciascun task attivo, permettendo di ricalcolare l'andamento reale dello sprint;
- **Integrazione con Kanban**: molti team Scrum applicano principi Kanban sul board, limitando il numero massimo di cartellini ammessi contemporaneamente nella colonna *In Progress* (*Work In Progress limits* o WIP limits) per evitare colli di bottiglia e favorire la conclusione rapida delle storie prima di iniziarne di nuove.

Il Rituale del Daily Scrum (o Stand-up Meeting). Il **Daily Scrum** è l'incontro quotidiano di coordinamento del team di sviluppo. Per conservare dinamismo ed efficacia, adotta regole stringenti:

- **Durata massima di 15 minuti**: si svolge preferibilmente sempre alla stessa ora e nello stesso luogo, rigorosamente **in piedi** (*stand-up*) per evitare che la discussione si prolunghi;
- **Nessun problem-solving**: non serve a risolvere problemi tecnici sul momento né a progettare soluzioni (queste attività vengono affrontate subito dopo, solo dalle persone direttamente interessate);

- **Aperto a tutti, ma parlano solo i "pigs"**: chiunque nell'organizzazione può assistere come osservatore silenzioso, ma hanno diritto di parola esclusivamente i membri operativi (*Development Team*, *Scrum Master*, *Product Owner*);
- **Non è un report per lo Scrum Master**: non si tratta di una rendicontazione gerarchica al capo, bensì di un momento di sincronizzazione orizzontale in cui ogni membro assume un impegno pubblico verso i colleghi rispondendo a tre domande:
  1. *Che cosa ho fatto ieri per aiutare il team a raggiungere lo Sprint Goal?*
  2. *Che cosa farò oggi?*
  3. *Quali ostacoli o impedimenti rallentano il mio lavoro?*

Artefatto: Il Burndown Chart e la Team Velocity. Il **Burndown Chart** è lo strumento visuale principe per monitorare in tempo reale l'avanzamento dello sprint. Mette in relazione l'**effort residuo da completare** (in ore o in story points) con il **tempo trascorso** (giorni lavorativi):



- **Linea Ideale**: congiunge lo sforzo iniziale allo zero atteso per l'ultimo giorno;
- **Curva Reale**: aggiornata quotidianamente dopo il Daily Scrum. Se staziona sopra la retta ideale il team è in ritardo, se sotto è in anticipo;
- **Team Velocity**: misura empirica calcolata come la media degli Story Points completati e conformi alla DoD per iterazione.

Comportamenti Tipici del Burndown Chart

L'andamento della curva reale riflette fedelmente le dinamiche operative del team:

- **Piatto nei primi giorni**: naturale periodo di studio architetturale e impostazione delle strutture dati;
- **Risalita improvvisa**: scoperta di complessità impreviste o inserimento di nuovi task tecnici non stimati nello sprint planning;

- **Crollo a scalino finale**: rilascio concentrato di più storie completate insieme solo verso gli ultimi giorni dello sprint;
- **Andamento asintotico / Mancato azzeramento**: stime iniziali eccessivamente ottimistiche o presenza di colli di bottiglia nei test di collaudo.

Artefatto: Potentially Shippable Product Increment. Al termine di ogni singolo sprint il risultato tangibile deve consistere in un **incremento di prodotto potenzialmente rilasciabile**:

- Comprende la somma di tutti gli elementi del backlog completati nello sprint corrente uniti a quelli delle iterazioni precedenti;
- È perfettamente compilato, integrato, privo di bug bloccanti e conforme al 100% alla *Definition of Done*;
- Si definisce "potenzialmente rilasciabile" perché spetta al Product Owner decidere se consegnarlo subito agli utenti finali o attendere l'accumulo di ulteriori incrementi, ma da un punto di vista puramente tecnico il software deve essere già pronto per il rilascio.

Il Rituale della Sprint Review (Ispezione del Prodotto). La **Sprint Review** è l'incontro formale di chiusura dello sprint dedicato all'ispezione dell'incremento software appena realizzato:

- **Durata e perimetro**: dura al massimo 3 ore (circa 60–90 minuti per sprint bisettimanali). È condotta dallo Scrum Team al completo (*Dev Team*, *SM*, *PO*); le slide del corso evidenziano inoltre la possibile estensione ad altri stakeholder, committenti e utenti finali interessati al processo;
- **Dimostrazione dal vivo (No Slides!)**: il team non presenta documenti teorici o diapositive riassuntive, bensì mostra il **software reale funzionante** attraverso una demo interattiva delle nuove funzionalità;
- **Verifica del Backlog**: il PO formalizza quali storie risultano completate con successo e quali eventualmente non hanno soddisfatto la DoD;
- **Feedback immediato**: la discussione serve a raccogliere le reazioni del cliente, sulla base delle quali il Product Owner adatta e riordina le priorità del Product Backlog per i cicli successivi.

Il Rituale della Sprint Retrospective (Miglioramento del Processo). A differenza della Review (focalizzata sul *prodotto*), la **Sprint Retrospective** è una sessione interna focalizzata esclusivamente sul **processo e sulle dinamiche del team**:

- **Durata e perimetro**: dura al massimo 3 ore per uno sprint di un mese (circa 60–90 minuti per sprint bisettimanali) e coinvolge strettamente i membri dello Scrum Team (*Development Team*, *Scrum Master*, *Product Owner*);
- **Oggetto di analisi**: esamina criticamente come si sono svolte le attività relativamente a **persone, relazioni interpersonali, metodologie e strumenti tecnici**;

- **Formato operativo (Start / Stop / Continue)**: il gruppo si confronta apertamente per identificare:
  1. **Start**: quali nuove pratiche o strumenti dobbiamo iniziare ad adottare per lavorare meglio?
  2. **Stop**: quali abitudini inefficienti o blocchi dobbiamo smettere di praticare?
  3. **Continue**: quali comportamenti si sono rivelati vincenti e vanno confermati?
- **Piano di miglioramento**: la riunione deve obbligatoriamente produrre un piano di azione concreto (*Improvement Plan*), ovvero 1–2 compiti di miglioramento mirati da inserire nello Sprint Backlog dell'iterazione successiva.

Artefatto e Attività: Product Backlog Refinement. Il Product Backlog non è un documento statico, ma evolve con l'esperienza acquisita dal team. Il **Backlog Refinement** (in passato denominato anche *Grooming*) è l'attività continuativa in cui il Product Owner e il Development Team collaborano per manutenere l'elenco dei requisiti:

- **Cosa si fa**: si scompongono le User Story troppo ampie (*Epic*) in storie più piccole, si aggiornano le stime di effort, si chiariscono i criteri di accettazione e si riordinano le priorità;
- **Impegno temporale**: non è una cerimonia formale a sé stante con durata fissa, ma un'attività regolare che non dovrebbe assorbire più del **10% del tempo complessivo** del team durante lo sprint;
- **Regola fondamentale**: il Product Owner ha piena facoltà di modificare, aggiungere o eliminare elementi dal Product Backlog in qualsiasi istante, purché non alteri i task dello sprint attualmente in corso.

5.2 User Stories

Anatomia di una User Story: Il Modello Standard (Mike Cohn / Connextra). Una **User Story** è una descrizione sintetica e informale di una caratteristica o funzionalità del sistema, formulata nella prospettiva della persona che desidera tale capacità. Non intende sostituire una specifica tecnica completa, ma agisce come un impegno a dialogare (*promise for a conversation*). La formulazione canonica, standardizzata da Mike Cohn e derivata dalle prassi di Connextra ed Extreme Programming, rispetta una precisa struttura tripartita:

`As a <user type / role>  I want <action / goal>  so that <business value>`

oppure, nella declinazione italiana:

`Come <ruolo>  voglio <funzione / scopo>  per <valore ottenuto>`

Ciascuna delle tre componenti assolve una funzione ingegneristica precisa:

1. **Ruolo (*As a ...*)**: identifica il tipo di utente o l'attore che beneficia della funzionalità. Evita l'errore sistematico di impiegare la dicitura generica e indistinta di "utente": specificare il ruolo (es. *utente della biblioteca*, *studente pendolare*, *amministratore contabile*) sviluppa empatia nel team di sviluppo e contestualizza bisogni, limitazioni e frequenza d'uso.

2. **Funzione o Scopo (*I want ...*)**: descrive l'azione che l'attore desidera compiere tramite il sistema software. Deve focalizzarsi sull'obiettivo dell'interazione e sull'intento comportamentale, astenendosi rigorosamente dall'indicare la tecnologia realizzativa o la veste grafica della soluzione.

3. **Valore Ottenuto (*So that ...*)**: costituisce la porzione concettualmente più rilevante e qualificata della storia. Esplicita la motivazione di business o il beneficio tangibile che l'attore riceve dal soddisfacimento della funzione. Permette al Product Owner e al team di giudicare l'effettiva utilità della richiesta: se non è possibile rintracciare un beneficio evidente, la storia non genera valore e deve essere espunta per scongiurare il cosiddetto fenomeno del *bloatware*.

Esempio di Riferimento: Sistema di Gestione Bibliotecaria

Si consideri il requisito di interrogazione dei testi in un ateneo universitario:

- **Ruolo**: *Come utente della biblioteca (studente o docente)*
- **Scopo / Funzione**: *Voglio consultare il catalogo dei testi disponibili online*
- **Valore**: *Per prendere a prestito un volume senza recarmi inutilmente presso la sede fisica*

La storia definisce chiaramente il chi, il cosa e il perché, lasciando ai tecnici la scelta implementativa migliore (motore di ricerca REST, interfaccia responsive, caching dei risultati).

Separazione Rigorosa: User Stories vs Task Tecnici. Uno dei malintesi più ricorrenti nei team alle prime armi con l'approccio agile consiste nel confondere le User Stories con i task di implementazione tecnica.

Regola Aurea di Non-Tecnicità delle User Stories

Le User Stories non sono e non devono mai essere espresse in termini tecnici.

- Una formulazione del tipo: *«Implementare la connessione al database PostgreSQL tramite JDBC»* oppure *«Configurare l'autenticazione mutua con certificati SSL»* **non è una User Story**.
- L'utente finale o il cliente non traggono alcun valore diretto dall'esistenza di una tabella relazionale o di una classe middleware: il loro interesse risiede nel registrare un ordine, prenotare una prestazione o visionare un saldo.
- Le decisioni tecniche sono conseguenze strumentali e appartengono ai **Task**, nei quali la storia viene scomposta dal team durante la fase di pianificazione esecutiva.

Quando una storia viene accolta nello sprint, il gruppo di sviluppo la scompone operativamente nell'insieme di task tecnici necessari a soddisfarla. Riprendendo la storia dell'iscrizione al portale di lavoro:

«Come utente del portale voglio iscrivermi inserendo il mio CV e lo stipendio desiderato...»

essa genera naturalmente una serie di compiti tecnici di dettaglio:

1. `Task 1`: Realizzare il flusso di autenticazione sicura federata con protocollo OAuth 2.0;
2. `Task 2`: Sviluppare il modulo di upload con validazione dei formati PDF/DOCX del CV e salvataggio su object storage;
3. `Task 3`: Implementare il servizio di scraping / web crawler per indicizzare le nuove offerte di lavoro dai siti partner;
4. `Task 4`: Progettare l'algoritmo di matching tra profilo del candidato e offerta retributiva;
5. `Task 5`: Configurare il broker di messaggistica asincrono per inviare le segnalazioni email o le notifiche push.

Partizionamento del Backlog: Slicing Orizzontale vs Verticale. Uno dei problemi più insidiosi nella scrittura delle User Stories e nella gestione del backlog risiede nella modalità con cui i requisiti vengono decomposti. La naturale inclinazione dei team di sviluppo, derivata dall'approccio ingegneristico tradizionale e dall'architettura a livelli (*layered architecture*), è quella di **partizionare il sistema in senso orizzontale**:

- Gli sviluppatori preferiscono strutturare il lavoro aggregando i compiti per strato tecnologico omogeneo: strato di presentazione/interfaccia grafica (UI), livello di autenticazione e sicurezza, logica di business e middleware, strato di persistenza e interfacciamento con database relazionali, protocolli di trasporto e reti di comunicazione.
- **Perché i tecnici prediligono lo slicing orizzontale**: viene percepito come maggiormente efficiente sul piano della concentrazione e della produttività locale (*local efficiency*), poiché consente a ciascuno specialista di rimanere nel proprio dominio di competenza (es. scrivere tutte le tabelle del database in una volta sola o disegnare tutte le maschere UI).
- **Il fallimento sul piano del valore**: un singolo strato orizzontale isolato non eroga alcun valore funzionante all'utente finale. Avere completato il 100% delle tabelle del database e dei servizi di trasporto non consente a un cliente di condurre una singola transazione. Di conseguenza, il rilascio slitta al termine di tutti gli strati, replicando esattamente i ritardi e l'assenza di feedback precoce del modello Waterfall.

La Metafora della Torta: "Slicing the Cake"

Per superare questa limitazione, l'approccio agile impone il partizionamento delle storie mediante **fette verticali** (*Vertical Slicing*):

- Tagliare una torta a strati orizzontali significa dare a un invitato solo la glassa, a un altro solo la crema e a un terzo solo il pan di spagna: nessuno di essi ha ricevuto una vera fetta di torta appetibile.
- La fetta verticale attraversa trasversalmente **tutti gli strati architetturali** (dalla UI al database), implementando il frammento minimo di ciascun livello necessario e

sufficiente a rendere una singola funzionalità concretamente utilizzabile ed erogabile al cliente.

La Definition of Done (DoD). Una problematica cruciale nei progetti software riguarda la definizione univoca di completamento: quando uno sviluppatore dichiara *«ho finito la storia»*, cosa intende realmente? Spesso intende unicamente di aver digitato l'ultima riga di codice sul proprio ambiente locale, ignorando se il codice compili senza warning, se esistano test a supporto, se la documentazione sia allineata o se la feature sia integrabile nella build principale.

Per sradicare qualsiasi ambiguità e allineare le aspettative del team, la metodologia agile formalizza la **Definition of Done (DoD)**:

Definizione Formale della Definition of Done

La **Definition of Done** rappresenta una serie condivisa di **criteri qualitativi oggettivi e verificabili** che stabiliscono se una User Story (o un qualsiasi Product Backlog Item) possa considerarsi autenticamente completata e di qualità rilasciabile (*releasable / acceptable quality*).

- Viene **stabilita dal team di sviluppo** al completo (concordata con il Product Owner);
- È convenzionalmente la **stessa per l'intero progetto**, garantendo uno standard omogeneo di qualità tra tutti i membri del gruppo;
- È un artefatto **dinamico ed evolutivo**: con il maturare del team e il perfezionamento della pipeline di integrazione continua, la DoD può essere inasprita durante le sessioni di retrospettiva per innalzare l'asticella della qualità.

Una tipica checklist di Definition of Done industriale include clausole quali:

- **Controllo di versione**: il codice della storia è stato committato, pushato su un branch dedicato e integrato nel branch principale tramite pull request approvata da almeno un pari (*peer review*);
- **Copertura dei test**: la suite di test unitari è stata aggiornata ed eseguita con successo, garantendo una copertura del codice (*code coverage*) pari ad almeno il 75%–80%;
- **Test funzionali**: tutti i criteri di accettazione formulati nella storia sono coperti da test automatici verdi;
- **Documentazione di codice**: tutte le interfacce, classi e metodi pubblici sono rigorosamente commentati secondo gli standard di progetto (es. Javadoc per Java, Pydoc per Python, JSDoc per TypeScript/JavaScript);
- **Standard di stile e qualità**: il codice rispetta le linee guida di stile (*coding style guidelines*) e supera senza violazioni i controlli degli analizzatori statici (linter, SonarQube);
- **Pipeline CI/CD**: la build automatica nel server di Continuous Integration è stata completata con successo (*green build*) e distribuita nell'ambiente di staging o collaudo.

Le User Stories come Frammenti di una Conversazione Continua. Uno degli assunti più profondi dell'ingegneria del software agile è che una User Story non costituisce un documento isolato o una specifica auto-consistente, bensì un **frammento di una conversazione ininterrotta**. La stesura della scheda rappresenta l'inizio del dialogo, non la sua conclusione.
Attorno a una medesima storia convergono le prospettive prismatiche e le esigenze conoscitive di tutte le figure professionali coinvolte nel progetto:

- **Utente finale**: *«Come posso descrivere chiaramente i miei desideri e le mie aspettative d'uso?»*
- **Stakeholder / Committente**: *«In che modo questa specifica funzionalità contribuirà al successo commerciale del prodotto o all'ottimizzazione del business?»*
- **Project Manager / Scrum Master**: *«Come posso tracciare, coordinare e pianificare questo compito all'interno dell'iterazione corrente per scongiurare colli di bottiglia?»*
- **Business Analyst**: *«Quali sono i dettagli fini, le regole di dominio, i vincoli normativi e le eccezioni che sottendono a questa feature?»*
- **UX Designer**: *«Quali sono i bisogni cognitivi, l'ergonomia e i flussi emotivi ed operativi dell'utente durante l'interazione?»*
- **Developer**: *«Quali task concreti di architettura, design e codifica devo eseguire ed orchestrare oggi per abilitare questo comportamento?»*
- **QA / Collaudatore**: *«Quali criteri e scenari di test devo predisporre per validare in modo inoppugnabile che il task completato rispetti le aspettative?»*

I Criteri di Qualità: Dagli Obiettivi S.M.A.R.T. all'Acronimo I.N.V.E.S.T. Proprio come nella letteratura esistono libri validi e libri mediocri, nell'ingegneria dei requisiti esistono storie efficaci e storie pessime. Per formulare obiettivi di progetto sensati si adotta storicamente la metrica aziendale degli obiettivi **S.M.A.R.T.**:

Specifici – **M**isurabili – **A**ccettati – **R**ealistici – **T**emporizzati

Tuttavia, per trasporre questi principi generali nella specificità delle User Stories agili, la letteratura di settore (formalizzata originariamente da Bill Wake e perfezionata da Mike Cohn) richiede di **investire** nella stesura dei requisiti adottando l'acronimo guida **I.N.V.E.S.T.**:

Il Modello I.N.V.E.S.T. per la Qualità delle User Stories

Independent **N**egotiable **V**aluable **E**stimable **S**ized right **T**estable

Analisi Dettagliata dei Sei Criteri I.N.V.E.S.T.:

1. **Independent (Indipendenti)**:
    - Le storie devono, per quanto possibile, risultare **ortogonali e disaccoppiate** l'una dall'altra.

- Se la storia B può essere implementata o rilasciata unicamente dopo aver concluso la storia A, la pianificazione diviene rigida, impedendo al Product Owner di riordinare liberamente il backlog in funzione del mutamento delle priorità di business.
- Se due storie presentano un legame intrinseco fortissimo, occorre accorparle in una storia singola oppure ristrutturarle separando la funzionalità base dalle sue estensioni opzionali.

2. **Negotiable (Negoziabili)**:
   - Le storie **non sono contratti legali scolpiti nella pietra**, né specifiche rigide da eseguire pedissequamente.
   - Descrivono un intento: i dettagli operativi, le modalità di implementazione e i confini precisi rimangono aperti alla continua contrattazione tra cliente e sviluppatori.
   - Se una storia risulta sovraccarica di minuzie tecniche e vincoli tassativi prima di essere discussa, la negoziabilità decade e si ricade nelle patologie dell'SRS; il team deve confrontarsi con il cliente per trovare il compromesso più economico ed efficace, preservando unicamente le limitazioni fisse imprescindibili.
3. **Valuable (Portatrici di Valore)**:
   - Ogni storia deve palesare in modo inequivocabile il **vantaggio o ritorno economico** per il cliente, per l'utente finale o per gli stakeholder di progetto.
   - Questa proprietà agisce come barriera contro l'infiltrazione di storie puramente tecniche: un requisito formulato come *«Creare il database relazionale»* o *«Refactoring del package com.service»* è privo di valore di business autonomo e deve essere declinato come task esecutivo subordinato a una storia di valore reale.
4. **Estimable (Stimabili)**:
   - La storia deve possedere un livello di chiarezza e dettaglio sufficiente a permettere al team di stimare il tempo, lo sforzo o la complessità necessari a completarla.
   - Se un team dichiara di non poter stimare una storia, ciò segnala una lacuna di comprensione del dominio (mancano informazioni sul perimetro) oppure una forte incertezza tecnologica (in tal caso si introduce uno *Spike* esplorativo preliminare).
   - La stima è sempre relativa e propria del team (scale lineari, sequenza di Fibonacci, taglie T-Shirt), tipicamente determinata mediante sessioni partecipative di *Planning Poker*.
5. **Sized right / Small (Di Giusta Taglia)**:
   - La dimensione della storia deve essere tale da poter essere pianificata, sviluppata, testata e considerata *Done* comodamente all'interno di una singola iterazione di sviluppo (sprint).
   - Storie eccessivamente voluminose mascherano complessità latenti e rischiano di non essere terminate entro la scadenza dell'iterazione.
   - I requisiti nascono spesso come macro-entità (epiche o temi) che devono essere progressivamente decomposte e affilate man mano che si avvicina il momento della loro effettiva lavorazione (il cosiddetto raffinamento a piramide).

6. **Testable (Collaudabili)**:

   - Una storia non può essere considerata valida se non è accompagnata da **criteri di accettazione espliciti** che ne consentano la verifica empirica.
   - I criteri di accettazione trasformano l'intento narrativo della storia in una realtà collaudabile, garantendo che il comportamento sviluppato corrisponda a quello preteso dal cliente.
   - Non si dovrebbe mai avviare la scrittura del codice di una User Story se i criteri di accettazione non sono stati chiariti e concordati preventivamente; i relativi test devono, ove possibile, essere completamente automatizzabili.

La Stima delle User Stories e gli Story Points. Nel paradigma agile tutte le User Stories destinate a entrare nel ciclo di sviluppo devono essere stimate dal team di ingegneria. A differenza dei modelli tradizionali, che tentavano di quantificare il lavoro direttamente in termini di tempo assoluto (ore o giornate-uomo), lo sviluppo agile adotta come unità di misura fondamentale gli **Story Points**:

- **Unità di misura astratta e relativa**: gli Story Points non misurano ore lavorative, ma esprimono una grandezza adimensionale che sintetizza congiuntamente la complessità algoritmica, l'ampiezza dello sforzo realizzativo e il grado di incertezza o rischio insito nella storia.
- **Stima per confronto e analogia**: anziché chiedersi quanto tempo richiederà una funzionalità in termini assoluti, il team valuta la dimensione di una storia confrontandola con altre storie già implementate o precedentemente calibrate (es. *«questa storia è circa due volte più complessa di quella di riferimento?»*).
- **Incoraggiamento della discussione costruttiva**: il valore primario della stima non risiede nella precisione numerica del punteggio finale, ma nel confronto partecipativo che essa genera; le divergenze di valutazione fanno emergere assunzioni implicite, rischi architetturali inespressi e lacune nella comprensione del dominio.
- **Neutralità rispetto alla velocità individuale**: stimare in ore risente pesantemente di chi implementerà concretamente il codice (uno sviluppatore senior impiega una frazione del tempo di un junior); gli Story Points prescindono dall'assegnazione della persona, misurando la taglia intrinseca del requisito per l'intero gruppo.

La Tecnica del Planning Poker. La metodologia di riferimento per condurre la stima partecipativa delle User Stories è il **Planning Poker** (noto anche come *Scrum Poker*), introdotto formalmente da James Grenning nel 2002 e successivamente popolarizzato da Mike Cohn:

- **Sintesi dei fattori decisionali**: riunisce in un unico processo rapido, strutturato e dinamico l'esperienza pregressa dei membri, la stima per analogia e i molteplici punti di vista disaggregati del team.
- **Partecipazione dell'intero team di sviluppo**: alla sessione di stima prende parte il gruppo di ingegneria al completo (programmatori, tester, architetti, DevOps); il Product Owner partecipa unicamente per illustrare le storie e dirimere dubbi sui requisiti, astenendosi categoricamente dal votare o influenzare le stime.

- **Mitigazione dei bias cognitivi**: elimina il fenomeno dell'ancoraggio psicologico (*anchoring bias*) e la subordinazione gerarchica; votando in modo segreto e simultaneo, si evita che l'opinione dello sviluppatore più carismatico o senior condizioni indebitamente il giudizio del resto del gruppo.

Procedura Esecutiva del Planning Poker. La sequenza di facilitazione del Planning Poker si articola in cinque passi ciclici ben definiti:

1. **Esposizione della storia**: il moderatore (tipicamente lo Scrum Master) o il Product Owner illustra la User Story candidata, leggendone descrizione e criteri di accettazione, e risponde alle domande di chiarimento del team;

2. **Selezione segreta**: ciascun componente sceglie dal proprio mazzo la carta corrispondente alla propria stima e la ripone coperta sul tavolo, senza mostrarla o comunicarla ai colleghi;

3. **Rivelazione simultanea**: al segnale convenuto del moderatore, tutti i partecipanti mostrano contemporaneamente la carta selezionata;

4. **Discussione degli outlier**: se le stime differiscono sensibilmente, gli *outliers* (coloro che hanno espresso la stima più bassa e la stima più alta) espongono pubblicamente le proprie motivazioni:

   - Chi ha votato il valore più basso può aver individuato una libreria o una soluzione preesistente che semplifica drasticamente il problema;
   - Chi ha votato il valore più alto può aver intravisto una complessità architetturale nascosta, un vincolo di sicurezza critico o un impatto su moduli terzi non percepito dagli altri;

5. **Iterazione e convergenza**: chiariti i punti di vista, si procede a un nuovo turno di voto segreto; il processo si ripete sino al raggiungimento di un consenso stabile condiviso (o di una convergenza stretta su valori adiacenti).

Il Product Backlog: Definizione e Ruolo nel Ciclo di Vita. Completata la formulazione e la stima partecipativa delle User Stories, l'insieme di tutti i requisiti viene consolidato all'interno di un unico inventario organizzato denominato **Product Backlog**:

Definizione di Product Backlog

Il **Product Backlog** è l'analogo agile dell'SRS tradizionale: costituisce una lista ordinata e continuamente aggiornata di tutto ciò che potrebbe essere inserito all'interno del prodotto software per accrescerne il valore.

A differenza della specifica classica di requisiti, il backlog non ha una natura statica:

- **Prioritizzazione multidimensionale**: le storie non sono archiviate in ordine cronologico di ideazione, ma sono costantemente ordinate per priorità in funzione di quattro variabili: il **valore di business** per il committente, la **dimensione** (stima in Story Points), i **rischi architetturali** emergenti e le **necessità di mercato** contingenti;

- **Target di Iterazione (Sprint Backlog)**: a ogni inizio di sprint, il team seleziona dal vertice del Product Backlog un sottoinsieme coerente di storie ad alta priorità commisurato alla propria Velocity; tale selezione costituisce l'obiettivo operativo vincolante dell'iterazione;
- **Singola sorgente di verità**: qualsiasi richiesta di nuova feature, modifica di flusso, segnalazione di anomalia o debito tecnico deve transitare obbligatoriamente attraverso il backlog per poter competere sulle priorità di sviluppo.

Valutazione Critica: Vantaggi e Svantaggi delle User Stories. L'adozione delle User Stories rappresenta uno dei cardini del successo dell'approccio agile, ma non costituisce una panacea universale priva di criticità. L'ingegnere del software deve saper discernere con rigore i contesti in cui esse eccellono rispetto ai domini in cui manifestano gravi limitazioni strutturali:

Vantaggi (PRO)	**Svantaggi e Limiti (CONTRO)**
Cicli di feedback continui: la brevità delle storie permette il rilascio rapido e la validazione frequente da parte degli utenti reali.	**Inadeguatezza per compiti puramente tecnici**: non si adattano naturalmente a refactoring strutturali, spike tecnologici o riparazione di bug di sistema.
Coinvolgimento diretto del cliente: abbattono la barriera burocratica alimentando una visione condivisa e una conversazione ininterrotta.	**Complessità contrattuali e legali**: la natura fluida e negoziabile contrasta con i bandi di gara tradizionali a prezzo e perimetro fissi (*fixed-price contracts*).
Prioritizzazione flessibile: facilitano il riordino continuo del lavoro al mutare dello scenario competitivo o dei vincoli di budget.	**Rischio di architettura miope (Design Povero)**: focalizzarsi unicamente su singole fette verticali rischia di far perdere la visione d'insieme del sistema, accumulando debito tecnico.
Massimizzazione dell'usabilità: costringono gli sviluppatori a guardare il software dalla prospettiva dell'attore che usa il sistema.	**Notevole difficoltà di stesura**: scrivere buone storie atomiche, indipendenti e collaudabili richiede grande disciplina e sensibilità metodologica.
Semplificazione delle stime: consentono l'adozione efficace di tecniche basate su analogia e scale relative (Planning Poker).	**Rischio di requisiti non funzionali ignorati**: qualità come prestazioni, scalabilità, concorrenza e sicurezza tendono a essere trascurate se non formalizzate con rigore.

Behavior-Driven Development (BDD): Cucumber e Gherkin. Uno dei rischi più frequenti nella gestione delle User Stories consiste nel disallineamento tra il linguaggio del business (usato nella storia) e il codice dei test automatizzati (scritto nel linguaggio di programmazione). Per colmare questo divario si è affermato il **Behavior-Driven Development (BDD)**, un'evoluzione metodologica fondata sulla convergenza di Test-Driven Development (TDD), Object-Oriented Programming (OOP) e Domain-Driven Design (DDD).
Lo scopo fondazionale del BDD è impiegare **tecniche di conversazione ed esempi concreti** per costruire una comprensione condivisa (*shared understanding*) sul comportamento atteso del sistema.

Cucumber e il Linguaggio Gherkin. Lo strumento di riferimento per attuare il BDD è **Cucumber**:

- Si appoggia a un Domain-Specific Language (DSL) descrittivo denominato **Gherkin**, strutturato per risultare quasi identico al linguaggio naturale e leggibile senza ambiguità sia dai clienti sia dagli sviluppatori.
- Consente di redigere script di collaudo ad alto livello che descrivono e convalidano il comportamento del software a fronte dei criteri di accettazione.
- Il motore di Cucumber collega ciascuna frase descrittiva al codice esecutivo sottostante (*glue code*) facendo largo uso di **Espressioni Regolari (Regex)** o espressioni Cucumber native, integrandosi nei framework di automazione della build come Apache Maven, Gradle, Ant o GNU Make.

Il Modello Strutturale di una Specifica Gherkin. Un file di specifica Gherkin (con estensione `.feature`) articola il requisito in funzionalità e molteplici scenari operativi:

```
Feature: Some terse yet descriptive text of what is desired
  In order to realize a named business value
  As an explicit system actor
  I want to gain some beneficial outcome which furthers the goal

  Scenario: Some determinable business situation
    Given some precondition
    And some other precondition
    When some action by the actor
    And some other action
    Then some testable outcome is achieved
    And something else we can check happens too

  Scenario: A different situation
    ...
```

La semantica delle clausole definisce un flusso causale rigoroso:

- **Given (Dato)**: fissa lo stato di partenza e le precondizioni del sistema prima che l'azione abbia luogo;
- **When (Quando)**: descrive l'evento scatenante, l'input o l'azione operativa eseguita dall'attore;
- **Then (Allora)**: esplicita il risultato osservabile e collaudabile atteso (l'asserzione logica);
- **And / But (E / Ma)**: congiunzioni per aggregare ulteriori precondizioni, azioni o controlli a valle.

Confronto Epistemologico: Use Cases (UML) vs User Stories (Agile). Nella teoria dell'ingegneria del software sorge spontaneo il confronto tra i due costrutti cardine per l'analisi dei requisiti funzionali: gli **Use Cases** del mondo object-oriented/UML e le **User Stories** dell'approccio agile.

Use Cases vs User Stories: Scopi e Prospettive

Sebbene entrambi mirino a comprendere le esigenze dell'utente evitando il gergo tecnico, adottano filosofie operative divergenti:

- **Use Case (UML)**: modella un'**interazione procedurale completa** e strutturata tra un attore e il sistema. Descrive l'intero cammino comportamentale comprensivo di flusso nominale primario, flussi alternativi e percorsi di errore. Ha un orientamento analitico, una notazione grafica standardizzata e ambisce a documentare formalmente la specifica funzionale del sistema.
- **User Story (Agile)**: è un'**unità atomica di pianificazione e valore**. Non intende documentare il sistema in dettaglio, ma serve a stimare, prioritizzare e programmare una fetta di lavoro all'interno di un'iterazione. È deliberatamente incompleta e demanda i dettagli operativi alla conversazione continua e ai criteri di accettazione.

Dimensione	Use Cases (UML)	User Stories (Agile)
Finalità Primaria	Comprensione, modellazione e specifica formale del sistema.	Pianificazione iterativa, stima e rilascio continuo di valore.
Granularità	Ampia: racchiude l'intero processo per raggiungere l'obiettivo.	Fine: fette verticali atomiche completabili in pochi giorni.
Contenuto	Precondizioni, postcondizioni, flusso nominale, eccezioni.	Ruolo, intento, valore di business e criteri di accettazione.
Rappresentazione	Diagramma UML (ellissi, attori, `«includes»`) + testo.	Scheda cartacea o card digitale su bacheca Kanban.
Ciclo di Vita	Durevole: aggiornato per riflettere il design del software.	Transitorio: viene archiviata una volta che è *Done*.
Rapporto	Uno Use Case può essere decomposto in **molteplici User Stories** per essere implementato a incrementi successivi lungo più sprint.	

5.3 I Processi Agili

Lo sviluppo del software non costituisce una mera sequenza di compiti tecnici di codifica, bensì un'**attività sociale complessa** che intreccia problematiche ingegneristiche e dinamiche organizzative. Comprendere e formalizzare i bisogni reali del committente richiede conversazioni continue, rinegoziazioni e coordinamento di competenze eterogenee. Un manufatto software non deve unicamente risultare privo di difetti sintattici o architetturali (*costruire il software correttamente*), ma deve risolvere un problema concreto generando valore tangibile per utenti e organizzazione (*costruire il software giusto*).

Lavorare per Progetti vs Lavorare per Prodotti. Nell'ingegneria del software coesistono due prospettive di gestione del lavoro, ciascuna caratterizzata da orizzonti temporali e criteri di successo differenti:

Dimensione	Prospettiva Progetto	Prospettiva Prodotto
Orizzonte Temporale	Iniziativa temporanea delimitata da una data di avvio e un traguardo prefissato.	Evoluzione a lungo termine che accompagna l'intero ciclo di vita utile del sistema.
Pianificazione	Vincolata alla gestione congiunta di ambito (*scope*), tempi di consegna e costi/risorse.	Orientata a obiettivi di business strategici, con backlog ordinato continuamente per valore generato.
Criterio di Successo	Rispettare i vincoli contrattuali concordati a priori (tempo, costo, specifiche).	Rilasciare e massimizzare valore reale per gli utenti e l'organizzazione nel tempo.

Queste due visioni non sono reciprocamente esclusive: un'iniziativa strutturata a progetto può impiegare tecniche agili mediante rilasci incrementali multipli, mentre un prodotto di successo è solitamente alimentato da una sequenza coordinata di progetti e iterazioni evolutive.

Pianificare in Presenza di Cambiamenti. I modelli di processo tradizionali (come il Waterfall rigido) assumono implicitamente che sia possibile congelare i requisiti all'avvio. La realtà dello sviluppo smentisce tale assunzione per motivi strutturali:

- **Conoscenza incompleta all'avvio**: nelle fasi iniziali di qualsiasi iniziativa, team e committenti conoscono solo una porzione limitata del problema e del suo dominio;
- **Evoluzione del contesto**: le condizioni di mercato, le tecnologie abilitanti, i vincoli normativi e gli obiettivi di business mutano frequentemente durante lo sviluppo;
- **Il feedback come driver**: la presa di contatto con un incremento software funzionante trasforma la comprensione che l'utente ha dei propri stessi bisogni.

In un processo agile, **pianificare significa formulare una previsione informata e aggiornarla sistematicamente man mano che nuove evidenze emergono**. L'agilità non è assenza di pianificazione, ma pianificazione frequente, adattiva e basata su riscontri oggettivi.

I Quattro Valori del Manifesto Agile (2001)

Sottoscritto da 17 rappresentanti delle metodologie di sviluppo leggero (*Lightweight Methods*), il Manifesto sancisce un netto cambio di priorità:

Attribuiamo maggiore valore a...		...pur riconoscendo il valore di
Gli individui e le interazioni	>	I processi e gli strumenti
Il software funzionante	>	La documentazione esaustiva
La collaborazione con il cliente	>	La negoziazione contrattuale
Rispondere al cambiamento	>	Seguire ciecamente un piano

Chiarimento metodologico: la preferenza accordata agli elementi di sinistra **non elimina** l'importanza di processi, contratti, piani o documentazione; stabilisce tuttavia che, in presenza di un trade-off operativo, il valore primario risiede nella risposta empirica e nelle persone.

I Dodici Principi: I Quattro Temi Fondamentali. I valori del Manifesto sono declinati operativamente in dodici principi guida, sintetizzabili in quattro pilastri:

1. **Valore e Adattamento**: la priorità assoluta è soddisfare il cliente mediante consegne precoci e continue di software funzionante. I requisiti che cambiano sono benvenuti anche a sviluppo avanzato, trasformando l'adattabilità in un vantaggio competitivo.

2. **Collaborazione e Autonomia**: le figure di business e gli sviluppatori lavorano a stretto contatto su base quotidiana. I team devono essere composti da persone motivate, operanti in ambienti fiduciari e dotati di auto-organizzazione decisionale. La conversazione diretta faccia a faccia costituisce il canale di comunicazione più efficace.

3. **Qualità e Sostenibilità**: gli sponsor, gli sviluppatori e gli utenti devono poter mantenere indefinitamente un **ritmo di lavoro costante e sostenibile**. La velocità non deve sacrificare l'eccellenza tecnica, il rigore ingegneristico e la qualità della progettazione.

4. **Semplicità e Miglioramento Continuo**: la semplicità è definita come **l'arte di massimizzare la quantità di lavoro non svolto** (evitare feature speculative o ridondanti). A intervalli regolari il team riflette sulle proprie modalità operative, adattando e perfezionando il proprio comportamento.

Minimum Viable Product (MVP) e Filosofia Lean Startup. Un errore diffuso consiste nel confondere l'incremento di codice con il rilascio di un **Minimum Viable Product (MVP)**. Concetto formalizzato originariamente all'interno dell'approccio Lean Startup, l'MVP rappresenta uno strumento conoscitivo:

- **Verifica di ipotesi**: un MVP serve a mettere alla prova un'ipotesi specifica sul prodotto, sul suo mercato e sui comportamenti reali degli utenti finali;

- **Criterio di decisione**: il risultato dell'esperimento orienta la strategia aziendale determinando se **perseverare** sulla direzione tracciata, **pivotare** (modificare rotta conservando quanto appreso) o **arrestare** l'iniziativa;

- **"Minimum" vs "Viable"**: il termine *minimo* esprime il minor dispendio di tempo e risorse necessario a ricavare un riscontro attendibile; il termine *viable* impone che l'esperienza d'uso offerta sia sufficientemente completa da rendere quel riscontro significativo e realistico.

Il numero assoluto di feature non determina la validità di un MVP. Nel celebre caso della nascita di **Dropbox**, l'MVP consistette in un semplice video dimostrativo di pochi minuti realizzato dal fondatore, che mostrava l'esperienza d'uso attesa del filesystem sincronizzato prima ancora di averne implementato l'intera architettura distribuita di rete. Il volume di adesioni raccolto certificò l'esistenza della domanda prima di avviare il pesante sforzo ingegneristico.

Mappa degli Approcci Collegati all'Agilità. Sotto l'ombrello dell'agilità e del pensiero snello ricadono framework e pratiche con focalizzazioni tra loro complementari:

- **Metodi e Framework di Sviluppo (XP, Scrum, Crystal, FDD, DSDM)**: disciplinano la cooperazione all'interno del singolo team e le pratiche di sviluppo tecnico o di organizzazione dei task;
- **Gestione del Flusso (Kanban, Lean Software Development)**: si focalizzano sull'eliminazione degli sprechi, sulla visualizzazione del carico di lavoro e sulla limitazione del lavoro in corso (*WIP – Work In Progress*);
- **Framework su Scala (SAFe, LeSS)**: affrontano il coordinamento sincrono e l'allineamento strategico tra decine o centinaia di team concorrenti all'interno di grandi aziende;
- **DevOps**: unisce la dimensione dello sviluppo (*Dev*) con le operazioni di gestione ed esercizio infrastrutturale (*Ops*), puntando sull'automazione del deployment e sulla continuità del servizio.

Extreme Programming (XP) e i Suoi Valori Fondamentali. Tra le metodologie agili emerse per affrontare progetti caratterizzati da requisiti volatili e forte incertezza, **Extreme Programming (XP)** si distingue per la sua marcata focalizzazione sull'eccellenza tecnica, sulla qualità del codice e sulle pratiche operative del team di sviluppo. XP porta all'estremo i principi del buon software engineering, elevandoli a regole quotidiane di lavoro guidate da cinque valori fondamentali:

- **Comunicazione**: rendere esplicite e visibili le conoscenze di dominio, le criticità architetturali e le decisioni tecniche attraverso il confronto continuo all'interno del team e con il committente;
- **Semplicità**: progettare e implementare unicamente ciò che serve a soddisfare i requisiti noti oggi, scegliendo la struttura più chiara e comprensibile ed evitando sovrastrutture speculative per il futuro;
- **Feedback**: raccogliere e valorizzare sistematicamente i riscontri generati dal codice compilato, dall'esecuzione dei test automatici, dalle revisioni dei colleghi e dalla valutazione degli utenti finali;
- **Coraggio**: avere l'audacia ingegneristica di rifattorizzare porzioni complesse di sistema per migliorarne il design, scartare soluzioni obsolete e dichiarare apertamente impedimenti e ritardi;
- **Rispetto**: riconoscere e valorizzare le competenze, il carico sostenibile e le esigenze di tutti i componenti del gruppo, garantendo che le scelte tecniche non compromettano il lavoro altrui.

Gestione dei Requisiti: Le User Stories (Il Modello delle 3C). Nei processi agili i requisiti funzionali abbandonano i pesanti documenti burocratici formali per tradursi in **User Stories**: descrizioni snelle e orientate all'utente che sintetizzano una necessità di business. Una user story efficace si articola sempre secondo la convenzione delle **3C** (formalizzata da Ron Jeffries):

1. **Card (Scheda)**: supporto fisico o sintetico che fissa in poche righe il perimetro del bisogno nel vocabolario dell'utente. Adotta frequentemente il modello narrativo:

*«Come **[ruolo utente]**, voglio **[funzionalità attesa]**, per **[beneficio/valore conseguito]**».*

Tale formato rappresenta un'utile guida comunicativa ma non un vincolo grammaticale universale;

2. **Conversation (Conversazione)**: momento di confronto dialogico diretto tra il team di sviluppo, il cliente e gli utenti. La scheda non è una specifica contrattuale congelata, bensì un *promemoria a conversare* per approfondire vincoli, ramificazioni, dettagli implementativi e percorsi alternativi;

3. **Confirmation (Conferma)**: l'insieme esplicito dei **criteri di accettazione** concordati, che definiscono in termini inequivocabili e verificabili le condizioni oggettive affinché la storia possa considerarsi completata e accettata dal cliente.

Il Planning Game e la Gestione dei Cambiamenti. La pianificazione adattiva in XP si concretizza nel rito del **Planning Game**, una sessione strutturata condotta all'inizio di ogni iterazione per allineare business e ingegneria:

Competenza del Cliente / Business	**Competenza degli Sviluppatori**
Chiarisce il valore economico e seleziona le priorità strategiche.	Chiariscono la fattibilità architetturale, stimano l'impegno relativo e valutano i rischi tecnici.
Valuta alternative funzionali di perimetro per massimizzare il ROI.	Identificano vincoli sistemici, dipendenze tecnologiche e complessità nascoste.
Esamina gli incrementi dimostrabili e fornisce feedback empirico.	Utilizzano i dati storici del lavoro completato per calibrare e migliorare le previsioni future.

Gestione di una Variazione durante l'Iterazione. Cosa accade se a metà ciclo il committente introduce una modifica urgente (ad esempio la richiesta inattesa: *«Vorrei introdurre subito l'abbonamento annuale»*)? L'approccio agile non rifiuta burocraticamente la variazione né la accoglie passivamente sovraccaricando gli sviluppatori. La risposta professionale prevede un protocollo disciplinato:

1. **Chiarimento del bisogno**: comprendere la natura della richiesta e la sua effettiva urgenza di business;

2. **Valutazione tecnica d'impatto**: stimare costo relativo, rischi architetturali e dipendenze con il team;

3. **Rinegoziazione del perimetro (*Trade-off*)**: concordare con il committente quale elemento di lavoro già pianificato debba essere anticipato, posticipato a iterazioni future o espunto dallo Sprint per fare spazio al nuovo requisito;

4. **Trasparenza decisionale**: aggiornare il piano operativo e rendere visibile a tutta l'organizzazione la rinegoziazione concordata, preservando la sostenibilità e la qualità del rilascio.

Criteri di Accettazione e Test di Accettazione. La componente di *Conferma* di una User Story definisce il confine tra l'intenzione del requisito e la sua effettiva realizzazione verificabile. A differenza dei requisiti ambigui o qualitativi tipici della documentazione tradizionale, una storia agile deve essere corredata da **criteri di accettazione oggettivi**, concordati esplicitamente tra il cliente e il team prima dell'avvio dello sviluppo.
Considerando il caso guida dell'abbonamento universitario alla sosta:

> *«Come studente, voglio acquistare un abbonamento al parcheggio per evitare il pagamento giornaliero»*.

I criteri di business stabiliscono regole univoche su durate, costi ed esiti delle transazioni (un mese costa 30 €, due mesi costano 60 €; il pagamento approvato attiva l'abbonamento, il pagamento rifiutato blocca l'emissione e segnala l'errore).
Per rendere tali regole sistematicamente collaudabili, i criteri vengono formalizzati sotto forma di **scenari di accettazione** strutturati secondo la semantica **Dato / Quando / Allora** (*Given / When / Then*):

Scenario	Dato (Contesto iniziale)	Quando (Evento scatenante)	Allora (Risultato osservabile)
Pagamento Approvato	Studente autenticato a sistema; selezionata durata di un mese.	La transazione di pagamento viene autorizzata dal circuito bancario.	Il sistema attiva l'abbonamento e restituisce ricevuta conforme a 30 €.
Pagamento Rifiutato	Studente autenticato a sistema; selezionata durata di un mese.	La transazione di pagamento viene respinta per fondi insufficienti.	Il sistema notifica l'errore, non attiva alcun titolo e mantiene lo stato coerente.

Il cliente e le figure di business chiariscono le regole di dominio; il team di sviluppo si assume la responsabilità ingegneristica di tradurle in **verifiche automatiche e ripetibili** all'interno della suite di test.

Test-Driven Development (TDD): Il Ciclo Red-Green-Refactor. Nel paradigma XP, i test non rappresentano una fase a valle della codifica, bensì il **motore primario di progettazione e implementazione del software**. Il **Test-Driven Development (TDD)** inverte il flusso convenzionale articolandosi attraverso micro-cicli iterativi composti da tre fasi rigorose:

Passo	Azione Ingegneristica	Controllo e Condizione di Uscita
1. Red	Scrivere un piccolo test automatico che descrive il comportamento successivo atteso.	Eseguire la suite: il test **deve fallire** per la ragione attesa (e non per errori di configurazione).
2. Green	Scrivere la quantità minimale di codice sorgente strettamente necessaria a soddisfare il test.	Eseguire la suite: il nuovo test e tutti i test preesistenti **devono passare** con successo.
3. Refactor	Ristrutturare e pulire il codice eliminando duplicazioni e migliorando i nomi.	Eseguire la suite: i test **devono rimanere verdi**, garantendo l'invarianza del comportamento.

Una singola user story non viene implementata in un blocco monolitico, ma scaturisce dalla concatenazione progressiva di molteplici cicli TDD a grana fine.

Piccoli Incrementi e Rilasci Frequenti. L'approccio agile stabilisce una netta distinzione concettuale tra due dimensioni del ciclo di sviluppo:

- **Iterazione (*Sprint / Iteration*)**: scandisce il ritmo interno di lavoro del team di sviluppo (tipicamente da 1 a 4 settimane), al termine del quale viene consolidato un incremento potenzialmente rilasciabile;
- **Rilascio (*Release*)**: l'atto operativo mediante cui una versione verificata e funzionante del software viene effettivamente distribuita e resa accessibile agli utenti finali.

Un incremento deve realizzare un comportamento completo, utilizzabile e collaudato lungo l'intero stack applicativo (*fetta verticale*). La prima versione messa in produzione può offrire una funzionalità essenziale (ad esempio l'acquisto di un abbonamento per un solo mese con un unico metodo di pagamento); i rilasci successivi amplieranno progressivamente le opzioni e le integrazioni sulla scorta del feedback reale ricavato dall'uso operativo. **Un ambito circoscritto e un'adeguata qualità ingegneristica devono obbligatoriamente convivere**: ridurre l'ambito non autorizza a rilasciare codice privo di test o architetturalmente degradato.

La Metafora del Sistema e il Glossario Comune. Nello sviluppo di software complessi, le divergenze terminologiche tra specialisti di dominio e programmatori generano disallineamenti gravi e costosi. Per ovviare a questo rischio, XP introduce la pratica della **Metafora del Sistema** (*System Metaphor*) affiancata dall'adozione rigorosa di un **linguaggio onnipresente** (*Ubiquitous Language*):

- **Visione d'assieme condivisa**: una metafora architetturale intuitiva consente a tutti i membri del gruppo di visualizzare immediatamente come i componenti cooperano e dove collocare una nuova responsabilità (ad esempio descrivere la struttura a livelli di un client web mediante l'analogia del "Bento Box" o suddividere i flussi computazionali secondo metafore fluide);
- **Glossario esplicito**: definisce in modo formale e privo di sinonimi i termini cardine del sistema, le entità di dominio, le loro responsabilità e le relazioni reciproche. La medesima denominazione concordata deve comparire senza scostamenti nelle conversazioni con il cliente, nelle schede delle user stories, nei nomi delle classi e nelle tabelle del database.

Progettazione Semplice e il Principio YAGNI. La tendenza spontanea di molti architetti software consiste nel sovra-ingegnerizzare le soluzioni, anticipando requisiti ipotetici mediante generalizzazioni speculative. Extreme Programming risponde con il principio del **Simple Design**, sintetizzato dall'acronimo **YAGNI**:

YAGNI: *«You Aren't Gonna Need It»* (Non ne avrai bisogno).

Una progettazione semplice non coincide con una codifica rozza o affrettata, bensì richiede un elevato giudizio tecnico ed è disciplinata da quattro regole cardine:

1. Il codice deve superare con successo tutti i test automatici di unità e di accettazione;

2. Esprime chiaramente l'intenzione del programmatore attraverso nomi appropriati e strutture limpide;
3. Elimina qualsiasi duplicazione o ridondanza concettuale non necessaria;
4. Contiene il minor numero possibile di classi, metodi e livelli di astrazione intermedi.

Il design deve sostenere con rigore i requisiti e i vincoli noti oggi. Anticipare scenari futuri non dimostrati appesantisce il sistema, moltiplica i costi di manutenzione e aumenta il debito tecnico. Qualora emergano rischi tecnici critici o incertezze su librerie esterne, il team non inserisce architetture speculative nel codice principale, bensì conduce brevi esperimenti mirati denominati **Spike architetturali**.

Pair Programming: Ruoli, Rotazione ed Evidenze Empiriche. La pratica del **Pair Programming** prevede che due sviluppatori lavorino congiuntamente alla progettazione, codifica e verifica del software condividendo un'unica postazione di lavoro. L'interazione si struttura mediante una chiara alternanza di ruoli operativi:

- **Driver**: siede alla tastiera, scrive il codice, controlla i dettagli sintattici e mantiene il passo esecutivo a breve termine;
- **Navigatore**: osserva il lavoro da una prospettiva strategica a medio termine, valuta percorsi alternativi, rileva errori logici, anticipa rischi architetturali e verifica l'allineamento con i criteri di accettazione;
- **Scambio continuo**: i ruoli non sono statici; i componenti della coppia discutono apertamente e si scambiano frequentemente la tastiera per mantenere attiva l'attenzione di entrambi. La rotazione periodica delle coppie all'interno del team favorisce inoltre la fertilizzazione incrociata delle competenze e abbatte i silos conoscitivi.

Cosa dimostra l'evidenza empirica. L'efficacia del pair programming è stata oggetto di numerosi studi sperimentali e industriali, sintetizzati nella meta-analisi di riferimento di Hannay et al. (2009):

Dimensione Osservata	**Sintesi delle Evidenze Empiriche (Hannay et al., 2009)**
Qualità del Software	**Miglioramento medio contenuto**: riduzione misurabile della densità di difetti rispetto allo sviluppo individuale, con beneficio massimo sui compiti complessi.
Durata del Compito (*Duration*)	**Riduzione media del tempo di completamento**: il compito viene concluso più rapidamente in termini di tempo di calendario (*elapsed time*).
Sforzo Totale (*Effort*)	**Aumento medio del lavoro totale**: incremento nell'impiego complessivo di ore-persona (+15% / +40%), poiché due professionisti lavorano simultaneamente sullo stesso modulo.

Valutazione ingegneristica: durata del compito e sforzo complessivo sono grandezze ortogonali. Il pair programming non è una pratica da imporre ciecamente su compiti banali o ripetitivi, ma si rivela economicamente vantaggioso sui task ad alta complessità concettuale, dove il costo dello sforzo aggiuntivo è ampiamente compensato dalla riduzione dei difetti e dalla tempestività di consegna.

Integrazione Continua (Continuous Integration). La pratica dell'**Integrazione Continua (CI)** prescrive che ciascun membro del team faccia confluire frequentemente le proprie modifiche nella linea di sviluppo principale e condivisa (*main trunk*).

- **Verifica automatica immediata**: a ogni commit o merge, una pipeline automatizzata provvede a compilare il codice, eseguire l'intera suite dei test unitari e di accettazione e generare report di analisi statica;
- **Visibilità e correzione del guasto**: se l'integrazione fallisce (*build broken*), l'anomalia diventa immediatamente visibile a tutto il gruppo; la correzione ha la precedenza assoluta su qualsiasi nuovo sviluppo per evitare l'accumulo di errori latenti;
- **Minimizzazione della divergenza**: prolungare rami di sviluppo isolati per giorni o settimane porta al cosiddetto "inferno delle integrazioni" (*merge hell*); commit frequenti e atomici riducono drasticamente i conflitti di versionamento e garantiscono che il sistema sia in ogni istante pronto per essere rilasciato.

Proprietà Collettiva e Convenzioni di Codifica. In XP non esistono moduli o file di codice "blindati" o di proprietà esclusiva di singoli sviluppatori; l'intero team detiene la **Proprietà Collettiva del Codice** (*Collective Codebase Ownership*):

- **Responsabilità condivisa**: chiunque rilevi un margine di miglioramento o un bug in qualsiasi componente ha il diritto e il dovere professionale di modificarlo e rifattorizzarlo;
- **Fattori abilitanti**: la proprietà collettiva non è anarchia; è resa sostenibile unicamente dalla presenza della rete di sicurezza dei test automatici, dall'integrazione continua e dal rispetto rigoroso degli **standard di codifica**;
- **Il ruolo dei commenti**: il codice deve essere auto-esplicativo tramite nomi precisi di classi e metodi. I commenti non devono parafrasare ciò che il codice fa già in modo ovvio, bensì spiegare **perché** è stata adottata una determinata scelta, documentare vincoli esterni non apparenti o motivare perché una soluzione alternativa apparentemente ovvia sia errata.

Ritmo Sostenibile (Sustainable Pace). Formulato storicamente come "la settimana di 40 ore", il principio del **ritmo sostenibile** tutela la risorsa più preziosa dell'ingegneria del software: la lucidità cognitiva delle persone. Lo sviluppo software richiede concentrazione profonda; stanchezza, straordinari cronici e sovraccarico moltiplicano gli errori di progettazione e aumentano la difettosità. Qualora il lavoro stimato ecceda il tempo disponibile, il team non accelera artificialmente a scapito della qualità, ma rinegozia l'ambito con il committente prima che l'urgenza degeneri in emergenza ingestibile.

Intelligenza Artificiale Generativa nei Processi Agili. L'introduzione dei modelli linguistici generativi (LLM) nello sviluppo agile offre nuove opportunità di automazione ma introduce rischi di conformità ingegneristica. L'IA può supportare le pratiche XP lungo diverse aree, purché sia sempre accompagnata da un rigoroso controllo umano:

Attività XP	Supporto Potenziale dell'LLM	Verifica Richiesta allo Sviluppatore
User Stories	Proporre formulazioni alternative, identificare casi limite e suggerire scomposizioni.	Validare bisogni reali, priorità di business e coerenza di contesto con cliente e team.
Testing	Generare dati di test sintetici, suggerire casi di confine ed estendere scenari parametrici.	Verificare la solidità dell'oracolo di correttezza e la pertinenza dei casi di collaudo.
Codifica	Proporre implementazioni algoritmiche boilerplate o funzioni di libreria.	Ispezionare correttezza logica, vulnerabilità di sicurezza e comprensibilità del codice.
Refactoring	Suggerire nomi più espressivi, estrazioni di metodi e semplificazioni sintattiche.	Assicurare che il comportamento sia preservato e che non si introducano astrazioni futili.

Il risparmio effettivo di tempo nell'uso dell'IA generativa dipende dal **costo di revisione critica** e dalla complessità degli errori sottili introdotti dai modelli.

L'Anti-Pattern dell'Errore Condiviso (Shared Error)

Il rischio più insidioso nell'adozione degli assistenti AI risiede nella **generazione simultanea di codice e relativi test**. Se il modello fraintende o allucina una regola di dominio, rifletterà la medesima assunzione errata sia nell'implementazione sia nell'asserzione di collaudo.

Esempio concreto: la regola di business concordata prescrive che un abbonamento di due mesi costi **60 €**. Il modello propone un'implementazione che applica uno sconto arbitrario non richiesto, affiancandovi il test corrispondente:

```
# Codice suggerito dall'LLM (sconto arbitrario non richiesto):
def prezzo(mesi):
    return 30 * mesi * 0.9

# Test suggerito dall'LLM (allineato all'errore del codice):
def test_due_mesi():
    assert prezzo(2) == 54
```

Il test viene eseguito e **risulta verde con successo**; tuttavia, la suite collauda unicamente la coerenza interna tra due allucinazioni.

Principio fondamentale: codice e test perfettamente coerenti tra loro possono essere entrambi completamente errati rispetto al bisogno reale. Il controllo della verità deve rimanere ancorato all'**oracolo esterno** concordato con il cliente nei criteri di accettazione (`assert prezzo(2) == 60`).

Organizzazione del Lavoro nel Progetto del Corso. A recepimento dei principi agili ed XP analizzati, l'organizzazione operativa per lo sviluppo dei progetti di laboratorio prevede standard chiari:

- **Repository e Tracciabilità**: codice versionato su repository centralizzato con permessi e policy di revisione espliciti; tracciamento rigoroso di storie, decisioni e difetti negli strumenti di issue tracking;
- **Sincronizzazione Quotidiana**: un breve momento di allineamento giornaliero (ispirato allo standup) per coordinare avanzamento, carichi e impedimenti emersi;
- **Automazione e Riscontro**: esecuzione automatica di build e test a ogni commit, associata a dimostrazioni periodiche del software funzionante e sessioni di riflessione per adattare continuamente il modo di lavorare del gruppo.

6 Architetture Software

6.1 Concetti di System Design

6.1.1 Introduzione e Concetto di System Design

La progettazione architetturale di un sistema (**System Design**) risponde all'interrogativo fondamentale: **come costruire il sistema?** Consiste nel prendere le decisioni strutturali di più alto livello, stabilendo l'**architettura globale**: come il software viene ripartito in macro-componenti, quali responsabilità sono assegnate a ciascuno di essi e attraverso quali canali e protocolli essi comunicano.

Il Principio del Divide et Impera. Il controllo della complessità di qualsiasi opera di ingegneria poggia sul principio latino del **Divide et Impera** (*divide and conquer*): il sistema complesso non viene affrontato come un blocco monolitico, bensì partizionato in **sottosistemi cooperanti**, ciascuno preposto a un sotto-problema delimitato e governabile.

6.1.2 Architettura di Sistema e Livelli di Astrazione

L'**Architectural Design** (progettazione architetturale) è formalmente l'attività di **identificazione, perimetrazione e definizione dei sottosistemi** che compongono l'intero manufatto, nonché delle relazioni e dei vincoli che ne regolano le interazioni.
I sottosistemi cooperanti non si collocano tutti sul medesimo piano, ma vengono definiti e osservati a **tre livelli di astrazione gerarchici**:

1. **Livello Hardware**: apparati fisici, topologia delle macchine, processori, memorie, interfacce di rete, sensori ed attuatori.
2. **Livello Software di Sistema (*System Software*)**: la piattaforma software di base che fa da ponte tra l'hardware e le applicazioni dell'utente. Comprende:
   - Sistemi Operativi (es. Linux, Windows);
   - Database Management Systems (DBMS relazionali, documentali o a oggetti);
   - Server di rete e web server (es. Apache, Nginx);
   - Middleware e standard di comunicazione distribuita (es. RPC, message broker).
3. **Livello Software Applicativo (*Application Software*)**: il software scritto ad-hoc per risolvere i problemi specifici del cliente o degli utenti. Si articola in moduli software, librerie, componenti logici con relative funzionalità esposte e interfacce contrattuali pubbliche (API).

Livello Architetturale	Elementi Costitutivi Tipici	Ruolo e Responsabilità
Software Applicativo	Moduli, classi, microservizi, package, interfacce API.	Realizza la logica di business e le funzioni richieste dagli utenti finali.
Software di Sistema	Sistema Operativo, DBMS, Web Server, Runtime, Middleware.	Offre astrazioni uniformi, gestione delle risorse hardware e servizi di rete.
Hardware	Server, workstation, dispositivi embedded, sensori, memorie.	Fornisce la potenza di calcolo fisica e la connettività materiale di trasmissione.

6.2 Stili Architetturali

6.2.1 Gli Stili Architetturali secondo Garlan e Shaw

A metà degli anni '90, la ricerca sull'ingegneria del software ha compiuto un salto fondamentale verso la formalizzazione delle architetture. Nel 1994 e nel 1996, **David Garlan** e **Mary Shaw** (della Carnegie Mellon University - CMU) hanno sistematizzato il modo in cui i sistemi software su larga scala vengono organizzati, definendo il concetto di **Stile Architetturale** (*Architectural Style*).

Definizione Formale di Stile Architetturale (Shaw e Garlan, 1996)

«Uno Stile Architetturale definisce una famiglia di sistemi in termini di uno schema di organizzazione strutturale. Più specificamente, uno stile architetturale definisce un ***vocabolario di componenti*** *e di* ***tipi di connettori****, e un* ***insieme di vincoli*** *su come essi possono essere combinati.»*

L'essenza di un'architettura si scompone quindi in tre elementi costitutivi:

- **Componenti (*Components*)**: le unità che svolgono la computazione o mantengono lo stato del sistema (es. filtri, oggetti, manager, database).
- **Connettori (*Connectors*)**: i canali che regolano la comunicazione, la coordinazione e il flusso informativo tra i componenti (es. pipe, chiamate di procedura, bus di eventi, messaggi di rete).
- **Vincoli (*Constraints*)**: le regole formali che limitano la topologia (es. "i filtri non condividono memoria", "uno strato comunica solo con lo strato adiacente").

I Sette Stili Architetturali Comuni. Shaw e Garlan identificano **sette stili architetturali classici** ricorrenti nell'industria del software:

1. **Pipes and Filters** (Tubi e Filtri)
2. **Data Abstraction and OO Organization** (Astrazione dei Dati e Organizzazione a Oggetti)

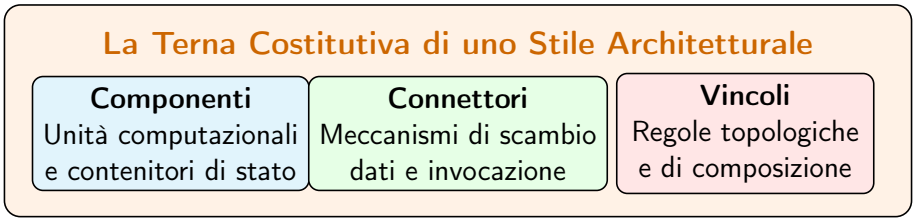


Figura 77: La definizione formale di stile architetturale secondo Shaw e Garlan.

3. **Layered Systems** (Sistemi Stratificati / a Strati)
4. **Implicit Invocation / Event-based** (Invocazione Implicita basata su Eventi)
5. **Repositories** (Repository / Lavagna - *Blackboard*)
6. **Interpreters** (Interpreti guidati da tabelle)
7. **Process Control** (Sistemi di Controllo di Processo)

6.2.2 Stile 1: Pipes and Filters (Tubi e Filtri)

Concetto e Meccanismo Operativo. Lo stile **Pipes and Filters** adotta la metafora di un impianto idraulico di elaborazione:

- **Componenti → Filtri (*Filters*)**: ciascun filtro è un modulo di calcolo autonomo. Legge uno o più flussi di dati in ingresso (*streams*), applica loro una trasformazione locale e produce flussi di dati in uscita.
- **Connettori → Pipe (*Tubi*)**: canali unidirezionali che trasportano il flusso di dati (tipicamente stream di byte o flussi di testo ASCII) dall'uscita di un filtro all'ingresso del filtro successivo.



Figura 78: Topologia Pipes and Filters: elaborazione sequenziale a stadi incrementali.

La Proprietà Chiave: Elaborazione Incrementale (Streaming)

Nei sistemi Pipes and Filters **l'emissione dell'output inizia prima che l'input sia stato completamente consumato**. Il filtro non attende la fine del file o dello stream per iniziare il proprio lavoro: trasforma i singoli caratteri o pacchetti di byte non appena disponibili, garantendo:

- Bassa latenza complessiva;
- Capacità di elaborare flussi di dimensioni teoricamente infinite con occupazione di

memoria limitata;

- Possibilità di eseguire i filtri in thread paralleli concorrenti (mentre il filtro B lavora sul blocco n, il filtro A sta già elaborando il blocco $n+1$).

Vincoli Architetturali sui Filtri. Affinché il sistema preservi la sua integrità modulare, devono essere rispettati tre vincoli stringenti:

1. **Indipendenza assoluta**: ciascun filtro deve operare come un'entità completamente autonoma.
2. **Nessuna condivisione di stato**: due filtri non devono **mai condividere variabili o memoria comune** tra loro; ciascuno mantiene solo lo stato strettamente necessario alla propria computazione locale.
3. **Ignoranza dell'identità dei vicini**: un filtro non deve assolutamente conoscere quale componente si trovi a monte (*upstream*) o a valle (*downstream*). Il filtro riceve semplicemente uno stream in ingresso e scrive su uno stream in uscita.

Esempi Noti:

- La pipeline della riga di comando UNIX: `cat elenco.txt | grep "Rossi" | sort | uniq`;
- Pipeline di compilazione: *Lexer* (analisi lessicale) → *Parser* (analisi sintattica) → *Optimizer* → *Code Generator*;
- Pipeline di elaborazione di segnali multimediali audio/video.

6.2.3 Stile 2: Data Abstraction and Object-Oriented Organization

Concetto e Meccanismo. Nello stile basato su **Astrazione dei Dati e Organizzazione a Oggetti**, il sistema è partizionato attorno alle strutture informative anziché attorno ai passi procedurali:

- **Componenti → Oggetti o ADT (Abstract Data Types)**: ciascun oggetto agisce come un componente **Manager** preposto a una risorsa o a un'entità di business.
- **Responsabilità del Manager**: l'oggetto è l'unico responsabile della preservazione dell'integrità della propria risorsa. Nasconde deliberatamente i dettagli di rappresentazione interna agli altri componenti (**Information Hiding**).
- **Connettori → Chiamate di Procedura (*Procedure Calls*)**: gli oggetti cooperano invocando reciprocamente le operazioni esposte dalle rispettive interfacce pubbliche (*method invocation / message passing*).

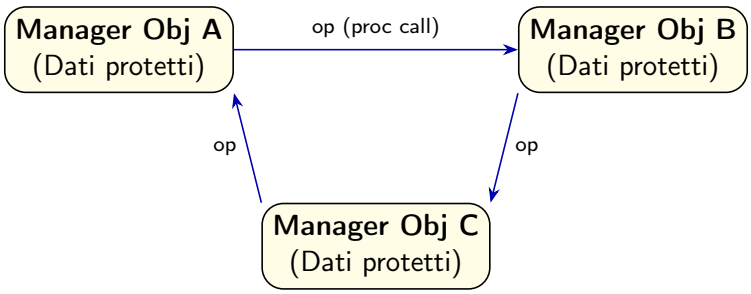


Figura 79: Topologia ad oggetti: componenti manager e chiamate di procedura esplicite.

La Criticità Chiave: Il Vincolo dell'Identità (Object IDs)

Sebbene l'incapsulamento ad oggetti garantisca alta coesione e manutenibilità interna, introduce uno svantaggio architetturale fondamentale:

- Per interagire con un oggetto, il modulo chiamante **deve conoscere a priori la sua identità specifica** (il suo puntatore, riferimento o ID);
- Quando l'identità di un fornitore di servizi cambia, o quando un oggetto deve essere rimpiazzato con un altro, tutti gli oggetti client che ne detengono il riferimento devono essere aggiornati;
- Questo legame nominale diretto genera un **accoppiamento rigido** tra gli oggetti, limitando la ricombinazione dinamica del sistema.

6.2.4 Stile 3: Layered Systems (Sistemi Stratificati / a Strati)

Concetto e Regola Architetturale. Nei **Layered Systems**, l'organizzazione strutturale è concepita come una pila gerarchica di livelli concentrici o sovrapposti (*abstract machines*):

- Ciascun livello (*layer*) definisce una macchina astratta a un determinato grado di granularità;
- **Regola di accesso**: le operazioni e i servizi di uno strato possono accedere unicamente ai servizi erogati dallo strato corrente o da quello **immediatamente inferiore (o interno)**;
- Gli strati interni sono completamente all'oscuro dell'esistenza e dei dettagli di implementazione degli strati esterni (*inner layers do not know outer layers*).

Connettori e Vantaggi: I connettori sono convenzionalmente **chiamate di procedura** (*procedure calls*). Il grande vantaggio risiede nella **sostituibilità modulare**: uno strato intermedio può essere completamente riscritto o sostituito da una nuova implementazione a patto di preservare l'interfaccia pubblica fornita agli strati sovrastanti.

6.2.5 Stile 4: Event-Based, Implicit Invocation (Invocazione Implicita)

Concetto e Origini Storiche. Lo stile basato su eventi ed **invocazione implicita** affonda le sue radici storiche nei sistemi ad agenti/attori, nei sistemi di soddisfacimento di vincoli (*constraint satisfaction*), nei demoni di sistema (*daemons*) e nelle reti a commutazione di pacchetto.

- A differenza della programmazione tradizionale in cui un componente chiama esplicitamente un metodo sapendo esattamente quale classe risponderà, nell'invocazione implicita il componente si limita a **segnalare il verificarsi di un evento**;
- **Interfacce dei componenti**: un componente espone sia un insieme di procedure ordinarie, sia un insieme di **eventi** che è in grado di annunciare.

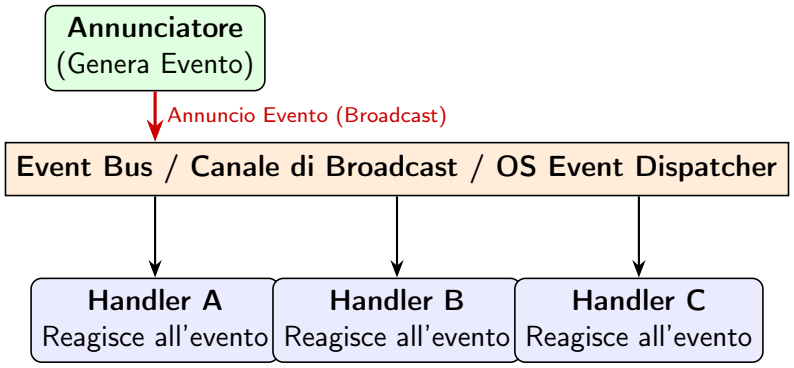


Figura 80: Invocazione implicita basata su eventi: disaccoppiamento totale tra annunciatore e gestori.

Il Principio di Disaccoppiamento e il Riuso Estremo

Il punto di forza architetturale risiede nel fatto che **chi annuncia un evento non ha idea di chi reagirà**:

- L'annunciatore trasmette l'evento nell'ambiente (in modalità *broadcast*);
- I componenti interessati registrano in precedenza le proprie procedure (*callbacks* / *listener*) presso il gestore degli eventi;
- **Altissimo supporto al riuso**: è possibile innestare nuovi gestori e nuovi comportamenti nel sistema **senza dover modificare neanche una riga di codice dell'annunciatore originale**.

Esempi Noti:

- La gestione degli eventi nelle moderne interfacce grafiche (GUI: click di mouse, pressione tasti, ridimensionamento finestre);
- Architetture guidate da eventi nel cloud (es. trigger di caricamento file in uno storage bucket che attiva elaborazioni asincrone);
- Sistemi Publish/Subscribe e messaggistica IoT (es. broker MQTT).

6.2.6 Quadro Riassuntivo dei Primi Quattro Stili Architetturali

A chiusura della prima porzione di trattazione, la seguente tabella sintetizza e confronta i primi quattro stili secondo il modello analitico di Shaw e Garlan:

Stile Architetturale	Componenti	Connettori	Vantaggio Principale	Criticità / Limite
Pipes and Filters	Filtri indipendenti e senza stato condiviso.	Pipe (canali di flusso continuo / stream).	Elaborazione streaming incrementale; eccellente parallelismo naturale.	Non adatto a sistemi interattivi complessi; parsing continuo dei formati.
Data Abstraction / OO	Oggetti manager che incapsulano dati.	Chiamate di procedura (invocazione metodi).	Information hiding; preservazione dell'integrità dei dati interni.	Accoppiamento rigido basato su identità/riferimenti (Object IDs).
Layered Systems	Strati a livelli gerarchici di astrazione.	Chiamate di procedura discendenti.	Chiara gerarchia; sostituibilità completa di uno strato se l'interfaccia resta fissa.	Possibile calo di prestazioni nell'attraversamento di tutti gli strati intermedi.
Implicit Invocation	Moduli registrati (annunciatori e handler).	Bus o dispatcher di eventi (broadcast).	Disaccoppiamento estremo; eccezionale estendibilità e riuso senza modifiche.	Nessun controllo deterministico sull'ordine di esecuzione delle reazioni.

6.2.7 Stile 5: Repositories (Repository e Architetture a Lavagna)

Concetto e Meccanismo. Lo stile architetturale basato su **Repository** organizza il sistema attorno a una memoria dati centrale condivisa, attorno alla quale gravitano molteplici componenti computazionali indipendenti che vi accedono per leggere, elaborare e memorizzare informazioni. Questo stile si suddivide storicamente in due grandi sottocategorie, distinte dal meccanismo di innesco (*trigger*) delle elaborazioni:

1. **Database Tradizionali**: l'innesco dell'elaborazione è determinato dalle **transazioni in ingresso** generate dai client esterni (flusso guidato dalle richieste esterne).
2. **Architetture Blackboard (a Lavagna)**: l'innesco dell'elaborazione è guidato dallo **stato corrente dei dati condivisi** memorizzati nella lavagna centrale. Quando lo stato della lavagna muta, i componenti specialistici valutano autonomamente se intervenire per far progredire la risoluzione del problema.

6.2.8 Stile 6: Table-Driven Interpreters (Interpreti Guidati da Tabelle)

Concetto e Meccanismo della Macchina Virtuale. Nello stile **Interpreti Guidati da Tabelle**, il software realizza una vera e propria **macchina virtuale** (*virtual machine*) programmabile. Il sistema non esegue direttamente le istruzioni native della CPU, ma decodifica ed esegue un linguaggio o set di istruzioni specializzato definito dal progettista:

- **Pseudoprogramma**: comprende il codice sorgente o byte-code del programma da simulare e i relativi record di attivazione dello stack;
- **Interpretation Engine (Motore di Interpretazione)**: il blocco logico che definisce le regole di decodifica dell'interprete e governa il ciclo di fetch/decode/execute.

6.2.9 Architettura a Repository Centrale (Central Repository)

Struttura Generale. L'architettura a **Repository Centrale** si adotta quando una moltitudine di utenti o applicazioni deve condividere un patrimonio informativo comune:

- Il **Data Store** (tipicamente implementato mediante un DBMS) costituisce il nucleo centripeto attorno al quale ruota l'intero sistema;
- Le applicazioni client gravitano attorno alla memoria centrale: leggono, modificano, inseriscono o cancellano record.

6.3 Architetture Client-Server, Multi-Livello e a Strati

6.3.1 Il Modello Client-Server (C/S)

Definizione e Topologia di Rete. Il modello **Client-Server** organizza l'elaborazione su una rete formata da molteplici processori e dispositivi informatici collegati:

- **Server**: entità che offrono servizi di varia natura (es. file server, database server, server di autenticazione o di stampa);
- **Client**: stazioni di lavoro o dispositivi utente che richiedono e consumano i servizi erogati dai server;
- **Concorrenza sulla stessa macchina**: client e server possono risiedere su nodi fisicamente distinti della rete, ma possono anche essere eseguiti contemporaneamente come processi concorrenti sulla **medesima macchina fisica**;
- **Ruoli dinamici**: la distinzione non è necessariamente rigida; un server può fungere a sua volta da client nei confronti di un altro server più specializzato (chiamate a catena).

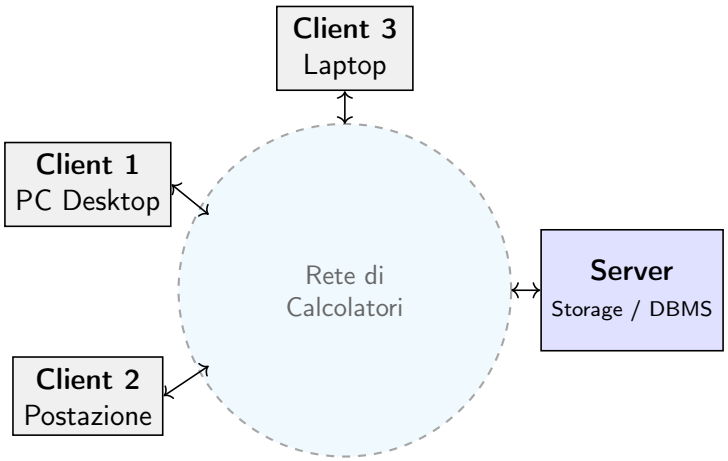


Figura 81: Topologia Client-Server: molteplici client interconnessi a un fornitore di servizi comune.

6.3.2 Architetture Basate su Internet e Intranet

Evoluzione dello Stile Client-Server. L'avvento degli standard aperti del World Wide Web ha trasformato le tradizionali architetture Client-Server nelle moderne architetture basate su **Internet e Intranet**:

- **Superamento dei vincoli geografici**: una volta connessi alla rete, la collocazione fisica di macchine, utenti e server cessa di costituire un vincolo progettuale;
- **Ampia disponibilità di strumenti**: ecosistema vastissimo di software di sistema, web server e framework gratuiti e open-source;
- **Natura bivalente delle applicazioni**: la medesima applicazione può agire contemporaneamente come server (ricevendo connessioni in ingresso ed erogando servizi) e come client (inviando richieste verso altri nodi della rete);
- **Fondamento sullo standard TCP/IP**: protocollo universale aperto, affidabile e interoperabile.

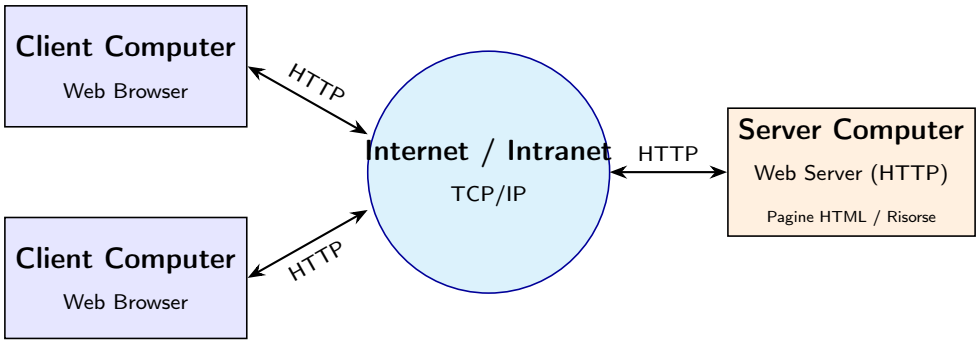


Figura 82: Architettura Internet-based: client browser che dialogano con server web via protocollo HTTP.

6.3.3 Architetture Multi-Livello: Dal Two-Tier all'n-Tier

Una Prospettiva Strutturale sul Modello C/S. Le architetture *n*-**tier** costituiscono una modalità strutturata di osservare e ripartire le responsabilità logiche e fisiche di un sistema Client-Server distribuito. La scomposizione lineare divide nettamente la **logica di presentazione** (interfaccia utente), la **logica di business** (regole applicative) e la **gestione dei dati** (archiviazione e persistenza).

1. **Architettura a Due Livelli (Two-Tier Architecture).** Nel modello classico a due livelli:
   - **Tier 1 (Client)**: ospita contemporaneamente sia la logica di presentazione grafica, sia le regole applicative e di business (*fat client*);
   - **Tier 2 (Server)**: ospita unicamente il database centrale e il motore di memorizzazione;
   - **Limite critico**: se cambiano le regole di business aziendali, è necessario reinstallare e ridistribuire l'intera applicazione su tutte le singole postazioni client.

2. Architettura a Tre Livelli (Three-Tier Architecture). Per superare la rigidità del modello a due livelli, la logica applicativa viene scorporata e collocata su un livello intermedio dedicato:

1. **Client Tier (Livello Client)**: implementa unicamente la presentazione grafica (*presentation logic*). È un thin client che raccoglie gli input dell'utente e mostra i risultati.
2. **Middle Tier Server (Livello Intermedio Applicativo)**: server dedicato che incapsula integralmente le regole di business e gli algoritmi di calcolo (*business rules*).
3. **Data Server (Livello Dati)**: server che conserva e gestisce i dati aziendali garantendone la persistenza e l'integrità.

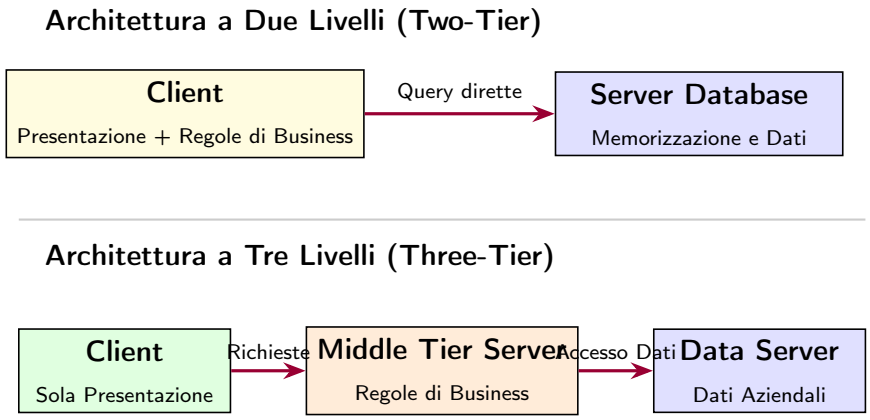


Figura 83: Confronto strutturale tra architetture Two-Tier e Three-Tier.

3. Architettura n-Tier (Multi-Tier). Quando il livello intermedio (*middle tier*) viene a sua volta frazionato e articolato su molteplici strati specializzati cooperanti (ad esempio: server web di front-end, microservizi applicativi, cache distribuita in-memory, bus di messaggistica e gateway di integrazione), il sistema assume formalmente la denominazione di **Architettura n-Tier**.

Proprietà Architetturali del Modello n-Tier

- **Flusso lineare dell'informazione**: il flusso dei dati e delle chiamate procede in modo sequenziale e ordinato da un tier al tier adiacente.
- **Sostituibilità e aggiornamento indipendente**: ciascun livello può essere aggiornato, modificato o sostituito (ad esempio passando da un database a un altro, o rinnovando la grafica del client) in modo totalmente isolato, senza alterare gli altri livelli.
- **Manutenibilità centralizzata**: le modifiche alle logiche di business vengono rilasciate unicamente sul middle-tier, rendendosi immediatamente attive per tutti gli utenti senza interventi sui terminali client.
- **Pro e Contro**: riflettono fedelmente quelli del paradigma Client-Server, garantendo eccellente modularità al prezzo di una maggiore complessità di configurazione e coordinamento.

6.3.4 Approfondimento sulle Architetture a Strati (Layered Architecture)

Il Concetto di Macchina Astratta. L'architettura a strati (**Layered Architecture**) modella il sistema come una successione ordinata di **macchine astratte** (*abstract machines*). Ciascun livello offre una vista semplificata e a maggior grado di astrazione rispetto al livello sottostante, nascondendone la complessità implementativa:

- Non si configura come un'alternativa distruttiva rispetto ai modelli visti in precedenza (come Client-Server o Repository), ma come una **modalità ortogonale e complementare** per strutturare e governare i moduli all'interno di tali architetture;
- **Regola fondamentale di accesso**: i moduli e le funzioni appartenenti a un determinato strato possono accedere **esclusivamente** alle operazioni definite all'interno del proprio livello o in quello **immediatamente inferiore (o interno)**;
- **Inconsapevolezza verso l'alto**: gli strati interni non hanno alcuna visibilità né consapevolezza dell'esistenza degli strati esterni (*inner layers are not aware of outer layers*).

6.3.5 Architetture di Rete a Strati: OSI e TCP/IP

Definizione di Architettura di Rete. Un'**architettura di rete a strati** definisce l'insieme formale di livelli e protocolli impiegati per abilitare la comunicazione trasparente tra processi paritetici (*peer processes*) residenti su nodi fisicamente distinti:

- La comunicazione orizzontale tra livelli corrispondenti su macchine diverse è una **comunicazione virtuale** regolata da protocolli di pari livello;
- La comunicazione effettiva avviene unicamente in verticale, scendendo lungo lo stack della macchina mittente fino al supporto fisico e risalendo lungo lo stack della macchina destinataria.

Il Modello di Riferimento OSI (Open Systems Interconnection). Elaborato dall'ente di standardizzazione internazionale **ISO**, il modello di riferimento OSI definisce una suddivisione canonica a **7 livelli**. Pur non essendosi affermato commercialmente come implementazione dominante, costituisce il quadro concettuale universale per discutere, progettare e confrontare le reti di calcolatori:

Livello	Nome Strato	Descrizione e Compiti Fondamentali
7	**Application Layer**	Interfaccia verso l'applicazione utente (es. trasferimento file, web, posta).
6	**Presentation Layer**	Formattazione, codifica, compressione e crittografia/decifratura dei dati.
5	**Session Layer**	Apertura, sincronizzazione, gestione e chiusura delle sessioni tra nodi.
4	**Transport Layer**	Segmentazione dei messaggi in pacchetti, riordino e controllo affidabilità end-to-end.
3	**Network Layer**	Instradamento (*routing*) dei pacchetti attraverso nodi intermedi e indirizzamento logico.
2	**Data Link Layer**	Controllo degli errori di trasmissione su singolo canale fisico, framing e indirizzi MAC.
1	**Physical Layer**	Trasmissione fisica dei segnali elettrici, ottici o radio sul mezzo di trasmissione.

6.4 Architettura Applicativa e Modelli di Controllo

6.4.1 Application Architecture e Modelli di Controllo

Definizione di Architettura Applicativa. L'**Application Architecture** identifica la struttura organizzativa interna del software applicativo, ovvero del codice realizzato direttamente dagli sviluppatori per soddisfare i requisiti degli utenti. In essa si distinguono due dimensioni primarie:

1. I **Modelli Architetturali**: la scomposizione logica del dominio in classi, funzioni o moduli;
2. I **Modelli di Controllo (*Control Models*)**: definiscono **come il flusso di esecuzione viene governato e orchestrato** tra i vari moduli componenti affinché operino in modo corretto e coordinato.

Il Controllo Centralizzato (*Centralized Control*). Nei sistemi a controllo centralizzato esiste un netto squilibrio gerarchico tra i componenti:

- I moduli ordinari sono entità puramente **passive**: non possono avviare autonomamente alcuna elaborazione;
- L'esecuzione di un modulo comincia solo quando viene esplicitamente comandata da un modulo di controllo supervisore;
- Al termine del proprio lavoro, il modulo restituisce il controllo e i risultati ottenuti al supervisore.

Si distinguono formalmente due modelli di controllo centralizzato: il **Call and Return Model** e il **Manager Model**.

6.4.2 Il Modello Call and Return

Funzionamento Procedurale ad Albero. Il modello **Call and Return** (Chiamata e Ritorno) costituisce lo schema classico dei programmi sequenziali strutturati:

1. Il programma principale (`main`) viene avviato dal sistema operativo;
2. Il main invoca una o più routine o funzioni di livello inferiore;
3. Ciascuna sotto-funzione può a sua volta invocare ulteriori funzioni più specializzate;
4. Il risultato di ciascuna funzione viene restituito al chiamante non appena l'elaborazione termina;
5. L'applicazione cessa la propria esecuzione quando il programma principale restituisce il controllo finale.

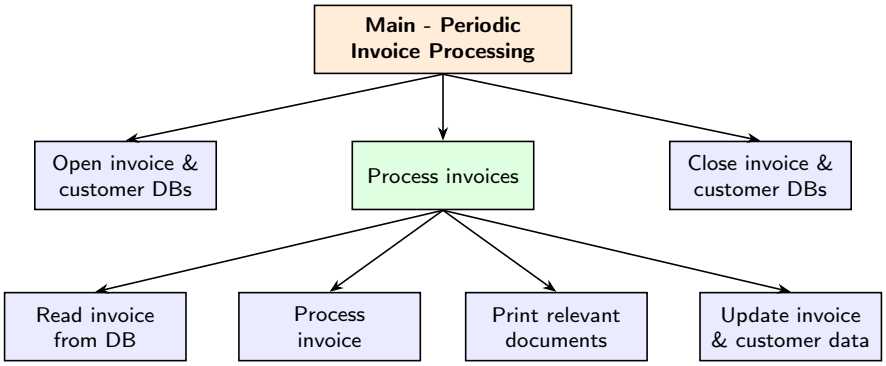


Figura 84: Esempio di modello Call and Return: albero di decomposizione gerarchica per la fatturazione.

Regole di Progettazione nel Modello Call and Return

- **Funzioni Non-Foglia come Supervisori**: qualsiasi routine che contenga chiamate verso altre funzioni agisce come modulo di controllo locale per le proprie sotto-funzioni;
- **Attenzione alla Ricorsione**: la ricorsione deve essere utilizzata con estrema cautela per prevenire la saturazione dello stack di memoria (*stack overflow*);
- **Estensione Distribuita**: il modello si applica identicamente ad ambienti di rete tramite le chiamate di procedura remota (**RPC**), dove il chiamante attende che il server remoto completi la routine e restituisca i dati.

6.4.3 Il Modello Manager

Funzionamento a Ciclo Infinito (Never-Ending Loop). Il modello **Manager** è specificamente concepito per sistemi industriali di automazione e sistemi **Real-Time**:

- Il modulo supervisore (**System Manager**) coordina l'avvio, l'arresto e la sincronizzazione di tutti gli altri moduli (sottosistemi, funzioni o oggetti periferici);

- Il supervisore esegue un **ciclo di controllo infinito** (*never-ending control loop*);
- A ogni iterazione del ciclo, il manager campiona e interroga periodicamente lo stato di sensori o sorgenti di input (*polling*), elabora le grandezze acquisite e impartisce comandi diretti agli attuatori.

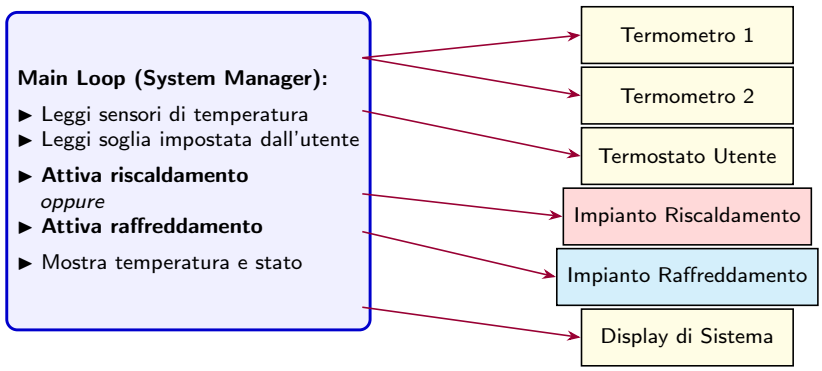


Figura 85: Esempio di modello Manager: ciclo di controllo termico su sensori, attuatori e display.

6.4.4 Modelli di Controllo Guidati da Eventi (Event-Driven Control)

Concetto di Evento e Registrazione dei Moduli. Nei modelli di controllo guidati da eventi (**Event-Driven Control**), il flusso di esecuzione non segue un percorso sequenziale prefissato, ma è determinato dal verificarsi asincrono di stimoli:

- **Definizione di Evento**: un accadimento che si verifica **al di fuori del controllo diretto** dei moduli preposti a gestirlo (es. pressione di un tasto, segnale di un timer hardware, arrivo di un pacchetto di rete);
- **Registrazione preventiva**: i moduli software interessati si registrano all'interno del sistema comunicando preventivamente quali tipologie di eventi sono in grado di gestire;
- **Event Handler (Gestore di Sistema)**: per orchestrare la ricezione e l'instradamento degli eventi è sempre richiesto un componente supervisore dedicato (fornito tipicamente dal **Sistema Operativo** o dal framework applicativo).

Si distinguono formalmente due modelli operativi: il **Broadcast Model** e l'**Interrupt-Driven Model**.

6.4.5 Il Modello Broadcast

Funzionamento in Diffusione. Nel **Broadcast Model**, quando un evento viene intercettato:

1. L'evento viene diffuso indistintamente (*broadcasted*) a tutti i moduli registrati capaci di trattare quella categoria di stimoli;
2. Ciascun modulo ricevente esamina i parametri dell'evento e valuta autonomamente la reale necessità di intraprendere un'azione;
3. Se la condizione è soddisfatta, il modulo esegue la propria procedura di risposta in modo indipendente.

Caratteristiche Chiave del Modello Broadcast

- **Elevata Modularità ed Estensibilità**: aggiungere nuovi componenti o definire nuovi tipi di evento è immediato; non è richiesto alcun intervento sui moduli già esistenti;
- **Complessità di Coordinamento**: non esiste una regia centrale sull'ordine con cui i vari moduli rispondono all'evento; la coordinazione globale tra le parti può risultare complessa;
- **Esempio Canonico**: la gestione degli eventi nelle moderne **interfacce grafiche utente (GUI)**, dove il clic del mouse viene propagato attraverso l'albero dei componenti visivi (finestre, pannelli, pulsanti).

6.4.6 Il Modello Guidato da Interrupt (Interrupt-Driven)

6.4.7 Processi Multi-Threading e Concorrenza

Parallelismo e Gestione dei Thread. Nei sistemi moderni, più flussi di esecuzione possono essere attivi concorrentemente:

- **Esecuzione Parallela Reale**: garantita fisicamente dalla presenza di hardware multiprocessore o CPU multi-core;
- **Esecuzione Concorrente su Singolo Processore**: realizzata dal sistema operativo mediante tecniche di *time-sharing* e commutazione di contesto rapida (*context switch*), guidata da priorità associate a ciascun processo.

6.4.8 Modelli di Decomposizione Modulare Applicativa

Quando si scompone un'applicazione software in moduli cooperanti, si adottano prevalentemente due stili di modellazione: il **Modello Data-Flow** e il **Modello Orientato agli Oggetti**.

6.4.9 Il Modello Data-Flow (Flusso di Dati)

Caratteristiche e Pipeline. Il modello **Data-Flow** (analogo allo stile Pipes and Filters) organizza la decomposizione orientandola attorno alla trasformazione progressiva delle informazioni:

- Il sistema si articola come una catena di blocchi di elaborazione (**Filtri**), connessi da canali unidirezionali (**Pipe**);
- I dati viaggiano dal blocco iniziale di input, subiscono manipolazioni successive e confluiscono nei canali di output;
- L'elaborazione può avvenire lungo un'unica linea sequenziale (**Batch Architecture**) oppure ramificarsi in percorsi paralleli e cicli di feedback (**Generic Data-Flow Architecture**).

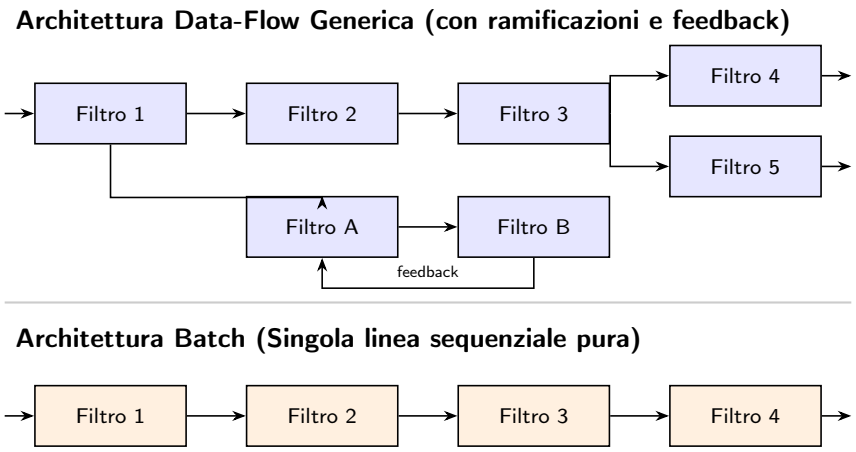


Figura 86: Confronto tra architettura Data-Flow generica (a grafo) e architettura Batch (sequenziale pura).

Valutazione del Modello Data-Flow

Vantaggi (Pros):

- **Semplicità e Intuitività**: il flusso di trasformazione corrisponde fedelmente al modo naturale di descrivere un processo di lavorazione;
- **Riusabilità dei Componenti**: ciascun filtro è indipendente e può essere riutilizzato all'interno di pipeline differenti;
- **Esecuzione Concorrente**: i filtri possono iniziare a produrre output parziale mentre l'input è ancora in elaborazione, operando in parallelo su thread distinti.

Svantaggi e Limiti (Cons):

- Non tutte le applicazioni possono essere modellate comodamente come una trasformazione lineare di flussi;
- **Gestione degli Errori Complessa**: se un'eccezione si verifica a metà catena, coordinare il rollback o notificare l'anomalia lungo la pipe risulta estremamente laborioso;
- **Incompatibilità dei Formati**: se i filtri intermedi adottano formati dati discordanti, si generano gravosi costi computazionali di parsing continuo.

6.4.10 Il Modello Orientato agli Oggetti (Object-Oriented)

Fondamenti del Modello. Nel modello orientato agli oggetti, la decomposizione del software è rigorosamente **centrata sui dati**:

- L'**Oggetto** costituisce l'unità elementare di scomposizione architetturale;

- Ciascun oggetto racchiude al proprio interno: dati di stato privati, operazioni eseguibili (*metodi*) e un'interfaccia pubblica di comunicazione;
- La cooperazione si realizza mediante **scambio di messaggi** (*message passing*);
- Poggia sui pilastri di:
  - **Incapsulamento**: totale occultamento dei dettagli algoritmici interni;
  - **Ereditarietà**: definizione di nuove classi a partire da classi esistenti specificandone unicamente le differenze.

Punti di Forza dell'Approccio a Oggetti.

- **Omogeneità Metodologica**: il medesimo modello concettuale a oggetti accompagna l'intero ciclo di vita (dall'analisi al design, fino alla codifica e ai test);
- **Stabilità della Specifica**: all'interno di un'organizzazione o dominio aziendale, le entità dei dati (oggetti) mutano molto più lentamente rispetto alle singole procedure o funzioni temporanee;
- **Isolamento delle Modifiche**: variazioni all'implementazione interna di un metodo hanno impatto trascurabile sul resto del sistema;
- **Riuso di Framework**: elevato potenziale di riuso tramite librerie e gerarchie di classi collaudate.

Criticità e l'Anti-Pattern del "Ravioli Code". Accanto ai benefici, il modello a oggetti introduce sfide architetturali non banali:

- **Overhead Prestazionale**: la frammentazione e il continuo passaggio di messaggi tra migliaia di istanze possono imporre un sensibile sovraccarico di tempo macchina;
- **Curva di Apprendimento**: la transizione dalla programmazione procedurale al paradigma a oggetti richiede un cambio di mentalità spesso ostico per gli sviluppatori.

L'Anti-Pattern Architetturale: "Ravioli Code"

Analogamente allo **Spaghetti Code** (tipico del paradigma procedurale, caratterizzato da un unico flusso contorto e inestricabile di salti e istruzioni), nel paradigma orientato agli oggetti si manifesta il rischio speculare del **Ravioli Code**:

- Si presenta quando il sistema viene iper-frammentato in centinaia di minuscole classi composte da pochissime righe di codice;
- Presa singolarmente, ciascuna classe appare perfetta e ben incapsulata (proprio come un raviolo);
- Tuttavia, **comprendere la logica globale del sistema diventa un incubo disordinato**: il flusso di esecuzione rimbalza incessantemente da un piccolo oggetto all'altro senza che esista una visione d'insieme chiara.

Regola: l'incapsulamento non deve mai compromettere la leggibilità del flusso globale.

Anti-Pattern	Paradigma d'Origine	Difetto Architetturale Principale
Spaghetti Code	Programmazione Procedurale	Assenza di modularità: blocchi monolitici contorti e privi di struttura.
Ravioli Code	Programmazione Orientata agli Oggetti	Eccesso di frammentazione: miriadi di classi minuscole che rendono opaco il flusso.

6.5 Principi Guida del Design

6.5.1 Principi Guida del Design: Astrazione, Scala e Strategie

I Due Concetti Fondamentali: Astrazione e Scala. Per governare sistemi software complessi, il progettista applica due leve cognitive essenziali:

- **Astrazione (*Abstraction*)**: consiste nell'isolare e prendere in considerazione **unicamente gli aspetti rilevanti** per risolvere il problema corrente, trascurando, nascondendo o posticipando tutti i dettagli accessori non pertinenti a quel livello di analisi.
- **Scala (*Scale*)**: capacità di osservare il medesimo sistema a **diversi ordini di grandezza** e ingrandimento, decomponendo il software in macro-sottosistemi, questi in sotto-sottosistemi, e così via fino ad arrivare alle singole istruzioni elementari.

Astrazione Procedurale vs Astrazione sui Dati. La formalizzazione del software si declina lungo due direttrici storiche:

- **Astrazione Procedurale (*Procedural Abstraction*)**:
  - Il sistema è scomposto in termini di **funzioni e procedure** operative;
  - Ciascuna funzione è univocamente definita dalla propria **segnatura** (*signature*: nome, lista argomenti e tipo di ritorno);
  - Il modulo chiamante invoca la funzione conoscendone lo scopo senza doversi preoccupare del corpo algoritmico interno.
- **Astrazione sui Dati (*Data Abstraction*)**:
  - Si concretizza nella formalizzazione dei **Tipi Astratti di Dati (ADT - Abstract Data Types)**;
  - Un tipo di dato viene definito formalmente **attraverso le operazioni** che è lecito applicare sui suoi oggetti;
  - Vengono specificate unicamente le segnature delle operazioni; costituisce la base su cui poggiano le classi nei linguaggi orientati agli oggetti.

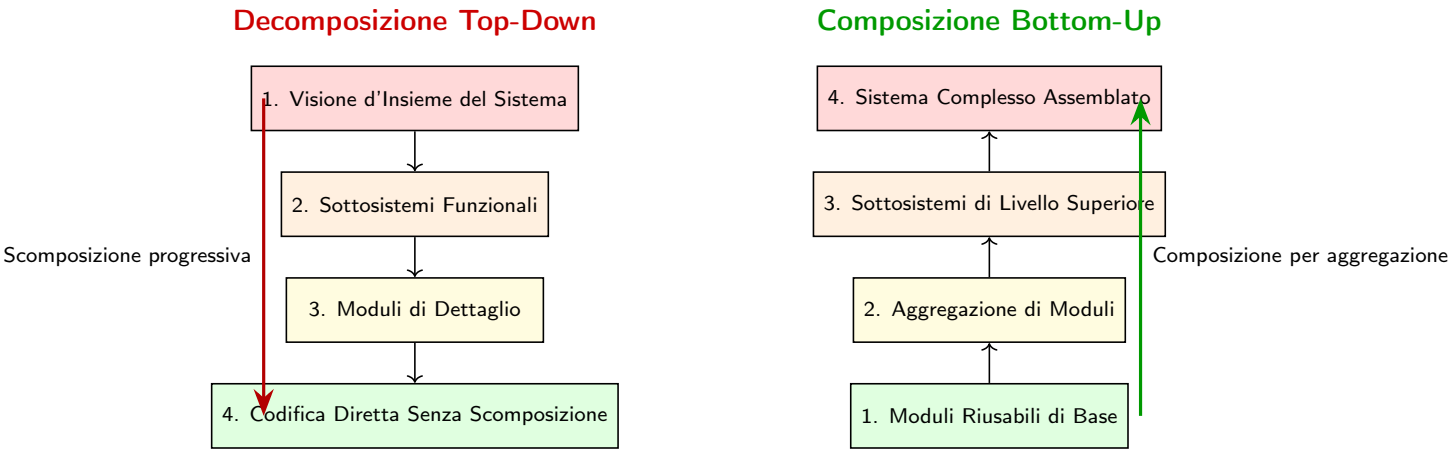


Figura 87: Confronto concettuale tra le strategie di sviluppo Top-Down e Bottom-Up.

Strategie di Decomposizione: Top-Down vs Bottom-Up. Nel condurre la scomposizione modulare di un'architettura si possono adottare due percorsi progettuali complementari:

- **Top-Down Design (Dall'Alto verso il Basso)**: si parte dalla specifica globale del sistema e la si decompone ricorsivamente in sottosistemi di granularità via via più fine, arrestando il processo quando i moduli elementari ottenuti possono essere implementati direttamente senza ulteriore scomposizione.
- **Bottom-Up Design (Dal Basso verso l'Alto)**: si parte da un insieme di moduli riusabili e collaudati già disponibili a basso livello, aggregandoli progressivamente per costruire componenti di astrazione superiore fino a completare il sistema finale.

6.5.2 Valutazione delle Alternative e Raffinamento Iterativo

Raffinamento Iterativo (Iterative Refinement). La definizione dell'architettura non scaturisce in forma perfetta al primo tentativo, ma richiede un processo di **raffinamento iterativo progressivo**:

- **Sulle Gerarchie di Classi**: si parte definendo astrazioni ampie e generali, aggiungendo iterativamente classi sempre più specializzate per accogliere nuove esigenze;
- **Sui Metodi e sulle Operazioni**: si parte dichiarando segnature di alto livello concettuale, raffinando progressivamente il loro comportamento in sequenze dettagliate di istruzioni concrete.

6.5.3 Quadro Sinottico Finale del Modulo 2

A coronamento dell'intero modulo didattico, la tabella seguente riassume la corrispondenza tra i livelli architetturali, i modelli strutturali analizzati e i rispettivi meccanismi di controllo:

Dimensione	Modello Architetturale	Meccanismo di Connessione	Modello di Controllo
Hardware	Single, Mainframe, C/S, Embedded, Parallel.	Bus, canali seriali, reti LAN, cavi fisici o wireless.	Driver, polling o gestione interrupt di sistema.
Software di Sistema	Layered, Repository, Web Server, Internet/Intranet.	Middleware (RPC, ORB), stack TCP/IP, protocollo HTTP.	Gestione centralizzata o instradamento asincrono.
Software Applicativo	Data-Flow (Pipes and Filters) vs Object-Oriented.	Stream di byte (Pipe) oppure invocazione metodi.	Call-and-Return, Manager, Broadcast o Interrupt.

7 Java Strategy Pattern

7.1 Motivazioni e Fondamenti: Polimorfismo e Scoping

L'implementazione del pattern **Strategy** in Java rappresenta uno dei punti di contatto più profondi tra i principi architetturali dell'ingegneria del software e i meccanismi concreti forniti dal linguaggio. A differenza di linguaggi procedurali o ibridi come il C e il C++ (in cui la selezione dinamica del comportamento è tradizionalmente affidata a **puntatori a funzione** o template a tempo di compilazione), Java adotta un modello puramente orientato agli oggetti:

- Il comportamento non viaggia isolato come indirizzo di memoria, ma viene incapsulato all'interno di **oggetti polimorfi**;
- Il meccanismo abilitante risiede nella combinazione di **overriding** dei metodi e **late binding** (risoluzione dinamica delle chiamate a runtime);
- L'obiettivo ingegneristico consiste nel padroneggiare *come* e *perché* determinate scelte architetturali vengono adottate per disaccoppiare la logica computazionale, evitando soluzioni grezze (*hacking*).

Recap dei Pattern Visti Finora.

- **Singleton**: vincola l'unicità dell'istanza di una classe;
- **Visitor**: applica una funzione a tutti gli elementi di una struttura (ad esempio la funzione `map`);
- **Strategy**: polimorfismo via *overriding*.

Il Riferimento Concettuale: Le Chiusure Lessicali di Smalltalk. Le radici concettuali del passaggio di comportamenti affondano nel linguaggio **Smalltalk**, dove blocchi di codice autonomi (*blocks*) agiscono come oggetti di prima classe dotati di chiusura lessicale (*lexical scoping*):

```
testScope
    | t |
    t := 42.
    self testBlock: [ Transcript show: t printString ]

testBlock: aBlock
    | t |
    t := nil.
    aBlock value
```

All'esecuzione, la chiamata a `testScope` stampa **42** (e non `nil`). Il blocco `[ ... ]` cattura stabilmente la variabile `t` del contesto in cui è stato **definito**, rimanendo del tutto immune alla ridefinizione locale presente in `testBlock:`. Questo modello di incapsulamento ed isolamento di esecuzione costituisce l'obiettivo ideale a cui mira la gestione dinamica delle strategie.
In Smalltalk il **chunk** (blocco) permette quindi di usare il corpo di una funzione come parametro da passare. Un'idea analoga, in forma molto più limitata, esiste già in C: l'espressione `a = b = c = d = 50` viene valutata da destra verso sinistra, perché l'assegnazione è essa stessa un'espressione che restituisce un valore (prima `d = 50`, poi `c = (d = 50)`, e così via). In Java e in C++ lo stesso

obiettivo si raggiunge con le classi annidate; in C++ però l'annidamento ha rilevanza solo per la **visibilità**, mentre in Java si distinguono i tre tipi di classe descritti di seguito (nested, inner, local).

7.2 Classi in Ambiti Annidati (Inner Scopes)

Differenza Concettuale tra C++ e Java. Sia Java che C++ consentono di dichiarare la definizione di una classe all'interno di un'altra classe:

- **In C++**: l'annidamento ha una rilevanza legata primariamente alla **visibilità** e al **namespacing**. Una classe annidata è concettualmente un'entità indipendente inserita nello scope di nomi della classe contenitrice; non possiede alcun legame implicito con le istanze di quest'ultima.
- **In Java**: il meccanismo è nettamente più articolato e distingue tre livelli di astrazione:
  1. **Static Nested Classes**: classi statiche annidate per ragioni di incapsulamento e raggruppamento logico;
  2. **Inner Classes**: classi non statiche le cui istanze dipendono intrinsecamente dall'esistenza di un'istanza della classe contenitrice;
  3. **Local Classes**: classi dichiarate localmente all'interno di un blocco di codice (es. all'interno di un metodo).

Tassonomia delle Classi in Ambiti Interni in Java

- **Nested Classes (Static)**: dichiarate con la parola chiave `static` nel corpo della classe principale. Non mantengono alcun riferimento implicito all'istanza dell'outer class e possono accedere solo ai suoi membri statici.
- **Inner Classes (Non-Static)**: dichiarate senza `static`. Ogni loro istanza è legata a una specifica istanza dell'outer class, di cui può leggere e modificare tutti i campi (anche privati).
- **Local Classes**: inner class definite all'interno del corpo di un metodo, costruttore o blocco esecutivo (`if`, `for`).
- **Anonymous Classes**: classi locali prive di identificatore, istanziate al volo all'interno di una singola espressione.

Tipologia	Punto di Dichiarazione	Dipendenza da Istanza Outer	Membri Statici Ammessi
Static Nested Class	Livello di classe (con `static`)	Nessuna (autonoma)	Sì, tutti.
Member Inner Class	Livello di classe (senza `static`)	Obbligatoria (`Outer.this`)	No (solo costanti `static final`).
Local Class	All'interno di un metodo/blocco	Se in contesto non-statico	No (solo costanti `static final`).
Anonymous Class	All'interno di un'espressione	Se in contesto non-statico	No (solo costanti `static final`).

7.3 Classi Statiche Annidate (Static Nested Classes)

Definizione e Regole di Visibilità. Una **static nested class** consente di nascondere l'esistenza di un tipo rispetto al mondo esterno e di raggruppare logicamente classi correlate all'interno del medesimo modulo.

- Il nome della classe annidata è locale allo scope della classe contenitrice (es. `Outer.Nested`);
- Può accedere **esclusivamente ai membri statici** della classe contenitrice;
- **Non ha accesso diretto** ai campi o metodi di istanza dell'outer class, poiché non possiede alcun riferimento implicito a un oggetto contenitore.

Nota Storica. Le classi annidate statiche sono funzionalmente simili ai `package`/`namespace`: furono introdotte perché il C++ era molto in voga ma all'epoca non esistevano ancora i namespace. Servono solo per **visibilità** (possono essere nascoste all'esterno della classe) e accedono unicamente alle variabili statiche della classe contenitrice.

Confronto di Sintassi: Java vs C++. Il frammento seguente evidenzia la differenza tra classi interne e classi statiche annidate in Java:

```
1  public class X {
2      int instanceVar;        // Appartiene a una specifica istanza di X
3      static int staticVar;   // Appartiene alla classe X
4
5      // 1. Inner class (non-statica): accede sia a stato d'istanza che
       statico
6      private class Inner {
7          int y;
8          Inner() {
9              y = instanceVar + staticVar; // Pienamente lecito
10         }
11     }
12
13     // 2. Static nested class: accede ESCLUSIVAMENTE ai membri statici
14     private static class Nested {
15         int x;
```

```
        Nested() {
            x = staticVar;              // Lecito
            // x += instanceVar;        // ERRORE DI COMPILAZIONE!
        }
    }
}
```

In C++, la semantica coincide naturalmente con la variante statica di Java:

```
class X {
public:
    int instanceVar;
    static int staticVar;

    class Nested {
        int x;
        Nested() {
            x = staticVar;          // Lecito
            // instanceVar++;       // ERRORE: nessun puntatore this a X
        }
    };
};
```

Pattern Applicativi delle Static Nested Classes. L'impiego delle classi statiche annidate risponde a tre esigenze primarie di ingegnerizzazione:

1. **Namespacing e Classi Helper**: raggruppare funzioni di utilità correlate sotto un unico namespace logico evitando l'inquinamento del package globale.

```
class Strings {
    static class Joiner {
        static String by(char sep, String... parts) {
            return String.join(String.valueOf(sep), parts);
        }
    }
    static class Trimmer {
        static String left(String s, int n) {
            return (s.length() <= n) ? s : s.substring(0, n);
        }
    }
}
// Utilizzo: Strings.Joiner.by(',', "a", "b");
```

2. **Builder Pattern**: disaccoppiare la configurazione dei parametri di creazione dall'oggetto immutabile finale, consentendo una sintassi fluente.

```
class HttpRequest {
    final String url;
    final int timeoutMs;

    private HttpRequest(String url, int timeoutMs) {
```

```
        this.url = url;
        this.timeoutMs = timeoutMs;
    }

    static class Builder {
        private String url;
        private int timeoutMs = 3000;

        Builder url(String u) { this.url = u; return this; }
        Builder timeout(int ms) { this.timeoutMs = ms; return this; }
        HttpRequest build() { return new HttpRequest(url, timeoutMs);
    }
    }
}
// Utilizzo: new HttpRequest.Builder().url("https://ex").timeout
   (1000).build();
```

3. **Private Type Hiding (Incapsulamento Rigoroso)**: celare strutture dati e tipi ausiliari all'esterno del componente client.

```
class Cache {
    private static class Entry {
        final String key, value;
        final long ts;
        Entry(String k, String v) {
            key = k; value = v; ts = System.currentTimeMillis();
        }
    }

    private final java.util.Map<String, Entry> map = new java.util.
   HashMap<>();

    void put(String k, String v) { map.put(k, new Entry(k, v)); }
    String get(String k) {
        Entry e = map.get(k);
        return (e == null) ? null : e.value;
    }
}
```

7.4 Classi Interne (Inner Classes)

Legame con l'Istanza Esterna. Una classe interna (**Member Inner Class**) non è decorata con `static` ed è strettamente associata all'oggetto contenitore:

- Ogni istanza della classe interna possiede un **riferimento implicito** all'istanza della classe esterna che l'ha generata;
- Può accedere direttamente a tutti i campi e metodi dell'istanza esterna, anche se privati;
- Per fare esplicito riferimento all'istanza della classe contenitrice si impiega la sintassi formale:

`NomeClasseEsterna.this`

- L'istanziazione diretta dall'esterno richiede obbligatoriamente un oggetto della classe outer:

```
Outer outer = new Outer();
Outer.Inner inner = outer.new Inner();
```

Vincoli Strutturali.

- Una inner class **non può dichiarare membri statici** (metodi, campi o blocchi statici), ad eccezione di costanti a tempo di compilazione (`static final`);
- Non può avere lo stesso identificatore della classe che la racchiude;
- La sua esistenza riflette una stretta cooperazione in memoria tra l'oggetto racchiuso e quello contenitore.

Perché Non Possono Avere Membri Statici. Una inner class esiste solo in presenza di un oggetto della classe contenitrice. La differenza con il C emerge dal modello di caricamento: in C il file eseguibile (`.out`) carica tutto all'avvio (in modo semplificato), mentre in Java ogni classe è compilata in un **ClassFile** e i suoi membri statici vengono inizializzati quando la **classe viene caricata**. Un membro statico di una inner class dovrebbe invece esistere prima di qualunque istanza dell'outer, e per questo non è ammesso. Le inner class possono essere **anonime** e **locali**.

Pattern Applicativi delle Inner Classes.

1. **Accesso Condiviso allo Stato dell'Outer Instance**:

```
class Counter {
    private int value = 0;

    class Incrementer {
        void inc() { value++; }              // Modifica il campo
   dell'outer class
        Counter outer() { return Counter.this; } // Restituisce l'
   istanza outer
    }

    int next() {
        new Incrementer().inc();
        return value;
    }
}
```

2. **Implementazione di Iteratori Personalizzati**: l'iteratore necessita di navigare le collezioni interne della classe preservando l'information hiding verso l'esterno.

```
class IntRange implements Iterable<Integer> {
    private final int start, end;
    IntRange(int s, int e) { start = s; end = e; }

    class It implements java.util.Iterator<Integer> {
        int cur = start;
        public boolean hasNext() { return cur <= end; }
        public Integer next() {
            if (!hasNext()) throw new java.util.
   NoSuchElementException();
            return cur++;
        }
    }

    public java.util.Iterator<Integer> iterator() { return new It();
   }
}
```

3. **Event Handler e GUI Listener**: gestione di reazioni grafiche che necessitano di aggiornare direttamente lo stato del pannello contenitore.

```
class ClickCounter {
    private int clicks = 0;

    class Handler implements java.awt.event.ActionListener {
        public void actionPerformed(java.awt.event.ActionEvent e) {
            clicks++;
            System.out.println("Clicks = " + clicks);
        }
    }

    javax.swing.JButton makeButton() {
        javax.swing.JButton b = new javax.swing.JButton("Click");
        b.addActionListener(new Handler());
        return b;
    }
}
```

7.5 Classi Locali (Local Classes)

Incapsulamento a Livello di Blocco. Una **classe locale** è una particolare variante di inner class dichiarata direttamente all'interno di un blocco di codice eseguibile: tipicamente nel corpo di un metodo, di un costruttore o persino all'interno di un costrutto decisionale o iterativo (`if`, `for`, `while`).

- **Scope circoscritto**: la classe è visibile e utilizzabile **soltanto all'interno del blocco** in cui è dichiarata;
- **Ciclo di vita dell'istanza**: sebbene il tipo sia noto solo nel blocco, un oggetto della classe locale può sopravvivere al metodo qualora venga restituito come interfaccia o sottomesso a un thread;

- **Incapsulamento funzionale**: consente di realizzare strutture dati o algoritmi helper temporanei senza renderli visibili al resto della classe o del package.

Funzioni dentro Funzioni. Una classe dentro una funzione è lecita; una *funzione* dentro una funzione è possibile ma complessa. In C++ si ottiene solo con un trucco, tramite un'estensione dell'ultima versione di GCC. Una classe locale non può essere `static`: non avrebbe un'istanza dell'outer da cui allocare memoria.

Regole Sintattiche e Compilazione.

- Non possono essere marcate con modificatori di accesso (`public`, `protected`, `private`);
- Non possono essere dichiarate `static`;
- Non possono dichiarare membri statici, salvo costanti `static final` inizializzate con espressioni costanti a tempo di compilazione;
- Il compilatore Java genera un file bytecode dedicato adottando una convenzione di denominazione numerata:

`OuterClass$1LocalClass.class`

Regola di Accesso: Membri Outer e Variabili Locali.

- Una classe locale ha pieno accesso a tutti i membri della classe che la racchiude (inclusi campi privati);
- Se la classe locale è dichiarata all'interno di un metodo **non-statico**, ha a disposizione il riferimento `Outer.this`; se dichiarata in un metodo **statico**, può accedere solo ai membri statici della classe outer;
- Può accedere ai parametri formali e alle variabili locali del metodo contenitore **unicamente se sono `final` o *effectively final*** (ovvero valorizzate una sola volta e mai riassegnate successivamente).

```
class UserService {
    boolean isUsernameOk(String u) {
        // Classe locale: visibile e utilizzabile solo dentro
   isUsernameOk
        class NonEmptyAlphaValidator {
            boolean ok(String s) {
                if (s == null || s.isEmpty()) return false;
                for (char c : s.toCharArray()) {
                    if (!Character.isLetter(c)) return false;
                }
                return true;
            }
        }

```

```
        NonEmptyAlphaValidator v = new NonEmptyAlphaValidator();
        return v.ok(u);
    }
}
```

7.6 Contesti Statici e Non-Statici nelle Classi Annidate

Esempio di Utilizzo: Validatore Ad-Hoc.

Panoramica e Tassonomia delle Inner Class. Le classi interne (non-statiche) sono dichiarate senza la parola chiave `static` e risultano intrinsecamente vincolate a un'istanza della classe che le racchiude. La gerarchia include tre varianti:

- **Member Inner Classes**: dichiarate a livello di corpo di classe, esattamente come un campo o un metodo d'istanza.
- **Local Classes**: dichiarate all'interno di un blocco di codice circoscritto (metodo, costruttore o blocco logico).
- **Anonymous Classes**: classi locali prive di identificatore, definite e istanziate istantaneamente all'interno di una singola espressione per compiti usa-e-getta.

Esecuzione in Contesto Statico vs Non-Statico. L'accesso ai membri della classe contenitrice dipende strettamente dal contesto in cui la classe interna viene dichiarata ed eseguita:

- **Contesto Non-Statico (metodi d'istanza, costruttori, inizializzatori)**: il codice viene eseguito con un riferimento `this` associato all'istanza dell'outer class. La classe interna dispone automaticamente di `Outer.this` e può accedere a tutti i membri d'istanza (campi e metodi, anche privati) della classe esterna.
- **Contesto Statico (metodi statici, inizializzatori statici)**: non esiste alcuna istanza outer associata. Di conseguenza, le classi locali o anonime dichiarate in un contesto statico non possono accedere ai membri d'istanza di `Outer`, ma unicamente ai suoi membri statici.

```
class Outer {
    private int v = 10;

    int f() {
        // Contesto non-statico: new L() ha accesso a v tramite Outer.
    this
        class L {
            int h() { return v; } // OK
        }
        return new L().h();
    }

    static int g() {
        // Contesto statico: nessuna istanza di Outer disponibile
        class L {
```

```
            // int h() { return v; } // ERRORE DI COMPILAZIONE: v non e statico
            int h() { return 0; }   // OK: usa solo membri statici o propri
        }
        return new L().h();
    }
}
```

7.7 Cattura delle Variabili Locali: Final ed Effectively Final

Il Principio di Cattura. Le classi locali e anonime possono utilizzare le variabili locali e i parametri definiti nel metodo circostante, ma a una condizione tassativa: tali variabili devono essere dichiarate `final` oppure essere ***effectively final***. Una variabile è *effectively final* se, pur priva del modificatore esplicito `final`, riceve un valore una sola volta e non viene mai più riassegnata nel corso della sua esistenza.

Perché le Variabili Catturate Devono Essere (Effectively) Final?

La motivazione risiede nel divario tra il ciclo di vita dello **stack** e dello **heap**:

- Le variabili locali di un metodo risiedono sullo **stack** di esecuzione e vengono deallocate immediatamente non appena il metodo termina.
- Un'istanza di una classe locale o anonima risiede nello **heap** e può sopravvivere alla fine del metodo (ad esempio se restituita all'esterno, registrata come callback o sottomessa a un pool di thread concorrenti).
- Per consentire all'oggetto di continuare ad accedere a tali valori, il compilatore crea una **copia sintetica** di ciascuna variabile catturata all'interno dei campi generati della classe interna.
- Se la variabile locale potesse essere riassegnata dopo la creazione dell'istanza, la copia interna e la variabile originale divergerebbero, distruggendo la consistenza dei dati. Il vincolo di finalità garantisce che il valore copiato sia immutabile e sempre coerente.

```
void run() {
    int base = 10; // effectively final
    class Task {
        void print() { System.out.println(base); } // OK: cattura base
    }
    new Task().print();
}

void fail() {
    int base = 10;
    base++; // La riassegnazione rompe l'effective finality!
    class Task {
```

```
        // ERRORE: local variables referenced from an inner class must be final
        void print() { System.out.println(base); }
    }
}
```

Strategie di Risoluzione del Vincolo (Fix Pattern). Nel codice concorrente o asincrono capita di dover modificare lo stato o aggiornare contatori prima dell'invio di un task a un esecutore. Esistono due soluzioni ingegneristiche per superare il blocco del compilatore:

1. **Snapshotting del Valore (Fix A)**: si assegna il valore corrente a una variabile locale esplicitamente marcata `final` prima della cattura. La variabile originale può continuare a mutare senza alterare lo snapshot catturato dal task.

```
Future<?> scheduleJob() {
    int tries = 0;
    final int snapshot = tries; // Snapshot immutabile per la classe locale

    class Job implements Runnable {
        public void run() { System.out.println(snapshot); }
    }
    tries++; // Lecito: non altera snapshot
    return pool.submit(new Job());
}
```

2. **Boxing dello Stato Mutabile (Fix B)**: si incapsula il dato all'interno di un contenitore reference immutabile, ad esempio un array a elemento singolo (`final int[] box = {0};`) o un wrapper. Il riferimento all'array rimane rigorosamente `final` (soddisfacendo il compilatore), mentre il suo contenuto interno (`box[0]`) può essere modificato liberamente sia dentro che fuori la classe locale.

```
Future<?> scheduleJob() {
    final int[] box = { 0 }; // Reference immutabile a contenuto mutabile

    class Job implements Runnable {
        public void run() { System.out.println(box[0]); }
    }
    box[0]++; // Lecito: muta il contenuto dell'array, non il puntatore
    return pool.submit(new Job());
}
```

7.8 Pattern ed Esempi Applicativi di Classi Locali

Le classi locali trovano impiego pratico in tre scenari principali:

- **Validatori e Helper Monouso**: incapsulano logiche di controllo interne a un metodo senza esporre classi accessorie all'esterno.

- **Algoritmi di Calcolo Parametrici**: classi matematiche temporanee che leggono parametri del metodo *effectively final* senza richiedere campi d'istanza dedicati.
- **Strategy Pattern Locale**: scelta runtime dell'algoritmo direttamente nello scope del metodo chiamante.

Esempio: Strategy Pattern Diretto all'Interno di un Metodo. Il seguente frammento dimostra come realizzare una variante compatta del pattern Strategy senza creare gerarchie pubbliche separate. Le classi concrete che implementano gli algoritmi di ordinamento (`Asc` e `Desc`) sono definite all'interno del metodo `sort` e selezionate dinamicamente:

```
class Sorter {
    void sort(int[] a, boolean descending) {
        class Asc {
            void apply(int[] x) { java.util.Arrays.sort(x); }
        }
        class Desc {
            void apply(int[] x) {
                java.util.Arrays.sort(x);
                for (int i = 0, j = x.length - 1; i < j; i++, j--) {
                    int t = x[i]; x[i] = x[j]; x[j] = t;
                }
            }
        }

        if (descending) {
            new Desc().apply(a);
        } else {
            new Asc().apply(a);
        }
    }
}
```

7.9 Significato di `static` e `final`

Data la dichiarazione `HttpRequestBuilder a = new HttpRequestBuilder()`, a lezione è stata commentata una tabella sul significato dei modificatori:

Modificatore	Su	Significato
`final`	Variabile	Costante: il compilatore non deve allocare spazio in memoria per essa. Protegge da modifiche future accidentali di un valore che deve restare costante.
`static`	Variabile	Condivisa da tutte le istanze: è membro della classe, non del singolo oggetto.
`static`	Metodo	Non richiede un'istanza dell'oggetto.
`static`	Classe	Solo per classi annidate: classe inserita in un'altra classe, senza riferimento all'outer.

Costruttore omesso: se non viene dichiarato, Java fornisce il costruttore di default (che invoca quello del padre).

Confronto con il C. In C `static` ha due significati: su una **variabile globale** la rende visibile solo nel file; su una **variabile locale** a una funzione la rende comune a tutte le chiamate (persiste tra una chiamata e l'altra). Con `extern int x;` si *dichiara* ma non si *definisce* la variabile (non viene allocato spazio).

`final` vs `static final`. Domanda posta a lezione dal docente. `final` da solo indica un campo non riassegnabile ma **per istanza** (può essere inizializzato nel costruttore e ogni oggetto ne ha una copia); `static final` indica una costante **di classe**, unica e condivisa, valorizzata una sola volta al caricamento della classe.

7.10 Errori Comuni e Correzioni: Guida Pratica

L'analisi dei tipici errori di compilazione legati allo scoping e all'istanziazione delle classi annidate permette di fissare le regole cardinali del modello Java:

1. Static Nested Class e Membri Non-Statici.

- **Errore comune**: tentare di accedere direttamente a campi d'istanza della classe contenitrice dall'interno di una classe statica annidata (`return secret + common;`). Poiché la classe è `static`, non dispone di alcun puntatore `this` verso l'esterno.
- **Correzione**: passare esplicitamente l'oggetto esterno come parametro al metodo della classe annidata (`int f(ENested outer) { return outer.secret + common; }`).
- **Risultato**: se `secret = 7` e `common = 3`, l'invocazione di `t.f(outer)` produce `10`, mentre la chiamata statica `Tool.g()` restituisce `6`.

2. Member Inner Class: Istanziazione e Membri Statici.

- **Errore comune**: dichiarare variabili statiche non costanti dentro una inner class (`static int k = 4;`) oppure tentare di istanziare la classe da un metodo statico con un generico `new In()`.
- **Correzione**:
  1. I campi statici nelle inner class sono ammessi solo se costanti di compilazione (`static final int K = 4;`).
  2. L'istanziazione da contesto statico deve ancorarsi esplicitamente a un'istanza esterna tramite la sintassi dedicata: `outer.new In()`.
- **Risultato**: con `v = 1` e costante `K = 4`, `in.h()` calcola `1 + 4 = 5`.

3. Local Class: Campi Statici e Rottura dell'Effective Finality.

- **Errore comune**: dichiarare costanti prive del modificatore `final` all'interno del metodo (`static int FACTOR = 10;`) e riassegnare variabili locali utilizzate dalla classe locale (`base = base + n;`).
- **Correzione**:
  1. Convertire il campo statico della classe locale in costante a tempo di compilazione: `static final int FACTOR = 10;`.
  2. Precalcolare il valore della variabile locale e mantenerlo rigorosamente immutabile: `final int base = 2 + n;` prima della dichiarazione della classe, eliminando successive riassegnazioni.
- **Risultato**: invocando `work(5)` con `label = "X"`, si ottiene `base = 7`; il calcolo `apply(3)` produce $3 \times 7 = 21$, generando l'output formattato `X:21:10`.

7.11 Interfacce: Definizione, Costanti ed Ereditarietà

Natura di un'Interfaccia in Java. Un'interfaccia introduce un nuovo **tipo di riferimento** (*reference type*) il cui scopo primario è definire un contratto formale di comportamento per classi altrimenti del tutto eterogenee e non correlate.

- **Assenza di stato d'istanza**: un'interfaccia non può dichiarare variabili di istanza né memorizzare alcuno stato mutabile;
- **Non istanziabilità**: non può essere istanziata direttamente tramite l'operatore `new`, ma può essere unicamente implementata da classi o estesa da altre interfacce;
- **Membri ammessi**: a partire dalle specifiche moderne di Java, un'interfaccia può contenere esclusivamente costanti, firme di metodi astratti, metodi `default`, metodi `static` e tipi annidati (*nested types*);
- **Disaccoppiamento**: elimina la necessità per classi correlate sul piano comportamentale di condividere una comune superclasse astratta nell'albero gerarchico, evitando di forzare metodi specifici all'interno della classe radice `Object`.

Dichiarazione, Costanti ed Ereditarietà Multipla. La sintassi di un'interfaccia supporta sia l'ereditarietà singola che l'**ereditarietà multipla** tramite la clausola `extends`: poiché le interfacce non contengono dati d'istanza, l'ereditarietà multipla non incorre nelle problematiche e ambiguità tipiche del C++ (come la duplicazione dei campi nel problema del diamante).

```
1 // Esempio base: contratto comportamentale
2 public interface Bicycle {
3     void changeCadence(int newValue);
4     void changeGear(int newValue);
5     void speedUp(int increment);
6     void applyBrakes(int decrement);
7 }
8
```

```java
// Ereditarieta multipla tra interfacce e costanti
public interface GroupedInterface extends Interface1, Interface2,
    Interface3 {
     // Costante implicita: compilata come public static final
     double E = 2.718282;

     // Firme di metodi astratti: implicitamente public e abstract
     void doSomething(int i, double x);
     int doSomethingElse(String s);
}
```

- **Costanti implicite**: tutte le variabili dichiarate all'interno di un'interfaccia sono implicitamente `public`, `static` e `final`;
- **Metodi impliciti**: tutte le firme prive di corpo, così come i metodi `default` e `static`, possiedono automaticamente visibilità `public`.

Implementazione Concreta. Quando una classe concreta implementa un'interfaccia, il compilatore impone che tutti i metodi astratti dichiarati siano provvisti di un'implementazione esplicita; l'assenza anche di una sola definizione comporta il fallimento della compilazione:

```java
class BianchiBicycle implements Bicycle {
     int cadence = 0, speed = 0, gear = 1;

     public void changeCadence(int newValue) { cadence = newValue; }
     public void changeGear(int newValue) { gear = newValue; }
     public void speedUp(int increment) { speed += increment; }
     public void applyBrakes(int decrement) { speed -= decrement; }

     void printStates() {
         System.out.println("cadence: " + cadence + " speed: " + speed + "
    gear: " + gear);
     }
}
```

7.12 Metodi nelle Interfacce e Risoluzione dei Conflitti

Metodi Default e Metodi Statici. Con l'introduzione di Java 8, le interfacce hanno acquisito la capacità di dichiarare metodi dotati di corpo:

- **Metodi Statici** (`static`): definiscono comportamenti generali o funzioni helper collegate al tipo. Non consentono l'accesso a dati d'istanza e non partecipano al meccanismo di polimorfismo/overriding (vengono invocati unicamente tramite il nome dell'interfaccia, es. `InterfaceName.method()`).
- **Metodi Default** (`default`): forniscono un'implementazione standard del metodo che viene ereditata dalle classi implementatrici. Possono essere ridefiniti liberamente (*overridden*) dalle classi derivate.

Le Due Regole Fondamentali di Risoluzione dei Conflitti. Quando una classe implementa molteplici interfacce o estende una superclasse ereditando definizioni concorrenti per un metodo con la medesima firma, il compilatore Java adotta due principi gerarchici deterministici per risolvere le ambiguità:

1. **I metodi d'istanza di classe vincono sui metodi default delle interfacce** (*"Classes beat interfaces"*).
2. **I metodi già sovrascritti da altri candidati nella gerarchia vengono ignorati** (*"More specific interfaces beat general ones"*).

Regola 1: Metodi d'Istanza di Classe vs Metodi Default. Se una superclasse concreta fornisce un metodo d'istanza e un'interfaccia fornisce un metodo `default` avente identica firma, l'implementazione della classe genitore ha sempre la precedenza:

```
public class Horse {
    public String identifyMyself() { return "I am a horse."; }
}

public interface Flyer {
    default public String identifyMyself() { return "I am able to fly.";
   }
}

public interface Mythical {
    default public String identifyMyself() { return "I am a mythical
   creature."; }
}

public class Pegasus extends Horse implements Flyer, Mythical {
    public static void main(String... args) {
        Pegasus myApp = new Pegasus();
        System.out.println(myApp.identifyMyself()); // Restituisce: "I am
    a horse."
    }
}
```

Regola 2: Metodi Default Ereditati e Gerarchie Concorrenti. Quando una classe implementa due interfacce che condividono un antenato comune, l'interfaccia che ha già effettuato l'override del metodo nasconde l'implementazione dell'interfaccia più generica:

```
public interface Animal {
    default public String identifyMyself() { return "I am an animal."; }
}

public interface EggLayer extends Animal {
    default public String identifyMyself() { return "I am able to lay
   eggs."; }
}

```

```java
public interface FireBreather extends Animal { }

public class Dragon implements EggLayer, FireBreather {
    public static void main(String... args) {
        Dragon myApp = new Dragon();
        System.out.println(myApp.identifyMyself()); // Restituisce: "I am able to lay eggs."
    }
}
```

Poiché `EggLayer` specializza `Animal` ridefinendo `identifyMyself()`, la sua versione è considerata più specifica rispetto a quella ereditata per via transitiva da `FireBreather`.

Conflitti Indipendenti e Disambiguazione Esplicita con `super`. Se due interfacce indipendenti forniscono implementazioni `default` per lo stesso metodo, oppure se un metodo `default` entra in conflitto con un metodo astratto privo di corpo, il compilatore genera un **errore di compilazione**. In questo scenario, la classe implementatrice è obbligata a effettuare l'override esplicito del metodo.
All'interno dell'override, è possibile richiamare una o entrambe le implementazioni di default facendo ricorso alla parola chiave `super` preceduta dal nome dell'interfaccia di riferimento:

```
NomeInterfaccia.super.nomeMetodo(parametri);
```

```java
public interface OperateCar {
    default public int startEngine(EncryptedKey key) {
        // Implementazione accensione auto convenzionale
        return 1;
    }
}

public interface FlyCar {
    default public int startEngine(EncryptedKey key) {
        // Implementazione accensione modulo di volo
        return 2;
    }
}

public class FlyingCar implements OperateCar, FlyCar {
    // Override obbligatorio: risoluzione del conflitto tra interfacce concorrenti
    public int startEngine(EncryptedKey key) {
        // Invocazione selettiva o combinata dei comportamenti originali
        OperateCar.super.startEngine(key);
        return FlyCar.super.startEngine(key);
    }
}
```

Metodi d'Istanza di Superclasse che Soddisfano Metodi Astratti. Un metodo d'istanza ereditato da una superclasse ordinaria può soddisfare direttamente la dichiarazione di un metodo

astratto presente in un'interfaccia implementata, senza richiedere la riscrittura del codice nella classe figlia:

```
public interface Mammal {
    String identifyMyself(); // Metodo astratto privo di corpo
}

public class Horse {
    public String identifyMyself() { return "I am a horse."; }
}

public class Mustang extends Horse implements Mammal {
    public static void main(String... args) {
        Mustang myApp = new Mustang();
        System.out.println(myApp.identifyMyself()); // Restituisce: "I am a horse."
    }
}
```

La classe `Mustang` eredita `identifyMyself()` da `Horse`, soddisfacendo automaticamente il contratto imposto dall'interfaccia `Mammal`.

Quadro Sinottico sull'Overriding dei Metodi. Il comportamento delle interazioni tra metodi statici e d'istanza tra superclassi e sottoclassi è formalizzato dalla seguente matrice:

	Superclass Instance Method	**Superclass Static Method**
Subclass Instance Method	**Overrides** (Polimorfismo dinamico)	**Errore di compilazione**
Subclass Static Method	**Errore di compilazione**	**Hides** (Nascondimento statico)

7.13 Evoluzione delle Interfacce e Retrocompatibilità

La Motivazione Architetturale: Interface Evolution. Prima dell'introduzione dei metodi `default`, l'evoluzione delle API pubbliche presentava una criticità strutturale: **aggiungere un nuovo metodo astratto a un'interfaccia già distribuita rompeva istantaneamente tutto il codice client preesistente**, poiché ogni classe concreta era forzata a implementarlo per poter compilare.

Il Problema dell'Evoluzione delle API (Esempio Oracle)

Si consideri un'interfaccia originaria rilasciata in una libreria di sistema:

```
public interface DoIt {
    void doSomething(int i, double x);
    int doSomethingElse(String s);
}
```

Per evolvere l'architettura aggiungendo un controllo di validazione (`didItWork`), l'aggiunta di un semplice metodo astratto causerebbe la rottura di tutti i progetti implementatori terzi:

```
public interface DoIt {
    void doSomething(int i, double x);
    int doSomethingElse(String s);
    // Errore nei client legacy: le vecchie classi non compilano piu!
    boolean didItWork(int i, double x, String s);
}
```

Con i metodi `default`, l'interfaccia può espandere il proprio perimetro funzionale garantendo la piena retrocompatibilità (*backward compatibility*): le classi storiche continuano a compilare ereditando l'implementazione predefinita, mentre le nuove classi possono ridefinirla secolarmente.

```
public interface DoIt {
    void doSomething(int i, double x);
    int doSomethingElse(String s);

    // Integrazione sicura: corpo fornito di default, nessun client esistente si rompe
    default boolean didItWork(int i, double x, String s) {
        // Implementazione di fallback predefinita
        return true;
    }
}
```

7.14 Classi Anonime (Anonymous Classes)

Definizione e Caratteristiche Fondamentali. Una **classe anonima** (*anonymous class*) è una classe locale priva di identificatore (senza nome), che viene **dichiarata e istanziata simultaneamente** all'interno di una singola espressione.

- **Scopo primario**: consente di rendere il codice più compatto e leggibile quando una determinata implementazione serve **una sola volta** (*one-off / disposable use*);
- **Dichiarazione in-line**: la definizione della classe è incorporata direttamente nell'espressione in cui l'oggetto viene creato e passato come parametro;
- **Eredità e Contratti**: una classe anonima non può estendere esplicitamente una classe e contemporaneamente implementare un'interfaccia; deve estendere esattamente una classe preesistente oppure implementare una specifica interfaccia;
- **Regole di Scoping**: condivide le medesime regole di visibilità delle classi locali (accesso ai membri d'istanza della classe contenitrice in contesti non-statici, cattura delle sole variabili locali `final` o *effectively final*).

7.15 Gestione degli Eventi e Listener in Java GUI

Architettura a Eventi nelle Interfacce Grafiche (Swing/AWT). Nelle interfacce grafiche orientate agli oggetti, il flusso di controllo è guidato dagli eventi (*Event-Driven*). I componenti grafici interagiscono secondo il pattern architetturale basato su:

- **Sorgenti di Evento (*Event Sources*)**: componenti UI (come `JButton`, `JTextField`, `JCheckBox`) in grado di rilevare interazioni fisiche dell'utente (click del mouse, digitazione di testo, ridimensionamento);
- **Ascoltatori di Evento (*Event Listeners*)**: oggetti preposti a intercettare tali notifiche e a eseguire la corrispondente logica di reazione;
- **Disaccoppiamento** $N : 1$: più sorgenti grafiche possono condividere il medesimo listener qualora debbano compiere azioni simili, così come un singolo componente può notificare una moltitudine di listener indipendenti.

L'Interfaccia `ActionListener`. Nel caso specifico della pressione di un pulsante (`JButton`), il contratto standard della libreria Java è rappresentato dall'interfaccia `ActionListener`:

```
1 public interface ActionListener extends EventListener {
2     public void actionPerformed(ActionEvent e);
3 }
```

- L'interfaccia definisce un unico metodo astratto: `actionPerformed(ActionEvent e)`;
- L'oggetto `ActionEvent` contiene i metadati associati all'accadimento: ad esempio, il metodo `e.getActionCommand()` restituisce la stringa di comando o l'etichetta testuale associata al pulsante premuto.

7.16 Caso Studio SimpleCalc: I Tre Approcci a Confronto

Per comprendere appieno la convergenza tra classi interne, scoping e il pattern Strategy, si analizza la realizzazione di una calcolatrice semplificata (`SimpleCalc`). L'interfaccia grafica comprende un campo di testo (`numberCalc`) e un insieme di pulsanti disposti su una griglia:

- Pulsanti numerici da `1` a `9`;
- Pulsanti operatore (`+` e `-`);
- Pulsante di calcolo finale (`=`).

Java consente di implementare l'interfaccia `ActionListener` per i pulsanti secondo tre strategie distinte, ciascuna appropriata per una specifica configurazione di riuso e accoppiamento:

1. Implementazione direttamente all'interno della **classe contenitrice** (*outer class as listener*);
2. Implementazione tramite **member inner class**;
3. Implementazione tramite **classe anonima** (*anonymous inner class*).

Approccio 1: Implementazione nella Classe Contenitrice. Quando una molteplicità di componenti grafici compie esattamente la medesima azione di base, la soluzione più lineare consiste nel far implementare `ActionListener` direttamente alla classe contenitrice principale (`SimpleCalc`). È il caso dei tasti numerici da 1 a 9: alla pressione di uno qualsiasi di essi, il numero corrispondente deve essere semplicemente visualizzato nel `JTextField`.

```
public class SimpleCalc implements ActionListener {
    private JFrame guiFrame;
    private JTextField numberCalc;
    private JPanel buttonPanel;

    public SimpleCalc() {
        guiFrame = new JFrame();
        numberCalc = new JTextField();
        buttonPanel = new JPanel();
        buttonPanel.setLayout(new GridLayout(4, 3));
        guiFrame.add(buttonPanel, BorderLayout.CENTER);

        // Creazione iterativa dei bottoni numerici
        for (int i = 1; i < 10; i++) {
            addButton(buttonPanel, String.valueOf(i));
        }
    }

    private void addButton(Container parent, String name) {
        JButton button = new JButton(name);
        button.setActionCommand(name);
        // Registrazione: la classe stessa gestisce l'evento tramite 'this'
        button.addActionListener(this);
        parent.add(button);
    }

    @Override
    public void actionPerformed(ActionEvent event) {
        // Intercetta l'etichetta del bottone premuto e aggiorna il display
        String action = event.getActionCommand();
        numberCalc.setText(action);
    }
}
```

- **Vantaggio**: centralizza la gestione in un unico punto senza creare classi ausiliarie;
- **Limite**: scala male se i pulsanti devono compiere compiti eterogenei, costringendo a lunghe catene di `if-else` per distinguere la sorgente dell'evento.

Approccio 2: Implementazione con Member Inner Class. Quando i componenti svolgono operazioni logiche affini che condividono una struttura comune ma differiscono per parametri interni, l'uso di una **member inner class** rappresenta la scelta architetturale più coesa.

I pulsanti di addizione (+) e sottrazione (-) memorizzano il primo operando e impostano il codice dell'operazione. La classe interna `OperatorAction` incapsula il discriminante numerico (`operator`), avendo al contempo accesso diretto ai campi privati dell'outer class (`currentCalc` e `calcOperation`):

```
public class SimpleCalc {
    private JTextField numberCalc;
    private int currentCalc = 0;
    private int calcOperation = 0; // 1 = addizione, 2 = sottrazione

    // Member Inner Class: parametrizzabile e dotata di accesso allo
   stato outer
    private class OperatorAction implements ActionListener {
        private int operator;

        public OperatorAction(int operator) {
            this.operator = operator;
        }

        @Override
        public void actionPerformed(ActionEvent event) {
            if (!numberCalc.getText().isEmpty()) {
                currentCalc = Integer.parseInt(numberCalc.getText());
                calcOperation = operator;
            }
        }
    }

    private void setupOperators(JPanel buttonPanel) {
        JButton additionButton = new JButton("+");
        additionButton.setActionCommand("+");
        additionButton.addActionListener(new OperatorAction(1));
        buttonPanel.add(additionButton);

        JButton subtractionButton = new JButton("-");
        subtractionButton.setActionCommand("-");
        subtractionButton.addActionListener(new OperatorAction(2));
        buttonPanel.add(subtractionButton);
    }
}
```

- **Vantaggio**: elimina la duplicazione di codice; l'operazione è parametrizzata tramite costruttore preservando l'information hiding verso l'esterno;
- **Legame d'istanza**: ogni istanza di `OperatorAction` è legata all'oggetto `SimpleCalc` chiamante e ne muta direttamente lo stato interno.

Approccio 3: Implementazione con Classe Anonima. Spesso un determinato pulsante ha un comportamento del tutto unico ed esclusivo nell'intero sistema. Creare una classe dedica-

ta (anche interna) per un'operazione invocata in un solo punto causerebbe una **proliferazione ingiustificata di classi nominali**, appesantendo la leggibilità del progetto.
Il pulsante di uguaglianza (=) deve prelevare il secondo operando, valutare `calcOperation`, calcolare il risultato finale e aggiornare il testo a video. Costituisce il candidato naturale per una **classe anonima**:

```
JButton equalsButton = new JButton("=");

// Dichiarazione e istanziazione in-line di una classe anonima
equalsButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent event) {
        if (!numberCalc.getText().isEmpty()) {
            int number = Integer.parseInt(numberCalc.getText());
            if (calcOperation == 1) {
                int result = currentCalc + number;
                numberCalc.setText(String.valueOf(result));
            } else if (calcOperation == 2) {
                int result = currentCalc - number;
                numberCalc.setText(String.valueOf(result));
            }
        }
    }
});
```

Riflessione Teorica: L'Illusione del Passaggio di Codice

Dal punto di vista della sintassi, il programmatore percepisce la classe anonima come la trasmissione di un **blocco di codice al volo** passato come argomento al metodo `addActionListener`.

La realtà computazionale sottostante:

- In Java non si stanno passando puntatori a funzione né indirizzi di memoria;
- Il compilatore sintetizza a tempo di compilazione una vera e propria classe anonima (denominata internamente `SimpleCalc$1.class`) che implementa l'interfaccia `ActionListener`;
- A runtime viene istanziato un oggetto concreto di tale classe anonima e il suo riferimento d'oggetto viene passato come parametro formale polimorfico.

7.17 Sintesi Architetturale: Dalle Classi Anonime alle Lambda

La Convergenza verso lo Strategy Pattern. L'evoluzione delle tecniche di scoping in Java analizzate nel modulo illustra come il linguaggio abbia progressivamente affinato il supporto al pattern Strategy:

- **C++ / C**: uso di puntatori a funzione o template (scelte rapide, ma prive di incapsulamento integrato o vincolate a tempo di compilazione);

- **Smalltalk**: chiusure lessicali pure (*blocks*) capaci di catturare l'ambiente circostante in modo del tutto trasparente;
- **Java Standard (Pre-Java 8)**: incapsulamento rigoroso tramite classi annidate, locali e anonime. La strategia algoritmica viene trattata come un **oggetto polimorfo** che implementa un'interfaccia contrattuale;
- **Java Moderno (Java 8+)**: introduzione delle **espressioni Lambda** e dei riferimenti a metodo (*Method References*), che offrono lo "zucchero sintattico" ideale per sintetizzare interfacce a singolo metodo astratto (*Functional Interfaces* o SAM – *Single Abstract Method*), colmando definitivamente il divario espressivo con i blocchi Smalltalk pur conservando la sicurezza del sistema di tipi orientato agli oggetti.

Strategia di Scoping	**Visibilità del Tipo**	**Accoppiamento con l'Outer Class**	**Caso d'Uso Ottimale**
Outer as Listener	Pubblica / Package	Totale (è la classe stessa a rispondere)	Molteplici componenti con comportamento identico (es. tastierino 1–9).
Member Inner Class	Privata / Limitata	Accede a campi d'istanza dell'outer via `Outer.this`	Operazioni correlate parametrizzate (es. operatori aritemetici `+` e `-`).
Anonymous Class	Locale all'espressione	Accede al contesto outer e a variabili (effectively) final	Comportamenti univoci e monouso (*one-off*), come il tasto `=`.